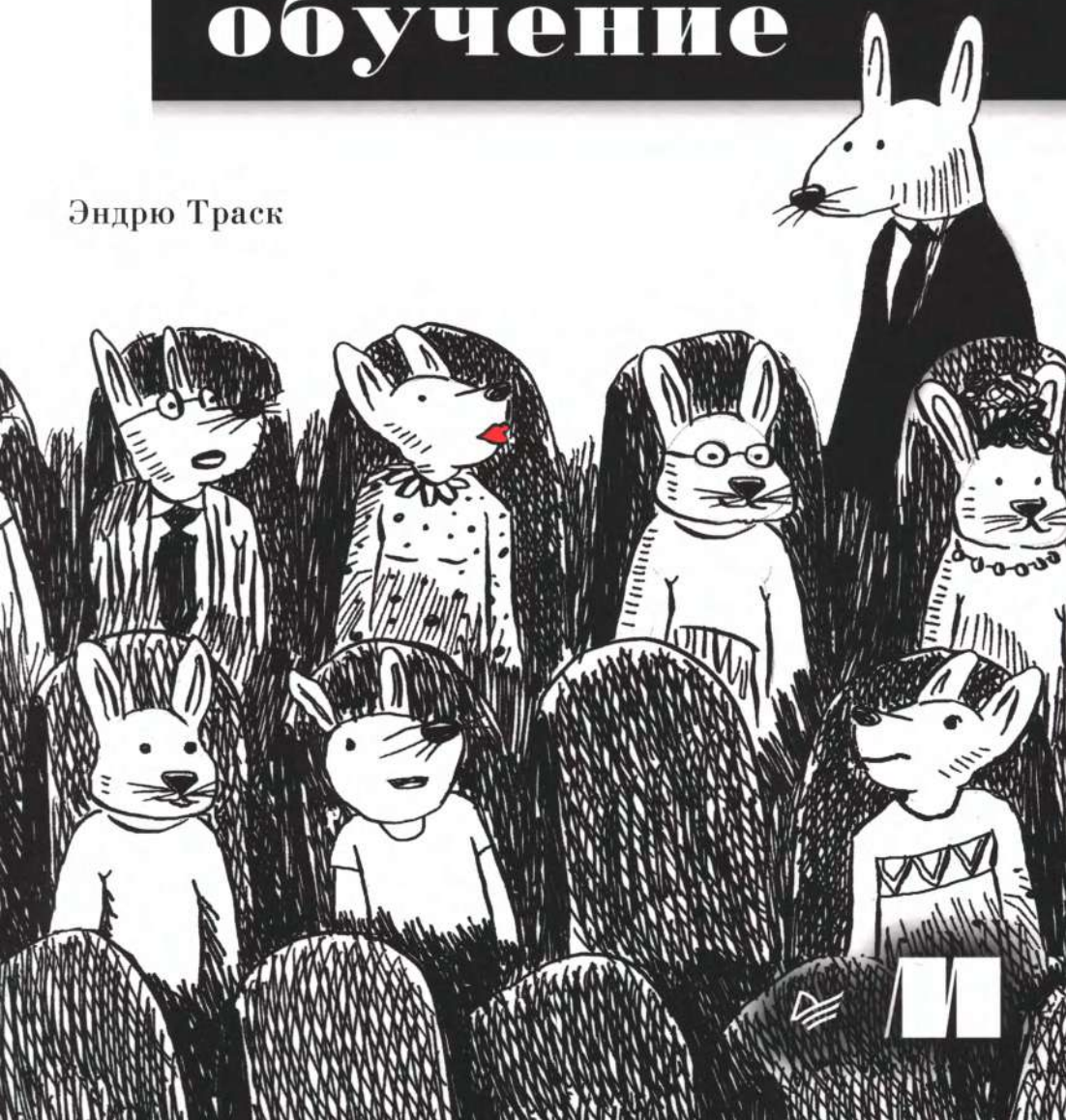


**грокаем**

# Глубокое обучение

Эндрю Траск



*Моей маме. Ты вложила так много сил и времени, чтобы дать хорошее образование нам с Тарой. И я надеюсь, в этой книге ты увидишь и свой вклад.*

*И папе. Спасибо, что любишь нас так сильно и нашел время, чтобы обучить меня программированию и технологиям в еще очень юном возрасте. Я бы не добился этого без тебя.*

*Для меня честь быть вашим сыном.*

# grokking **Deep Learning**

---

Andrew W. Trask



MANNING

SHELTER ISLAND

# **играем Глубокое обучение**

Эндрю Траск



Санкт-Петербург • Москва • Екатеринбург • Воронеж  
Нижний Новгород • Ростов-на-Дону • Самара • Минск

2019

Эндрю Траск

## Грокаем глубокое обучение

Серия «Библиотека программиста»

Перевел с английского А. Киселев

|                         |                          |
|-------------------------|--------------------------|
| Заведующая редакцией    | Ю. Сергиенко             |
| Ведущий редактор        | К. Тульцева              |
| Литературный редактор   | А. Бульченко             |
| Художественный редактор | В. Мостипан              |
| Корректоры              | С. Беляева, М. Молчанова |
| Верстка                 | Л. Егорова               |

ББК 32.813+32.973.23-018

УДК 004.89

Траск Эндрю

Т65 Грокаем глубокое обучение. — СПб.: Питер, 2019. — 352 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-1334-7

Глубокое обучение — это раздел искусственного интеллекта, цель которого научить компьютеры обучаться с помощью нейронных сетей — технологии, созданной по образу и подобию человеческого мозга. Онлайн-переводчики, беспилотные автомобили, рекомендации по выбору товаров именно для вас и виртуальные голосовые помощники — вот лишь несколько достижений, которые стали возможны благодаря глубокому обучению.

«Грокаем глубокое обучение» научит конструировать нейронные сети с нуля! Эндрю Траск знакомит со всеми деталями и тонкостями этой нелегкой задачи. Python и библиотека NumPy способны научить ваши нейронные сети видеть и распознавать изображения, переводить любые тексты на все языки мира и даже писать не хуже Шекспира!

**16+** (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-1617293702 англ.  
ISBN 978-5-4461-1334-7

© 2019 by Manning Publications Co. All rights reserved.

© Перевод на русский язык ООО Издательство «Питер», 2019

© Издание на русском языке, оформление ООО Издательство «Питер», 2019

© Серия «Библиотека программиста», 2019

Права на издание получены по соглашению с Manning Publications Co. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,

Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 07.2019. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 25.06.19. Формат 70×100/16. Бумага офсетная. Усл. п. л. 28,380. Тираж 3000. Заказ № ВЗК-04461-19

Отпечатано в АО «Первая Образцовая типография», филиал «Дом печати — ВЯТКА»  
610033, г. Киров, ул. Московская, 122.



# Оглавление

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Предисловие</b>                                       | <b>12</b> |
| <b>Благодарности</b>                                     | <b>14</b> |
| <b>О книге</b>                                           | <b>16</b> |
| Кому адресована книга                                    | 16        |
| Структура                                                | 16        |
| Соглашения об оформлении кода и его загрузке             | 18        |
| Форум книги                                              | 18        |
| <b>Об авторе</b>                                         | <b>19</b> |
| От издательства                                          | 19        |
| <b>1 Введение в глубокое обучение: зачем его изучать</b> | <b>20</b> |
| Добро пожаловать в «Грожаем глубокое обучение»!          | 21        |
| Почему вам стоит изучать глубокое обучение               | 21        |
| Этому трудно учиться?                                    | 22        |
| Почему вы должны прочитать эту книгу                     | 23        |
| Что нужно для начала                                     | 25        |
| Возможно, вам потребуется знание Python                  | 26        |
| Итоги                                                    | 26        |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>2 Основные понятия: как учатся машины? . . . . .</b>                        | <b>27</b> |
| Что такое глубокое обучение? . . . . .                                         | 28        |
| Что такое машинное обучение? . . . . .                                         | 29        |
| Машинное обучение с учителем . . . . .                                         | 30        |
| Машинное обучение без учителя . . . . .                                        | 31        |
| Параметрическое и непараметрическое обучение . . . . .                         | 32        |
| Параметрическое обучение с учителем . . . . .                                  | 33        |
| Параметрическое обучение без учителя . . . . .                                 | 35        |
| Непараметрическое обучение . . . . .                                           | 37        |
| Итоги . . . . .                                                                | 38        |
| <b>3 Введение в нейронное прогнозирование: прямое распространение . . . .</b>  | <b>39</b> |
| Шаг 1: прогнозирование . . . . .                                               | 40        |
| Простая нейронная сеть, делающая прогноз . . . . .                             | 42        |
| Что такое нейронная сеть? . . . . .                                            | 43        |
| Что делает эта нейронная сеть? . . . . .                                       | 44        |
| Прогнозирование с несколькими входами . . . . .                                | 47        |
| Несколько входов: что делает эта нейронная сеть? . . . . .                     | 49        |
| Несколько входов: полный выполняемый код . . . . .                             | 54        |
| Прогнозирование с несколькими выходами . . . . .                               | 56        |
| Прогнозирование с несколькими входами и выходами . . . . .                     | 58        |
| Несколько входов и выходов: как это работает? . . . . .                        | 60        |
| Прогнозирование на основе прогнозов . . . . .                                  | 62        |
| Короткий пример использования NumPy . . . . .                                  | 64        |
| Итоги . . . . .                                                                | 67        |
| <b>4 Введение в нейронное обучение: градиентный спуск . . . . .</b>            | <b>69</b> |
| Предсказание, сравнение и обучение . . . . .                                   | 70        |
| Сравнение . . . . .                                                            | 70        |
| Обучение . . . . .                                                             | 71        |
| Сравнение: способны ли нейронные сети делать точные прогнозы? . . . . .        | 71        |
| Зачем измерять ошибку? . . . . .                                               | 72        |
| Как выглядит простейшая форма нейронного обучения? . . . . .                   | 74        |
| Обучение методом «холодно/горячо» . . . . .                                    | 76        |
| Особенности обучения методом «холодно/горячо» . . . . .                        | 77        |
| Вычисление направления и величины из ошибки . . . . .                          | 79        |
| Одна итерация градиентного спуска . . . . .                                    | 81        |
| Обучение просто уменьшает ошибку . . . . .                                     | 83        |
| Рассмотрим несколько циклов обучения . . . . .                                 | 86        |
| Как это работает? Что такое <code>weight_delta</code> на самом деле? . . . . . | 88        |
| Узкий взгляд на одно понятие . . . . .                                         | 90        |

|                                                                                       |            |
|---------------------------------------------------------------------------------------|------------|
| Коробка со стержнями                                                                  | 91         |
| Производные: второй пример                                                            | 92         |
| Что действительно необходимо знать                                                    | 94         |
| Что знать необязательно                                                               | 94         |
| Как использовать производные для обучения                                             | 95         |
| Выглядит знакомо?                                                                     | 97         |
| Ломаем градиентный спуск                                                              | 98         |
| Визуальное представление избыточной коррекции                                         | 99         |
| Расхождение                                                                           | 100        |
| Знакомьтесь: альфа-коэффициент                                                        | 101        |
| Альфа-коэффициент в коде                                                              | 102        |
| Запоминание                                                                           | 103        |
| <b>5 Корректировка сразу нескольких весов: обобщение градиентного спуска</b>          | <b>104</b> |
| Обучение методом градиентного спуска с несколькими входами                            | 105        |
| Градиентный спуск с несколькими входами, описание                                     | 107        |
| Рассмотрим несколько шагов обучения                                                   | 113        |
| Замораживание одного веса: для чего?                                                  | 115        |
| Обучение методом градиентного спуска с несколькими выходами                           | 117        |
| Обучение методом градиентного спуска с несколькими входами и выходами                 | 120        |
| Чему обучаются эти веса?                                                              | 121        |
| Визуализация значений весов                                                           | 124        |
| Визуализация скалярных произведений (сумм весов)                                      | 125        |
| Итоги                                                                                 | 126        |
| <b>6 Создание первой глубокой нейронной сети: введение в обратное распространение</b> | <b>127</b> |
| Задача о светофоре                                                                    | 128        |
| Подготовка данных                                                                     | 130        |
| Матрицы и матричные отношения                                                         | 131        |
| Создание матриц в Python                                                              | 134        |
| Создание нейронной сети                                                               | 135        |
| Обучение на полном наборе данных                                                      | 137        |
| Полный, пакетный и стохастический градиентный спуск                                   | 138        |
| Нейронные сети изучают корреляцию                                                     | 139        |
| Повышающее и понижающее давление                                                      | 140        |
| Пограничный случай: переобучение                                                      | 142        |
| Пограничный случай: конфликт давлений                                                 | 143        |
| Определение косвенной корреляции                                                      | 145        |
| Создание корреляции                                                                   | 146        |
| Объединение нейронных сетей в стек: обзор                                             | 147        |



|                                                                                                      |            |
|------------------------------------------------------------------------------------------------------|------------|
| Обратное распространение: определение причин ошибок на расстоянии . . . . .                          | 148        |
| Обратное распространение: как это работает? . . . . .                                                | 150        |
| Линейность и нелинейность . . . . .                                                                  | 151        |
| Почему составная нейронная сеть не работает . . . . .                                                | 152        |
| Тайна эпизодической корреляции . . . . .                                                             | 153        |
| Короткий перерыв . . . . .                                                                           | 154        |
| Ваша первая глубокая нейронная сеть . . . . .                                                        | 155        |
| Обратное распространение в коде . . . . .                                                            | 156        |
| Одна итерация обратного распространения . . . . .                                                    | 159        |
| Объединяем все вместе . . . . .                                                                      | 161        |
| Почему глубокие сети важны для нас? . . . . .                                                        | 162        |
| <b>7 Как изобразить нейронную сеть: в голове и на бумаге . . . . .</b>                               | <b>164</b> |
| Время упрощать . . . . .                                                                             | 165        |
| Обобщение корреляции . . . . .                                                                       | 166        |
| Прежняя усложненная визуализация . . . . .                                                           | 167        |
| Упрощенная визуализация . . . . .                                                                    | 169        |
| Еще более упрощенная визуализация . . . . .                                                          | 170        |
| Посмотрим, как эта сеть получает прогноз . . . . .                                                   | 171        |
| Визуализация с использованием букв вместо картинок . . . . .                                         | 173        |
| Связывание переменных . . . . .                                                                      | 174        |
| Сравнение разных способов визуализации . . . . .                                                     | 175        |
| Важность инструментов визуализации . . . . .                                                         | 175        |
| <b>8 Усиление сигнала и игнорирование шума: введение<br/>в регуляризацию и группировку . . . . .</b> | <b>177</b> |
| Трехслойная сеть для классификации набора данных MNIST . . . . .                                     | 178        |
| Это было просто . . . . .                                                                            | 180        |
| Запоминание и обобщение . . . . .                                                                    | 181        |
| Переобучение нейронных сетей . . . . .                                                               | 182        |
| Причины переобучения . . . . .                                                                       | 184        |
| Простейшая регуляризация: ранняя остановка . . . . .                                                 | 185        |
| Стандартный способ регуляризации: прореживание (дропаут) . . . . .                                   | 186        |
| Как работает прореживание: в работе участвуют ансамбли . . . . .                                     | 187        |
| Прореживание в коде . . . . .                                                                        | 188        |
| Влияние прореживания на модель MNIST . . . . .                                                       | 191        |
| Пакетный градиентный спуск . . . . .                                                                 | 192        |
| Итоги . . . . .                                                                                      | 194        |
| <b>9 Моделирование случайности и нелинейности: функции активации . . .</b>                           | <b>195</b> |
| Что такое функция активации? . . . . .                                                               | 196        |
| Стандартные функции активации для скрытых слоев . . . . .                                            | 200        |

|                                                                                                    |            |
|----------------------------------------------------------------------------------------------------|------------|
| Стандартные функции активации для выходного слоя                                                   | 201        |
| Главная проблема: входные данные могут быть схожи между собой                                      | 204        |
| Вычисление softmax                                                                                 | 205        |
| Инструкции по внедрению функций активации                                                          | 207        |
| Умножение разности на производную                                                                  | 209        |
| Преобразование выхода в наклон (производную)                                                       | 211        |
| Усовершенствование сети MNIST                                                                      | 212        |
| <b>10 Края и углы нейронного обучения: введение в сверточные нейронные сети</b>                    | <b>215</b> |
| Повторное использование весов в нескольких местах                                                  | 216        |
| Сверточный слой                                                                                    | 217        |
| Простая реализация в NumPy                                                                         | 220        |
| Итоги                                                                                              | 224        |
| <b>11 Нейронные сети, понимающие человеческий язык: король – мужчина + женщина == ?</b>            | <b>226</b> |
| Что значит понимать человеческий язык?                                                             | 227        |
| Обработка естественного языка (NLP)                                                                | 228        |
| Обработка естественного языка с учителем                                                           | 229        |
| Набор данных IMDB с обзорами фильмов                                                               | 230        |
| Выявление корреляции слов во входных данных                                                        | 231        |
| Прогнозирование обзоров фильмов                                                                    | 232        |
| Введение в слой с векторным представлением                                                         | 234        |
| Интерпретация результата                                                                           | 236        |
| Нейронная архитектура                                                                              | 237        |
| Сравнение векторных представлений слов                                                             | 240        |
| В чем заключается смысл нейрона?                                                                   | 241        |
| Подстановка пропущенных слов                                                                       | 242        |
| Смысл определяется потерями                                                                        | 244        |
| Король – мужчина + женщина ≈ королева                                                              | 248        |
| Словесные аналогии                                                                                 | 249        |
| Итоги                                                                                              | 251        |
| <b>12 Нейронные сети, которые пишут как Шекспир: рекуррентные слои для данных переменной длины</b> | <b>252</b> |
| Проблема произвольной длины                                                                        | 253        |
| Действительно ли сравнение имеет значение?                                                         | 254        |
| Удивительная мощь усредненных векторов слов                                                        | 255        |
| Как векторные представления хранят информацию?                                                     | 257        |
| Как нейронная сеть использует векторные представления?                                             | 258        |
| Ограничение векторов в модели «мешок слов»                                                         | 259        |

|                                                                                         |            |
|-----------------------------------------------------------------------------------------|------------|
| Объединение векторных представлений слов с использованием единичной матрицы             | 261        |
| Матрицы, которые ничего не меняют                                                       | 262        |
| Определение переходных матриц                                                           | 264        |
| Обучение созданию векторов предложений                                                  | 265        |
| Прямое распространение на Python                                                        | 266        |
| Как добавить сюда обратное распространение?                                             | 267        |
| Обучим ее!                                                                              | 268        |
| Подготовка                                                                              | 269        |
| Прямое распространение с данными произвольной длины                                     | 271        |
| Обратное распространение с данными произвольной длины                                   | 272        |
| Корректировка весов с данными произвольной длины                                        | 273        |
| Запуск и анализ результатов                                                             | 274        |
| Итоги                                                                                   | 277        |
| <b>13 Введение в автоматическую оптимизацию: создание фреймворка глубокого обучения</b> | <b>278</b> |
| Что такое фреймворк глубокого обучения?                                                 | 279        |
| Введение в тензоры                                                                      | 280        |
| Введение в автоматическое вычисление градиента (autograd)                               | 281        |
| Контрольная точка                                                                       | 283        |
| Тензоры, используемые многократно                                                       | 284        |
| Добавление поддержки тензоров многократного использования в реализацию autograd         | 286        |
| Как работает сложение в обратном распространении?                                       | 288        |
| Добавление поддержки отрицания                                                          | 289        |
| Добавление поддержки других операций                                                    | 290        |
| Использование autograd в обучении нейронной сети                                        | 295        |
| Добавление автоматической оптимизации                                                   | 297        |
| Добавление поддержки слоев разных типов                                                 | 298        |
| Слои, содержащие другие слои                                                            | 299        |
| Слои с функцией потерь                                                                  | 300        |
| Как научиться пользоваться фреймворком                                                  | 301        |
| Нелинейные слои                                                                         | 302        |
| Слой с векторным представлением                                                         | 304        |
| Добавление индексирования в autograd                                                    | 305        |
| Слой с векторным представлением (повтор)                                                | 306        |
| Слой с перекрестной энтропией                                                           | 307        |
| Рекуррентный слой                                                                       | 309        |
| Итоги                                                                                   | 313        |

|                                                                                                            |            |
|------------------------------------------------------------------------------------------------------------|------------|
| <b>14 Обучаем сеть писать как Шекспир: долгая краткосрочная память . . . .</b>                             | <b>314</b> |
| Моделирование языка символов . . . . .                                                                     | 315        |
| Необходимо усеченное обратное распространение . . . . .                                                    | 316        |
| Усеченное обратное распространение . . . . .                                                               | 317        |
| Образец вывода . . . . .                                                                                   | 321        |
| Затухающие и взрывные градиенты . . . . .                                                                  | 322        |
| Упрощенный пример обратного распространения в RNN . . . . .                                                | 323        |
| Ячейки долгой краткосрочной памяти (LSTM) . . . . .                                                        | 324        |
| Аналогия, помогающая понять идею вентилей LSTM . . . . .                                                   | 325        |
| Слой долгой краткосрочной памяти . . . . .                                                                 | 326        |
| Усовершенствование модели языка символов . . . . .                                                         | 328        |
| Обучение LSTM-модели языка символов . . . . .                                                              | 329        |
| Настройка LSTM-модели языка символов . . . . .                                                             | 330        |
| Итоги . . . . .                                                                                            | 331        |
| <b>15 Глубокое обучение на конфиденциальных данных: введение<br/>    в федеративное обучение . . . . .</b> | <b>332</b> |
| Проблема конфиденциальности в глубоком обучении . . . . .                                                  | 333        |
| Федеративное обучение . . . . .                                                                            | 334        |
| Обучаем выявлять спам . . . . .                                                                            | 335        |
| Сделаем модель федеративной . . . . .                                                                      | 337        |
| Взламываем федеративную модель . . . . .                                                                   | 338        |
| Безопасное агрегирование . . . . .                                                                         | 340        |
| Гомоморфное шифрование . . . . .                                                                           | 341        |
| Федеративное обучение с гомоморфным шифрованием . . . . .                                                  | 342        |
| Итоги . . . . .                                                                                            | 343        |
| <b>16 Куда пойти дальше: краткий путеводитель . . . . .</b>                                                | <b>345</b> |
| Поздравляю! . . . . .                                                                                      | 346        |
| Шаг 1: начните изучать PyTorch . . . . .                                                                   | 346        |
| Шаг 2: начните изучать следующий курс по глубокому обучению . . . . .                                      | 347        |
| Шаг 3: купите учебник по математике глубокого обучения . . . . .                                           | 347        |
| Шаг 4: заведите блог и рассказывайте в нем о глубоком обучении . . . . .                                   | 348        |
| Шаг 5: Twitter . . . . .                                                                                   | 349        |
| Шаг 6: напишите руководство на основе академической статьи . . . . .                                       | 350        |
| Шаг 7: получите доступ к GPU . . . . .                                                                     | 350        |
| Шаг 8: найдите оплачиваемую работу, связанную с глубоким обучением . . . . .                               | 351        |
| Шаг 9: присоединитесь к открытому проекту . . . . .                                                        | 351        |
| Шаг 10: ищите единомышленников . . . . .                                                                   | 352        |



## Предисловие

«Грокаем глубокое обучение» — это результат трехлетнего напряженного труда. Чтобы создать книгу, которую вы держите в руках, мне пришлось написать вдвое больше страниц, чем вы видите. Полдесятка глав три или четыре раза были переписаны заново, и только после этого я решил, что они готовы к публикации. Кроме этого, попутно были добавлены новые важные главы, отсутствовавшие в изначальном плане.

Что еще более важно, я в самом начале принял два решения, делающие мою книгу особенно ценной: эта книга не требует от читателя специальной математической подготовки, кроме знания основ арифметики, и не опирается на высокоуровневые библиотеки, которые могут скрывать происходящее в коде. Иначе говоря, любой сможет прочитать эту книгу и понять, как в действительности работает глубокое обучение. Для этого мне пришлось придумать новые способы описания и разъяснения основных идей и приемов, не прибегая к сложному математическому аппарату или замысловатому программному коду, написанному кем-то другим.

Работая над «Грокаем глубокое обучение», я преследовал цель максимально уменьшить порог входа в практику глубокого обучения. Вы не просто прочитаете теорию — вы откроете ее заново. А чтобы помочь вам в этом, я написал много кода и постарался выстроить объяснения в правильном порядке, чтобы фрагменты кода, необходимые для демонстрации, не были лишены смысла.

Эти знания, в сочетании с теорией, кодом и примерами, которые вы будете изучать в книге, помогут вам намного быстрее выполнять все наши эксперименты. Вы быстро добьетесь успеха и расширите свои практические навыки, а кроме того, вам проще будет освоить более сложные понятия глубокого обучения.

В последние три года я не только писал эту книгу, но еще и поступил в аспирантуру Оксфордского университета, присоединился к команде Google и стал одним из инициаторов проекта OpenMined — децентрализованной платформы искусственного интеллекта. Эта книга является кульминацией многих лет размышлений, обучения и преподавания.

Есть много источников знаний о глубоком обучении. И я рад, что вы выбрали именно этот.



## Благодарности

Я чрезвычайно благодарен всем, кто принял участие в работе над книгой. Прежде всего я хочу поблагодарить удивительную команду издательства Manning: Берта Бейтса (Bert Bates), научившего меня писать; Кристину Тейлор (Christina Taylor), терпеливо поддерживавшую меня в течение трех лет; Майкла Стивенса (Michael Stephens), чье творческое мышление позволило книге стать успешной еще до публикации; Марьяна Бейса (Marjan Vase), чья поддержка была определяющей во время задержек.

«Грожаем глубокое обучение» не получилась бы такой, какой получилась, без значительного вклада первых ее читателей, приславших отзывы по электронной почте, в Twitter и GitHub. Я очень признателен Яше Суишеру (Jascha Swisher), Варуну Судхакару (Varun Sudhakar), Франсуа Шолле (Francois Chollet), Фредерику Виторино (Frederico Vitorino), Коди Хаммонду (Cody Hammond), Маурисио Марото Арриете (Mauricio Maroto Arrieta), Александару Драгосавлевичу (Aleksandar Dragosavljevic), Алану Картеру (Alan Carter), Френку Хинеку (Frank Hinek), Николасу Бенджамину Хокеру (Nicolas Benjamin Hocker), Хенку Мейссе (Hank Meisse), Уотеру Хибме (Wouter Hibma), Йоргу Розенкранцу (Joerg Rosenkranz), Алексу Виейре (Alex Vieira) и Чарли Харрингтону (Charlie Harrington) за их помощь в улучшении текста и кода в онлайн-репозитории.

Хочу также поблагодарить рецензентов, выкроивших время, чтобы прочитать рукопись на разных этапах работы: Александра А. Мыльцева (Alexander

А. Myltsev), Амита Ламба (Amit Lamba), Ананда Саха (Anand Saha), Эндрю Хамора (Andrew Hamor), Кристиана Барриентоса (Cristian Barrientos), Монтойя (Montoya), Еремея Валетова (Eremey Valetov), Джеральда Мака (Gerald Mack), Яна Стирка (Ian Stirk), Каляна Редди (Kalyan Reddy), Камала Раджа (Kamal Raj), Кельвина Д. Микса (Kelvin D. Meeks), Марко Пауло душ Сантуш Ногейра (Marco Paulo dos Santos Nogueira), Мартина Бира (Martin Beer), Массимо Иларио (Massimo Ilario), Ненси У. Греди (Nancy W. Grady), Питера Хемптона (Peter Hampton), Себастьяна Мальдонада (Sebastian Maldonado), Шашанка Гулты (Shashank Gupta), Тимотеуша Воложко (Tymoteusz Wołodźko), Кумара Унникришнана (Kumar Unnikrishnan), Випула Гулты (Vipul Gupta), Уилла Фугера (Will Fuger) и Уильяма Уилера (William Wheeler).

Я чрезвычайно благодарен Мэту (Mat) и Нико (Niko) из Udacity, которые включили книгу в свой курс обучения Deep Learning Nanodegree, что очень помогло популяризации книги среди молодых специалистов по глубокому обучению.

Я должен поблагодарить доктора Уильяма Хупера (Dr. William Hooper), позволившего мне зайти в его кабинет и поспорить по вопросам информатики, сделавшего исключение и давшего мне возможность попасть на его курс (где уже не было мест) по программированию и вдохновившего меня на карьеру в области глубокого обучения. Я чрезвычайно благодарен за терпение, проявленное ко мне с самого начала. Вы были безмерно щедры ко мне.

Наконец, я хочу сказать спасибо моей жене за то, что терпела, когда я ночи напролет работал над книгой. Спасибо, что много раз исправляла ошибки в тексте, а также создавала и настраивала репозиторий кода на GitHub.





## О книге

Книга «Грокаем глубокое обучение» закладывает фундамент для дальнейшего овладения технологией глубокого обучения. Она начинается с описания основ нейронных сетей и затем подробно рассматривает дополнительные уровни и архитектуры.

## Кому адресована книга

Я специально писал книгу с намерением обеспечить минимально возможный порог входа. Вам не требуются знания линейной алгебры, численных методов, выпуклых оптимизаций и даже машинного обучения. Все, что потребуется для понимания глубокого обучения, будет разъясняться по ходу дела. Если вы окончили среднюю школу и владеете языком Python, этого будет вполне достаточно для чтения книги.

## Структура

Книга состоит из 16 глав:

- ❑ Глава 1 рассказывает, зачем необходимо изучать глубокое обучение и что вам потребуется на начальном этапе.
- ❑ Глава 2 начинает погружение в основные понятия, такие как машинное обучение, параметрические и непараметрические модели и обучение с учителем

и без учителя. Она также знакомит с парадигмой «предсказание, сравнение, обучение», рассмотрение которой будет продолжено в следующих главах.

- ❑ Глава 3 показывает примеры использования простых сетей для предсказания, а также впервые знакомит с нейронными сетями.
- ❑ Глава 4 научит оценивать прогнозы, сделанные в главе 3, и выявлять ошибки, что позволит продолжить обучение моделям на следующем этапе.
- ❑ Глава 5 акцентирует внимание на части «обучение» в парадигме «предсказание, сравнение, обучение». Эта глава рассматривает процесс обучения на более обширном примере.
- ❑ В главе 6 вы создадите свою первую «глубокую» нейронную сеть, напишете код и сделаете все остальное.
- ❑ Глава 7 содержит общий обзор нейронных сетей, что поможет вам сформировать свое представление.
- ❑ Глава 8 познакомит вас с такими понятиями, как переобучение, регуляризация и пакетный градиентный спуск, а также научит, как классифицировать набор своих данных в новой, только что созданной сети.
- ❑ Глава 9 расскажет о функциях активации и как их использовать при моделировании вероятностей.
- ❑ Глава 10 знакомит со сверточными нейронными сетями, акцентируя внимание на приемах для предотвращения переобучения.
- ❑ Глава 11 посвящена обработке естественного языка (natural language processing, NLP) и определяет базовый словарь и понятия в области глубокого обучения.
- ❑ Глава 12 рассматривает рекуррентные нейронные сети, современный метод глубокого обучения, используемый почти во всех областях моделирования последовательностей, один из самых популярных инструментов в отрасли.
- ❑ Глава 13 кратко рассказывает, как создать свой фреймворк глубокого обучения с нуля и стать опытным пользователем таких фреймворков.
- ❑ В главе 14 вы используете свою рекуррентную нейронную сеть для решения более сложной задачи: моделирования текста на естественном языке.
- ❑ Глава 15 затрагивает вопросы конфиденциальности данных, знакомит с базовыми понятиями конфиденциальности, такими как федеративное

обучение, гомоморфное шифрование, и идеями, имеющими отношение к дифференцированной конфиденциальности и безопасности многосторонних вычислений.

- ❑ Глава 16 познакомит вас с инструментами и ресурсами, необходимыми для дальнейшего путешествия по миру глубокого обучения.

## Соглашения об оформлении кода и его загрузке

Весь программный код в книге набран моноширинным шрифтом, как здесь, чтобы его проще было отличить от обычного текста. Некоторые листинги кода сопровождаются комментариями, подчеркивающими важные понятия.

Код примеров в книге можно загрузить с сайта издательства: [www.manning.com/books/grokking-deep-learning](http://www.manning.com/books/grokking-deep-learning) или из репозитория GitHub: <https://github.com/iamtrask/grokking-deep-learning>.

## Форум книги

Приобретая книгу «Грокаем глубокое обучение», вы получаете бесплатный доступ на частный веб-форум на английском языке издательства Manning Publications, где сможете оставлять отзывы о книге, задавать вопросы и получать помощь от авторов и других пользователей. Чтобы получить доступ к форуму и зарегистрироваться на нем, откройте в браузере страницу <https://forums.manning.com/forums/grokking-deep-learning>. Кроме того, на странице <https://forums.manning.com/forums/about> вы можете узнать больше подробностей о форумах Manning и правилах поведения на них.

Издательство Manning обязуется предоставить своим читателям место встречи, где может состояться содержательный диалог между отдельными читателями и между читателями и автором. Но со стороны автора отсутствуют какие-либо обязательства уделять форуму какое-то определенное внимание — его присутствие на форуме остается добровольным (и неоплачиваемым). Мы предлагаем задавать автору стимулирующие вопросы, чтобы его интерес не угасал! Форум и архивы предыдущих дискуссий будут оставаться доступными, пока книга продолжает издаваться.



## Об авторе

Эндрю Траск (Andrew Trask) — один из основателей лаборатории машинного обучения в Digital Reasoning, где изучаются методы глубокого обучения и их применение к обработке естественного языка, распознаванию образов и преобразованию речи в текст. В течение нескольких месяцев Эндрю и его коллегам удалось превзойти лучшие опубликованные результаты в области классификации эмоциональной окраски и маркировки частей речи. Он обучил крупнейшую нейронную сеть более чем со 160 миллиардами параметров и вместе со своим соавтором представил результаты на международной конференции по машинному обучению. Эти результаты были опубликованы в журнале *Journal of Machine Learning*. В настоящее время Эндрю руководит направлением анализа текста и речи в Digital Reasoning и отвечает за разработку планов анализа для платформы когнитивных вычислений Synthesys, в которой глубокое обучение является ключевой основой.

## От издательства

Ваши замечания, предложения, вопросы отправляйте по адресу [comp@piter.com](mailto:comp@piter.com) (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства [www.piter.com](http://www.piter.com) вы найдете подробную информацию о наших книгах.

# 1

## Введение в глубокое обучение: зачем его изучать



### В этой главе

- ✓ Почему вам стоит изучать глубокое обучение.
  - ✓ Почему вы должны прочитать эту книгу.
  - ✓ Что потребуется для начала.
- .....

Не беспокойтесь о ваших сложностях с математикой.  
Могу вас заверить, что мои сложности куда больше.

*Альберт Эйнштейн*

## Добро пожаловать в «Грокаем глубокое обучение»!

Вы приступаете к овладению одним из самых ценных навыков века!

Рад приветствовать вас! Думаю, вы тоже рады! Глубокое обучение — это захватывающее пересечение машинного обучения и искусственного интеллекта, а также значимый прорыв для общества и промышленности. Методы, рассматриваемые в этой книге, меняют окружающий мир. Глубокое обучение используется везде: от оптимизации двигателя вашего автомобиля до выбора контента для просмотра в социальных сетях. Оно открывает широкие возможности, и, к счастью, его изучение доставляет массу удовольствия!

## Почему вам стоит изучать глубокое обучение

Это мощный инструмент для постепенной автоматизации интеллектуальных задач

С незапамятных времен люди создавали все более качественные инструменты, помогающие понять окружающий мир и управлять им. Глубокое обучение является новейшей главой в этой истории инноваций.

Особенно захватывающей эту главу делает тот факт, что данная область лежит в сфере *умственных* инноваций, а не *механических*. Глубокое обучение, как и родственные ему области машинного обучения, стремится *автоматизировать интеллект* постепенно. В последние несколько лет были достигнуты большие успехи в этой сфере, превосходящие предыдущие достижения в области компьютерного зрения, распознавания речи, машинного перевода и в решении многих других задач.

Самое необычное, что для достижения всех этих успехов системы глубокого обучения используют почти *тот же алгоритм, по которому работает мозг* (нейронные сети). Даже при том, что глубокое обучение все еще остается сферой активных исследований с множеством проблем, последние разработки вызвали большое волнение: похоже, что мы нашли не просто хороший инструмент, а окно в наши собственные умы.

## У глубокого обучения хороший потенциал для автоматизации умственного труда

О потенциальном влиянии глубокого обучения, если оно будет развиваться с той или иной скоростью, было сделано много шокирующих предсказаний. Многие из предсказателей явно переусердствовали, но я полагаю, что одно из них заслуживает вашего внимания: сокращение рабочих мест. Я думаю, что в отличие от остальных это утверждение имеет под собой реальную основу, потому что даже если развитие глубокого обучения прекратится прямо *сегодня*, то оно *уже* оказало большое влияние на квалификацию труда по всему миру. Операторы информационно-справочных служб, водители такси и бизнес-аналитики низшего звена являются убедительными примерами, когда глубокое обучение способно обеспечить недорогую альтернативу.

К счастью, экономика не способна в одночасье совершить крутой разворот; но во многих сферах мы уже столкнулись с неприятностями, обусловленными развитием технологий. Надеюсь, что вы (и ваши знакомые) сможете с помощью этой книги перейти из одной отрасли, столкнувшейся с потрясениями, в другую, где наблюдается рост и процветание, имя которому глубокое обучение.

## Это весело и интересно. Пытаясь симитировать творчество и интеллект, вы многое узнаете о том, что значит быть человеком

Лично я занялся глубоким обучением потому, что это очень увлекательно. Это удивительное пересечение человека и машины. Процесс познания, означающий мысли, рассуждения и творчество, оказался для меня поучительным, увлекательным и вдохновляющим. Только представьте, что вам удалось собрать коллекцию картин, когда-либо нарисованных человеком, и теперь вы можете научить машину рисовать как Моне. Невероятно, но возможно. А наблюдение за тем, как это происходит, вызывает непередаваемые эмоции.

## Этому трудно учиться?

Насколько усердно придется поработать, прежде чем наступит «весело и интересно»?

Это мой любимый вопрос. В моем понимании «весело и интересно» — это опыт переживания чего-то, чему я *научился*. Есть что-то удивительное, когда видишь, как твоё творение делает что-то необычное. Если вам знакомо это чувство, тог-

да ответ прост. На нескольких страницах в главе 3 вы создадите свою первую нейронную сеть. Единственное, над чем вам придется потрудиться, — прочитать страницы, отделяющие вас от этого.

*Следующий* забавный этап после главы 3, если вам интересно узнать, наступит в главе 4, когда вы после знакомства с небольшим фрагментом кода доберетесь до середины главы. Примерно так построены все главы: знакомство с небольшим фрагментом кода в предыдущей главе, чтение следующей главы и новый всплеск интереса от знакомства с новой нейронной сетью.

## Почему вы должны прочитать эту книгу

### Устанавливает очень низкий порог входа

Причина, по которой вы должны прочитать эту книгу, совпадает с причиной, по которой я ее написал. Я не знаю другого источника знаний (книги, курса, серии статей в блоге), который рассказывал бы о глубоком обучении, *не предполагая наличия у читателя углубленных знаний математики* (которые даются, например, в университете).

Не поймите меня неправильно: есть более чем веские причины для преподавания этого предмета с привлечением математики. В конце концов, математика — это язык, и намного *эффективнее* изучать глубокое обучение с использованием этого языка, однако я не считаю, что углубленные знания математики абсолютно необходимы, чтобы стать опытным и знающим практиком, который четко понимает, *как* происходит глубокое обучение.

Итак, почему вы должны изучать глубокое обучение с помощью этой книги? Я предполагаю, что у вас за плечами лишь школьный курс математики (и кое-что уже позабылось), и собираюсь *объяснять все остальное по ходу дела*. Помните таблицу умножения? Помните графики с системой координат  $x/y$  (такие квадраты с линиями на них)? Отлично! Вам этого будет достаточно.

### Помогает понять, что находится внутри фреймворка (Torch, TensorFlow и других)

Есть две разновидности учебных материалов по глубокому обучению (книги или курсы). В материалах из одной группы рассказывается, как использовать популярные фреймворки и библиотеки, такие как Torch, TensorFlow, Keras и другие. А материалы из другой группы описывают собственно принципы глубокого обучения, то есть *теорию*, на которой эти фреймворки основываются.



Важно знать *и то и другое*. Если бы вы захотели стать пилотом NASCAR, то вам было бы нужно знать не только конкретную модель болида со всеми ее тонкостями (фреймворк), но и уметь управлять им (теория/навыки). Простое изучение фреймворков напоминает изучение достоинств и недостатков Chevrolet SS шестого поколения до знакомства с рычагом переключения передач. Эта книга познакомит вас с глубоким обучением и подготовит к изучению фреймворков.

## Все связанное с математикой будет подкрепляться простыми и понятными аналогиями

Всякий раз, встречаясь с математической формулой в дикой природе, я использую двухэтапный подход. Сначала подбираю понятную *аналогию* из реального мира. Я почти никогда не принимаю формулы на веру и всегда разбиваю их на *части*, каждую со своей историей. Этот же подход я использовал и здесь. Каждый раз, подходя к какому-то математическому понятию, я буду предлагать аналогию тому, что фактически делает формула.

*Все должно быть простым, насколько возможно,  
но не проще.*

*Приписывается Альберту Эйнштейну*

## Все после вступительных глав основано на понятии проекта

Если и есть что-то, что мне не нравится при изучении чего-то нового, так это недосказанность в отношении полезности или уместности изучаемого. Если кто-то во всех подробностях рассказывает мне об устройстве молотка, но не берет меня за руку и не учит забивать гвозди, я считаю, что он не учит меня владению молотком. Я знаю, что встречу что-то, не связанное между собой, и если окажусь в реальном мире с молотком, коробкой гвоздей и парой досок, мне придется кое о чем догадываться самому.

Эта книга *сначала* даст вам доски, гвозди и молоток, а потом расскажет, что с ними делать. Каждый урок описывает, какие инструменты выбрать и как с их помощью что-то сконструировать, а также объясняет, как все это работает. При таком подходе вы не просто будете иметь список фактов о разных инструментах глубокого обучения, но сможете их использовать для решения задач. Кроме того, вы будете знать самое важное: когда, почему и для каких задач следует использовать каждый инструмент. Благодаря этому знанию вы сможете продолжить карьеру в области исследований или в промышленности.

## Что нужно для начала

### Установите Jupyter Notebook и библиотеку NumPy для Python

Моим самым любимым рабочим инструментом является Jupyter Notebook. Я считаю, что при освоении глубокого обучения очень важно иметь возможность остановить процесс обучения сети и разобрать ее на составные части, чтобы увидеть, как она устроена. А для этого нет удобнее инструмента, чем Jupyter Notebook.

Ничто не скроется от нашего внимания, так как в книге используется единственная библиотека — библиотека матричных вычислений NumPy. Благодаря такому подходу вы узнаете саму *суть* глубокого обучения, а не только список функций фреймворка и порядок их вызова. Эта книга рассказывает о глубоком обучении всё: от начала и до конца.

Инструкции по установке этих инструментов можно найти по адресам: <http://jupyter.org> (для Jupyter) и <http://numpy.org> (для NumPy). Все примеры я создавал в Python 2.7, но также проверил их в Python 3. Чтобы Reddy упростить установку, рекомендую использовать фреймворк Anaconda: <https://docs.continuum.io/anaconda/install>.

### Повторите школьный курс математики

Как уже отмечалось, эта книга не предполагает наличие у читателя специальной математической подготовки и моя цель — поведать вам о глубоком обучении, предполагая лишь базовые знания школьной алгебры.

### Найдите задачу, которая интересна лично вам

Это условие кажется «необязательным» для начала. Может быть и так, но я вполне серьезно советую найти такую задачу. У всех, кого я знаю и кто добился успеха на этом поприще, была своя задача, которую они пытались решить. Изучение глубокого обучения было лишь «зависимостью», необходимой для решения некоторой другой интересной задачи.

Моей задачей было использование Twitter для предсказания котировок на фондовой бирже. Я просто посчитал такую задачу увлекательной. Именно она заставила меня сесть, прочитать следующую главу и создать прототип.

И, как выяснилось, эта область *настолько нова* и меняется *настолько быстро*, что если вы потратите пару следующих лет, стараясь реализовать один проект

с помощью этих инструментов, то быстро войдете в число ведущих экспертов по этой *конкретной теме*, намного быстрее, чем кажется сейчас. В моем случае такая погоня за идеей всего за 18 месяцев превратила меня, человека почти ничего не знающего о программировании, в опытного специалиста, который был удостоен гранта хедж-фонда! Для освоения глубокого обучения важно иметь задачу, увлекающую вас и предусматривающую использование одного набора данных для прогнозирования другого. Обязательно найдите такую задачу!

## Возможно, вам потребуется знание Python

Python — мой любимый язык для обучения, но я покажу примеры на некоторых других интерактивных языках

Python — удивительно простой и понятный язык. Более того, я считаю его самым популярным и понятным из всех языков, созданных когда-либо. Сообщество пользователей Python испытывает неодолимую страсть к простоте. По этим причинам я написал все примеры на Python (точнее на Python 2.7). В исходном коде примеров, доступном для загрузки на сайте издательства [www.manning.com/books/grokking-deep-learning](http://www.manning.com/books/grokking-deep-learning), а также в репозитории на GitHub <https://github.com/iamtrask/Grokking-Deep-Learning>, я представлю примеры на некоторых других интерактивных языках.

## Насколько большой опыт программирования требуется?

Загляните на страницу курса Python Codecademy ([www.codecademy.com/learn/python](http://www.codecademy.com/learn/python)). Если вы прочитали оглавление и все упомянутые там термины вам знакомы, значит, всё в порядке! Если нет, то пройдите курс до конца и возвращайтесь к книге, когда закончите. Он ориентирован на начинающих и очень хорошо продуман.

## Итоги

Если вы установили Jupyter Notebook и знакомы с основами программирования на Python, тогда вы готовы перейти к следующей главе. Глава 2 — это последняя глава в этой книге, где мы с вами будем просто беседовать, ничего не создавая. Ее цель — познакомить вас с терминологией и с базовыми понятиями в области искусственного интеллекта, машинного обучения и, самое главное, глубокого обучения.

# 2

## Основные понятия: как учатся машины?



### В этой главе

- ✓ Что такое глубокое обучение, машинное обучение и искусственный интеллект.
  - ✓ Что такое параметрические модели и непараметрические модели.
  - ✓ Что такое обучение с учителем и обучение без учителя.
  - ✓ Как учатся машины.
- .....

Через пять лет машинное обучение обеспечит успех  
каждого первичного размещения акций.

Эрик Шмидт (Eric Schmidt),  
председатель правления Google,  
из вступительной речи на конференции  
Cloud Computing Platform conference в 2016 году

## Что такое глубокое обучение?

**Глубокое обучение — это подмножество методов машинного обучения**

Глубокое обучение (deep learning) — это подмножество методов машинного обучения, области изучения и создания машин, которые могут обучаться (иногда с целью достичь уровня искусственного интеллекта).

Глубокое обучение используется в промышленности для решения практических задач в самых разных областях, таких как компьютерное зрение (изображения), обработка естественного языка (текст) и автоматическое распознавание речи. Проще говоря, глубокое обучение — это подмножество *методов* машинного обучения, главным образом основанных на применении *искусственных нейронных сетей*, которые представляют класс алгоритмов, подражающих человеческому мозгу.



Обратите внимание, что глубокое обучение, как показано на этом рисунке, не полностью входит в область искусственного интеллекта (разумные машины, как в кино). Эта технология часто используется для решения широкого круга практических задач. Цель этой книги — познакомить с основами глубокого обучения, на которые опираются и передовые исследования, и практические применения, и подготовить вас к приложению своих сил в том или ином направлении.

## Что такое машинное обучение?

Область исследований, которая дает компьютерам способность обучаться без непосредственного программирования.

Приписывается Артуру Сэмюэлю (Arthur Samuel)

Если глубокое обучение является подмножеством методов машинного обучения, тогда что такое машинное обучение? В общем и целом это именно то, что подразумевается. Машинное обучение — это область computer science, в которой *машины учатся* решать задачи, для которых они *не были запрограммированы непосредственно*. Проще говоря, машины наблюдают закономерности и пытаются прямо или косвенно некоторым способом имитировать их.

Машинное обучение  $\sim$  Обезьяна видит,  
обезьяна делает

Я упомянул прямую и косвенную имитации как параллель с двумя основными видам машинного обучения: *с учителем* и *без учителя*. Машинное обучение с учителем — это прямая имитация закономерностей, имеющих место между двумя наборами данных. В нем всегда входной набор данных преобразуется в выходной. Часто это невероятно мощный и полезный метод. Рассмотрим следующие примеры (**входные** данные выделены жирным шрифтом, а **выходные** — курсивом):

- ❑ Использование **пикселей** изображения для определения *присутствия или отсутствия кота*.
- ❑ Использование списка **понравившихся фильмов** для выбора *фильмов, которые могут понравиться*.
- ❑ Использование **слов в сообщении**, чтобы предсказать, *счастливы* ли их автор или *расстроен*.
- ❑ Использование **данных** с метеорологических приборов для предсказания *вероятности дождя*.
- ❑ Использование **датчиков** автомобильного двигателя для определения *оптимальных настроек*.
- ❑ Использование **новостей** для предсказания завтрашних *котировок* на бирже.

- Использование входного **числа** для предсказания удвоенного **числа**.
- Использование **аудиофайла** для получения *транскрипции* речи, содержащейся в нем.

Все это — задачи машинного обучения с учителем. Во всех случаях алгоритм машинного обучения пытается выявить такие закономерности между двумя наборами данных, чтобы *по одному можно было спрогнозировать другой*. А теперь представьте, что для любого из этих примеров вы получили возможность предсказать *результат*, имея только **входной** набор данных. Значимость такой возможности трудно было бы переоценить.

## Машинное обучение с учителем

### Машинное обучение с учителем преобразует наборы данных

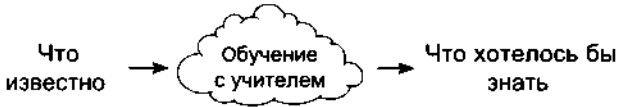
Машинное обучение с учителем — это метод преобразования одного набора данных в другой. Например, если представить, что имеется один набор данных «Котировки на бирже в понедельник», в котором записаны все котировки, имевшие место в каждый понедельник в течение последних 10 лет, и второй набор «Котировки на бирже во вторник» с котировками за тот же период, то алгоритм машинного обучения с учителем может попытаться использовать первый, чтобы предсказать второй.



Если вам удастся успешно обучить алгоритм машинного обучения с учителем на 10-летних наборах данных с котировками по понедельникам и по вторникам, то вы сможете предсказывать котировки в любой вторник в будущем, имея котировки за предшествующий понедельник. А теперь давайте остановимся и немного поразмышляем.

Машинное обучение с учителем лежит в основе прикладного искусственного интеллекта (также известного как ограниченный ИИ). Его удобно использовать, когда на входе имеется *нечто известное* и требуется быстро преобразовать его в то, *что хотелось бы знать*. Это позволяет алгоритмам машинного обучения с учителем расширять человеческий интеллект едва ли не до бесконечности.

Основным результатом машинного обучения является обученный классификатор некоторого типа. Даже машинное обучение без учителя (с которым мы познакомимся чуть ниже) обычно выполняется для разработки точного алгоритма машинного обучения с учителем.



Далее мы будем создавать алгоритмы, принимающие входные данные, которые можно наблюдать и записывать, то есть *знать*, и преобразующие их в выходные данные, требующие логической оценки. В этом сила машинного обучения с учителем.

## Машинное обучение без учителя

### Обучение без учителя группирует данные

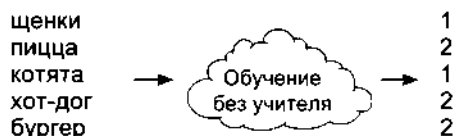
Обучение без учителя и обучение с учителем обладают одним общим свойством: они оба преобразуют один набор данных в другой. Но в обучении без учителя набор данных, в который происходит преобразование, *прежде не был известен*. В отличие от обучения с учителем, здесь нет «правильного ответа», который модель должна воспроизвести. Вы просто даете команду алгоритму «найти закономерности в этих данных и сообщить о них».

Например, *кластеризация набора данных на группы* — это разновидность обучения без учителя. Кластеризация преобразует последовательность *точек данных* в последовательность *меток кластеров*. Часто роль меток играют последовательные целые числа, например, в случае 10 кластерами будут созданы метки от 1 до 10. Каждая точка данных получит метку, в зависимости от того, к какому кластеру она будет отнесена. Набор данных, состоящий из точек, превратится в набор меток. Почему в качестве меток часто выбираются числа? Алгоритм ничего не может сказать о природе кластеров, он лишь сообщает вам: «Я обнаружил тут некоторые закономерности. Похоже, что ваши данные делятся на группы. Вот они!»





Могу вас обрадовать! Эту идею кластеризации с полным правом можно считать определением обучения без учителя. Несмотря на большое разнообразие форм обучения без учителя, *их все можно рассматривать как разновидности кластеризации*. Далее в книге мы еще не раз будем рассматривать эту тему.



Взгляните на этот пример. Сможете ли вы понять, по какому принципу алгоритм объединил слова, несмотря на то что он ничего не сообщает о природе кластеров? (Ответ: 1 == няшки и 2 == вкусняшки.) Позже мы выясним, что другие формы обучения без учителя тоже являются лишь разновидностями кластеризации, и узнаем, как эти кластеры могут пригодиться в обучении с учителем.

## Параметрическое и непараметрическое обучение

Упрощенно: обучение методом проб и ошибок и вычисления и вероятность

На предыдущих двух страницах мы выяснили, что все алгоритмы машинного обучения делятся на две группы: с учителем и без учителя. Теперь обсудим деление на две группы по другим признакам: параметрические и непараметрические. Можно представить, что облако алгоритмов машинного обучения имеет два переключателя:



Как видите, в действительности существует четыре разных типа алгоритмов. Алгоритмы бывают с учителем или без, а также параметрические или непараметрические. Если, как говорилось выше, наличие или отсутствие учителя (обучающей выборки данных) определяет *тип выявляемых закономерностей*, то *параметричность* задает способ *хранения* результатов обучения и зачастую *метод обучения*. Для начала рассмотрим формальное определение параметрических и непараметрических моделей. Справедливости ради следует отметить, что споры, касающиеся точных отличий, продолжаются до сих пор.

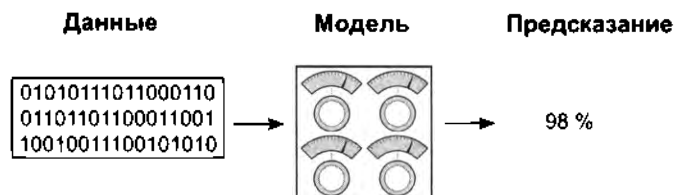
Параметрическая модель характеризуется наличием фиксированного числа параметров, тогда как непараметрическая модель имеет *бесконечное* число параметров (определяется данными).

В качестве примера возьмем задачу, в которой требуется вставить колышек с квадратным сечением в правильное (квадратное) отверстие. Некоторые люди (например, маленькие дети) просто пытаются вставить колышек во все отверстия, пока он не встанет (параметрический подход). Ребенок постарше уже может подсчитать число сторон (четыре) у колышка и найти отверстие с тем же числом сторон (непараметрический подход). Параметрические модели обычно используют метод проб и ошибок, тогда как непараметрические модели обычно основаны на вычислениях. А теперь рассмотрим эти модели подробнее.

## Параметрическое обучение с учителем

### Упрощенно: обучение методом проб и ошибок с использованием регуляторов

Параметрические модели обучения с учителем — это модели, имеющие фиксированное число регуляторов (это параметрическая часть таких моделей), обучение которых происходит путем поворота регуляторов. Входные данные обрабатываются согласно углу поворота регуляторов и преобразуются в *предсказание*.



Обучение осуществляется поворотом регуляторов на разные углы. Если вы попытаетесь предсказать вероятность выигрыша мировой серии бейсбольной командой Red Sox, тогда эта модель сначала примет исходные данные (такие, как статистика побед/поражений или среднее число игр, сыгранных игроками) и сделает прогноз (например, вероятность 98 %). Затем модель проверит, действительно ли команда Red Sox выиграла мировую серию. Затем, зная результат, алгоритм обучения *повернет регуляторы*, чтобы в следующий раз, когда он получит *те же или похожие исходные данные*, можно было дать более точный прогноз.

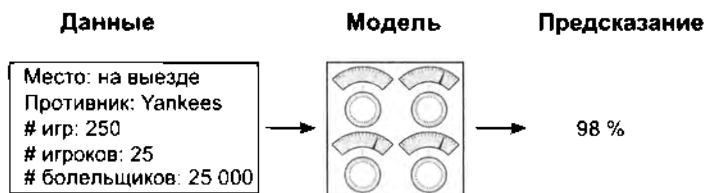
Возможно, он «повернет» в сторону увеличения регулятор «учета побед/поражений», если победы и поражения команды окажутся хорошим прогнозирующим признаком. И наоборот, он может «повернуть» в сторону уменьшения регулятор «среднего числа игр на игрока», если этот признак окажется слабо влияющим на прогноз. Именно так обучаются параметрические модели!

Обратите внимание, что результат обучения модели в любой момент можно зафиксировать по положениям регуляторов. Этот способ обучения модели также можно представить как алгоритм поиска. Вы стараетесь «найти» подходящие положения регуляторов, настраивая их и повторяя попытки.

Отметьте также, что понятие «метод проб и ошибок» не является формальным определением, но считается общим свойством параметрических моделей (за некоторым исключением). Когда имеется некоторое фиксированное число регуляторов, для определения оптимальной конфигурации требуется выполнить некоторый поиск. Непараметрические модели, напротив, часто основаны на вычислениях (в той или иной степени) и добавляют новые регуляторы, когда обнаруживается что-то новое, пригодное для использования в вычислениях. Давайте разобьем параметрическое обучение с учителем на три этапа.

## Этап 1: прогноз

Для иллюстрации параметрического обучения с учителем продолжим аналогию со спортивными состязаниями и попыткой предсказать победу Red Sox в мировой серии. На первом шаге, как уже упоминалось, производится сбор статистики, ввод ее в машину и прогнозирование вероятности победы Red Sox.



## Этап 2: сравнение с истиной

Второй этап — сравнение прогноза (98 %) с истиной (выиграла ли команда Red Sox на самом деле). К сожалению, они проиграли, поэтому

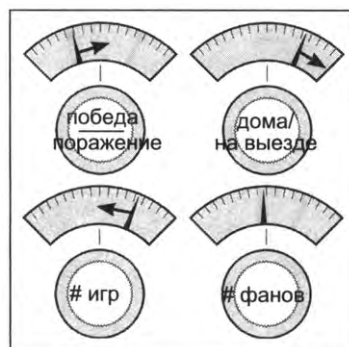
Прогноз : 98 % > Истина : 0 %

На этом этапе выясняется, что если бы модель выдала прогноз 0 %, то отлично предсказала бы грядущий проигрыш команды. Нам нужно повысить точность прогноза, поэтому переходим к этапу 3.

## Этап 3: обучение

На этом этапе модель поворачивает регуляторы, учитывая величину ошибки (98 %) и исходные данные на момент прогноза (статистика состязаний), чтобы уточнить прогноз по заданным исходным данным.

**Корректировка чувствительности  
поворотом регуляторов**



Теоретически, когда эта модель увидит ту же статистику состязаний, она вернет прогноз меньше 98 %. Обратите вниманиис, что каждый регулятор представляет *чувствительность прогноза к разным типам входных данных*. Именно они меняются при «обучении».

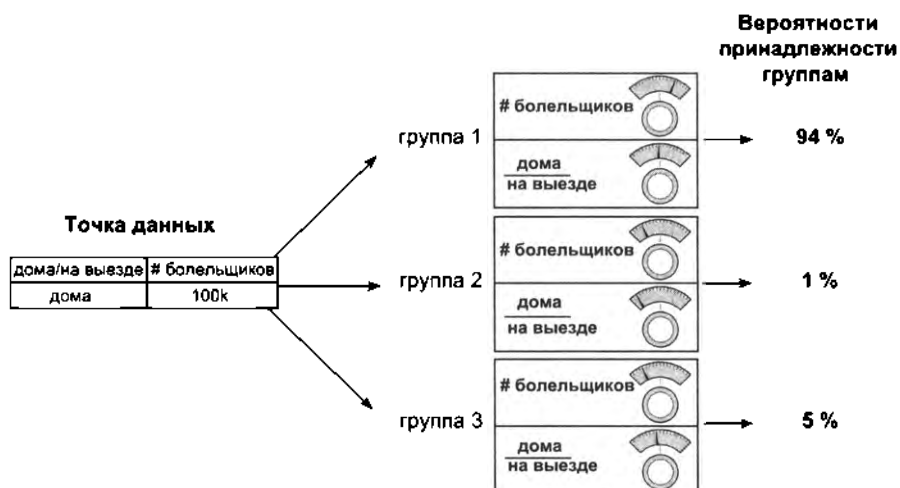
## Параметрическое обучение без учителя

В параметрическом обучении без учителя используется схожий подход. Давайте рассмотрим в общих чертах этапы такого обучения. Не забывайте, что, по сути, обучение без учителя осуществляет группировку данных. В *параме-*

трическом обучении без учителя регуляторы используются для группировки данных. В этом случае обычно имеется несколько регуляторов по числу групп, каждый из которых отражает близость входных данных к конкретной группе (с некоторыми исключениями и нюансами не забывайте, что это всего лишь обобщенное описание). Рассмотрим пример, предполагающий деление данных на три группы.

| Дома или на выезде | Болельщиков |
|--------------------|-------------|
| Дома               | 100k        |
| На выезде          | 50k         |
| Дома               | 100k        |
| Дома               | 99k         |
| На выезде          | 50k         |
| На выезде          | 10k         |
| На выезде          | 11k         |

В этом наборе данных я выделил разным шрифтом три кластера — **группа 1**, **группа 2** и **группа 3**, — которые должна выявить параметрическая модель. Передадим первую точку данных модели, обученной без учителя, как показано ниже. Обратите внимание, что она наиболее близка **группе 1**.



Для каждой группы модель пытается преобразовать входные данные в число от 0 до 1, сообщая *вероятность принадлежности данных к этой группе*. Модели могут обучаться самыми разными способами и получать самые разные свой-

ства, но, в общем и целом, они просто корректируют параметры преобразования входных данных в группы.

## Непараметрическое обучение

### Упрощенно: методы на основе вычислений

Непараметрическое обучение — это класс алгоритмов, в которых число параметров зависит от данных (то есть не **предопределено**). Это позволяет использовать методы, выполняющие некоторые вычисления и увеличивающие число параметров, исходя из числа признаков, выявленных в данных. В обучении с учителем, например, непараметрическая модель может подсчитать, сколько раз конкретная секция светофора вызвала «движение» автомобилей. Подсчитав лишь несколько примеров, эта модель может затем предсказать, что включение *средней* секции всегда (100 %) вызывает движение автомобилей, а секции *справа* — только иногда (50 %).



Обратите внимание, что эта модель будет иметь три параметра: три счетчика, определяющих, сколько раз включалась каждая секция и какое количество автомобилей проехало (возможно, деленное на общее число наблюдений). Если бы в светофоре было пять секций, модель создала бы пять счетчиков (пять параметров). *Непараметрической* эту простую модель делает то обстоятельство, что число параметров меняется в зависимости от данных (в данном случае от числа огней в светофоре). Этим непараметрические модели отличаются от параметрических, которые изначально имеют **предопределенное** число параметров, и, что особенно важно, число параметров определяется исключительно человеком, управляющим обучением модели (и не зависит от данных).

При близком рассмотрении эта идея может вызвать вопросы. Похоже, что предыдущая параметрическая модель имела регуляторы для каждой входной точки данных. Большинство параметрических моделей все еще должны иметь некоторый *вход*, в зависимости от числа классов в данных. То есть между параметрическими и непараметрическими алгоритмами имеется *серая зона*. Даже

параметрические алгоритмы в некоторой степени зависят от числа классов в данных, даже при том, что они явно не подсчитывают имеющиеся закономерности.

Из вышесказанного вытекает, что *параметры* — это обобщенный термин, относящийся лишь к множеству чисел, используемых для моделирования закономерностей (без каких-либо ограничений в отношении использования этих чисел). Счетчики — это параметры. Веса — это параметры. Нормализованные значения счетчиков или весов — это параметры. Коэффициенты корреляции тоже могут быть параметрами. Этот термин обозначает набор чисел, используемых для моделирования. Кстати, глубокое обучение является классом параметрических моделей. В этой книге мы больше не будем возвращаться к непараметрическим моделям, но имейте в виду, что они представляют интересный и обширный класс алгоритмов.

## Итоги

Мы рассмотрели некоторые виды машинного обучения. Вы узнали, что машинное обучение может быть с учителем или без учителя, а также параметрическим или непараметрическим. Мы рассмотрели отличительные черты этих четырех групп алгоритмов. Вы узнали, что машинное обучение с учителем — это класс алгоритмов, обучающихся предсказанию одного набора данных по другому, а обучение без учителя фактически сводится к делению набора данных на группы. Вы также узнали, что параметрические алгоритмы имеют фиксированное число *параметров*, а непараметрические алгоритмы выбирают число параметров, основываясь на данных.

Для обучения с учителем и без учителя в глубоком обучении используются нейронные сети. До сих пор мы оставались на концептуальном уровне и пытались сориентироваться и понять, где находимся. В следующей главе мы создадим свою первую нейронную сеть, и все последующие главы будут сопровождаться *разработкой проектов*. Итак, доставайте свой блокнот Jupyter Notebook и приступим!

# 3

## Введение в нейронное прогнозирование: прямое распространение



### В этой главе

- ✓ Простая сеть, делающая прогноз.
- ✓ Что такое нейронная сеть и что она делает.
- ✓ Прогнозирование с несколькими входами.
- ✓ Прогнозирование с несколькими выходами.
- ✓ Прогнозирование с несколькими входами и выходами.
- ✓ Прогнозирование на основе прогнозов.

Я стараюсь не давать прогнозов. Это самый простой способ поставить себя в идиотское положение.

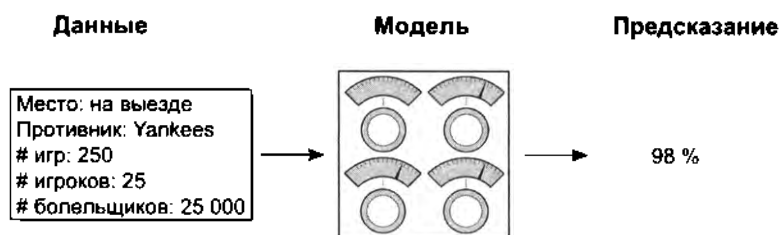
Уоррен Эллис (Warren Ellis),  
автор комиксов, прозаик и сценарист



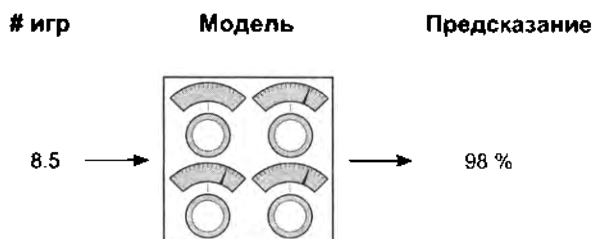
## Шаг 1: прогнозирование

### Эта глава о прогнозировании

В предыдущей главе вы познакомились с парадигмой *предсказание, сравнение, обучение*. В этой главе мы подробнее рассмотрим первый шаг: *предсказание*, или *прогнозирование*. Возможно, вы помните, что этап прогнозирования выглядит примерно так:



В этой главе вы узнаете больше об этих трех компонентах прогнозирующих нейронных сетей. Начнем с первого: с данных. В своей первой нейронной сети вы будете выполнять прогнозирование по одной точке данных за раз, например:

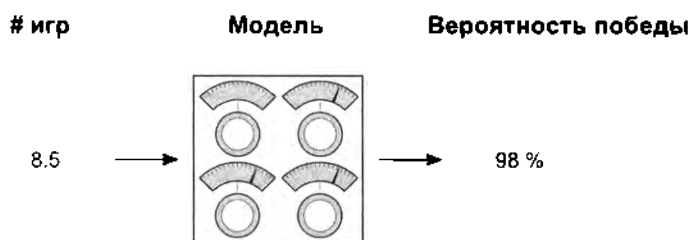


Позднее вы узнаете, что количество точек данных, обрабатываемых одновременно, оказывает значительное влияние на устройство сети. Возможно, вам интересно знать, как определить, сколько точек данных передавать одновременно. Ответ зависит от того, какой объем данных необходим сети для точного предсказания.

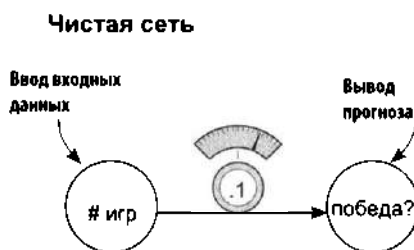
Например, если требуется определить присутствие кошки на фотографии, то я определенно должен передать в сеть сразу все пиксели, составляющие изображение. Почему? Представьте, что я отправил вам только один пиксел: смогли бы вы с уверенностью определить присутствие или отсутствие кошки

на изображении? Я тоже не смог бы! (Кстати, это универсальное эмпирическое правило: всегда передавать в сеть достаточный объем информации, где «достаточность» определяется довольно свободно, например, сколько потребуется человеку, чтобы сделать тот же прогноз.)

Но давайте пока отложим сеть в сторону. Как оказывается, сеть можно создать, только поняв форму входного и выходного наборов данных (в данном случае под *формой* подразумевается «число столбцов» или «число точек данных, обрабатываемых одновременно»). Остановимся пока на одном прогнозе — вероятности победы бейсбольной команды:



Теперь, зная, что на вход будет подаваться одна точка данных и на выходе возвращаться один прогноз, можно создать нейронную сеть. Поскольку на входе и на выходе имеется только одна точка данных, мы построим сеть с единственным регулятором, отражающим одну входную точку в одну выходную. Эти «регуляторы» называют *весами* или *весовыми коэффициентами*, и с этого момента я так и буду их называть. Итак, вот ваша первая нейронная сеть, с единственным весовым коэффициентом, отражающим вход «число игр» в выход «вероятность победы»:



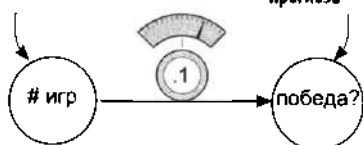
Как видите, при одном весовом коэффициенте эта сеть принимает по одной точке данных (среднее число игр, сыгранных игроками в бейсбольной команде) и выводит один прогноз (считает ли она вероятной победу команды).

# Простая нейронная сеть, делающая прогноз

Начнем с самой простой нейронной сети, какая только возможна

## 1. Чистая сеть

Ввод входных  
данных



Вывод  
прогноза

weight = 0.1

```
def neural_network(input, weight):
    prediction = input * weight
    return prediction
```

## 2. Передача одной точки данных

Входные данные  
(# игр)



number\_of\_toes = [8.5, 9.5, 10, 9]

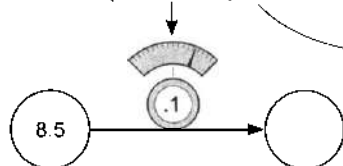
input = number\_of\_toes[0]

pred = neural\_network(input, weight)

print(pred)

## 3. Умножение входного значения на весовой коэффициент

$(8.5 * 0.1 = 0.85)$



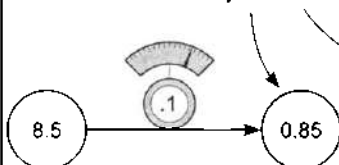
def neural\_network(input, weight):

prediction = input \* weight

return prediction

## 4. Получение прогноза

Прогноз



number\_of\_toes = [8.5, 9.5, 10, 9]

input = number\_of\_toes[0]

pred = neural\_network(input, weight)

# Что такое нейронная сеть?

## Ваша первая нейронная сеть

Чтобы начать создание нейронной сети, откройте Jupyter Notebook и выполните следующий код:

```
weight = 0.1
def neural_network(input, weight):
    prediction = input * weight
    return prediction
```

Сеть

Теперь выполните этот код:

```
number_of_toes = [8.5, 9.5, 10, 9]
input = number_of_toes[0]
pred = neural_network(input, weight)
print(pred)
```

Так используется сеть, чтобы получить прогноз

Вы только что создали свою первую нейронную сеть и использовали ее для получения прогноза! Поздравляю! Последняя строка выводит прогноз (`pred`). Это должно быть число `0.85`. Так что же такое нейронная сеть? На данный момент это один или несколько *весовых коэффициентов*, на которые можно умножить *входные данные* и получить *прогноз*.

### ЧТО ТАКОЕ ВХОДНЫЕ ДАННЫЕ?

Это число, записанное где-то в реальном мире. Обычно это что-то легко узнаваемое, например: сегодняшняя температура воздуха, средний уровень бейсболиста или вчерашняя цена акций на бирже.

### ЧТО ТАКОЕ ПРОГНОЗ?

*Прогноз*, или *предсказание*, — это то, что возвращает нейронная сеть после получения *входных данных*, например: «с учетом указанной температуры, вероятность того, что люди наденут сегодня теплую одежду, равна **0 %**» или «с учетом среднего уровня бейсболистов вероятность добиться успеха равна **30 %**» или «с учетом вчерашней цены на акции сегодня цена составит **101.52**».

### ВСЕГДА ЛИ ВЕРЕН ПРОГНОЗ?

Нет. Иногда нейронная сеть допускает ошибки, но она способна на них учиться. Например, если предсказанная величина слишком высока, она уменьшит вес, чтобы в следующий раз получить меньшее прогнозное значение, и наоборот.

### КАК СЕТЬ ОБУЧАЕТСЯ?

Методом проб и ошибок! Сначала она пытается сделать прогноз. Затем проверяет, насколько завышенной или заниженной получилась прогнозная величина. Наконец, она изменяет весовой коэффициент (вверх или вниз), чтобы в следующий раз, когда она увидит те же данные, получить более точный прогноз.

## Что делает эта нейронная сеть?

Она умножает входное значение на весовой коэффициент, «масштабирует» входное значение на определенную величину

В предыдущем разделе вы сделали свой первый прогноз с помощью нейронной сети. Нейронная сеть в своей простейшей форме использует операцию *умножения*. Она принимает входную точку данных (в данном случае 8.5) и умножает ее на *весовой коэффициент*. Если коэффициент будет равен 2, тогда нейронная сеть *удвоит входное значение*. Если коэффициент будет равен 0.01, тогда сеть *разделит* входное значение на 100. Как видите, некоторые весовые коэффициенты *увеличивают* входное значение, а некоторые — *уменьшают*.

#### 1. Чистая сеть

Ввод входных  
данных



weight = 0.1

```
def neural_network(input, weight):
    prediction = input * weight
    return prediction
```

Нейронная сеть имеет простой интерфейс. Она принимает переменную *input* с *исходной информацией*, переменную *weight*, отражающую *знание*, и возвращает *прогноз prediction*. Все нейронные сети, которые вам доведется увидеть, действуют подобным образом. Они используют *знание* для взвешивания и интерпретации *исходной информации* во входных данных. Нейронные сети, которые мы будем далее рассматривать, будут принимать более сложные и обширные значения *input* и *weight*, однако это упрощенное описание в равной степени применимо и к ним.

## 2. Передача одной точки данных

Входные данные  
(# игр)



```
number_of_toes = [8.5, 9.5, 10, 9]
```

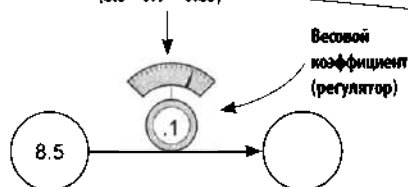
```
input = number_of_toes[0]
```

```
pred = neural_network(input, weight)
```

В данном случае исходной информацией является среднее число игр, проведенных игроками команды перед данной игрой. Обратите внимание на несколько обстоятельств. Во-первых, нейронная сеть *не* имеет никакой другой информации, кроме единственного экземпляра. Если после получения прогноза вы передадите в сеть `number_of_toes[1]`, она не вспомнит предыдущий прогноз. Нейронная сеть знает только то, что вы передадите ей на вход. Все остальное она забывает. Далее вы узнаете, как оснастить нейронную сеть «кратковременной памятью», передавая на вход сразу несколько значений.

## 3. Умножение входного значения на весовой коэффициент

$(8.5 * 0.1 = 0.85)$

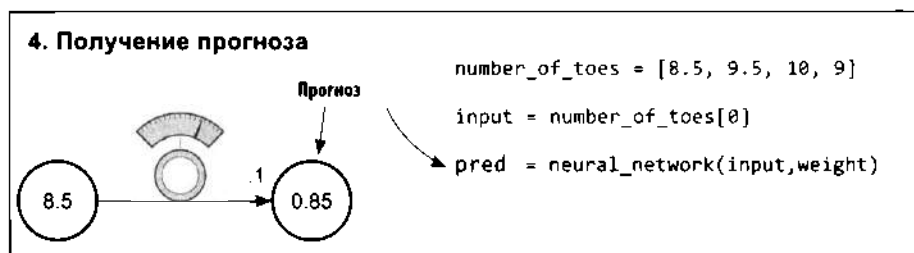


```
def neural_network(input, weight):
```

```
    prediction = input * weight
```

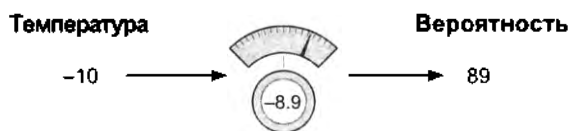
```
    return prediction
```

Весовой коэффициент в нейронной сети можно также представить как меру *чувствительности* прогноза к входным данным. Если вес слишком высок, тогда даже очень маленькое входное значение может породить очень большое прогнозное значение! Если вес слишком мал, тогда даже очень большое входное значение даст на выходе маленькое прогнозное значение. Такая чувствительность сродни *громкости*. «Прибавка веса» усиливает прогнозное значение относительно входа: вес — это регулятор громкости.



В этом случае нейронная сеть фактически применяет *регулировку громкости* к переменной `number_of_toes`. Теоретически этот регулятор громкости может сообщить вероятность победы команды, исходя из среднего числа игр, сыгранных игроками. Прогноз при этом может оказаться верным или ошибочным. Впрочем, совершенно очевидно, что если у всех игроков в команде за плечами будет 0 игр, они наверняка сыграют ужасно. Но бейсбол намного сложнее. В следующем разделе мы попробуем передать нейронной сети больше информации, чтобы она смогла принять более обоснованное решение.

Обратите внимание, что нейронные сети могут прогнозировать не только положительные числа, но и *отрицательные*, и даже принимать *отрицательные числа на входе*. Представьте, что вам захотелось предсказать вероятность того, что сегодня люди будут выходить на улицу в пальто. Если температура будет равна  $-10$  градусов Цельсия, тогда отрицательный вес предскажет высокую вероятность, что люди наденут пальто.

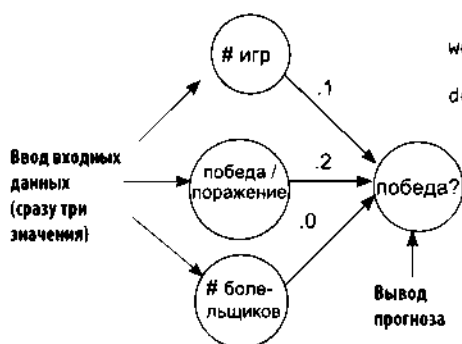


## Прогнозирование с несколькими входами

Нейронные сети могут объединять информацию из нескольких точек данных

Предыдущая нейронная сеть принимала на входе одну точку данных и, опираясь на нее, возвращала один прогноз. Возможно, у вас возник вопрос: «Действительно ли число сыгранных игр является хорошим прогнозирующим признаком?» Если это так, то вы на верном пути. А можно ли передать в нейронную сеть больше информации (одновременно), чем простое среднее число игр, сыгранных игроками? В этом случае сеть теоретически должна давать более точные прогнозы. Как оказывается, сеть действительно может принимать сразу несколько точек данных. Взгляните на следующий прогноз:

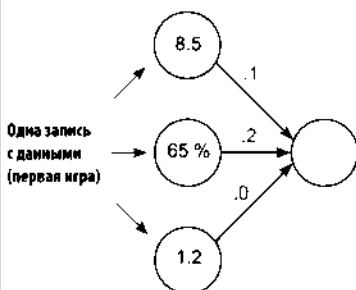
### 1. Чистая сеть с несколькими входами



weights = [0.1, 0.2, 0]

```
def neural_network(input, weights):
    pred = w_sum(input, weights)
    return pred
```

### 2. Передача одной точки данных



Этот набор данных определяет текущее состояние перед началом каждой из первых четырех игр в сезоне:  
 toes = текущее среднее число игр, сыгранных игроками.  
 wlrec = текущая доля игр, окончившихся победой (процент)  
 nfans = число болельщиков (в миллионах)

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

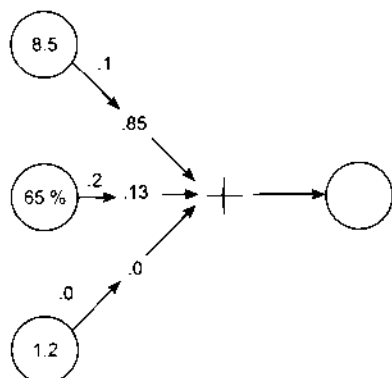
```
input = [toes[0], wlrec[0], nfans[0]]
```

```
pred = neural_network(input, weights)
```

В переменной input передается запись, соответствующая первой игре в сезоне



## 3. Вычисление взвешенной суммы входов



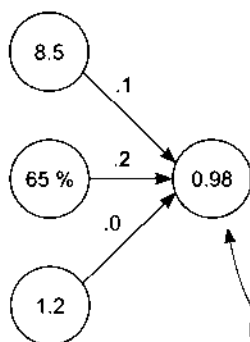
```
def w_sum(a,b):
    assert(len(a) == len(b))
    output = 0
    for i in range(len(a)):
        output += (a[i] * b[i])
    return output
```

```
def neural_network(input, weights):
    pred = w_sum(input,weights)
    return pred
```

| Входы        | Веса | =    | Частные прогнозы               |  |
|--------------|------|------|--------------------------------|--|
| (8.50 * 0.1) | =    | 0.85 | = прогноз по числу игр         |  |
| (0.65 * 0.2) | =    | 0.13 | = прогноз по доле побед        |  |
| (1.20 * 0.0) | =    | 0.00 | = прогноз по числу болельщиков |  |

прогноз по числу игр + прогноз по доле побед + прогноз по числу болельщиков = суммарный прогноз  
 0.85 + 0.13 + 0.00 = 0.98

## 4. Получение прогноза



В переменной input передается запись, соответствующая первой игре в сезоне

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

```
input = [toes[0],wlrec[0],nfans[0]]
```

```
pred = neural_network(input,weights)
```

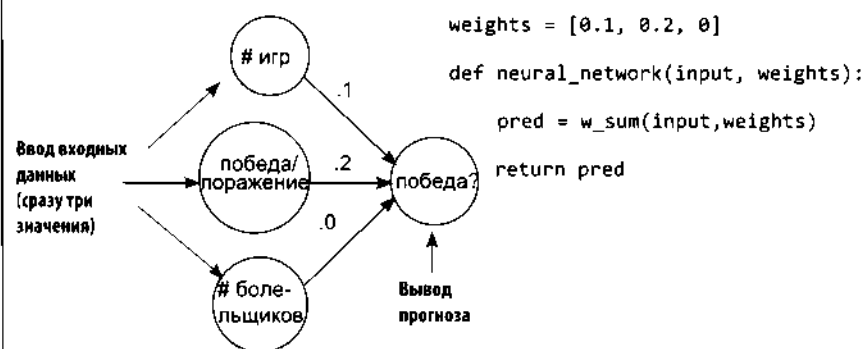
```
print(pred)
```

## Несколько входов: что делает эта нейронная сеть?

Умножает три входных значения на три весовых коэффициента и суммирует результаты. Это взвешенная сумма

В конце предыдущего раздела мы определили причину ограниченных возможностей нашей простой сети: она была всего лишь регулятором громкости, воздействующим на единственную точку данных. Роль этой точки данных играло среднее число игр, сыгранных игроками команды. Потом мы узнали, что для получения более точного прогноза нужно создать нейронную сеть, которая могла бы *объединить сразу несколько входов*. К счастью, нейронные сети способны на это.

### 1. Чистая сеть с несколькими входами

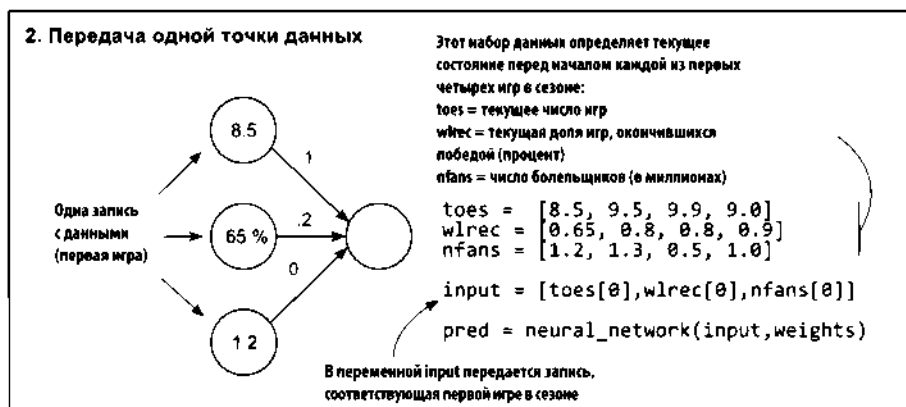


Эта новая нейронная сеть принимает *сразу несколько входных данных*, что позволяет ей объединять разные сведения и принимать более обоснованное решение. Но сам механизм использования весов при этом не изменился. Как и прежде, к каждому входному значению применяется свой регулятор громкости. Проще говоря, каждое входное значение умножается на свой весовой коэффициент.

Новой здесь является необходимость суммирования частных прогнозов для каждого входного значения. Каждый вход умножается на соответствующий ему вес, после чего результаты суммируются. Окончательный результат называется *взвешенной суммой входов*, или просто *взвешенной суммой*. Иногда взвешенную сумму называют *скалярным произведением*.

## НЕОБХОДИМОЕ НАПОМИНАНИЕ

Нейронная сеть имеет простой интерфейс: она принимает переменную `input` с исходной информацией, переменную `weights`, отражающую знание, и возвращает прогноз `prediction`.



Новая потребность обрабатывать сразу несколько входов объясняет необходимость использования нового инструмента. Он называется *вектором*, и если вы следовали за примерами в Jupyter Notebook, значит, вы уже использовали его. Вектор — это всего лишь *список чисел*. В данном примере `input` — это вектор, и `weights` тоже вектор. Сможете ли вы сами найти другие векторы в примере выше? (Там всего три вектора.)

Векторы невероятно удобны, когда в операциях участвуют группы чисел. В этом примере вычисляется взвешенная сумма (скалярное произведение) по двум векторам. Вы берете два вектора одинаковой длины (`input` и `weights`), перемножаете соответствующие элементы этих векторов (первый элемент в `input` умножается на первый элемент в `weights`, и так далее), а затем суммируете результаты.

Всякий раз, выполняя математическую операцию с двумя векторами равной длины, вы попарно объединяете значения соответствующих элементов этих векторов (и снова: первый с первым, второй со вторым и так далее). Такие операции называются *поэлементными* (*elementwise*). *Поэлементное сложение* складывает два вектора, а *поэлементное умножение* — умножает.

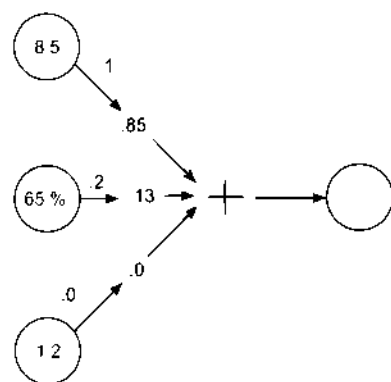
### ЗАДАЧА: ВЕКТОРНАЯ МАТЕМАТИКА

Умение выполнять операции с векторами является краеугольным камнем в глубоком обучении. Попробуйте самостоятельно написать функции, реализующие следующие операции:

- `def elementwise_multiplication(vec_a, vec_b)`
- `def elementwise_addition(vec_a, vec_b)`
- `def vector_sum(vec_a)`
- `def vector_average(vec_a)`

Затем попробуйте использовать две из них для вычисления скалярного произведения.

### 3. Вычисление взвешенной суммы входов



```
def w_sum(a,b):
    assert(len(a) == len(b))
    output = 0
    for i in range(len(a)):
        output += (a[i] * b[i])
    return output
```

```
def neural_network(input, weights):
    pred = w_sum(input,weights)
    return pred
```

| Входы                                                                                           | Веса | Частные прогнозы |                                |
|-------------------------------------------------------------------------------------------------|------|------------------|--------------------------------|
| (8.50 * 0.1)                                                                                    | =    | 0.85             | = прогноз по числу игр         |
| (0.65 * 0.2)                                                                                    | =    | 0.13             | = прогноз по доле побед        |
| (1.20 * 0.0)                                                                                    | =    | 0.00             | = прогноз по числу болельщиков |
| прогноз по числу игр + прогноз по доле побед + прогноз по числу болельщиков = суммарный прогноз |      |                  |                                |
| 0.85                                                                                            | +    | 0.13             | + 0.00 = 0.98                  |

Не зная свойств скалярного произведения (взвешенной суммы), невозможно по-настоящему понять, как нейронная сеть получает прогноз. Если говорить

простыми словами, скалярное произведение позволяет получить *представление о сходстве* двух векторов. Взгляните на следующие примеры:

|                      |                     |
|----------------------|---------------------|
| $a = [0, 1, 0, 1]$   | $w\_sum(a, b) = 0$  |
| $b = [1, 0, 1, 0]$   | $w\_sum(b, c) = 1$  |
| $c = [0, 1, 1, 0]$   | $w\_sum(b, d) = 1$  |
| $d = [.5, 0, .5, 0]$ | $w\_sum(c, c) = 2$  |
| $e = [0, 1, -1, 0]$  | $w\_sum(d, d) = .5$ |
|                      | $w\_sum(c, e) = 0$  |

Самой большой оказалась взвешенная сумма ( $w\_sum(c, c)$ ) двух абсолютно идентичных векторов. Напротив, так как векторы  $a$  и  $b$  имеют прямо противоположные веса, их скалярное произведение равно нулю. Наибольший интерес представляет, пожалуй, взвешенная сумма векторов  $c$  и  $e$ , потому что  $e$  имеет один отрицательный вес. Этот отрицательный вес уравнивает положительное сходство между ними. Однако скалярное произведение вектора  $e$  на самого себя дает в результате 2, несмотря на отрицательный вес (как известно, минус на минус дает плюс).

А теперь познакомимся с некоторыми свойствами скалярного произведения.

Иногда скалярное произведение можно сравнить с логической операцией AND. Возьмем векторы  $a$  и  $b$ :

$a = [0, 1, 0, 1]$   
 $b = [1, 0, 1, 0]$

Если вы спросите, имеют ли оба элемента  $a[0]$  AND  $b[0]$  ненулевые значения, ответ будет «нет». Если вы спросите, имеют ли оба элемента  $a[1]$  AND  $b[1]$  ненулевые значения, ответ снова будет «нет». И так как этот ответ вы получите для всех четырех элементов, окончательный результат будет равен 0. Ни одна пара элементов не преодолет логическую операцию AND.

$b = [1, 0, 1, 0]$   
 $c = [0, 1, 1, 0]$

Векторы  $b$  и  $c$ , однако, имеют в одном столбце одинаковые значения. Они преодолеют логическую операцию AND, потому что  $b[2]$  AND  $c[2]$  вернет ненулевой вес. Этот (и только этот) столбец поднимет оценку до 1.

$c = [0, 1, 1, 0]$   
 $d = [.5, 0, .5, 0]$

К счастью, нейронные сети способны моделировать частичную операцию AND. В данном случае *c* и *d* имеют в одном столбце ненулевые значения, но, так как *d* имеет в этом столбце вес 0.5, окончательная оценка получается равной 0.5. Это свойство широко используется в нейронных сетях для моделирования вероятностей.

```
d = [.5, 0, .5, 0]
e = [-1, 1, 0, 0]
```

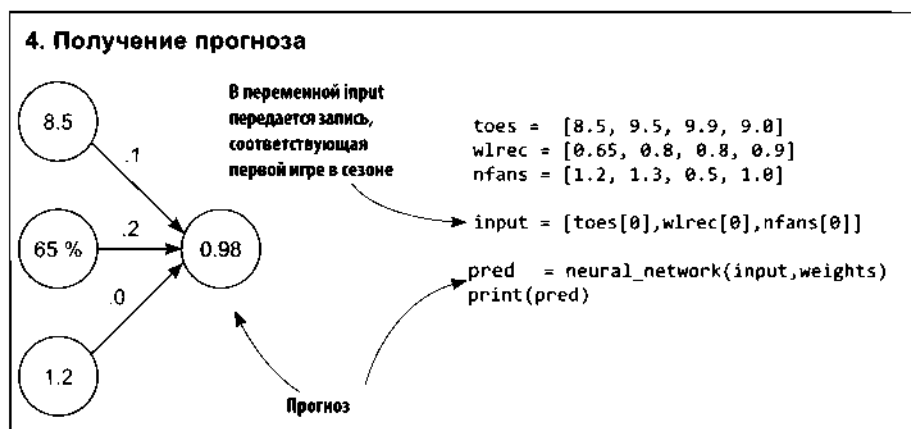
Согласно этой аналогии, отрицательные веса, как правило, подразумевают логическую операцию NOT. Положительный вес в паре с отрицательным даст в результате снижение оценки. Кроме того, если оба вектора имеют отрицательные веса (как в случае `w_sum(e, e)`), минус на минус даст плюс и нейронная сеть сложит два веса, получив положительный результат. Также можно сказать, что скалярное произведение — это операция OR, следующая за операцией AND, потому что если в любом столбце результата получится ненулевой вес, это повлияет на окончательную оценку. Если (`a[0] AND b[0]`) OR (`a[1] AND b[1]`), и так далее, даст ненулевой результат, тогда `w_sum(a, b)` вернет положительную оценку. А если один из столбцов будет иметь отрицательное значение, тогда к нему применится операция NOT.

Самое интересное, что такой подход дает нам грубый язык выражения весов. Попробуйте прочитав несколько примеров и скажите: разве я не прав? Здесь предполагается выполнение операции `w_sum(input, weights)`, а под оператором «then» в этих инструкциях if подразумевается абстрактное «тогда дать высокую оценку»:

```
weights = [ 1, 0, 1] => if input[0] OR input[2]
weights = [ 0, 0, 1] => if input[2]
weights = [ 1, 0, -1] => if input[0] OR NOT input[2]
weights = [ -1, 0, -1] => if NOT input[0] OR NOT input[2]
weights = [ 0.5, 0, 1] => if BIG input[0] or input[2]
```

Обратите внимание: `weight[0] = 0.5` в последней строке означает, что соответствующее значение `input[0]` должно быть больше, чтобы компенсировать меньший вес. Но, как я уже говорил, это *очень* грубый язык. Однако я считаю, что его вполне можно использовать, чтобы получить общее представление о происходящем за кулисами. Знание этого языка поможет вам в будущем, особенно при объединении сетей более сложными способами.

Но какое значение все это имеет для получения прогноза? Упрощенно говоря, следуя описанной логике, нейронная сеть оценивает входы, *исходя из степени их сходства с весами*. Обратите внимание, что в следующем примере значение `nfans` никак не влияет на результат прогнозирования, потому что соответствующий ему вес равен 0. Наиболее существенным прогнозным признаком является `wlrec`, потому что он имеет вес 0.2. Но наиболее существенный вклад в прогноз вносит число игр (`ntoes`), не потому, что ему соответствует самый высокий вес, а потому, что вход, объединяемый с соответствующим весом, имеет самое большое значение.



Вот еще несколько важных аспектов, которые стоит запомнить на будущее. Вы не можете перемешать веса: они должны находиться в определенных позициях. Кроме того, на итоговую оценку влияют обе величины: значение веса и входное значение. Наконец, отрицательный вес приведет к тому, что соответствующее ему входное значение уменьшит прогнозную оценку (и наоборот).

## Несколько входов: полный выполняемый код

Предыдущие фрагменты кода из этого примера можно объединить вместе и получить программу, которая создает и использует нейронную сеть. В целях упрощения я использовал только основные конструкции языка Python (списки и числа). Но есть более эффективный способ реализации, который мы используем в будущих примерах.

## Предыдущий код

```
def w_sum(a,b):
    assert(len(a) == len(b))
    output = 0
    for i in range(len(a)):
        output += (a[i] * b[i])
    return output
weights = [0.1, 0.2, 0]
def neural_network(input, weights):
    pred = w_sum(input,weights)
    return pred

toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]

input = [toes[0],wlrec[0],nfans[0]]
pred = neural_network(input,weights)
print(pred)
```

В переменной `input` передается запись, соответствующая первой игре в сезоне.

Для Python имеется библиотека, которая называется NumPy (от англ. *numerical Python* – «численные методы для Python»). Она включает очень эффективную реализацию операций с векторами (таких, как вычисление скалярного произведения). Ниже показано, как реализовать ту же программу с использованием NumPy.

## Код с использованием NumPy

```
import numpy as np
weights = np.array([0.1, 0.2, 0])
def neural_network(input, weights):
    pred = input.dot(weights)
    return pred

toes = np.array([8.5, 9.5, 9.9, 9.0])
wlrec = np.array([0.65, 0.8, 0.8, 0.9])
nfans = np.array([1.2, 1.3, 0.5, 1.0])

input = np.array([toes[0],wlrec[0],nfans[0]])
pred = neural_network(input,weights)
print(pred)
```

В переменной `input` передается запись, соответствующая первой игре в сезоне.

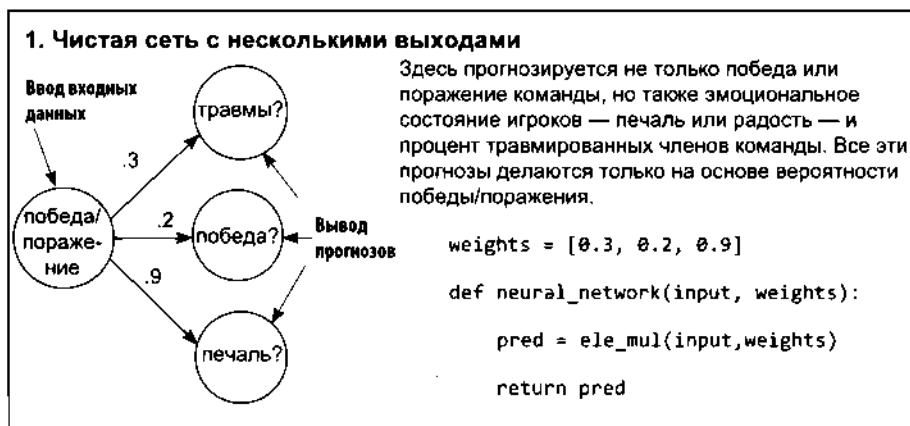


Обе программы должны вывести число 0.98. Обратите внимание, что при использовании библиотеки NumPy отпала необходимость определять функцию `w_sum`. Вместо нее можно использовать функцию `dot` из библиотеки NumPy (сокращенно от «dot product» — «скалярное произведение»). Многие функции, которые вам понадобятся в будущем, имеют аналоги в библиотеке NumPy.

## Прогнозирование с несколькими выходами

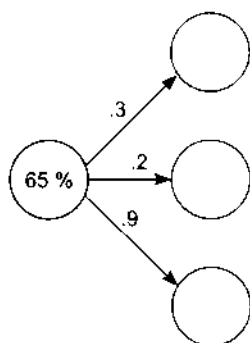
### Сети способны возвращать несколько прогнозов для единственного входа

Реализовать получение нескольких выходов, пожалуй, проще, чем прием нескольких входов. В этом случае прогнозирование выполняется так, как если бы имелось несколько независимых нейронных сетей с единственным весовым коэффициентом в каждой.



Обратите внимание, что все три прогноза совершенно разные. В отличие от нейронных сетей с несколькими входами и единственным выходом, где все взаимосвязано, эта сеть действует так, как если бы состояла из трех независимых компонентов, каждый из которых получает одни и те же входные данные. Это упрощает реализацию сети.

## 2. Передача одной точки данных

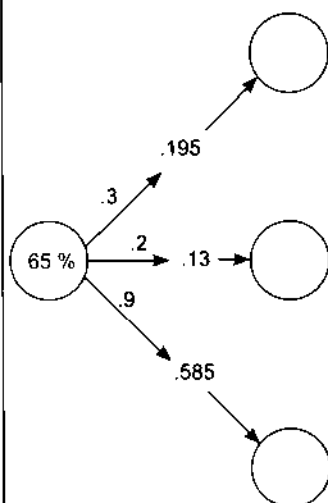


```
wlrec = [0.65, 0.8, 0.8, 0.9]
```

```
input = wlrec[0]
```

```
pred = neural_network(input, weights)
```

## 3. Выполнение поэлементного умножения



```
def ele_mul(number, vector):
```

```
    output = [0, 0, 0]
```

```
    assert(len(output) == len(vector))
```

```
    for i in range(len(vector)):
        output[i] = number * vector[i]
```

```
    return output
```

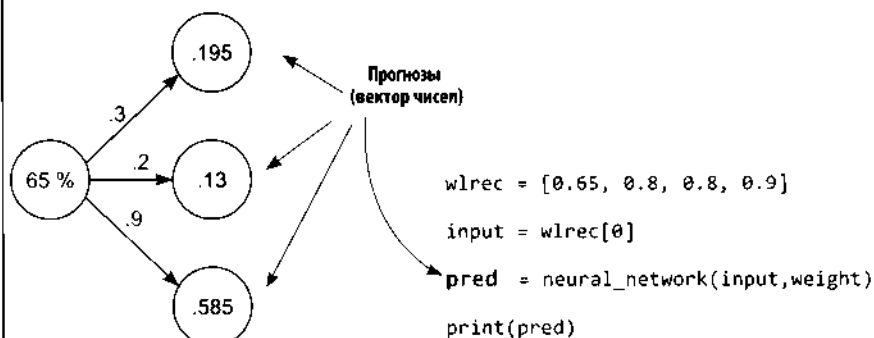
```
def neural_network(input, weights):
```

```
    pred = ele_mul(input, weights)
```

```
    return pred
```

| Входы        | Веса | Итоговые прогнозы |                                 |
|--------------|------|-------------------|---------------------------------|
| (0.65 * 0.3) | =    | 0.195             | = прогноз вероятности травм     |
| (0.65 * 0.2) | =    | 0.13              | = прогноз вероятности победы    |
| (0.65 * 0.9) | =    | 0.585             | = прогноз вероятности огорчения |

## 4. Получение прогноза



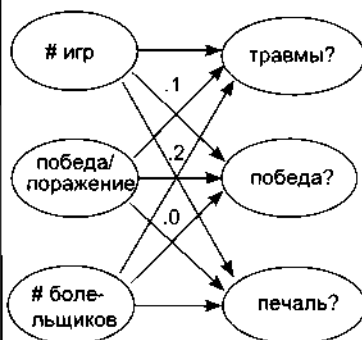
## Прогнозирование с несколькими входами и выходами

Сети способны возвращать несколько прогнозов для нескольких входов

Наконец, подходы к созданию сетей с несколькими входами и несколькими выходами можно объединить и создать сеть, принимающую несколько значений на входе и возвращающую несколько прогнозов. Как и прежде, каждый вес в этой сети связывает каждое входное значение с каждым выходным значением, и прогнозирование происходит обычным образом.

## 1. Чистая сеть с несколькими входами и выходами

**Входы**      **Прогнозы**



```

# игр % побед # болельщиков
weights = [ [0.1, 0.1, -0.3], # травмы?
            [0.1, 0.2, 0.0], # победа?
            [0.0, 1.3, 0.1] ] # печаль?

```

```

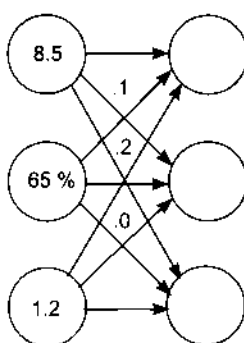
def neural_network(input, weights):
    pred = vect_mat_mul(input, weights)
    return pred

```

## 2. Передача одной точки данных

Входы

Прогнозы



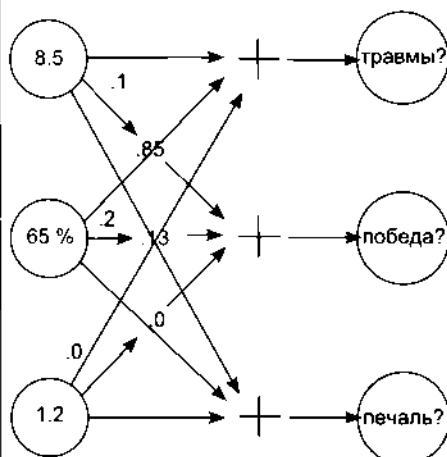
Этот набор данных определяет текущее состояние перед началом каждой из первых четырех игр в сезоне:  
 toes = текущее среднее число игр, сыгранных игроками  
 wlrcc = текущая доля игр, окончившихся победой (процент)  
 fans = число болельщиков (в миллионах)

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrcc = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

```
input = [toes[0], wlrcc[0], nfans[0]]
pred = neural_network(input, weights)
```

В переменной input передается запись, соответствующая первой игре в сезоне

## 3. Для каждого выхода вычисляется взвешенная сумма входов



```
def w_sum(a,b):
    assert(len(a) == len(b))
    output = 0
    for i in range(len(a)):
        output += (a[i] * b[i])
    return output
```

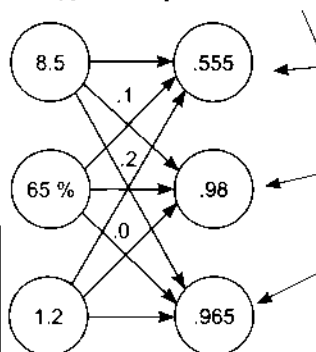
```
def vect_mat_mul(vect, matrix):
    assert(len(vect) == len(matrix))
    output = [0,0,0]
    for i in range(len(vect)):
        output[i] = w_sum(vect, matrix[i])
    return output
```

```
def neural_network(input, weights):
    pred = vect_mat_mul(input, weights)
    return pred
```

| # игр                                     | % побед | # болельщиков                   |  |
|-------------------------------------------|---------|---------------------------------|--|
| (8.5 * 0.1) + (0.65 * 0.1) + (1.2 * -0.3) | = 0.555 | = прогноз вероятности травм     |  |
| (8.5 * 0.1) + (0.65 * 0.2) + (1.2 * 0.0)  | = 0.98  | = прогноз вероятности победы    |  |
| (8.5 * 0.0) + (0.65 * 1.3) + (1.2 * 0.1)  | = 0.965 | = прогноз вероятности огорчения |  |

## 4. Получение прогноза

Входы Прогнозы



В переменной input передается запись, соответствующая первой игре в сезоне

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]

input = [toes[0], wlrec[0], nfans[0]]
pred = neural_network(input, weight)
```

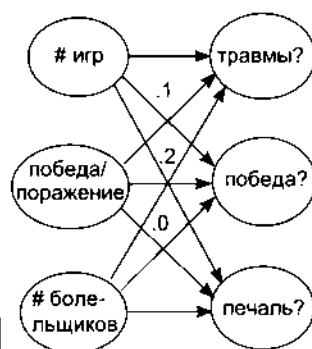
## Несколько входов и выходов: как это работает?

Для получения трех прогнозов вычисляется три независимых взвешенных суммы входов

На эту архитектуру можно взглянуть с двух точек зрения: как на три веса, исходящих из каждого входного узла, или как на три веса, входящих в каждый выходной узел. Последняя мне кажется наиболее удобной. Представьте эту нейронную сеть как три независимых скалярных произведения: три независимые взвешенные суммы входов. Каждый выходной узел получает свою взвешенную сумму входов и выдает прогноз.

## 1. Чистая сеть с несколькими входами и выходами

Входы Прогнозы

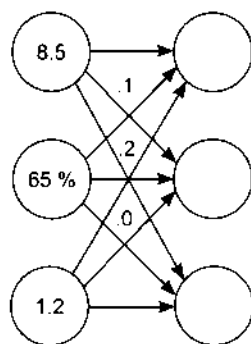


```
# игр % побед # болельщиков
weights = [ [0.1, 0.1, -0.3], # травмы?
            [0.1, 0.2, 0.0], # победа?
            [0.0, 1.3, 0.1] ] # печаль?
```

```
def neural_network(input, weights):
    pred = vect_mat_mul(input, weights)
    return pred
```

## 2. Передача одной точки данных

Входы Прогнозы



Этот набор данных определяет текущее состояние перед началом каждой из первых четырех игр в сезоне: *toes* = текущее среднее число игр, сыгранных игроками  
*wlrec* = текущая доля игр, окончившихся победой (процент)  
*fans* = число болельщиков (в миллионах)

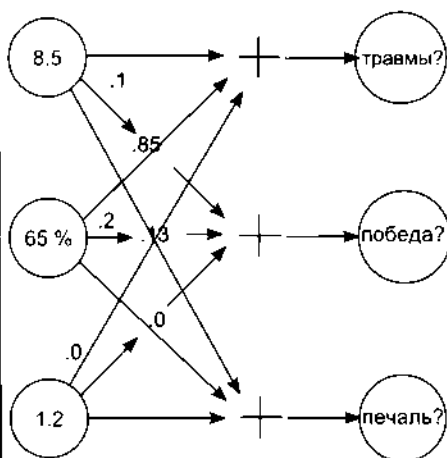
```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

```
input = [toes[0], wlrec[0], nfans[0]]
```

```
pred = neural_network(input, weights)
```

В переменной *input* передается запись, соответствующая первой игре в сезоне

## 3. Для каждого выхода вычисляется взвешенная сумма входов



```
def w_sum(a,b):
    assert(len(a) == len(b))
    output = 0
    for i in range(len(a)):
        output += (a[i] * b[i])
    return output
```

```
def vect_mat_mul(vect, matrix):
    assert(len(vect) == len(matrix))
    output = [0,0,0]

    for i in range(len(vect)):
        output[i] = w_sum(vect, matrix[i])

    return output
```

```
def neural_network(input, weights):
    pred = vect_mat_mul(input, weights)
    return pred
```

```
# игр      % побед    # болельщиков
(8.5 * 0.1) + (0.65 * 0.1) + (1.2 * -0.3) = 0.555 = прогноз вероятности травм
(8.5 * 0.1) + (0.65 * 0.2) + (1.2 * 0.0) = 0.98 = прогноз вероятности победы
(8.5 * 0.0) + (0.65 * 1.3) + (1.2 * 0.1) = 0.965 = прогноз вероятности огорчения
```

Мы решили, что будем рассматривать эту сеть как серию нескольких взвешенных сумм. Поэтому в предыдущем коде была определена новая функция *vect\_mat\_mul*. Она выполняет обход всех векторов весов и вычисляет прогноз

с помощью функции `w_sum`. Фактически, она последовательно находит три взвешенных суммы и сохраняет результаты в векторе `output`. На этот раз в вычислениях участвует намного больше весов, но сами вычисления ненамного сложнее, чем в предыдущих примерах.

Я воспользуюсь этим *списком векторов* и логикой вычисления *серии взвешенных сумм*, чтобы познакомить вас с двумя новыми понятиями. Взгляните на определение переменной `weights` на шаге 1. Это список векторов. Список векторов называют *матрицей*. В матрицах нет ничего сложного. Матрицы используются во многих операциях. Одна из таких операций называется *векторно-матричным умножением*. Именно она используется для вычисления серии взвешенных сумм: она берет вектор и находит скалярное произведение между ним и каждой строкой в матрице.<sup>1</sup> Как вы узнаете в следующем разделе, в библиотеке NumPy имеется специальная функция, реализующая эту операцию.

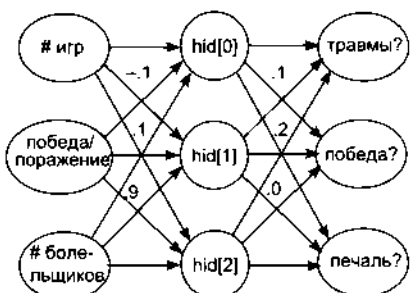
## Прогнозирование на основе прогнозов

Нейронные сети можно накладывать друг на друга!

Как показано на следующих рисунках, выход одной сети можно передать на вход другой. В результате получится цепочка векторно-матричных умножений. Поясню для тех, кому не совсем понятно, где такой способ прогнозирования может пригодиться: некоторые наборы данных (например, изображения) содержат закономерности, слишком сложные для единственной матрицы весов. Позднее мы обсудим природу этих закономерностей, а пока просто имейте в виду, что такое возможно.

### 1. Чистая сеть с несколькими входами и выходами

**Входы**   **Скрытый слой**   **Прогнозы**



```
# игр % побед # болельщиков
ih_wgt = [ [0.1, 0.2, -0.1], # hid[0]
            [-0.1, 0.1, 0.9], # hid[1]
            [0.1, 0.4, 0.1] ] # hid[2]

#hid[0] hid[1] hid[2]
hp_wgt = [ [0.3, 1.1, -0.3], # травмы?
            [0.1, 0.2, 0.0], # победа?
            [0.0, 1.3, 0.1] ] # печаль?

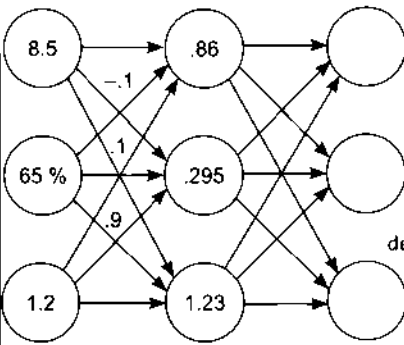
weights = [ih_wgt, hp_wgt]

def neural_network(input, weights):
    hid = vect_mat_mul(input, weights[0])
    pred = vect_mat_mul(hid, weights[1])
    return pred
```

<sup>1</sup> Формально в линейной алгебре векторы весов хранятся/обрабатываются как векторы-столбцы (а не как векторы-строки). Но об этом чуть ниже.

## 2. Прогнозирование в скрытом слое

Входы      Скрытый слой      Прогнозы



В переменной input передается запись, соответствующая первой игре в сезоне

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]

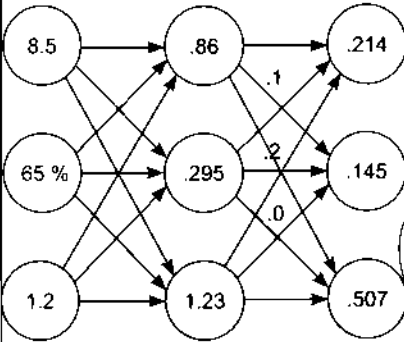
input = [toes[0], wlrec[0], nfans[0]]

pred = neural_network(input, weights)

def neural_network(input, weights):
    hid = vect_mat_mul(input, weights[0])
    pred = vect_mat_mul(hid, weights[1])
    return pred
```

## 3. Прогнозирование в выходном слое (и вывод прогноза)

Входы      Скрытый слой      Прогнозы



def neural\_network(input, weights):

```
    hid = vect_mat_mul(input, weights[0])
    pred = vect_mat_mul(hid, weights[1])
    return pred
```

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

```
input = [toes[0], wlrec[0], nfans[0]]
```

```
pred = neural_network(input, weights)
print(pred)
```

В переменной input передается запись, соответствующая первой игре в сезоне

Следующий листинг демонстрирует, как ту же операцию, описанную выше, можно реализовать с использованием удобной библиотеки NumPy для Python. Библиотеки, такие как NumPy, делают программный код более быстрым и читаемым.

## Версия с NumPy

```
import numpy as np
```

```
# игр % побед болельщиков
```

```
ih_wgt = np.array([
    [0.1, 0.2, -0.1], # hid[0]
    [-0.1, 0.1, 0.9], # hid[1]
    [0.1, 0.4, 0.1]]) # hid[2]
```



```
# hid[0] hid[1] hid[2]
hp_wgt = np.array([
    [0.3, 1.1, -0.3], # травмы?
    [0.1, 0.2, 0.0], # победа?
    [0.0, 1.3, 0.1] ]).T # печаль?

weights = [ih_wgt, hp_wgt]

def neural_network(input, weights):

    hid = input.dot(weights[0])
    pred = hid.dot(weights[1])
    return pred

toes = np.array([8.5, 9.5, 9.9, 9.0])
wlrec = np.array([0.65, 0.8, 0.8, 0.9])
nfans = np.array([1.2, 1.3, 0.5, 1.0])

input = np.array([toes[0], wlrec[0], nfans[0]])

pred = neural_network(input, weights)
print(pred)
```

## Короткий пример использования NumPy

### NumPy может многое. Раскроем ее секреты

К настоящему моменту мы рассмотрели два новых математических инструмента: векторы и матрицы. Вы познакомились с некоторыми операциями над векторами и матрицами, включая скалярное произведение, поэлементное умножение и сложение, а также векторно-матричное умножение. Для этих операций мы написали функции на Python, оперирующие простыми объектами списков.

В ближайшее время я продолжу писать и использовать эти функции, чтобы дать вам возможность до конца понять происходящее. Но теперь, когда я уже упомянул библиотеку NumPy и несколько крупных операций, реализованных в ней, я хотел бы кратко познакомить вас с основами использования NumPy, чтобы подготовить к переходу к главам, где все сложные операции выполняются только с помощью NumPy. Начнем с азов: векторов и матриц.

```
import numpy as np

a = np.array([0,1,2,3])  ← Вектор
b = np.array([4,5,6,7])  ← Еще один вектор
c = np.array([[0,1,2,3],  ← Матрица
               [4,5,6,7]])
```

```

d = np.zeros((2,4))  ← Матрица 2 × 4, заполненная нулями
e = np.random.rand(2,5) ← Матрица 2 × 5, заполненная случайными числами от 0 до 1

print(a)
print(b)
print(c)
print(d)
print(e)

```

### Вывод:

```

[0 1 2 3]
[4 5 6 7]
[[0 1 2 3]
 [4 5 6 7]]
[[ 0.  0.  0.  0.]
 [ 0.  0.  0.  0.]]
[[ 0.22717119  0.39712632  0.0627734  0.08431724  0.53469141]
 [ 0.09675954  0.99012254  0.45922775  0.3273326  0.28617742]]

```

Создавать матрицы и векторы в NumPy можно разными способами. Типичные и широко используемые при работе с нейронными сетями показаны в предыдущем листинге. Обратите внимание, что процессы создания вектора и матрицы идентичны. При создании матрицы с единственной строкой получится вектор. И, как и везде в математике, при создании матрицы сначала указывается число строк, а следом число столбцов. Я говорю это только для того, чтобы вы запомнили порядок: сначала строки, а потом столбцы. Давайте посмотрим, какие операции можно выполнять с этими векторами и матрицами:

```

Каждый элемент матрицы c
будет умножен на 0.2
→ print(a * 0.1)
→ print(c * 0.2)
→ print(a * b)
→ print(a * b * 0.2)
→ print(a * c)

Каждый элемент вектора a
будет умножен на 0.1
→ print(a * 0.1)

Позлементное
умножение a на b
→ print(a * b)

Позлементное умножение
векторов, затем каждый элемент
результата будет умножен на 0.2
→ print(a * b * 0.2)

Позлементное умножение
вектора a с каждой строкой
матрицы c, поскольку c имеет
то же число столбцов, что и a
→ print(a * c)

Так как a и e имеют разное число столбцов, эта операция
возбудит исключение «Value Error: operands could not be
broadcast together with...» («Ошибка в значении: операнды
не могут передаваться вместе с...»)
→ print(a * e)

```

Сделайте еще шаг и запустите предыдущий код. Сразу отметим первые чудеса, «сначала непонятные, но потом очевидные». При умножении двух переменных с помощью оператора `*`, NumPy автоматически определит их типы и попробует

понять, какую операцию вы пытаетесь выполнить. Иногда это очень удобно, но порой усложняет чтение кода. Старайтесь внимательно следить за типами переменных по мере движения.

Главное правило, касающееся поэлементных операций (+, -, \*, /), — обе переменные должны иметь *одинаковое* число столбцов или одна из переменных должна иметь только один столбец. Например, `print(a * 0.1)` умножает вектор на единственное число (скаляр). NumPy говорит: «Ага! Бьюсь об заклад, что здесь имеет место векторно-скалярное умножение», — а затем умножает каждый элемент вектора на скаляр (0.1). Аналогично выполняется операция `print(c * 0.2)`, с той лишь разницей, что библиотека NumPy знает, что `c` — это матрица. Она выполняет матрично-скалярное умножение, умножая каждый элемент матрицы `c` на 0.2. Так как скаляр имеет единственный столбец, на него можно умножить все, что угодно (или разделить, сложить или вычесть).

Далее: `print(a * b)`. NumPy сначала определит, что имеет дело с двумя векторами. Так как в каждом векторе число столбцов больше одного, NumPy проверит, совпадает ли число столбцов в обоих векторах. А поскольку число столбцов в обоих векторах одинаковое, NumPy выполнит попарное умножение элементов в соответствии с их позициями. То же верно в отношении сложения, вычитания и деления.

Инструкция `print(a * c)` является, пожалуй, самой непонятной. Переменная `a` — это вектор с четырьмя столбцами, а `c` — матрица ( $2 \times 4$ ). В обоих переменных больше одного столбца, поэтому дальше NumPy сравнит число столбцов в них. Оно одинаково, поэтому NumPy умножит вектор `a` на каждую строку в `c` (как если бы выполнялось поэлементное умножение вектора на каждую строку).

Сложность заключается в том, что все эти операции выглядят совершенно одинаковыми, если не знать, какие переменные являются скалярами, векторами или матрицами. Читая код, использующий NumPy, вы на самом деле должны не только читать операции, но и помнить *форму* (число строк и столбцов) каждого операнда, а для этого необходима некоторая практика. Но рано или поздно это войдет в привычку. Рассмотрим еще несколько примеров умножения матриц в NumPy, попутно запоминая формы операндов и результатов.

```
a = np.zeros((1,4))
b = np.zeros((4,3))

c = a.dot(b)  ← Вектор с длиной 4
print(c.shape) ← Матрица с 4 строками и 3 столбцами
```

**Вывод**

(1,3)

Есть одно золотое правило, касающееся использования функции `dot`: если поместить рядом описания (строки, столбцы) двух переменных, которые вы пытаетесь передать в функцию `dot`, соседние числа всегда должны совпадать. В данном случае мы пытаемся получить скалярное произведение матриц (1, 4) и (4, 3). Эта операция будет успешно выполнена, и в результате получится матрица (1, 3). В терминах формы переменных это правило можно перефразировать так: независимо от типов операндов — векторы это или матрицы — их *формы* (число строк и столбцов) должны совпадать. Число столбцов в матрице слева должно совпадать с числом строк в матрице справа, и в результате  $(a, b) \cdot dot(b, c) = (a, c)$ .

```

a = np.zeros((2,4))  ← Матрица с 2 строками и 4 столбцами
b = np.zeros((4,3))  ← Матрица с 4 строками и 3 столбцами

c = a.dot(b)
print(c.shape)  ← Выведет (2,3)

e = np.zeros((2,1))  ← Матрица с 2 строками и 1 столбцом
f = np.zeros((1,3))  ← Матрица с 1 строкой и 3 столбцами

g = e.dot(f)
print(g.shape)  ← Выведет (2,3)

h = np.zeros((5,4)).T  ← Матрица с 4 строками и 5 столбцами
i = np.zeros((5,6))    ← Матрица с 6 строками и 5 столбцами

j = h.dot(i)
print(j.shape)  ← Выведет (4,6)

h = np.zeros((5,4))  ← Матрица с 5 строками и 4 столбцами
i = np.zeros((5,6))  ← Матрица с 5 строками и 6 столбцами
j = h.dot(i)
print(j.shape)  ← Сгенерирует ошибку

```

Операция `.T` повернет (транспонирует) матрицу, поменяв строки и столбцы местами

## Итоги

Чтобы получить прогноз, нейронные сети многократно вычисляют взвешенную сумму для входных данных

В этой главе вы увидели постепенно усложняющиеся разновидности нейронных сетей. Я надеюсь, мне удалось показать, что, используя относительно небольшой набор простых правил, можно создавать большие и сложные нейронные сети. Прогнозирующие возможности сети зависят от значений весов, которые вы ей передадите.

Все сети, которые мы создали в этой главе, называются сетями *прямого пространства* – они принимают входные данные и получают прогноз. Такое название они получили потому, что информация в них *распространяется в прямом направлении*. В этих примерах информацией являются все числа, не являющиеся весами и уникальные для каждого прогноза.

В следующей главе вы узнаете, как настраивать веса, чтобы повысить точность прогноза, получаемого нейронной сетью. Так же как прогнозирование основано на нескольких простых приемах, повторяющихся и накладываемых друг на друга, настройка *весов*, или *обучение*, реализуется как комбинация простых приемов, которые многократно объединяются и образуют архитектуру. Увидимся там!

# 4

## Введение в нейронное обучение: градиентный спуск



### В этой главе

- ✓ Способны ли нейронные сети делать точные прогнозы?
- ✓ Зачем измерять ошибку?
- ✓ Обучение методом «холодно/горячо».
- ✓ Вычисление направления и величины из ошибки.
- ✓ Градиентный спуск.
- ✓ Обучение просто уменьшает ошибку.
- ✓ Производные и как их использовать для обучения.
- ✓ Расхождение и альфа-коэффициент.

Единственный надежный способ проверить гипотезу —  
сравнить ее прогноз с экспериментальными данными.

*Милтон Фридман (Milton Friedman),  
Essays in Positive Economics  
(издательство Чикагского университета, 1953)*

## Предсказание, сравнение и обучение

В главе 3 вы познакомились с парадигмой «предсказание, сравнение, обучение» и углубились в изучение первого шага: *предсказания*. Попутно вы узнали много нового, включая основные компоненты нейронных сетей (узлы и веса), как организуется соответствие наборов данных и сети (количество точек данных, одновременно подаваемых на вход) и как нейронная сеть получает прогноз.

Возможно, в процессе чтения у вас возник вопрос: «Как выбрать значения весов так, чтобы сеть получала точные прогнозы?» Ответ на этот вопрос является главной темой этой главы, и здесь мы рассмотрим следующие два шага парадигмы: *сравнение* и *обучение*.

## Сравнение

**Сравнение позволяет оценить, насколько прогноз «промахнулся»**

Следующий шаг после получения прогноза — оценка его качества. Это может показаться простым делом, но, как вы убедитесь сами, выбор хорошего способа измерения ошибки — одна из самых сложных и важных задач в глубоком обучении.

Есть много разных подходов к измерению ошибок, которые вы наверняка использовали в своей жизни, даже не подозревая об этом. Например, вы (или ваш знакомый) могли преувеличивать большие ошибки и игнорировать мелкие. В этой главе вы познакомитесь с математическим аппаратом, который поможет научить сеть это делать. Вы также узнаете, что ошибка всегда положительна! В качестве аналогии возьмем стрельбу из лука по мишени: если стрела попала в мишень на дюйм выше или ниже, в обоих случаях ошибка составит 1 дюйм. На этапе *сравнения* результатов, полученных от нейронной сети, необходимо учитывать это при оценке ошибки.

Сразу отмечу, что в этой главе мы будем оценивать только один простой способ измерения ошибки: вычисление *среднеквадратической ошибки*. Это лишь один из способов оценки точности нейронной сети.

Этот шаг поможет вам получить представление, насколько вы промахнулись, однако этого недостаточно для обучения. Результатом логики *сравнения* является сигнал «горячо/холодно». Мера ошибки, вычисленная по результатам прогноза, сообщит вам, «насколько сильно» вы промахнулись, но она ничего

не скажет, почему случился промах, в какую сторону вы промахнулись или что нужно сделать, чтобы исправить ошибку, — она лишь скажет «сильно промахнулись», «мало промахнулись» или «попали точно в цель». Исправление ошибки — это уже задача следующего этапа: *обучения*.

## Обучение

Процесс обучения определяет, как изменить каждый вес, чтобы уменьшить ошибку

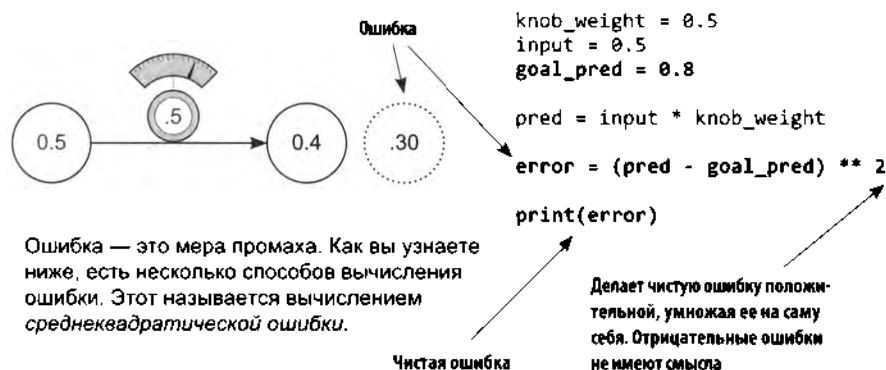
Обучение — это процесс *определения причин ошибок*, или искусство выяснения вклада каждого веса в общую ошибку. Это главная задача глубокого обучения. В этой главе мы много времени уделим изучению одного из самых популярных способов решения этой задачи: *градиентного спуска*.

Этот способ позволяет вычислить для каждого веса некоторое число, определяющее, насколько этот вес должен быть выше или ниже, чтобы уменьшить ошибку. После этого вам останется только изменить вес на это число, и дело в шляпе.

## Сравнение: способны ли нейронные сети делать точные прогнозы?

Измерим ошибку и узнаем!

Выполните следующий код в своем блокноте Jupyter Notebook. Он должен вывести 0.3025:





**ЧТО ЭТО ЗА ПЕРЕМЕННАЯ `goal_pred`?**

Переменная `goal_pred`, так же как `input`, хранит число, полученное в реальном мире путем наблюдений, иногда очень сложных, как, например, «процент людей, надевших теплую одежду» при данной температуре воздуха; или «попал ли отбивающий в хоум-ран»<sup>1</sup>.

**ПОЧЕМУ ОШИБКА ВОЗВОДИТСЯ В КВАДРАТ?**

Представьте лучника, стреляющего в мишень. Допустим, стрела попала в мишень на 2 дюйма выше центра. Насколько промахнулся лучник? А если на 2 дюйма ниже? В обоих случаях лучник промахнулся на 2 дюйма. Основная причина возведения в квадрат «величины промаха» заключается в получении *положительного* числа. Выражение  $(pred - goal\_pred)$  может дать отрицательный результат, в отличие от фактической ошибки.

**РАЗВЕ ВОЗВЕДЕНИЕ В КВАДРАТ НЕ УВЕЛИЧИВАЕТ БОЛЬШИЕ ОШИБКИ ( $>1$ ) И НЕ УМЕНЬШАЕТ МАЛЕНЬКИЕ ( $<1$ )?**

Да... Это немного странный способ измерения ошибки, но, как оказывается, *преувеличение* больших ошибок и *преуменьшение* маленьких — это нормально. Позднее вы будете использовать эту ошибку для обучения сети, поэтому лучше сосредоточить внимание на больших ошибках и игнорировать маленькие. Так же поступают хорошие родители: они не замечают мелких ошибок своих детей (например, сломанный грифель карандаша), но могут взорваться в случае большой ошибки (например, если сын или дочь разбили автомобиль). Теперь понимаете, почему возведение в квадрат может быть полезным?

## Зачем измерять ошибку?

### Измерение ошибки упрощает задачу

Цель обучения нейронной сети — получение достоверных прогнозов. Это наше желание. И в нашем прагматичном мире (как отмечалось в предыдущей главе) хотелось бы иметь сеть, принимающую входные данные, которые легко получить (например, сегодняшние цены на акции), и предсказывающую что-то, что трудно вычислить (завтрашние цены на акции). Это то, что делает нейронные сети полезными.

<sup>1</sup> [https://ru.wikipedia.org/wiki/Бейсбольная\\_терминология](https://ru.wikipedia.org/wiki/Бейсбольная_терминология).

Как оказывается, правильно подобрать значение `knob_weight`, чтобы сеть верно предсказывала `goal_prediction`, несколько сложнее, чем просто присвоить `knob_weight` такое значение, при котором `error == 0`. Есть какой-то недостаток при таком взгляде на проблему. В целом оба утверждения говорят об одном и том же, но попытка *получить ошибку, равную 0*, выглядит более прямолинейным решением.

## Разные способы измерения ошибки по-разному оценивают важность ошибок

Не волнуйтесь, если смысл этой фразы покажется вам непонятным, просто вспомните, что я говорил выше: при *возведении ошибки в квадрат* значения меньше 1 *уменьшаются*, а значения больше 1 *увеличиваются*. Наша цель — изменить то, что я называю *чистой ошибкой* (`pred - goal_pred`), чтобы большие ошибки стали *еще больше*, а маленькие — еще меньше и незначительнее.

Оценивая ошибки таким способом, можно сделать большие ошибки важнее маленьких. Когда имеется некоторая большая чистая ошибка (например, 10), вы говорите себе, что у вас имеется *очень* большая ошибка ( $10^{**2} == 100$ ); и наоборот, в отношении маленькой чистой ошибки (например, 0.01), вы говорите себе, что она *очень* незначительная ( $0.01^{**2} == 0.0001$ ). Видите, что я имел в виду под оцениванием важности ошибок? Такой прием преувеличения больших ошибок и преуменьшения маленьких просто меняет ваше *представление о том, что можно считать ошибкой*.

Напротив, если вместо квадрата ошибки взять ее *абсолютное значение*, вам будет сложнее оценить их важность. Ошибка просто станет положительной версией чистой ошибки — тоже неплохая оценка, но другая. Но давайте пока остановимся и вернемся к этой теме позже.

## Почему ошибка должна быть только положительной?

Рано или поздно вам придется работать с миллионами пар `input -> goal_prediction` и перед вами будет стоять все та же цель — обеспечить достоверность прогноза. Вы попытаетесь уменьшить *среднюю ошибку* до 0.

Если ошибка сможет принимать положительные и отрицательные значения, это может породить проблему. Представьте, что вы пытаетесь заставить нейронную сеть выдавать достоверный прогноз по двум точкам данных — двум парам `input -> goal_prediction`. Если для первой пары ошибка составит 1000, а для второй — 1000, тогда *средняя ошибка* получится равной *нулю*! Фактически

вы обманете сами себя, считая свой прогноз идеальным, когда на самом деле каждый раз промахивались на целую 1000! Поэтому ошибки для *каждого прогноза* всегда должны иметь *положительное значение*, чтобы они случайно не компенсировали друг друга при усреднении.

## Как выглядит простейшая форма нейронного обучения?

### Обучение методом «холодно/горячо»

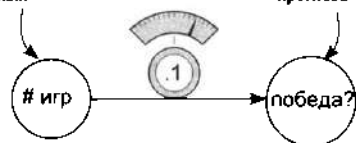
На самом деле обучение сводится к постепенному изменению `knob_weight` вверх или вниз, способствующему уменьшению ошибки. Если в какой-то момент ошибка станет равной 0, обучение можно считать законченным! Но как узнать, в какую сторону повернуть регулятор, вверх или вниз? Попробуйте *повернуть в обе стороны* и посмотрите, в каком случае ошибка уменьшилась. Определив направление изменения, обновите `knob_weight`. Это действительно простой способ, однако неэффективный. Повторите эту операцию много раз, в итоге ошибка станет равной 0, и это будет означать, что сеть идеально справляется с задачей прогнозирования.

#### ОБУЧЕНИЕ МЕТОДОМ «ХОЛОДНО/ГОРЯЧО»

Метод обучения «холодно/горячо» предполагает изменение весов в разных направлениях, чтобы определить, какое из них ведет к наибольшему уменьшению ошибки, корректировку весов в этом направлении и повторение процедуры сначала, пока ошибка не достигнет 0.

##### 1. Чистая сеть

Ввод входных данных



Вывод прогноза

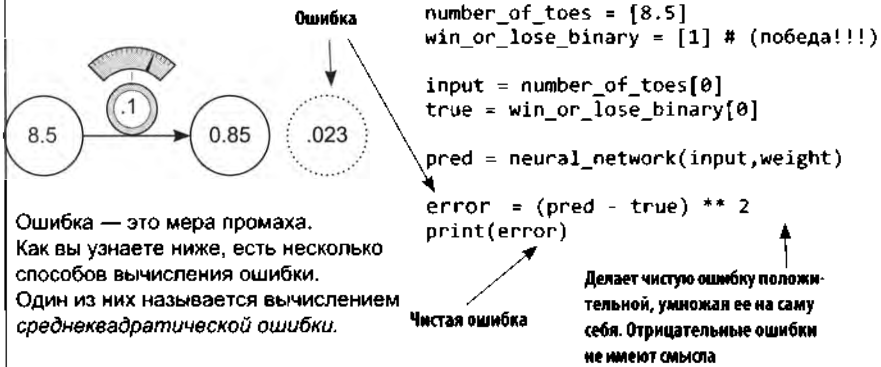
```
weight = 0.1
```

```
lr = 0.01
```

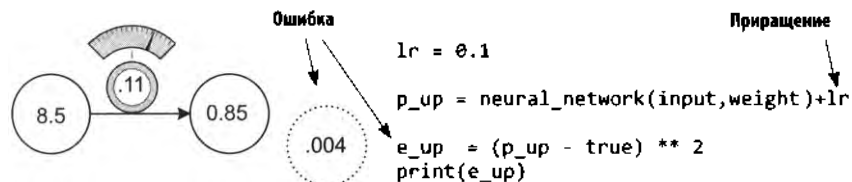
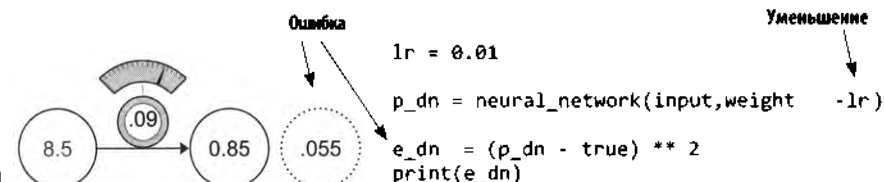
```
def neural_network(input, weight):
```

```
    prediction = input * weight
```

```
    return prediction
```

**2. ПРОГНОЗИРОВАНИЕ: получение прогноза и вычисление ошибки****3. СРАВНЕНИЕ: Получение прогноза с увеличенным значением веса и вычисление ошибки**

Чтобы уменьшить ошибку, нужно изменить вес. Попробуем сначала увеличить, а потом уменьшить его, передав в сеть  $\text{weight} + \text{lr}$  и  $\text{weight} - \text{lr}$ , и посмотрим, в каком случае получится самая низкая ошибка.

**4. СРАВНЕНИЕ: получение прогноза с уменьшенным значением веса и вычисление ошибки**

### 5. СРАВНЕНИЕ + ОБУЧЕНИЕ: сравнение ошибок и выбор нового значения веса



Эти последние пять шагов представляют одну итерацию метода обучения «холодно/горячо». Нам повезло, и эта итерация значительно приблизила нас к правильному ответу (новая ошибка составляет всего 0.004). Но чаще приходится повторять этот процесс много раз, чтобы найти правильные веса. Некоторым приходится обучать свои сети несколько недель или даже месяцев, прежде чем будет достигнуто хорошее сочетание весов.

Этот пример наглядно показывает, что обучение нейронных сетей в действительности является *задачей поиска*. Обучение сводится к поиску такой комбинации весов, при которой ошибка сети упадет до 0 (и будет обеспечена достоверность предсказания). Как и во всех других формах поиска, есть риск не найти искомое, и даже если то, что вы ищете, действительно существует, поиск может занять много времени. Далее мы используем метод обучения «холодно/горячо» для более сложного предсказания, чтобы вы могли увидеть процесс поиска в действии!

## Обучение методом «холодно/горячо»

Пожалуй, самая простая форма обучения

Выполните следующий код в Jupyter Notebook. (Изменения в реализации сети выделены **жирным**.) Этот код пытается предсказать правильный результат 0.8:

```

weight = 0.5
input = 0.5
goal_prediction = 0.8
step_amount = 0.001

for iteration in range(1101):
  
```

Шаг изменения веса в каждой итерации

Повторить обучение много раз, чтобы получить наименьшую ошибку

```

prediction = input * weight
error = (prediction - goal_prediction) ** 2

print("Error:" + str(error) + " Prediction:" + str(prediction))

up_prediction = input * (weight + step_amount)  ← Попробовать увеличить!
up_error = (goal_prediction - up_prediction) ** 2

down_prediction = input * (weight - step_amount)  ← Попробовать уменьшить!
down_error = (goal_prediction - down_prediction) ** 2

if(down_error < up_error):
    weight = weight - step_amount  ← Если уменьшение дало лучший
                                   результат, уменьшить!

if(down_error > up_error):
    weight = weight + step_amount  ← Если увеличение дало лучший результат,
                                   увеличить!

```

Запустив этот код, я получил следующие результаты:

```

Error:0.3025 Prediction:0.25
Error:0.30195025 Prediction:0.2505
....
Error:2.500000000033e-07 Prediction:0.7995
Error:1.07995057925e-27 Prediction:0.8  ← На последнем шаге получен точный прогноз 0.8!

```

## Особенности обучения методом «холодно/горячо»

### Он прост

Метод обучения «холодно/горячо» очень прост. После получения прогноза вычисляются еще два прогноза, в одном случае с немного увеличенным весом, а в другом — с немного уменьшенным. Затем производится изменение веса в том направлении, которое дало наименьшую ошибку. Многократное повторение этой процедуры в итоге уменьшило ошибку до 0.

#### ПОЧЕМУ Я ВЫБРАЛ ЧИСЛО ИТЕРАЦИЙ, РАВНОЕ 1101?

Нейронная сеть в этом примере достигает прогноза 0.8 точно через это число итераций. Если продолжить выполнять итерации, результат прогнозирования будет колебаться около 0.8 — то чуть выше, то чуть ниже, и вывод этого сценария не будет выглядеть столь вдохновляющим, как выше. Если хотите, попробуйте и убедитесь в этом сами.

## Проблема 1: он неэффективен

Этот метод требует вычислить прогноз *несколько раз*, чтобы один раз изменить `knob_weight`. Это решение выглядит очень неэффективным.

## Проблема 2: иногда невозможно добиться идеальной точности прогнозирования

При выбранном значении `step_amount`, если только идеальное значение веса не равно  $n \cdot \text{step\_amount}$ , сеть достигнет значения, отстоящего от точного прогноза на некоторую величину меньше `step_amount`, и начнет колебаться вокруг `goal_prediction` вверх и вниз. Присвойте переменной `step_amount` значение 0.2, чтобы убедиться в этом. Если для `step_amount` выбрать значение 10, вы фактически нарушите работу сети. Попробовав сделать так, вы увидите, что алгоритм даже не пытается приблизиться к 0.8!

```
Error:0.3025 Prediction:0.25
Error:19.8025 Prediction:5.25
Error:0.3025 Prediction:0.25
Error:19.8025 Prediction:5.25
Error:0.3025 Prediction:0.25
....
....и так до бесконечности...
```

Проблема в том, что, зная правильное *направление* для изменения веса, мы не знаем правильной *величины* этого изменения. Вместо этого мы произвольно выбираем некоторое фиксированное значение (`step_amount`). Кроме того, эта величина никак не связана с ошибкой. И для большой, и для маленькой ошибки величина `step_amount` остается неизменной. Метод обучения «холодно/горячо» далеко не лучший. Он неэффективен, потому что для каждого изменения веса приходится трижды вычислять прогноз, и `step_amount` выбирается произвольно, что может помешать узнать правильное значение веса.

А что, если бы у нас была возможность определить не только направление, но и величину изменения для каждого веса, не вычисляя при этом дополнительных прогнозов?

## Вычисление направления и величины из ошибки

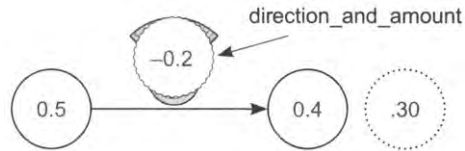
Измерим ошибку и определим направление и величину изменения!

Выполните следующий код в Jupyter Notebook:

```
weight = 0.5
goal_pred = 0.8
input = 0.5
```

```
for iteration in range(20):
    pred = input * weight
    error = (pred - goal_pred) ** 2
    direction_and_amount = (pred - goal_pred) * input
    weight = weight - direction_and_amount

    print("Error:" + str(error) + " Prediction:" + str(pred))
```



1 Чистая ошибка

2 Масштабирование, обращение знака и остановка

Перед вами более эффективный метод обучения, известный как *градиентный спуск*. Этот метод позволяет в одной строке кода (выделена **жирным**) вычислить сразу и *направление*, и *величину* изменения веса `weight` для уменьшения ошибки `error`.

### ЧТО ХРАНИТ ПЕРЕМЕННАЯ DIRECTION\_AND\_AMOUNT?

Переменная `direction_and_amount` представляет, как должен измениться вес. Чтобы получить ее значение, код сначала ❶ вычисляет *чистую ошибку*  $(pred - goal\_pred)$ . (Подробнее об этом чуть ниже.) А затем ❷ умножает ее на `input`, чтобы выполнить масштабирование, обращение знака и остановку, превращая чистую ошибку в значение, пригодное для изменения `weight`.

### ЧТО ТАКОЕ ЧИСТАЯ ОШИБКА?

Чистая ошибка — это разность  $(pred - goal\_pred)$ , определяющая направление и величину промаха. Если это *положительное* число, значит, прогноз слишком *велик*, и наоборот. Если это *большое* число, значит, вы *сильно* промахнулись, и так далее.



### ЧТО ТАКОЕ МАСШТАБИРОВАНИЕ, ОБРАЩЕНИЕ ЗНАКА И ОСТАНОВКА?

Эти три характеристики описывают общий эффект преобразования чистой ошибки в абсолютную величину изменения веса. Это необходимо для обхода трех основных крайних случаев, когда чистой ошибки недостаточно для выбора хорошей величины изменения веса.

#### ЧТО ТАКОЕ ОСТАНОВКА

Остановка — это первый (и самый простой) эффект, обусловленный умножением чистой ошибки на `input`. Представьте, что вы подключили проигрыватель компакт-дисков к своей стереосистеме. Если теперь включить полную громкость, но не включить проигрыватель, изменение громкости ничего не даст. Остановка — это похожий эффект в нейронных сетях. Если `input` получит значение 0, тогда `direction_and_amount` также получит значение 0. Обучения (изменения громкости) не будет происходить, когда переменная `input` будет равна 0, потому что нечему будет учиться. Каждое значение `weight` будет давать одну и ту же ошибку `error`, и попытка изменить ее не будет давать результата, потому что `pred` всегда будет равна 0.

#### ЧТО ТАКОЕ ОБРАЩЕНИЕ ЗНАКА?

Это самый сложный и важный, как мне кажется, эффект. Обычно (когда `input` имеет положительное значение) смещение веса вверх влечет смещение прогноза тоже вверх. Но если `input` получит отрицательное значение, вес начнет изменяться в другом направлении! При отрицательном значении `input` смещение веса *вверх* заставит прогноз сместиться *вниз*. Это и есть обращение знака! Как этого добиться? Все просто, умножение чистой ошибки на `input` *меняет знак* `direction_and_amount`, если `input` имеет отрицательное значение. Такое *обращение знака* гарантирует изменение веса в правильном направлении, даже когда `input` имеет отрицательное значение.

#### ЧТО ТАКОЕ МАСШТАБИРОВАНИЕ?

Масштабирование — это третий эффект, вызываемый умножением чистой ошибки на `input`. По логике, если `input` имеет большое значение, значит, и вес нужно изменить на большую величину. Это в большей степени побочный эффект, потому что часто выходит из-под контроля. Позднее мы познакомимся с *альфа-коэффициентом*, который будем использовать, когда такое случится.

Запустив предыдущий код, вы увидите следующий вывод:

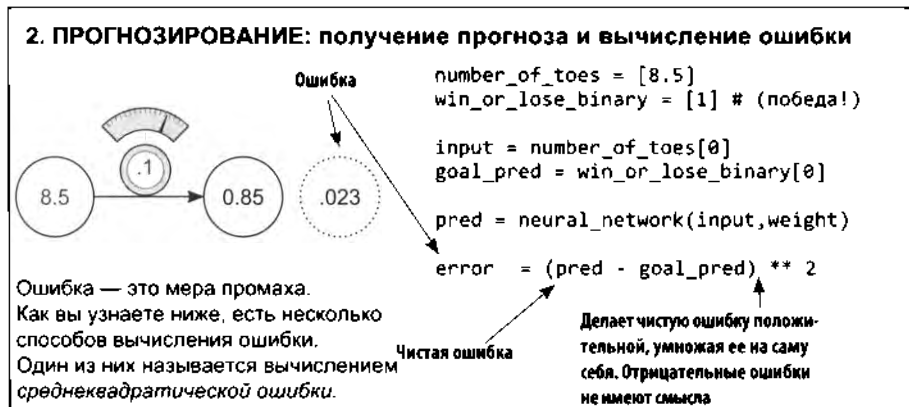
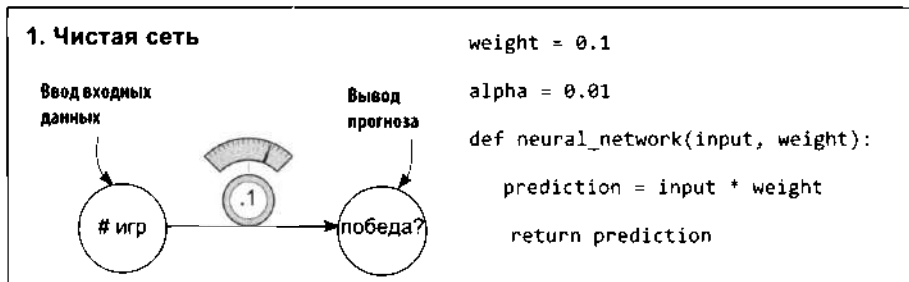
```
Error:0.3025 Prediction:0.25
Error:0.17015625 Prediction:0.3875
Error:0.095712890625 Prediction:0.490625
...
Error:1.7092608064e-05 Prediction:0.79586567925
Error:9.61459203602e-06 Prediction:0.796899259437
Error:5.40820802026e-06 Prediction:0.797674444578
```

После последнего шага мы  
близко подошли к точному  
прогнозу 0.8!

В этом примере вы увидели, как работает градиентный спуск, хотя и в упрощенной среде. Далее вы увидите его в более естественном окружении. Терминология немного изменится, но я постараюсь использовать ее применительно к другим типам сетей (например, с несколькими входами и выходами).

## Одна итерация градиентного спуска

Изменяет вес на примере одной обучающей пары  
(вход->истина)



### 3. СРАВНЕНИЕ: вычисление разности на выходе между прогнозом и истиной



```
number_of_toes = [8.5]
win_or_lose_binary = [1] # (победа!)

input = number_of_toes[0]
goal_pred = win_or_lose_binary[0]

pred = neural_network(input, weight)

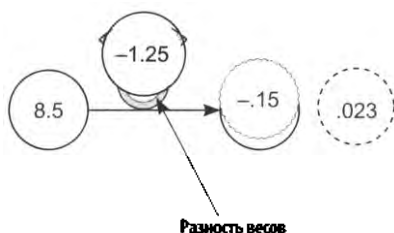
error = (pred - goal_pred) ** 2

delta = pred - goal_pred
```

Здесь в `delta` записывается величина промаха. Истинный прогноз равен 1.0, а сеть вернула прогноз 0.85, то есть прогноз сети оказался на 0.15 меньше истины. Соответственно разность `delta` равна *минус* 0.15.

Основное отличие этой реализации от градиентного спуска заключается в новой переменной `delta`. Она определяет чистую разность между прогнозом и истинным значением. Вместо непосредственного вычисления `direction_and_amount` мы сначала находим величину, на которую отличается прогноз от истины, и только потом вычисляем `direction_and_amount` для изменения веса (в шаге 4, но теперь переменная `weight` переименована в `weight_delta`):

### 4. ОБУЧЕНИЕ: вычисление разности весов



```
number_of_toes = [8.5]
win_or_lose_binary = [1] # (победа!)

input = number_of_toes[0]
goal_pred = win_or_lose_binary[0]

pred = neural_network(input, weight)

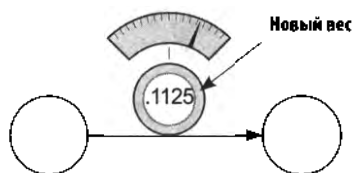
error = (pred - goal_pred) ** 2

delta = pred - goal_pred

weight_delta = input * delta
```

`weight_delta` определяет величину изменения веса, обусловленную промахом сети. Она вычисляется как произведение разности на выходе и взвешиваемого входа `input`. То есть каждое значение для `weight_delta` получается *масштабированием* разности на выходе взвешиваемым входом. Это обеспечит учет упоминавшихся выше трех свойств `direction_and_amount`: масштабирования, обращения знака и остановки.

## 5. ОБУЧЕНИЕ: корректировка веса



Фиксируется перед обучением



```
number_of_toes = [8.5]
win_or_lose_binary = [1] # (победа!)

input = number_of_toes[0]
goal_pred = win_or_lose_binary[0]
pred = neural_network(input, weight)

error = (pred - goal_pred) ** 2
delta = pred - goal_pred
weight_delta = input * delta

alpha = 0.01
weight -= weight_delta * alpha
```

Перед фактической корректировкой веса `weight` переменная `weight_delta` умножается на небольшой коэффициент `alpha`. Этот коэффициент помогает управлять скоростью обучения сети. Слишком быстрое обучение влечет слишком агрессивную корректировку весовых коэффициентов и приводит к большим промахам. (Подробнее об этом мы поговорим ниже.) Обратите внимание, что здесь корректировка веса производится так же, как в методе «холодно/горячо» — небольшими приращениями.

## Обучение просто уменьшает ошибку

### Уменьшить ошибку можно изменением веса

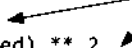
Собрав вместе код с предыдущих страниц, получаем следующее:

```
weight, goal_pred, input = (0.0, 0.8, 0.5)
```

```
for iteration in range(4):
```

```
    pred = input * weight
    error = (pred - goal_pred) ** 2
    delta = pred - goal_pred
    weight_delta = delta * input
    weight = weight - weight_delta
    print("Error:" + str(error) + " Prediction:" + str(pred))
```

В этих строках кроется секрет



**ЗОЛОТОЕ ПРАВИЛО ОБУЧЕНИЯ**

Описанный подход помогает корректировать каждый вес в правильном направлении и на правильную величину, чтобы в итоге уменьшить ошибку до 0.

Суть обучения заключается в определении правильного направления и величины изменения веса для уменьшения ошибки. Секрет заключается в вычислении прогноза `pred` и ошибки `error`. Обратите внимание, что прогноз `pred` используется в вычислении ошибки. Теперь заменим переменную `pred` кодом, который ее вычисляет:

```
error = ((input * weight) - goal_pred) ** 2
```

Значение `error` от этого никак не изменится! Здесь мы просто объединили две строки кода в одну и вычислили ошибку непосредственно. Напомню, что `input` и `goal_prediction` имеют фиксированные значения, 0.5 и 0.8 соответственно (они определяются до начала обучения сети). То есть если заменить имена переменных в выражении их значениями, секрет станет очевиден:

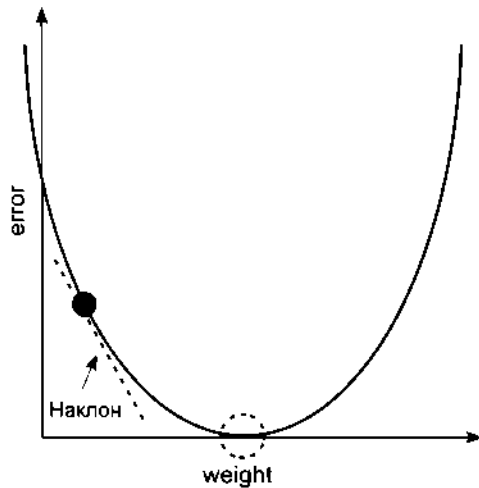
```
error = ((0.5 * weight) - 0.8) ** 2
```

**СЕКРЕТ**

Для любых `input` и `goal_pred` *точное отношение* между `error` и `weight` определяется комбинацией формул прогнозирования и вычисления ошибки. В данном случае:

```
error = ((0.5 * weight) - 0.8) ** 2
```

Допустим, вы изменили `weight` на 0.5. Зная точное отношение между `error` и `weight`, вы без труда сможете рассчитать, на какую величину изменится `error`. А что если нужно сместить ошибку `error` в конкретном направлении? Это можно рассчитать?



На этом рисунке изображен график изменения ошибки с изменением веса, соответствующий отношению в предыдущей формуле. Обратите внимание на идеальную параболическую форму кривой. Черная точка на графике соответствует текущим значениям *weight* и *error*. Точка, изображенная пунктирной окружностью, — это наша конечная цель ( $\text{error} == 0$ ).

#### ВАЖНОЕ ЗАМЕЧАНИЕ

Наклон касательной указывает направление к самой *нижней* точке графика (соответствующей наименьшему значению *error*), независимо от того, с какой стороны графика проведена эта касательная. Направление наклона можно использовать, чтобы помочь нейронной сети уменьшить ошибку.

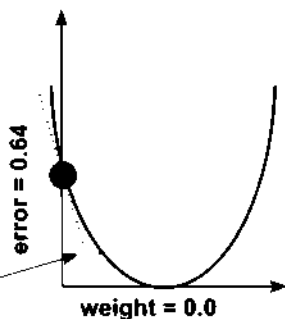
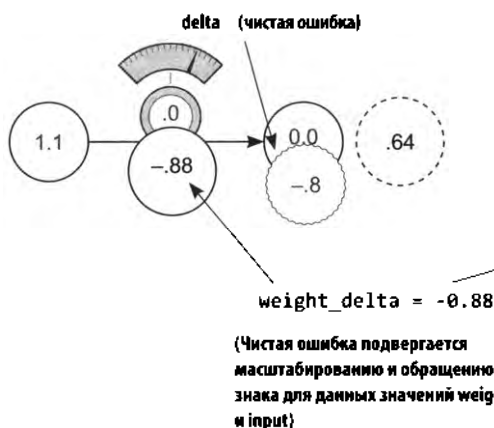
## Рассмотрим несколько циклов обучения

Удастся ли найти нижнюю точку на графике?

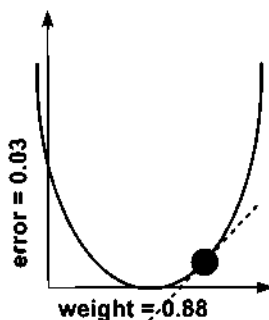
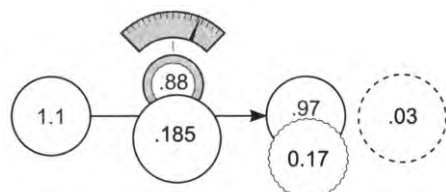
```
weight, goal_pred, input = (0.0, 0.8, 1.1)
```

```
for iteration in range(4):
    print("-----\nWeight:" + str(weight))
    pred = input * weight
    error = (pred - goal_pred) ** 2
    delta = pred - goal_pred
    weight_delta = delta * input
    weight = weight - weight_delta
    print("Error:" + str(error) + " Prediction:" + str(pred))
    print("Delta:" + str(delta) + " Weight Delta:" + str(weight_delta))
```

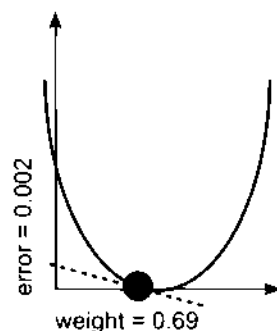
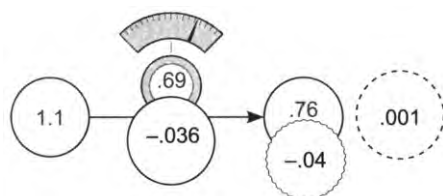
### 1. Увеличение веса на большую величину



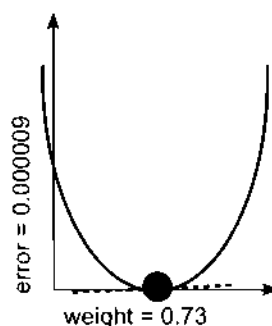
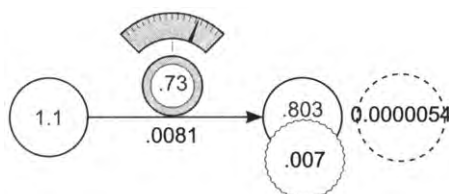
### 2. Перелет; сделаем шаг в обратную сторону



## 3. Снова перелет! Сделаем шаг обратно, но уже меньше



## 4. Отлично, мы почти попали в цель



## 5. Вывод кода

```

-----
Weight:0.0
Error:0.64 Prediction:0.0
Delta:-0.8 Weight Delta:-0.88
-----
Weight:0.88
Error:0.028224 Prediction:0.968
Delta:0.168 Weight Delta:0.1848
-----
Weight:0.6952
Error:0.0012446784 Prediction:0.76472
Delta:-0.03528 Weight Delta:-0.038808
-----
Weight:0.734008
Error:5.489031744e-05 Prediction:0.8074088
Delta:0.0074088 Weight Delta:0.00814968

```



## Как это работает? Что такое `weight_delta` на самом деле?

Вернемся и поговорим о функциях. Что такое функции? Как они работают?

Рассмотрим простую функцию:

```
def my_function(x):
    return x * 2
```

Функция принимает некоторые числа на входе и возвращает результат — еще одно число. Функция определяет зависимость между входным числом (или числами) и результатом. Возможность изучения зависимости играет важную роль: она позволяет взять некоторые числа (например, пиксели) и преобразовать их в другие числа (например, вероятность присутствия кошки на изображении).

Каждая функция имеет то, что можно назвать *движущимися частями*: части, которые можно менять или настраивать, чтобы получить от функции другой результат. Рассмотрим функцию `my_function` из предыдущего примера. Ответьте на вопрос: что определяет связь между входом и выходом этой функции? Правильный ответ: число 2. Ответьте на тот же вопрос в отношении следующей функции:

```
error = ((input * weight) - goal_pred) ** 2
```

Что определяет связь между входом (`input`) и выходом (`error`)? Здесь таких параметров больше — эта функция немного сложнее! В вычислении ошибки участвуют `goal_pred`, `input`, `**2`, `weight` и все круглые скобки и математические операции (сложение, вычитание и так далее). *Изменение* любого из этих элементов приведет к *изменению* результата. Важно учитывать это.

В качестве упражнения подумайте, как, изменяя `goal_pred`, можно уменьшить ошибку. Это бессмысленно, но возможно. В реальной жизни это называется «сдаться» (скорректировать цели в зависимости от своих возможностей). Вы отрицаете свой промах! Но мы не пойдем этим путем.

А если изменять `input`, пока `error` не достигнет 0? Это все равно что видеть мир таким, каким вы хотите его видеть, а не таким, какой он есть на самом деле. В этом случае вы продолжаете изменять входные данные, пока не получите желаемый прогноз (это объясняет в общих чертах, как работает *инцепционизм*).

Теперь рассмотрим возможность изменения числа 2 или операций сложения, вычитания или умножения. Прежде всего это поменяет способ вычисления ошибки. Вычислять ошибку имеет смысл, только если это позволит точно оценить величину промаха (с учетом свойств, упомянутых несколькими страницами выше). Но при изменении любого из перечисленных аспектов о точности говорить уже не приходится.

И что у нас остается? Единственная переменная `weight`. Ее изменение не меняет вашего восприятия мира, не влияет на цель и не ухудшает точность вычисления ошибки. Изменяя вес `weight`, функция *подстраивается под закономерности в данных*. Гарантируя неизменность остальной части функции, вы обеспечиваете правильное моделирование некоторой закономерности в данных. То есть допускается изменять только то, что влияет на *прогноз* сети.

Итак: вы изменяете определенные части функции вычисления ошибки, пока ее значение не достигнет 0. Вычисления в функции ошибки производятся с участием комбинации переменных, одну часть из которых можно менять (весовые коэффициенты), а другую — нет (исходные данные, выходные данные и логика вычисления ошибки):

```
weight = 0.5
goal_pred = 0.8
input = 0.5

for iteration in range(20):
    pred = input * weight
    error = (pred - goal_pred) ** 2
    direction_and_amount = (pred - goal_pred) * input
    weight = weight - direction_and_amount

    print("Error:" + str(error) + " Prediction:" + str(pred))
```

### ВАЖНОЕ ЗАМЕЧАНИЕ

При вычислении прогноза `pred` можно изменить все, кроме исходных данных `input`.

Мы потратим оставшуюся часть книги (а многие исследователи глубокого обучения — всю оставшуюся жизнь), пробуя использовать для вычисления `pred` все, что только можно представить, и обеспечить достоверность прогнозов. Суть обучения сводится к автоматическому изменению функции прогнозирования, так чтобы в итоге она научилась давать точные прогнозы, то есть чтобы величина ошибки стремилась к нулю.

А теперь, зная, что разрешено изменять, как воспользоваться этим знанием? Это ценное знание. И на нем основывается машинное обучение, верно? В следующем разделе мы поговорим именно об этом.

## Узкий взгляд на одно понятие

**Понятие: обучение** — это корректировка веса для уменьшения ошибки до 0

Выше в этой главе мы уже говорили об идее, что обучение фактически сводится к корректировке веса для уменьшения ошибки до 0. Это секретный соус нашего блюда. По правде говоря, знание, как это сделать, заключается в понимании *связи* между весом и ошибкой. Поняв эту связь, вы узнаете, как скорректировать вес, чтобы уменьшить ошибку.

Что я имею в виду под «пониманием связи»? Ничего особенно сложного: понимание связи между двумя переменными означает понимание, *как изменение одной переменной влияет на изменение другой*. В этом смысле особый интерес для нас представляет *чувствительность* к изменениям. Чувствительность — это еще одно название направления и величины изменения. Нам нужно знать, насколько чувствительна ошибка к изменению веса. Нам важно знать — насколько и в каком направлении изменится ошибка при некотором изменении веса. Это — наша главная цель. К настоящему моменту вы уже видели два разных метода, помогающих понять эту связь.

Меняя вес взад-вперед (в методе «горячо/холодно») и исследуя его влияние на ошибку, мы экспериментальным способом пытались понять связь между этими двумя переменными. Это как войти в комнату с 15 разными неподписанными выключателями. Вы начинаете включать и выключать их, чтобы понять, как они влияют на освещение в комнате. То же самое мы делали, изучая связь между весом и ошибкой: мы меняли вес взад-вперед и наблюдали за изменением ошибки. Затем, поняв зависимость, мы смогли изменить вес в правильном направлении, используя для этого две условных инструкции `if`:

```
if(down_error < up_error):
    weight = weight - step_amount

if(down_error > up_error):
    weight = weight + step_amount
```

Теперь вернемся к формуле, объединяющей логику вычисления `pred` и `error`. Как уже отмечалось, она точно определяет связь между ошибкой `error` и весом `weight`:

```
error = ((input * weight) - goal_pred) ** 2
```

Леди и джентльмены, эта строка — секрет. Это формула успеха. Это описание связи между ошибкой и весом. Это точная связь. Она вычисляемая. Она универсальная. Так есть и так будет всегда.

Но зная формулу, как использовать ее, чтобы узнать, как изменить вес, чтобы сместить ошибку в нужном нам направлении? *Это* хороший вопрос. Но не спешите. Я прошу вас. Остановитесь и оцените этот момент. Эта формула точно описывает связь между двумя переменными, и теперь мы собираемся выяснить, как изменить одну переменную, чтобы сместить другую в нужном направлении.

Как оказывается, есть способ сделать это для любой формулы, и мы будем использовать его для уменьшения ошибки.

## Коробка со стержнями

Представьте, что перед вами на столе картонная коробка, из которой торчат два стержня. Синий стержень, торчащий на 2 дюйма, и красный, торчащий на 4 дюйма. Допустим, вам сказали, что эти стержни связаны между собой, но не сказали, как именно. Вы должны выяснить эту связь экспериментальным путем.

Итак, вы беретесь за синий стержень, вдвигаете его в коробку на 1 дюйм и видите, что красный стержень выдвинулся на 2 дюйма. Затем вы вытягиваете синий стержень на 1 дюйм и видите, что красный стержень вытянулся на 2 дюйма. Что вы узнали в ходе этого эксперимента? Прежде всего стержни действительно *связаны* между собой. Насколько бы вы ни вдвинули или выдвинули синий стержень, красный стержень будет вдвигаться и выдвигаться на в два раза большую величину. Вы могли бы выразить это отношение так:

$$\text{red\_length} = \text{blue\_length} * 2$$

Как оказывается, есть формальное определение для ответа на вопрос: «Насколько сместится та часть, когда я передвину эту?» — *производная*. Фактически производная определяет, «насколько сместится стержень X при смещении стержня Y».

В примере с синим и красным стержнями производная, отвечающая на вопрос «насколько сместится стержень X при смещении стержня Y?», равна 2. Просто 2. Почему 2? Потому что это отношение определяется формулой:

$$\text{red\_length} = \text{blue\_length} * 2 \quad \leftarrow \text{Производная}$$

Обратите внимание, что *между двумя переменными* всегда находится производная. Вам всегда важно знать, как изменится одна переменная при изменении другой. Если производная положительная, тогда при изменении одной переменной другая будет изменяться *в том же* направлении. Если производная *отрицательная*, тогда при изменении одной переменной другая будет изменяться *в противоположном* направлении.

Рассмотрим несколько примеров. Поскольку производная, связывающая `red_length` с `blue_length`, равна 2, оба числа будут изменяться в одном направлении. Если говорить точнее, красный стержень будет двигаться на в два раза большую величину и в одном направлении с синим. Если бы производная была равна  $-1$ , тогда красный стержень двигался бы в противоположном направлении на одну и ту же величину, что и синий. То есть производная представляет не только величину, но и направление изменения одной переменной при изменении другой. Это именно то, что нам нужно.

## Производные: второй пример

Все еще сомневаетесь? Тогда посмотрим с другой стороны

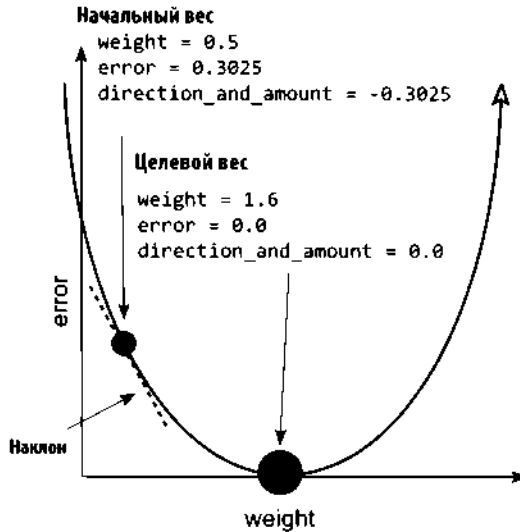
От разных людей я слышал два объяснения производной. Одни говорят, что производная определяет изменение одной переменной в зависимости от изменения другой. Другие говорят, что производная — это наклон прямой или кривой в данной точке. Как оказывается, если нарисовать график функции, наклон линии графика в данной точке будет *в точности* отражать, «насколько одна переменная изменится при изменении другой». Позвольте мне продемонстрировать это на графике нашей функции:

```
error = ((input * weight) - goal_pred) ** 2
```

Как вы помните, `goal_pred` и `input` имеют фиксированные значения, поэтому эту функцию можно переписать так:

```
error = ((0.5 * weight) - 0.8) ** 2
```

Поскольку для изменения нам доступны только две переменные (остальные члены формулы фиксированы), мы можем попробовать разные значения `weight`, вычислить ошибку `error`, которую они дают, и нарисовать график.



Как видите, график имеет U-образную форму. Обратите внимание, что в середине графика имеется точка, где `error == 0`. Также отметьте, что справа от этой точки кривая имеет положительный наклон, а слева — отрицательный. Но самое интересное, что чем дальше от *целевого веса*, тем круче наклон.

Это очень полезные свойства. Знак наклона дает нам направление, а крутизна — величину изменения. Мы можем воспользоваться этими свойствами, чтобы найти целевой вес.

Даже сейчас, глядя на эту кривую, я легко забываю, что она представляет. Это похоже на метод обучения «холодно/горячо». Если попробовать разные значения веса и построить график, вы получите эту кривую.

Что особенно примечательно в производных, так это то, что они способны заглядывать за большую формулу вычисления ошибки (в начале этого раздела) и видеть эту кривую. Вы можете вычислить наклон (производную) линии для любого значения веса и использовать этот наклон (производную) для определения направления изменения, при котором ошибка уменьшится. Но самое замечательное, что, опираясь на крутизну, можно получить хоть какое-то представление о том, насколько далеко вы находитесь от оптимальной точки, где наклон равен нулю (хотя и не точно, но об этом чуть позже).

## Что действительно необходимо знать

Благодаря производным можно выбрать любые две переменные в любой формуле и узнать, как они взаимосвязаны

Рассмотрим следующую большую *функцию*:

$$y = (((\text{beta} * \text{gamma}) ** 2) + (\text{epsilon} + 22 - x)) ** (1/2)$$

Вот что нам нужно знать о производных. Для любой функции (даже для этой) можно выбрать любые две переменные и узнать, как они взаимосвязаны. Для любой функции можно выбрать любые две переменные и построить график в системе координат X/Y, как это было сделано выше. Для любой функции можно выбрать любые две переменные и вычислить, насколько изменится одна из них при изменении другой. То есть для любой функции можно узнать, как следует изменить одну из переменных, чтобы сместить другую в нужном направлении. Прошу прощения за все эти подробности, но знать их действительно очень важно.

Итог: в этой книге мы будем строить нейронные сети. Нейронные сети — это всего лишь наборы весовых коэффициентов, используемых для вычисления функции ошибки. И для любой функции ошибки (какой бы сложной она ни была) можно вычислить отношение между любым весом и окончательной ошибкой сети. Теперь, зная это, мы можем изменить каждый вес в нейронной сети, чтобы уменьшить ошибку до 0. Именно этим мы и займемся.

## Что знать необязательно

### Дифференциальное исчисление

На изучение всех методов взятия любых двух переменных из любой функции и вычисления их отношения в вузе отводится три семестра. По правде говоря, после изучения глубокого обучения в течение трех семестров вы использовали бы лишь малую часть полученных знаний. Изучение дифференциального исчисления фактически сводится к запоминанию и опробованию на практике всех правил вычисления производных для всех возможных функций.

В этой книге я буду делать то же самое, что делаю в обычной жизни (потому что я ленивый, читай — эффективный): искать производные в справочных таблицах. Вам действительно достаточно знать лишь какую роль играет производная. Она описывает связь между двумя переменными в функции и позволяет узнать, насколько изменится одна переменная при изменении другой. Это просто чувствительность одной переменной к изменению другой.

Я понимаю, что вывалил на вас слишком много информации, чтобы в конце сказать: «Это просто чувствительность одной переменной к изменению другой», — но это действительно так. Обратите внимание, что чувствительность может быть *положительной* (когда переменные изменяются в одном направлении), *отрицательной* (когда они изменяются в разных направлениях) и *нулевой* (когда изменение одной переменной никак не отражается на другой). Например,  $y = 0 * x$ . При любом изменении  $x$  переменная  $y$  всегда будет получать значение 0.

Но хватит о производных! Вернемся к градиентному спуску.

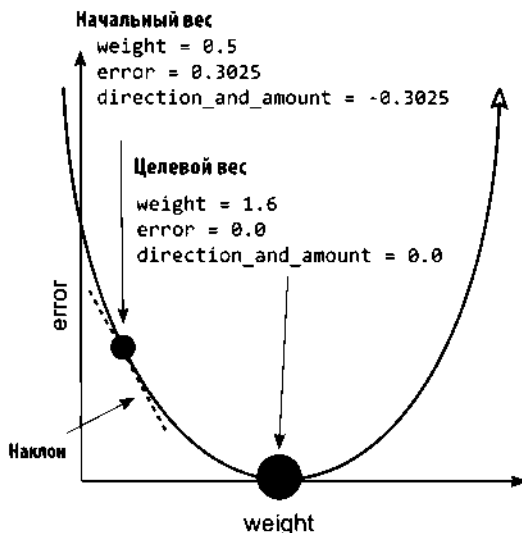
## Как использовать производные для обучения

`weight_delta` — это наша производная

В чем разница между ошибкой и производной от ошибки и веса? Ошибка определяет величину промаха. А производная определяет отношение между каждым весом и величиной промаха. Иначе говоря, производная говорит, какой вклад вносит в ошибку изменение веса. Итак, как теперь использовать это знание для смещения ошибки в нужном нам направлении?

Вы узнали, как взаимосвязаны две переменные в функции, но как воспользоваться этим знанием? Как оказывается, в этом нет ничего сложного. Взгляните еще раз на график ошибки. Черная точка отмечает начальное значение веса: (0.5). Пунктирная окружность отмечает точку, куда мы должны попасть: целевой вес. Видите прямую пунктирную линию, касательную к кривой графика в черной точке? Она определяет наклон, или производную. То есть касательная в этой точке подсказывает, насколько изменится ошибка при изменении веса. Обратите внимание, что касательная наклонена вниз: это отрицательный наклон.





Наклон кривой графика всегда указывает в направлении, противоположном направлению от самой нижней точки на кривой. То есть чтобы при отрицательном наклоне уменьшить ошибку, мы должны увеличить вес. Проверьте это.

Итак, как же использовать производную, чтобы найти минимальную ошибку (нижнюю точку на графике)? Мы должны двигаться в направлении, противоположном наклону — противоположном производной. Вы можете взять любое значение веса, вычислить производную с учетом ошибки (то есть сравнить две переменные: *weight* и *error*) и затем изменить вес в направлении, противоположном получившемуся наклону. В результате вы сделаете шаг в направлении минимума.

Вернемся к нашей цели: мы должны определить направление и величину изменения веса, которые уменьшают ошибку. Производная определяет, как связаны любые две переменные в функции. И мы используем производную, чтобы определить связь между весом и ошибкой. Затем смещаем вес в направлении, противоположном производной, и получаем уменьшенную ошибку. Вот и все! Нейронная сеть сделала один шаг в обучении.

Этот метод обучения (поиск минимума ошибки) называется *градиентным спуском*. Это название говорит само за себя. Вы перемещаете значение веса в направлении, противоположном значению градиента, и приближаете ошибку к 0. То есть вы увеличиваете вес при отрицательном градиенте, и наоборот. Очень похоже на действие гравитации.

## Выглядит знакомо?

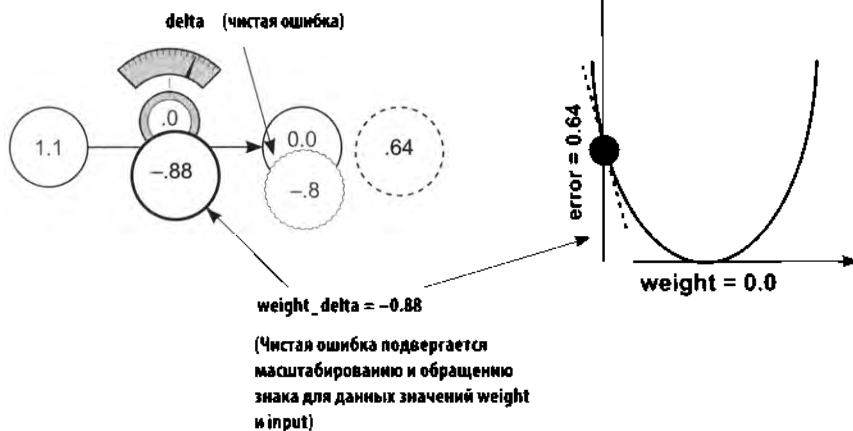
```
weight = 0.0
goal_pred = 0.8
input = 1.1
```

```
for iteration in range(4):
    pred = input * weight
    error = (pred - goal_pred) ** 2
    delta = pred - goal_pred
    weight_delta = delta * input
    weight = weight - weight_delta
```

Производная (как быстро изменяется ошибка при изменении веса)

```
print("Error:" + str(error) + " Prediction:" + str(pred))
```

### 1. Слишком большое изменение веса



### 2. Перелет; сделаем шаг в обратную сторону



## Ломаем градиентный спуск

Просто дайте мне код!

```
weight = 0.5
goal_pred = 0.8
input = 0.5

for iteration in range(20):
    pred = input * weight
    error = (pred - goal_pred) ** 2
    delta = pred - goal_pred
    weight_delta = input * delta
    weight = weight - weight_delta
    print("Error:" + str(error) + " Prediction:" + str(pred))
```

Выполнив этот код, я получил следующий вывод:

```
Error:0.3025 Prediction:0.25
Error:0.17015625 Prediction:0.3875
Error:0.095712890625 Prediction:0.490625
...
Error:1.7092608064e-05 Prediction:0.79586567925
Error:9.61459203602e-06 Prediction:0.796899259437
Error:5.40820802026e-06 Prediction:0.797674444578
```

Теперь, получив действующий код, попытаемся его сломать. Попробуйте разные начальные значения для `weight`, `goal_pred` и `input`. Вы можете присвоить им любые начальные значения, и нейронная сеть определит, как получить достоверный прогноз для данного входного значения и веса. Сможете ли вы подобрать такую комбинацию, при которой нейронная сеть не сможет дать прогноз? Я считаю, что подобные попытки что-то сломать помогают лучше понять предмет обсуждения.

Попробуем присвоить переменной `input` число 2, но сохраним целевой прогноз равным 0.8. Что из этого получится? Взгляните сами:

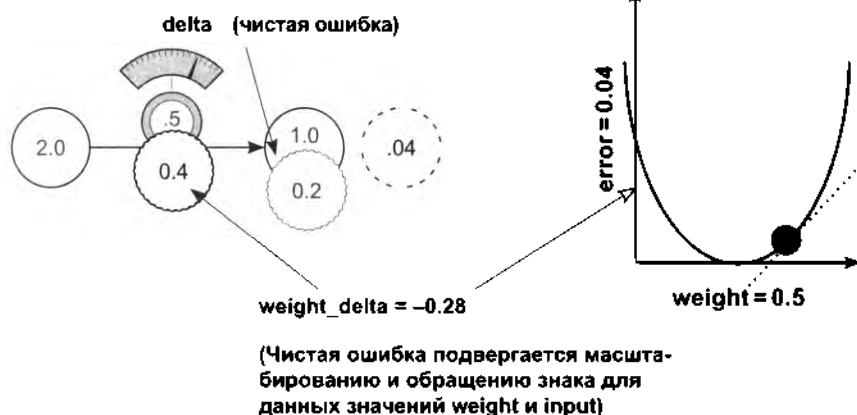
```
Error:0.04 Prediction:1.0
Error:0.36 Prediction:0.2
Error:3.24 Prediction:2.6
...
Error:6.67087267987e+14 Prediction:-25828031.8
Error:6.00378541188e+15 Prediction:77484098.6
Error:5.40340687069e+16 Prediction:-232452292.6
```

М-да! Это совсем не то, что нам хотелось. Наш алгоритм пошел вразнос! Результаты прыгают из отрицательных значений в положительные и из по-

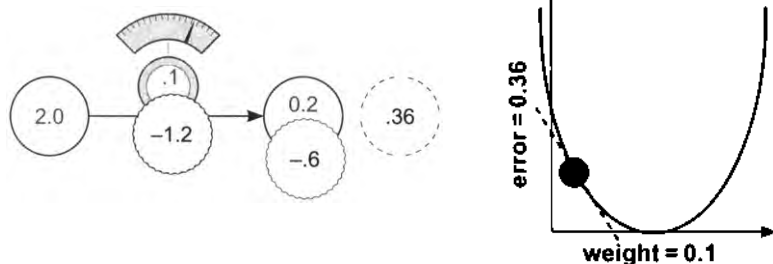
ложительных в отрицательные, все дальше уходя от истины. Иначе говоря, происходит все более избыточная коррекция веса. В следующем разделе вы узнаете, как бороться с этим феноменом.

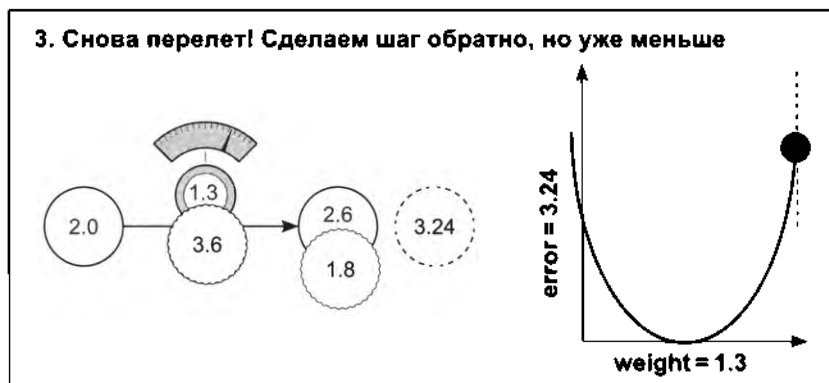
## Визуальное представление избыточной коррекции

### 1. Слишком большое изменение веса



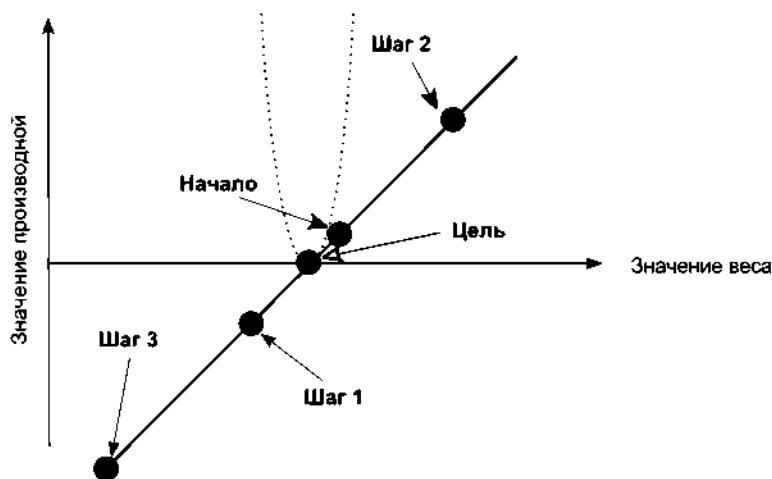
### 2. Перелет; сделаем шаг в обратную сторону





## Расхождение

Иногда нейронные сети идут вразнос. Почему?



Что же случилось на самом деле? Взрывное расхождение оценки ошибки вызвано увеличением входного значения. Посмотрим, как происходит изменение веса в этом случае:

```
weight = weight - (input * (pred - goal_pred))
```

Если входное значение достаточно велико, это может вызвать значительное увеличение веса даже при маленьком значении ошибки. Что происходит, когда вес изменяется на значительную величину при маленьком значении ошибки?

А происходит избыточная коррекция веса в сети. Если новая ошибка станет больше, коррекция окажется еще более избыточной. Это явление, которое мы наблюдали выше, называется *расхождением*.

При очень больших входных значениях прогноз становится очень чувствительным к изменениям веса (согласно формуле  $\text{pred} = \text{input} * \text{weight}$ ). Это может вызвать избыточную коррекцию сети. Иначе говоря, даже притом, что начальный вес остается равным 0.5, производная в этой точке имеет очень крутой наклон. Посмотрите, насколько близко друг к другу находятся ветви U-образной кривой на графике.

Теперь многое становится понятно. Как мы вычисляем прогноз? Умножаем входное значение на вес. То есть если входное значение достаточно велико, небольшое изменение веса вызывает значительное изменение в прогнозе. Ошибка очень чувствительна к весу. Иначе говоря, производная имеет очень большое значение. Можно ли его уменьшить?

## Знакомьтесь: альфа-коэффициент

### Это самый простой способ предотвратить избыточную коррекцию весов

В чем выражается проблема, которую мы пытаемся решить? При большом входном значении может происходить избыточная коррекция весов. Как это проявляется? Величина производной в новой точке будет превосходить величину прежней производной (хотя и с другим знаком).

А теперь остановимся и подумаем. Взгляните еще раз на график в предыдущем разделе, чтобы понять симптомы. Выполнив шаг 2, мы оказываемся от цели дальше, чем были, а это означает, что абсолютная величина производной увеличилась. В результате, выполнив шаг 3, мы оказываемся еще дальше от цели, чем даже на шаге 2, и нейронная сеть продолжает отклоняться все больше и больше, демонстрируя расхождение.

Увеличение расхождения служит явным симптомом. Решение проблемы заключается в том, чтобы умножить величину изменения веса на некоторый коэффициент, чтобы сделать ее меньше. В большинстве случаев достаточно умножить величину изменения веса на одно вещественное число в диапазоне от 0 до 1. Часто это число называют *альфа-коэффициентом*. Обратите внимание: это решение никак не влияет на саму проблему, которая обусловлена слишком большим входным значением. Оно также уменьшает величину изменения веса для не слишком больших входных значений.

Выбор соответствующего альфа-коэффициента, даже для современных нейронных сетей, часто производится наугад. Вы смотрите, как изменяется ошибка с каждой итерацией, и если начинает наблюдаться расхождение, значит, альфа-коэффициент слишком велик и его следует уменьшить. Если обучение происходит слишком медленно, значит, альфа-коэффициент слишком мал и его следует увеличить. Есть и другие методы обучения, кроме градиентного спуска, которые пытаются противостоять этому явлению, но градиентный спуск продолжает пользоваться большой популярностью.

## Альфа-коэффициент в коде

В каком месте «альфа-параметр» должен вступать в игру?

Вы только что узнали, что альфа-коэффициент уменьшает величину изменения веса и препятствует его избыточной коррекции. Но где в коде он должен использоваться? Давайте посмотрим. Итак, мы корректируем вес в соответствии со следующей формулой:

```
weight = weight - derivative
```

Чтобы уменьшить величину изменения, нужно добавить альфа-коэффициент, как показано ниже. Обратите внимание, что если альфа-коэффициент будет очень мал (например, 0.01), он существенно уменьшит величину изменения веса и предотвратит появление эффекта расхождения:

```
weight = weight - (alpha * derivative)
```

Как видите, ничего сложного. А теперь добавим альфа-коэффициент в нашу реализацию, представленную в начале главы, и опробуем ее с входным значением `input = 2` (которое прежде вызвало расхождение):

```
weight = 0.5
goal_pred = 0.8
input = 2
alpha = 0.1
```

← Что случится, если сделать альфа-коэффициент слишком большим или слишком маленьким? А если сделать его отрицательным?

```
for iteration in range(20):
    pred = input * weight
    error = (pred - goal_pred) ** 2
    derivative = input * (pred - goal_pred)
    weight = weight - (alpha * derivative)

    print("Error:" + str(error) + " Prediction:" + str(pred))
```

```
Error:0.0144 Prediction:0.92
Error:0.005184 Prediction:0.872
```

```
...
```

```
Error:1.14604719983e-09 Prediction:0.800033853319
```

Вуаля! Наша маленькая нейронная сеть теперь снова может давать хорошие прогнозы. Интересно, как я выбрал значение 0.1 для альфа-коэффициента? Если честно, я просто попробовал — и это сработало. Несмотря на сумасшедшие достижения в области глубокого обучения в последние несколько лет, многие просто пробуют использовать альфа-коэффициенты со значениями разных порядков (10, 1, 0.1, 0.01, 0.001, 0.0001) и выбирают тот, который подходит лучше всего. Выбор альфа-коэффициента — больше искусство, чем наука. Конечно, есть более продвинутые способы решения этой задачи, которые мы рассмотрим позже, а пока попробуйте разные значения для альфа-коэффициента и выберите то, которое вам покажется лучше других. Поэкспериментируйте с ним.

## Запоминание

### Пришло время начать изучение этой темы по-настоящему

Моя просьба может показаться вам чрезмерной, но я считаю это упражнение очень важным: попробуйте по памяти воспроизвести код из предыдущего раздела в Jupyter Notebook (или, если хотите, в .py файле). Я знаю, что это может выглядеть излишним, но у меня лично ничего не «щелкнуло» в голове, пока я не попробовал самостоятельно решить эту задачу с нейронной сетью.

Зачем это нужно? Прежде всего, единственный способ узнать, сумели ли вы запомнить все, о чем рассказывалось в этой главе, — это попробовать воспроизвести решение по памяти. В нейронных сетях очень много важных мелочей, которые легко упустить из виду.

Важно ли это для остальной части книги? В следующих главах я лишь мельком буду ссылаться на идеи и понятия, которые затронул в этой главе, чтобы больше времени уделить новому материалу. Поэтому очень важно, чтобы вы сразу же поняли, что я имею в виду, увидев, например, фразу: «Добавьте альфа-коэффициент к приращению веса».

Запоминание маленьких фрагментов кода нейронной сети было чрезвычайно полезным для меня лично, а также для многих из тех, кто в прошлом пользовался моими советами по этому вопросу.



# 5

## Корректировка сразу нескольких весов: обобщение градиентного спуска



### В этой главе

- ✓ Обучение методом градиентного спуска с несколькими входами.
- ✓ Замораживание одного веса: для чего?
- ✓ Обучение методом градиентного спуска с несколькими выходами.
- ✓ Обучение методом градиентного спуска с несколькими входами и выходами.
- ✓ Визуализация значений весов.
- ✓ Визуализация скалярных произведений.

Нельзя научиться ходить, просто следуя правилам.  
Мы учимся, пробуя и падая.

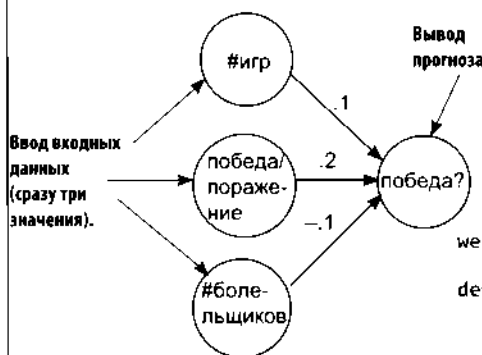
Ричард Брэнсон (Richard Branson), <http://mng.bz/oVgd>

## Обучение методом градиентного спуска с несколькими входами

Градиентный спуск можно использовать также с несколькими входами

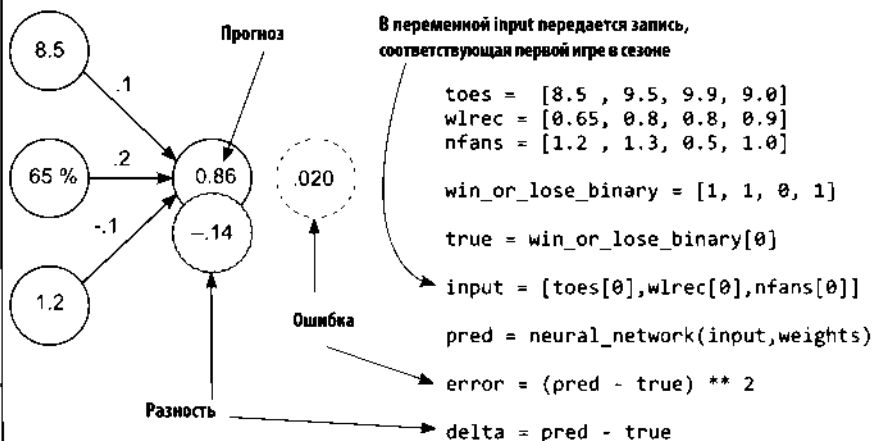
В предыдущей главе вы узнали, как корректировать вес с помощью метода градиентного спуска. В этой главе мы посмотрим, как тот же метод использовать для обучения сети с несколькими весами. Я предлагаю без долгих вступлений сразу взяться за дело, согласны? Следующая диаграмма иллюстрирует процесс обучения сети с несколькими входами.

### 1. Чистая сеть с несколькими входами

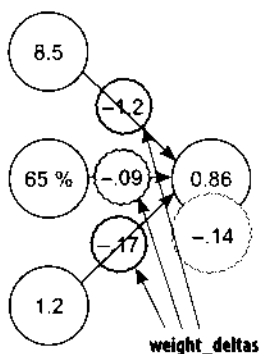


```
def w_sum(a,b):  
    assert(len(a) == len(b))  
    output = 0  
    for i in range(len(a)):  
        output += (a[i] * b[i])  
    return output  
  
weights = [0.1, 0.2, -.1]  
  
def neural_network(input, weights):  
    pred = w_sum(input,weights)  
    return pred
```

## 2. ПРОГНОЗ + СРАВНЕНИЕ: получение прогноза, вычисление ошибки и разности



## 3. ОБУЧЕНИЕ: вычисление всех приращений weight\_delta и добавление их в каждый вес



```

def ele_mul(number, vector):

    output = [0, 0, 0]

    assert(len(output) == len(vector))

    for i in range(len(vector)):
        output[i] = number * vector[i]

    return output

input = [toes[0], wlrec[0], nfans[0]]
pred = neural_network(input, weight)
error = (pred - true) ** 2
delta = pred - true
weight_deltas = ele_mul(delta, input)

```

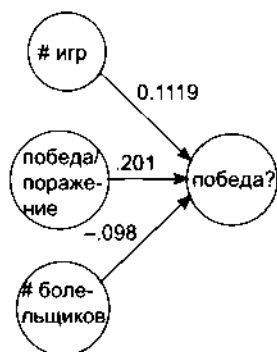
```

8.5 * -.14 = -1.19 = weight_deltas[0]
0.65 * -.14 = -0.091 = weight_deltas[1]
1.2 * -.14 = -0.168 = weight_deltas[2]

```

На этой диаграмме нет ничего нового. Каждое приращение `weight_delta` вычисляется умножением разности прогноза и истины на соответствующее входное значение. В данном случае в вычислении единственного выходного значения участвуют три веса, поэтому в вычислении приращения для каждого из них используется одна и та же разность прогноза и истины. Но из-за разных входных значений приращения весов получаются разными. Также отметьте, что здесь для умножения каждого входного значения на разность повторно используется функция `ele_mul`, которую мы написали раньше.

#### 4. ОБУЧЕНИЕ: корректировка весов



```
input = [toes[0],wlrec[0],nfans[0]]

pred = neural_network(input,weight)
error = (pred - true) ** 2
delta = pred - true

weight_deltas = ele_mul(delta,input)

alpha = 0.01

for i in range(len(weights)):
    weights[i] -= alpha * weight_deltas[i]
    print("Weights:" + str(weights))
    print("Weight Deltas:" + str(weight_deltas))
```

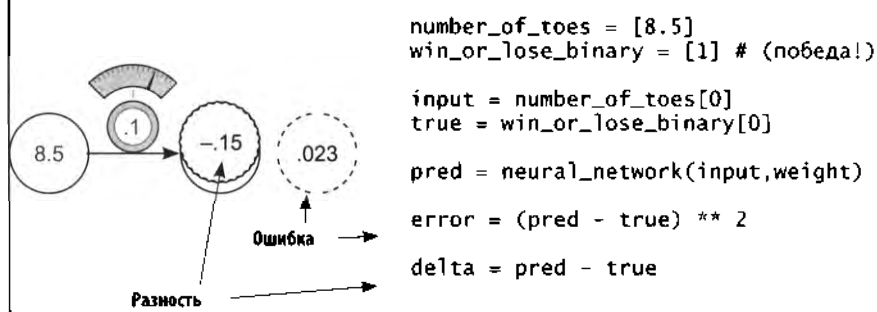
```
0.1 - (-1.19 * 0.01) = 0.1119 = weights[0]
0.2 - (-.091 * 0.01) = 0.2009 = weights[1]
-0.1 - (-.168 * 0.01) = -0.098 = weights[2]
```

## Градиентный спуск с несколькими входами, описание

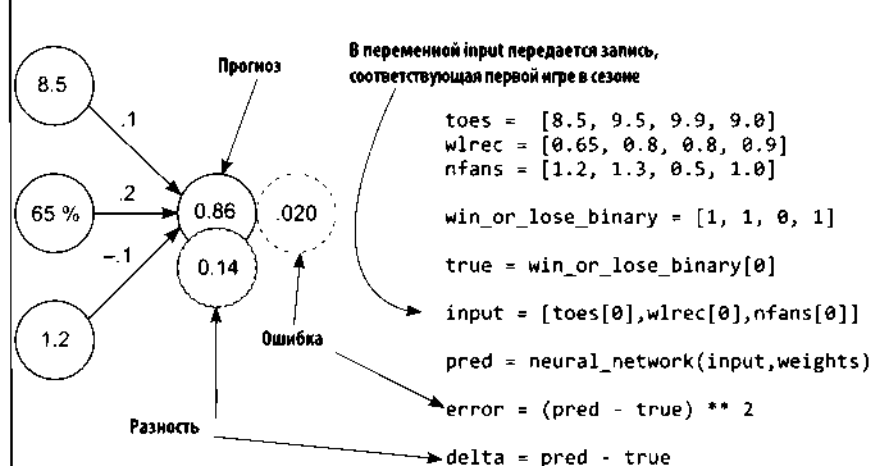
### Прост в реализации и увлекателен в изучении

При сопоставлении с нейронной сетью с единственным весом реализация градиентного спуска с несколькими входами кажется достаточно очевидной. Однако она обладает рядом интересных особенностей, на которых стоит остановиться. Для начала поместим оба подхода рядом друг с другом.

### 1. Единственный вход: получение прогноза и вычисление ошибки и разности



### 2. Несколько входов: получение прогноза и вычисление ошибки и разности



Вплоть до вычисления разности между прогнозом и истинным значением, градиентный спуск с единственным и несколькими входами выглядят совершенно идентично (кроме различий в вычислении прогноза, которые рассматривались в главе 3). В обоих случаях мы получаем прогноз и вычисляем ошибку и разность одинаковым способом. Но дальше возникает проблема: когда у нас имелся только один вес, мы имели только одно входное значение (и вычисляли одно

приращение `weight_delta`). Теперь у нас три веса. Как в этом случае получить три приращения `weight_deltas`?

### Как превратить одну разность (для одного прогноза) в три приращения `weight_delta`?

Вспомним определение и назначение разности `delta` и приращения `weight_delta`. Разность `delta` определяет величину, на которую желательно скорректировать выходное значение. В данном случае она вычисляется прямым вычитанием истинного значения из прогноза (`pred - true`). Положительное значение `delta` указывает, что прогноз имеет слишком большое значение, а отрицательное — что слишком маленькое.

#### РАЗНОСТЬ

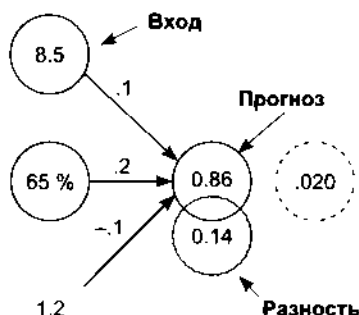
Определяет, насколько больше или меньше должно быть значение на выходе, чтобы можно было считать прогноз идеальным для данного обучающего экземпляра.

С другой стороны, приращение `weight_delta` является *оценкой* величины и направления смещения веса для уменьшения разности `delta` и определяется через производную. Как преобразовать `delta` в `weight_delta`? Умножением `delta` на вход, соответствующий весу.

#### ПРИРАЩЕНИЕ

Оценка на основе производной величины и направления смещения веса для уменьшения разности `delta`, учитывающая масштабирование, обращение знака и остановку.

Посмотрим на это с точки зрения единственного веса:



**delta:** Эй, на входе! Да, вы трое. В следующий раз прогноз должен быть чуть выше.

**Единственный вес:** Хм-м, если бы на входе был 0, тогда бы вес не имел значения, и я не смог бы ничего изменить (*остановка*). Если бы на входе было отрицательное число, тогда бы вес следовало уменьшить, а не увеличить (*обращение знака*). Но на входе положительное число и довольно большое, то есть можно *ожидать*, что оно имеет большое значение для общего результата. Я сильно увеличу свой вес для компенсации ошибки (*масштабирование*).

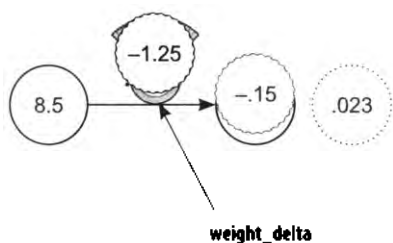
Единственный вес увеличивает свое значение.

О чем на самом деле говорят эти свойства/утверждения? Все они (*остановка*, *обращение знака* и *масштабирование*) отмечают роль веса в разности, обусловленной входным значением. То есть каждое приращение `weight_delta` является своего рода модифицированной входом версией разности `delta`.

Это возвращает нас к первоначальному вопросу: как превратить единственное значение разности `delta` в три приращения `weight_delta`? Итак, поскольку для каждого веса имеется свое входное значение и общая разность, мы умножаем вход `input`, соответствующий весу, на разность `delta` и получаем соответствующее приращение `weight_delta`. Рассмотрим этот процесс в действии.

На следующих двух рисунках можно видеть, как вычисляются переменные `weight_delta` для сети с одним входом и для новой сети с несколькими входами. Сходство особенно заметно, если взглянуть на псевдокод внизу каждого рисунка. Обратите внимание, что в версии с несколькими весами значение `delta` (0.14) умножается на каждый вход и в результате получаются разные приращения `weight_delta`. Процесс очень прост.

### 3. Единственный вход: вычисление приращения `weight_delta` и корректировка веса



```

number_of_toes = [8.5]
win_or_lose_binary = [1] # (победа!)

input = number_of_toes[0]
true = win_or_lose_binary[0]

pred = neural_network(input,weight)

error = (pred - true) ** 2

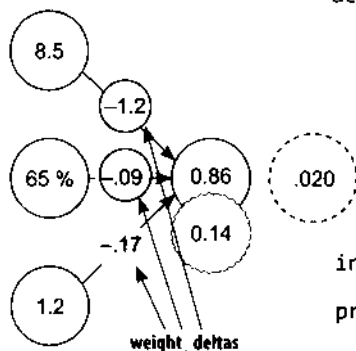
delta = pred - true

weight_delta = input * delta

8.5 * -0.15 = -1.25 => weight_delta

```

### 4. Несколько входов: вычисление приращения `weight_delta` и корректировка каждого веса



```

def ele_mul(number,vector):
    output = [0,0,0]
    assert(len(output) == len(vector))
    for i in range(len(vector)):
        output[i] = number * vector[i]
    return output

input = [toes[0],wlrec[0],nfans[0]]
pred = neural_network(input,weights)
error = (pred - true) ** 2

delta = pred - true

weight_deltas = ele_mul(delta,input)

```

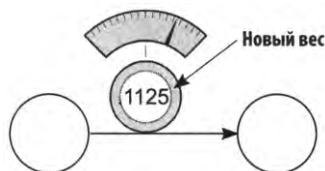
```

8.5 * 0.14 = -1.2 => weight_deltas[0]
0.65 * 0.14 = -.09 => weight_deltas[1]
1.2 * 0.14 = -.17 => weight_deltas[2]

```



## 5. Корректировка веса



Перед корректировкой веса приращение `weight_delta` умножается на небольшое число `alpha`. Это позволяет управлять скоростью обучения сети. Если обучение протекает слишком быстро, вес может корректироваться слишком агрессивно и вызвать расхождение. Обратите внимание, что корректировка веса изменена (уменьшенное приращение), так же как в методе обучения «холодно/горячо».

```
number_of_toes = [8.5]
win_or_lose_binary = [1] # (победа!)
```

```
input = number_of_toes[0]
true = win_or_lose_binary[0]

pred = neural_network(input, weight)
```

```
error = (pred - true) ** 2
```

```
delta = pred - true
```

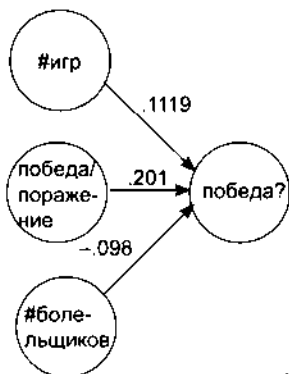
```
weight_delta = input * delta
```

```
alpha = 0.01
```

← Поправка перед обучением

```
weight -= weight_delta * alpha
```

## 6. Корректировка весов



```
input = [toes[0], wlrec[0], nfans[0]]
```

```
pred = neural_network(input, weights)
```

```
error = (pred - true) ** 2
```

```
delta = pred - true
```

```
weight_deltas = ele_mul(delta, input)
```

```
alpha = 0.01
```

```
for i in range(len(weights)):
    weights[i] -= alpha * weight_deltas[i]
```

```
0.1 - (.119 * 0.01) = 0.1119 = weights[0]
0.2 - (.091 * 0.01) = 0.2009 = weights[1]
-0.1 - (.168 * 0.01) = -0.098 = weights[2]
```

Последний шаг тоже выполняется почти в точности как в сети с единственным входом. После получения значений `weight_delta` они умножаются на альфа-коэффициент и вычитаются из текущих значений весов. Это фактически тот же процесс, только повторяется в отдельности для каждого веса.

## Рассмотрим несколько шагов обучения

```
def neural_network(input, weights):
    out = 0
    for i in range(len(input)):
        out += (input[i] * weights[i])
    return out
```

```
def ele_mul(scalar, vector):
    out = [0,0,0]
    for i in range(len(out)):
        out[i] = vector[i] * scalar
    return out
```

```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

```
win_or_lose_binary = [1, 1, 0, 1]
true = win_or_lose_binary[0]
```

```
alpha = 0.01
weights = [0.1, 0.2, -.1]
input = [toes[0],wlrec[0],nfans[0]]
```

(продолжение)

```
for iter in range(3):
```

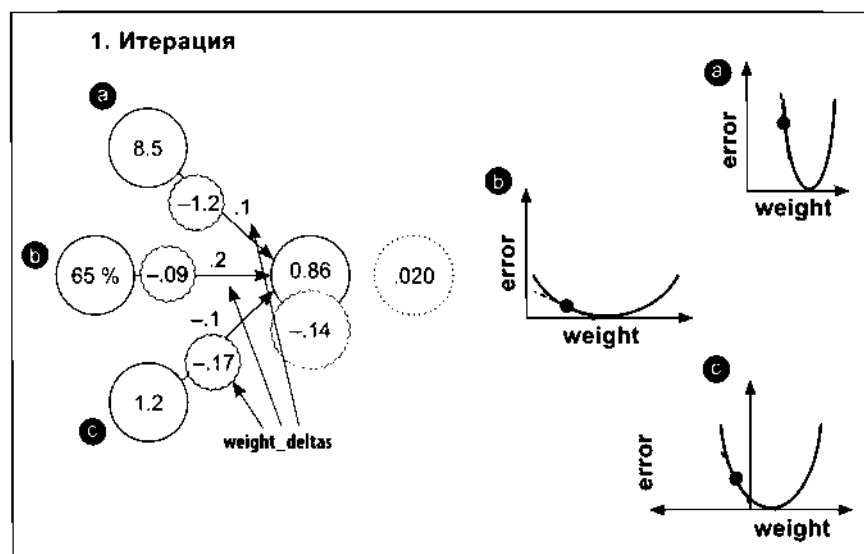
```
    pred = neural_network(input,weights)
```

```
    error = (pred - true) ** 2
    delta = pred - true
```

```
    weight_deltas=ele_mul(delta,input)
```

```
    print("Iteration:" + str(iter+1))
    print("Pred:" + str(pred))
    print("Error:" + str(error))
    print("Delta:" + str(delta))
    print("Weights:" + str(weights))
    print("Weight_Deltas:")
    print(str(weight_deltas))
    print(
    )
```

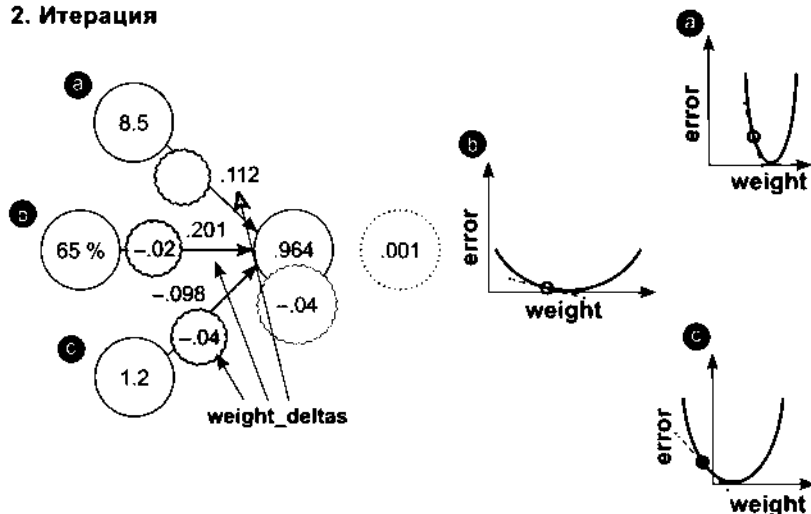
```
    for i in range(len(weights)):
        weights[i] -= alpha*weight_deltas[i]
```



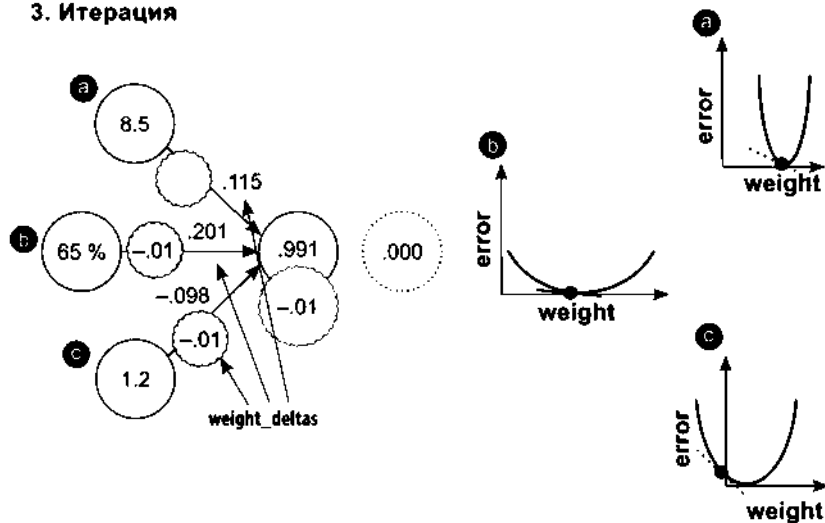
Мы можем нарисовать три графика «ошибка/вес», по одному для каждого веса. Как и прежде, на значения `weight_delta` влияют наклоны кривых (обозначены

пунктирными касательными). Обратите внимание, что наклон `weight_delta` на графике (a) больше, чем на других. Почему он оказался больше, ведь все значения `weight_delta` вычисляются на основе одной и той же разности `delta` и оценки ошибки `error`? Дело в том, что для (a) входное значение существенно больше, а значит, и производная больше.

## 2. Итерация



## 3. Итерация



Из описанного процесса обучения можно сделать несколько выводов. Основное обучение (наиболее существенный вклад в изменение веса) обусловлено входом (a), потому что соответствующая ему кривая имеет самый большой наклон. Однако это не всегда оправданно. Применение метода, называемого *нормализацией*, помогает сделать обучение более равномерным по всем весам, даже в таких наборах данных, как этот. Такое существенное различие в наклонах вынудило меня использовать меньший альфа-коэффициент, чем хотелось (0.01 вместо 0.1). Попробуйте установить альфа-коэффициент равным 0.1 и посмотрите, вызовет ли это расхождение.

## Замораживание одного веса: для чего?

Следующий эксперимент немного сложнее с точки зрения теории, но, как мне кажется, он поможет вам понять, как веса влияют друг на друга. Здесь мы повторим тот же процесс обучения, но на этот раз не будем корректировать вес (a). Мы попытаемся обучить сеть, используя только веса (b) и (c) (`weights[1]` и `weights[2]`).

```
def neural_network(input, weights):
    out = 0
    for i in range(len(input)):
        out += (input[i] * weights[i])
    return out

def ele_mul(scalar, vector):
    out = [0,0,0]
    for i in range(len(out)):
        out[i] = vector[i] * scalar
    return out

toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]

win_or_lose_binary = [1, 1, 0, 1]
true = win_or_lose_binary[0]

alpha = 0.3
weights = [0.1, 0.2, -0.1]
input = [toes[0],wlrec[0],nfans[0]]

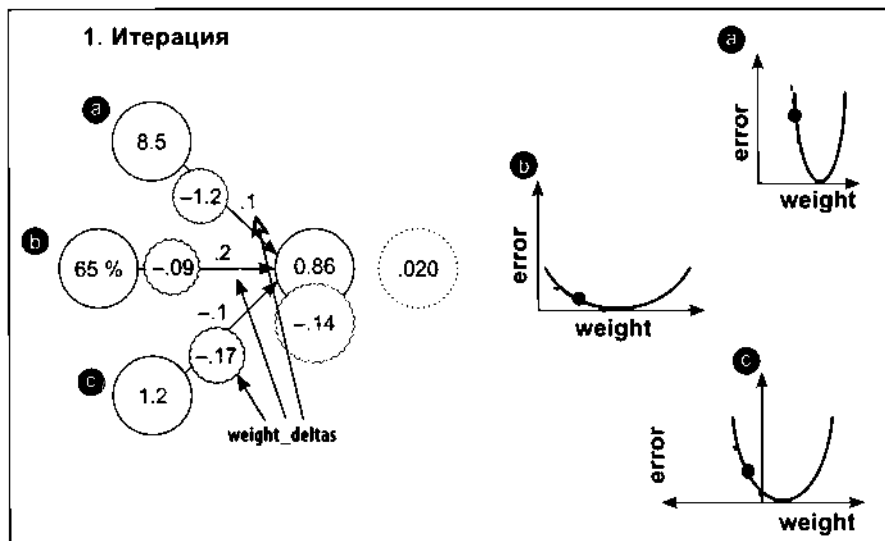
# (продолжение)
for iter in range(3):
    pred = neural_network(input,weights)

    error = (pred - true) ** 2
    delta = pred - true

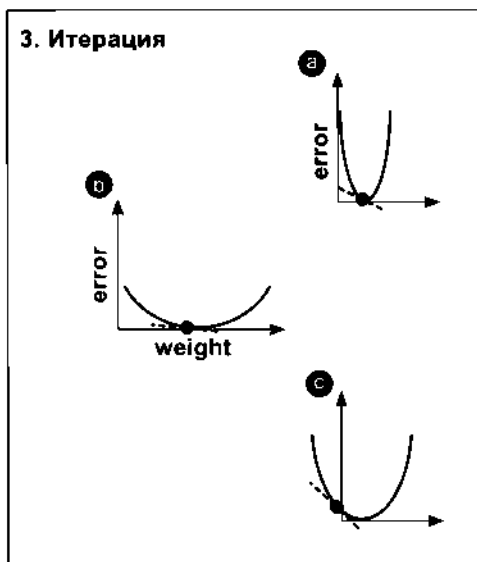
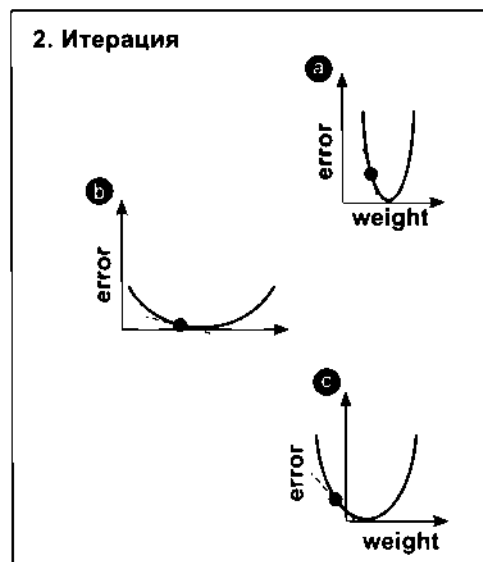
    weight_deltas=ele_mul(delta,input)
    weight_deltas[0] = 0

    print("Iteration:" + str(iter+1))
    print("Pred:" + str(pred))
    print("Error:" + str(error))
    print("Delta:" + str(delta))
    print("Weights:" + str(weights))
    print("Weight_Deltas:")
    print(str(weight_deltas))
    print(
)

    for i in range(len(weights)):
        weights[i]-=alpha*weight_deltas[i]
```



Возможно, вы удивитесь, что ошибка по-прежнему оказывается в самой нижней точке на графике (a). Чем это объясняется? Дело в том, что каждый отдельный вес оценивается относительно глобальной ошибки. Из-за того что ошибка является общей, когда для какого-то из весов она оказывается в нижней точке, она также оказывается в нижней точке для всех остальных весов.



Это чрезвычайно важный вывод. Во-первых, если достигнуто схождение ( $\text{error} = 0$ ) с весами (b) и (c), то последующая попытка выполнить обучение веса (a) ровным счетом ничего не даст. Почему? Потому что  $\text{error} = 0$ , а значит  $\text{weight\_delta}$  получит значение 0. Это обстоятельство раскрывает потенциально разрушительное свойство нейронных сетей: у вас может иметься мощный вход с большой предсказательной способностью, но если сеть случайно выяснит, как получить точный прогноз на обучающих данных без его участия, она никогда не научится включать его в прогноз.

Также обратите внимание, как точка на графике (a) оказывается внизу на кривой. Здесь смещается не черная точка, а сама кривая. Что это значит? Черная точка может смещаться по горизонтали только при изменении веса. Поскольку вес (a) в этом эксперименте был заморожен, горизонтальная координата точки остается фиксированной. Но сама ошибка  $\text{error}$  уменьшилась до 0.

Это говорит нам об истинной природе графиков. По правде говоря, это двумерные срезы четырехмерной фигуры. Три измерения соответствуют весам, а четвертое — ошибке. Эта фигура называется *поверхностью ошибки*, и, хотите верьте, хотите нет, ее кривизна определяется обучающими данными. Почему?

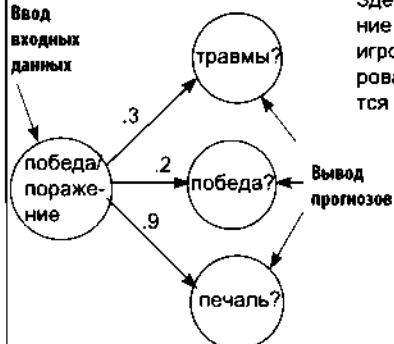
Величина ошибки определяется обучающими данными. Любая сеть может иметь любое значение веса, но значение ошибки при любой комбинации весов на 100 % определяется данными. Вы уже видели (в нескольких случаях), как входные данные влияют на крутизну U-образной кривой. Обучая нейронную сеть, мы фактически пытаемся найти самую нижнюю точку на поверхности ошибки, где нижняя точка соответствует наименьшей ошибке. Интересно?! Мы еще вернемся к этой идее, а пока отложим ее в сторону.

## Обучение методом градиентного спуска с несколькими выходами

Нейронные сети способны также возвращать несколько прогнозов для единственного входа

Возможно, решение покажется вам очевидным: для каждого выхода нужно вычислить свою разность  $\text{delta}$  и затем умножить их на единственный вход. В результате получатся приращения  $\text{weight\_delta}$  для всех весов. Сейчас вам должно быть понятно, что для обучения широкого разнообразия архитектур последовательно используется простой механизм (стохастический градиентный спуск).

## 1. Чистая сеть с несколькими выходами

Ввод  
входных  
данных

Здесь прогнозируется не только победа или поражение команды, но также эмоциональное состояние игроков — печаль или радость — и процент травмированных членов команды. Все эти прогнозы делаются только на основе вероятности победы/поражения.

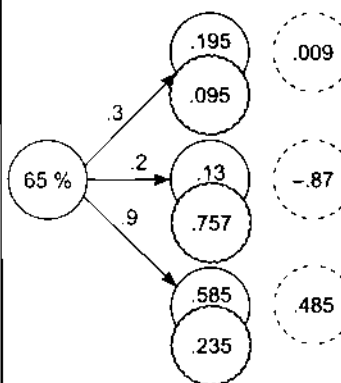
```
weights = [0.3, 0.2, 0.9]
```

```
def neural_network(input, weights):
```

```
    pred = ele_mul(input, weights)
```

```
    return pred
```

## 2. ПРОГНОЗ: получение прогноза и вычисление ошибки и разности



```
wlrec = [0.65, 1.0, 1.0, 0.9]
```

```
hurt = [0.1, 0.0, 0.0, 0.1]
```

```
win = [1, 1, 0, 1]
```

```
sad = [0.1, 0.0, 0.1, 0.2]
```

```
input = wlrec[0]
```

```
true = {hurt[0], win[0], sad[0]}
```

```
pred = neural_network(input, weights)
```

```
error = [0, 0, 0]
```

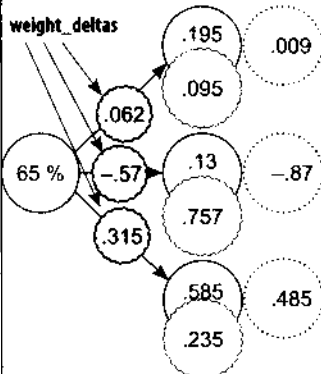
```
delta = [0, 0, 0]
```

```
for i in range(len(true)):
```

```
    error[i] = (pred[i] - true[i]) ** 2
```

```
    delta[i] = pred[i] - true[i]
```

### 3. СРАВНЕНИЕ: вычисление каждого приращения `weight_delta` и коррекция каждого веса



Как и прежде, значения приращений `weight_delta` вычисляются умножением разности прогноза и истины на соответствующее входное значение. В данном случае в вычислении приращений `weight_delta` участвуют единственное входное значение и соответствующие разности (`delta`). Также отметьте, что здесь повторно используется функция `ele_mul`.

```
def scalar_ele_mul(number,vector):
    output = [0,0,0]
    assert(len(output) == len(vector))

    for i in range(len(vector)):
        output[i] = number * vector[i]

    return output

wlrec = [0.65, 1.0, 1.0, 0.9]
hurt = [0.1, 0.0, 0.0, 0.1]
win = [ 1, 1, 0, 1]
sad = [0.1, 0.0, 0.1, 0.2]
```

```
input = wlrec[0]
true = [hurt[0], win[0], sad[0]]

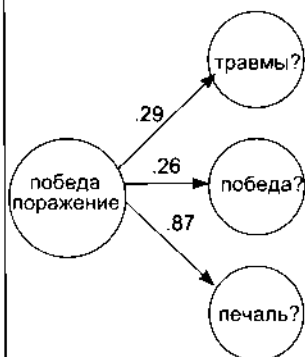
pred = neural_network(input,weights)

error = [0, 0, 0]
delta = [0, 0, 0]

for i in range(len(true)):
    error[i] = (pred[i] - true[i]) ** 2
    delta[i] = pred[i] - true[i]

weight_deltas = scalar_ele_mul(input,weights)
```

### 4. ОБУЧЕНИЕ: корректировка весов



```
input = wlrec[0]
true = [hurt[0], win[0], sad[0]]
pred = neural_network(input,weights)

error = [0, 0, 0]
delta = [0, 0, 0]

for i in range(len(true)):
    error[i] = (pred[i] - true[i]) ** 2
    delta[i] = pred[i] - true[i]

weight_deltas = scalar_ele_mul(input,weights)
alpha = 0.1

for i in range(len(weights)):
    weights[i] -= (weight_deltas[i] * alpha)

print("Weights:" + str(weights))
print("Weight Deltas:" + str(weight_deltas))
```

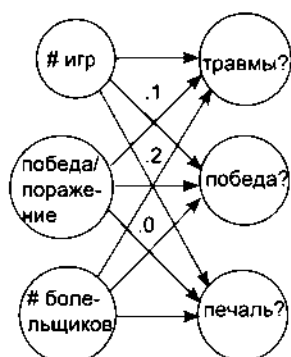


# Обучение методом градиентного спуска с несколькими входами и выходами

Обобщение градиентного спуска для обучения сетей произвольной величины

## 1. Чистая сеть с несколькими входами и выходами

**Входы**      **Прогнозы**



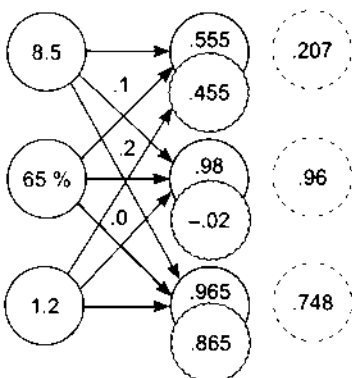
```
weights = # игр %победа # болельщики
          [ [0.1, 0.1, -0.3], # травмы?
            [0.1, 0.2, 0.0], # победа?
            [0.0, 1.3, 0.1] ]# печаль?
```

```
def vect_mat_mul(vect, matrix):
    assert(len(vect) == len(matrix))
    output = [0,0,0]
    for i in range(len(vect)):
        output[i] = w_sum(vect, matrix[i])
    return output
```

```
def neural_network(input, weights):
    pred = vect_mat_mul(input, weights)
    return pred
```

## 2. ПРОГНОЗ: получение прогноза и вычисление ошибки и разности

**Входы**      **Прогноз**      **Ошибки**



```
toes = [8.5, 9.5, 9.9, 9.0]
wlrec = [0.65, 0.8, 0.8, 0.9]
nfans = [1.2, 1.3, 0.5, 1.0]
```

```
hurt = [0.1, 0.0, 0.0, 0.1]
win = [1, 1, 0, 1]
sad = [0.1, 0.0, 0.1, 0.2]
```

```
alpha = 0.01
```

```
input = [toes[0], wlrec[0], nfans[0]]
true = [hurt[0], win[0], sad[0]]
```

```
pred = neural_network(input, weights)
```

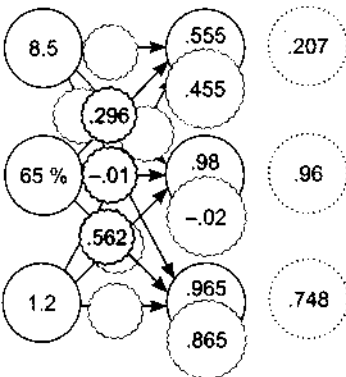
```
error = [0, 0, 0]
delta = [0, 0, 0]
```

```
for i in range(len(true)):
```

```
    error[i] = (pred[i] - true[i]) ** 2
    delta = pred[i] - true[i]
```

### 3. СРАВНЕНИЕ: вычисление каждого приращения `weight_delta` и коррекция каждого веса

**Входы**      **Прогноз**      **Ошибки**



(для экономии места здесь показано приращение `weight_delta` только для одного входа)

```
def outer_prod(vec_a, vec_b):
    out = zeros_matrix(len(a),len(b))
    for i in range(len(a)):
        for j in range(len(b)):
            out[i][j] = vec_a[i]*vec_b[j]
    return out

input = [toes[0],wlrec[0],nfans[0]]
true = [hurt[0], win[0], sad[0]]

pred = neural_network(input,weights)

error = [0, 0, 0]
delta = [0, 0, 0]

for i in range(len(true)):
    error[i] = (pred[i] - true[i]) ** 2
    delta = pred[i] - true[i]

weight_deltas = outer_prod(input,delta)
```

## Чему обучаются эти веса?

Каждый вес стремится уменьшить ошибку, но чему они учатся в совокупности?

Поздравляю! Вы достигли точки в повествовании, в которой мы перейдем к обработке первого набора данных из реального мира. Так удачно совпало, что он имеет историческое значение.

Он называется модифицированным набором данных Национального института стандартов и технологий (Modified National Institute of Standards and Technology, MNIST) и состоит из набора черно-белых изображений рукописных цифр, много лет тому назад написанных учениками старших классов и сотрудниками Бюро переписи США. Каждое изображение сопровождается фактическим числом (0–9). В последние несколько десятилетий люди использовали этот набор для обучения нейронных сетей распознаванию рукописного текста, и сегодня мы тоже предпримем такую попытку.

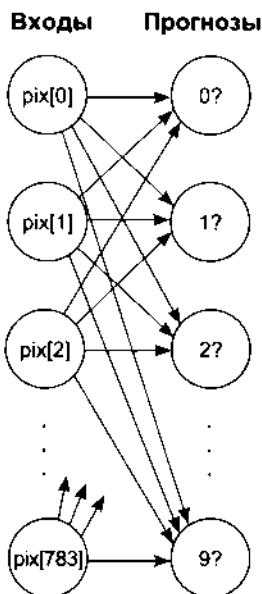


Каждое изображение состоит из 784 пикселей ( $28 \times 28$ ). Учитывая, что на входе имеется 784 пикселя и на выходе 10 возможных меток, уже можно представить форму нейронной сети: каждый обучающий образец содержит 784 значения (по одному на каждый пиксел), поэтому нейронная сеть должна иметь 784 входа. Довольно просто, не так ли? Мы выбрали число входных узлов по числу точек данных в каждом обучающем образце. Нам нужно предсказать *10 вероятностей*: по одной для каждой цифры. Получив изображение, нейронная сеть должна вернуть эти 10 вероятностей, сообщая, какая цифра вероятнее всего изображена.

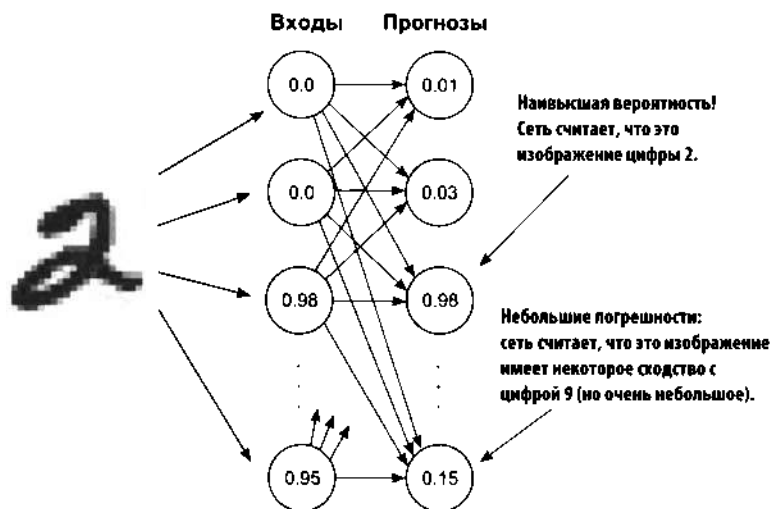
Как настроить нейронную сеть для получения 10 вероятностей? В предыдущем разделе вы видели диаграмму нейронной сети, способной принимать несколько входов и возвращать несколько прогнозов. Мы можем изменить эту сеть, чтобы создать нужное число входов и выходов для решения новой задачи анализа данных в наборе MNIST. Итак, скорректируем сеть, создав 784 входа и 10 выходов.

В блокноте `MNISTPreprocessor` имеется сценарий, выполняющий предварительную обработку набора данных MNIST и загружающий первую 1000 изображений и меток в две матрицы NumPy с именами `images` и `labels`. Возможно, вам интересно узнать, как двумерные изображения размером  $28 \times 28$  пикселей загрузить в плоскую нейронную сеть. Ответ прост: каждое изображение преобразуется в вектор  $1 \times 784$ . Сначала в вектор записывается первый ряд пикселей, затем второй, третий и так далее, пока не получится единый список пикселей изображения (длиной 784 пикселя).

На следующем рисунке изображена новая нейронная сеть, выполняющая классификацию данных из набора MNIST. Она очень похожа на сеть с несколькими входами и выходами, которую мы обучали выше. Разница лишь в количестве входов и выходов, которое существенно увеличилось. Эта сеть имеет 784 входа (по одному для каждого пикселя в изображении  $28 \times 28$ ) и 10 выходов (по одному для каждой возможной цифры).



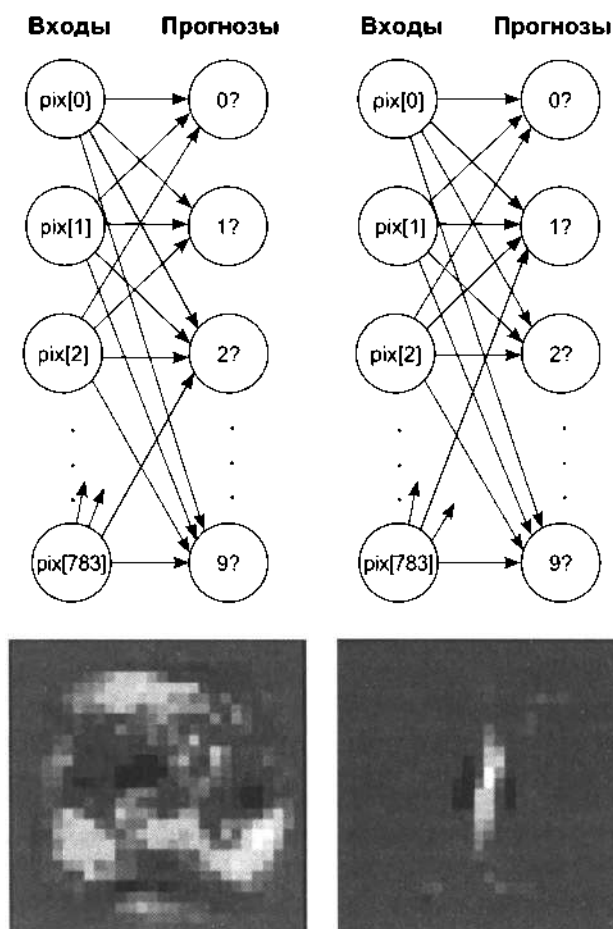
Если бы сеть была способна давать точный прогноз, тогда, получив пиксели изображения (например, с цифрой 2, как на следующем рисунке), она вернула бы прогнозную вероятность 1.0 в соответствующем выходе (в данном случае в третьем) и 0 во всех остальных. Если сеть сумеет дать правильный прогноз для всех изображений в наборе данных, тогда она будет иметь ошибку, равную нулю.



В процессе обучения сеть будет корректировать веса между входами и выходами, стремясь уменьшить ошибку до 0. Но как это происходит? Как происходит корректировка кучи весов при изучении совокупности закономерностей?

## Визуализация значений весов

Простым и интересным способом исследования нейронных сетей (особенно предназначенных для классификации изображений) является визуализация весов, как если бы они представляли пиксели изображений. Взглянув на следующую диаграмму, вы поймете, о чем речь.



Каждому выходному узлу соответствуют веса, исходящие из пикселей в исходном изображении. Например, узел 2? имеет 784 входящих веса, каждый из которых отражает связь между пикселем и цифрой 2.

Что это за связь? Все просто: если вес имеет высокое значение, значит, модель считает, что между этим пикселем и цифрой 2 существует сильная *корреляция*. Если вес имеет очень низкое (отрицательное) значение, значит, сеть полагает, что корреляция между этим пикселем и цифрой 2 очень слабая (или даже отрицательная).

Если вывести все эти веса в виде изображения, имеющего ту же форму, что и изображения из исходного набора данных, можно увидеть, какие пиксели имеют наивысшую корреляцию с конкретным выходным узлом. В нашем примере на изображениях, построенных для 2 и 1 с использованием весов, можно видеть очень расплывчатые цифры 2 и 1 соответственно. Яркие области соответствуют высоким весам, а темные — отрицательным. Нейтральные области представляют 0 в матрице весов. Эти изображения показывают, как нейронная сеть представляет форму цифр 2 и 1.

Как это получилось? Чтобы понять, нужно вернуться к уроку о скалярном произведении. Рассмотрим кратко его суть.

## Визуализация скалярных произведений (сумм весов)

Напомню, как вычисляется скалярное произведение. В этой операции участвуют два вектора. Сначала векторы перемножаются друг на друга (поэлементно), а затем произведения суммируются. Взгляните на следующий пример:

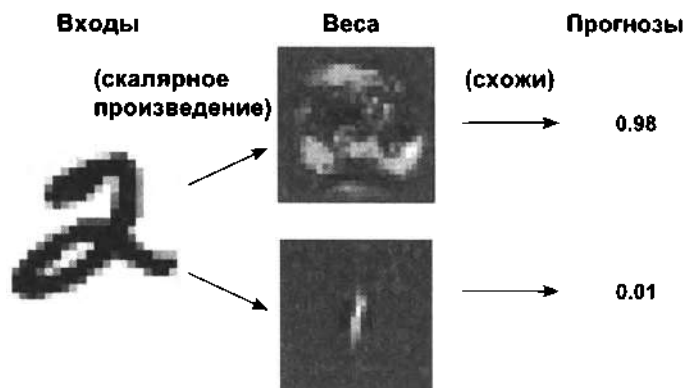
```
a = [ 0, 1, 0, 1 ]
b = [ 1, 0, 1, 0 ]
[ 0, 0, 0, 0 ] -> 0 ← Результат
```

Здесь сначала перемножаются пары соответствующих элементов в *a* и *b*, в результате чего получается промежуточный вектор, состоящий из нулей. После этого элементы промежуточного вектора складываются, что дает окончательный результат 0. Почему? Потому что векторы не имеют ничего общего между собой.

```
c = [ 0, 1, 1, 0 ]      b = [ 1, 0, 1, 0 ]
d = [ .5, 0, .5, 0 ]    c = [ 0, 1, 1, 0 ]
```

Но скалярное произведение с  $d$  вернет более высокий результат, потому что в столбцах, имеющих положительные значения, есть совпадение. Скалярное произведение между идентичными векторами также даст высокое значение в результате. И какой вывод? *Скалярное произведение может служить своеобразной оценкой сходства двух векторов.*

Что это значит для весов и входов в нашей нейронной сети? Если вектор весов имеет сходство с входным вектором, представляющим цифру 2, тогда, благодаря этому сходству, на выходе цифра 2 получит высокую оценку вероятности. И наоборот, если вектор весов *не* имеет сходства с входным вектором для 2, на выходе эта цифра получит низкую оценку. Это наглядно иллюстрирует следующий рисунок. Сможете теперь сами объяснить, почему наибольшая оценка (0.98) выше наименьшей (0.01)?



## Итоги

### Градиентный спуск — универсальный алгоритм машинного обучения

Самый важный, пожалуй, вывод этой главы — градиентный спуск является очень гибким алгоритмом машинного обучения. Если скомбинировать веса способом, позволяющим вычислить функцию ошибки и разность  $\delta$ , то алгоритм градиентного спуска сможет подсказать вам, как следует скорректировать веса, чтобы уменьшить ошибку. В оставшейся части книги мы займемся исследованием разных комбинаций весов и функций ошибки, где может пригодиться градиентный спуск. И начнем прямо со следующей главы.

# 6

## Создание первой глубокой нейронной сети: введение в обратное распространение



### В этой главе

- ✓ Задача о светофоре.
- ✓ Матрицы и матричные отношения.
- ✓ Полный, пакетный и стохастический градиентный спуск.
- ✓ Нейронные сети изучают корреляцию.
- ✓ Переобучение.
- ✓ Определение собственной корреляции.
- ✓ Обратное распространение: определение причин ошибок на расстоянии.
- ✓ Линейность и нелинейность.
- ✓ Иногда корреляция может быть тайной.
- ✓ Ваша первая глубокая сеть.
- ✓ Обратное распространение в коде: собираем все воедино.

О, компьютер Глубокий Замысел, задача, ради решения которой тебя разработали, следующая. Мы хотим, чтобы ты нам сказал... ОТВЕТ!

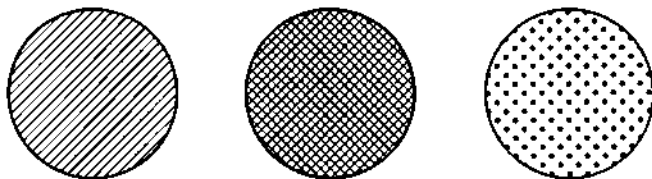
*Дуглас Адамс (Douglas Adams).  
«Автостопом по галактике»*



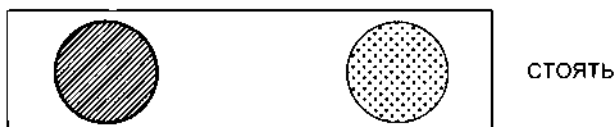
## Задача о светофоре

Эта простая задача поможет нам узнать, как обучаются сети на наборах данных

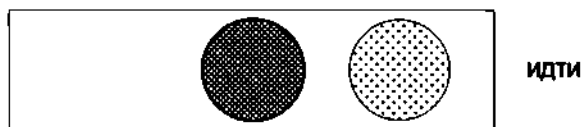
Представьте, что вы идете по улице в незнакомой стране. Подойдя к перекрестку, поднимаете глаза и видите светофор незнакомой конструкции. Как узнать, по какому сигналу можно безопасно пересечь проезжую часть?



Чтобы разобраться, когда безопасно пересечь улицу, нужно понимать значение огней светофора. Но в данном случае это неизвестно. Какая комбинация огней разрешает *переход*? Какая требует *остановиться*? Чтобы решить эту задачу, можно постоять перед перекрестком несколько минут и понаблюдать за комбинациями огней и за тем, как ведут себя окружающие — идут или стоят. Итак, вы достаете блокнот и зарисовываете следующую схему:

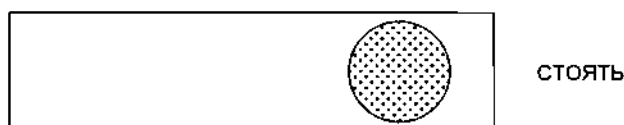


При этой комбинации никто не переходит улицу. В этот момент у вас рождается мысль: «Схема работы светофора не так проста. Включение левой или правой секции может означать требование остановиться, а включение средней — разрешать движение». Но пока вы не можете знать этого наверняка. Продолжим наблюдение:

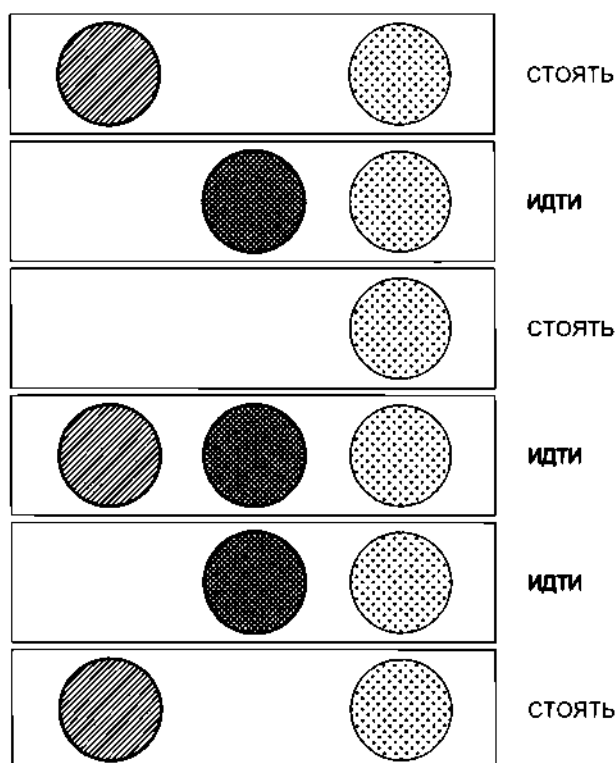


После включения этой комбинации огней люди пошли. Единственное, в чем вы теперь уверены, что правая секция, похоже, ничего не запрещает и ничего

не разрешает. Возможно, она вообще не имеет никакого значения. Понаблюдаем еще:



Еще одна комбинация. На этот раз изменилась только средняя секция, и вы отметили противоположное поведение. У вас появляется рабочая гипотеза, что *средняя* секция указывает, когда безопасно перейти перекресток. В течение следующих нескольких минут вы записываете следующие шесть комбинаций огней, отметив напротив каждой, когда люди шли или стояли. Теперь вы можете определить общий шаблон?



Как и предполагалось, между средней секцией и возможностью безопасно пересечь перекресток имеется *идеальная корреляция*. Вам удалось выявить эту за-

кономерность, наблюдая отдельные комбинации огней и выявляя связь между возможностью перехода. Именно этому мы будем обучать нейронную сеть.

## Подготовка данных

### Нейронные сети не умеют распознавать сигналы светофора

В предыдущих главах вы познакомились с алгоритмами обучения с учителем. Вы узнали, что они способны преобразовать один набор данных в другой. И, что особенно важно, они могут принимать набор данных, представляющий то, *что вы знаете*, и превращать его в набор данных, представляющий то, *что вы хотели бы знать*.

Как обучить нейронную сеть, реализующую алгоритм обучения с учителем? Вы должны передать ей два набора данных и попросить ее научиться преобразовывать один в другой. Вспомните задачу со светофором. Сможете сами определить два набора данных? Какой из них представляет то, что вы знаете, а какой — то, что вы хотели бы знать?

У нас действительно есть два набора данных. Один из них — шесть комбинаций огней светофора, а другой — шесть наблюдений за поведением пешеходов. Они и послужат нам двумя наборами данных, необходимыми для обучения сети.

Мы можем обучить нейронную сеть преобразовывать набор данных, представляющий то, что мы *знаем*, в набор данных, представляющий то, что мы *хотим знать*. В данном конкретном примере мы знаем шесть комбинаций огней светофора, а хотим знать — какая комбинация разрешает переход.

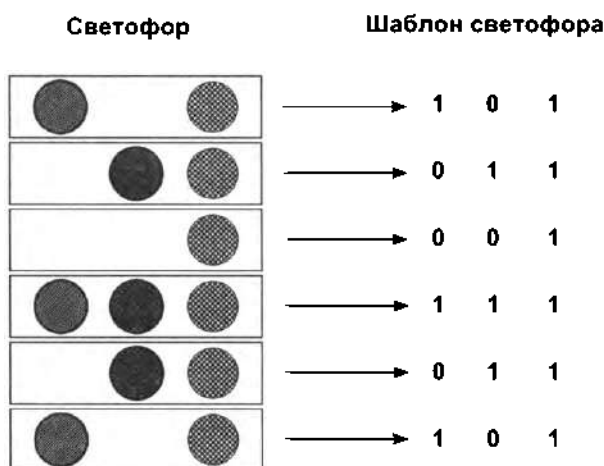
| Что мы знаем                                                                        | Что мы хотим<br>знать |
|-------------------------------------------------------------------------------------|-----------------------|
|  | СТОЯТЬ                |
|  | ИДТИ                  |
|  | СТОЯТЬ                |
|  | ИДТИ                  |
|  | ИДТИ                  |
|  | СТОЯТЬ                |

Чтобы подготовить эти данные для передачи в сеть, мы должны сначала разбить их на две группы (что мы знаем и что мы хотим знать). Обратите внимание, что в данном случае можно пойти в обратном направлении, поменяв группы местами. Для некоторых задач это работает.

## Матрицы и матричные отношения

### Преобразование сигналов светофора в цифровое представление

В математике нет такого понятия, как «сигнал светофора». Как отмечалось в предыдущем разделе, нам нужно обучить нейронную сеть преобразовывать комбинации сигналов светофора в шаблон стоять/идти. Главное слово здесь — *шаблон*. То есть нам нужно представить сигналы светофора в форме числового шаблона. Поясню, что я имею в виду.



Обратите внимание, что цифровой шаблон на этом рисунке имитирует состояние секций светофора, обозначая его как 1 (включено) и 0 (выключено). Каждой секции соответствует свой столбец (всего три столбца, потому что светофор имеет три секции). Отметьте также, что всего имеется шесть строк, представляющих шесть комбинаций сигналов, которые мы наблюдали.

Эта структура нулей и единиц называется *матрицей*. А связь между строками и столбцами — распространенное явление в мире матриц, особенно матриц данных (таких, как сигналы светофора).

В матрицах данных принято отдавать для каждого *наблюдаемого образца* отдельную *строку*, а для каждого *наблюдаемого признака* — отдельный *столбец*. Это упрощает чтение матриц.

Итак, каждый столбец в нашей матрице представляет состояния каждого признака. В данном случае каждый столбец содержит зафиксированные состояния включено/выключено каждой секции светофора. Каждая строка содержит состояние всех трех секций в определенный момент времени. Как уже говорилось, такая схема широко используется в матрицах.

### Хорошие матрицы данных идеально имитируют реальный мир

Матрица данных обязательно должна содержать только нули и единицы. Представьте, например, что огни светофора могут гореть с разной степенью яркости. Тогда матрица состояний светофора могла бы выглядеть примерно так:

| Светофор                                                                            | Матрица A,<br>представляющая светофор |
|-------------------------------------------------------------------------------------|---------------------------------------|
|    | → .9 .0 1                             |
|    | → .2 .8 1                             |
|    | → .1 .0 1                             |
|   | → .8 .9 1                             |
|  | → .1 .7 1                             |
|  | → .9 .1 0                             |

Матрица A вполне возможна. Она имитирует шаблоны (сигналы), используемые в реальности (светофор), соответственно мы можем попросить компьютер интерпретировать их. Возможна ли следующая матрица?

| Светофор                                                                          | Матрица В,<br>представляющая светофор |
|-----------------------------------------------------------------------------------|---------------------------------------|
|  | → 9 0 10                              |
|  | → 2 8 10                              |
|  | → 1 0 10                              |
|  | → 8 9 10                              |
|  | → 1 7 10                              |
|  | → 9 1 0                               |

Да, эта матрица (В) тоже *возможна*. Она достаточно точно отражает отношение между разными обучающими примерами (строками) и сигналами (столбцами). Обратите внимание, что  $A * 10 == B$ . Это означает, что матрицы *скалярно кратны* друг другу.

Обе матрицы, А и В, представляют один и тот же шаблон

Из вышесказанного следует важный вывод: существует *бесконечное* число матриц, идеально отражающих шаблоны светофора в виде наборов данных. Идеальна даже следующая матрица.

| Светофор                                                                            | Матрица С,<br>представляющая светофор |
|-------------------------------------------------------------------------------------|---------------------------------------|
|  | → 18 0 20                             |
|  | → 4 16 20                             |
|  | → 2 0 20                              |
|  | → 16 18 20                            |
|  | → 2 14 20                             |
|  | → 18 2 0                              |

Важно понимать, что фактический шаблон и матрица, построенная на его основе, — это не одно и то же. Это *особенность* матриц. Фактически эта особенность характерна для всех трех матриц (А, В и С). Шаблон — это то, что *выражает* каждая из этих матриц. А также шаблон — это схема работы светофора.

Этот *шаблон входных данных* представляет тот набор, который нейронная сеть должна научиться преобразовывать в *шаблон выходных данных*. Но чтобы передать шаблон выходных данных, его тоже нужно преобразовать в форму матрицы, как показано ниже.

|        |   |   |
|--------|---|---|
| стоять | → | 0 |
| идти   | → | 1 |
| стоять | → | 0 |
| идти   | → | 1 |
| идти   | → | 1 |
| стоять | → | 0 |

Обратите внимание, что здесь можно поменять нули и единицы местами и при этом выходная матрица по-прежнему будет точно отражать шаблон «стоять/идти». Потому что независимо от того, какому поведению, «стоять» или «идти», будет присвоена единица, вы всегда сможете декодировать нули и единицы в фактический шаблон «стоять/идти».

Получившаяся матрица называется *представлением без потерь*, потому что позволяет безошибочно преобразовывать метки стоять/идти в матрицу и обратно.

## Создание матриц в Python

### Импорт матриц в Python

Мы преобразовали шаблон работы светофора в матрицу (с нулями и единицами). Теперь создадим эту матрицу (и, что особенно важно, фактический шаблон) в коде на Python, чтобы нейронная сеть могла прочитать ее. Библиотека NumPy для Python (представлена в главе 3) специально создавалась для работы с матрицами. Посмотрим, как она действует:

```
import numpy as np
streetlights = np.array( [ [ 1, 0, 1 ],
                           [ 0, 1, 1 ],
                           [ 0, 0, 1 ],
                           [ 1, 1, 1 ],
                           [ 0, 1, 1 ],
                           [ 1, 0, 1 ] ] )
```

Если вы не особенно искушенный пользователь Python, этот код может показаться вам поразительным. Оказывается, что матрица — это всего лишь список списков. Это массив массивов. Что такое NumPy? NumPy — это на самом деле всего лишь очень удобная обертка вокруг массива массивов, которая предлагает множество функций для выполнения операций с матрицами. Давайте создадим также матрицу NumPy для выходных данных:

```
walk_vs_stop = np.array( [ [ 0 ],
                           [ 1 ],
                           [ 0 ],
                           [ 1 ],
                           [ 1 ],
                           [ 0 ] ] )
```

Чего мы хотим от нейронной сети? Чтобы она взяла матрицу `streetlights` и научилась преобразовывать ее в матрицу `walk_vs_stop`. Более того, мы хотим, чтобы нейронная сеть принимала *любую матрицу, отражающую тот же шаблон*, что и `streetlights`, и преобразовывала ее в матрицу, отражающую шаблон `walk_vs_stop`. Подробнее об этом мы поговорим ниже, а сейчас начнем с попытки преобразовать `streetlights` в `walk_vs_stop` с помощью нейронной сети.



## Создание нейронной сети

Мы изучаем нейронные сети вот уже несколько глав подряд. У нас есть новый набор данных, и мы собираемся создать нейронную сеть, которая будет обучаться на нем. Ниже приводится пример кода, реализующий обучение на первой комбинации сигналов светофора. Многие в нем должны быть вам знакомы:



```

import numpy as np
weights = np.array([0.5,0.48,-0.7])
alpha = 0.1

streetlights = np.array( [ [ 1, 0, 1 ],
                             [ 0, 1, 1 ],
                             [ 0, 0, 1 ],
                             [ 1, 1, 1 ],
                             [ 0, 1, 1 ],
                             [ 1, 0, 1 ] ] )

walk_vs_stop = np.array( [ 0, 1, 0, 1, 1, 0 ] )

input = streetlights[0]  ← [1,0,1]
goal_prediction = walk_vs_stop[0]  ← Содержит 0 (стоять)

for iteration in range(20):
    prediction = input.dot(weights)
    error = (goal_prediction - prediction) ** 2
    delta = prediction - goal_prediction
    weights = weights - (alpha * (input * delta))

    print("Error:" + str(error) + " Prediction:" + str(prediction))

```

Этот пример демонстрирует некоторые тонкости, описанные в главе 3. В первых, здесь используется функция `dot`, которая находит скалярное произведение (взвешенную сумму) двух векторов. Но в главе 3 не говорилось, что с матрицами NumPy можно выполнять поэлементное сложение и умножение:

```

import numpy as np

a = np.array([0,1,2,1])
b = np.array([2,2,2,3])

print(a*b)  ← Поэлементное умножение
print(a+b)  ← Поэлементное сложение
print(a * 0.5)  ← Умножение вектора на скаляр
print(a + 0.5)  ← Сложение вектора со скаляром

```

Библиотека NumPy упрощает выполнение этих операций. Когда вы ставите знак «плюс» (+) между двумя векторами, библиотека выполняет сложение двух векторов. Кроме использования этих удобных операторов NumPy и новых наборов данных, в остальном показанная здесь нейронная сеть ничем не отличается от предыдущих.

## Обучение на полном наборе данных

Мы обучили нейронную сеть распознавать одну комбинацию сигналов, но нам нужно, чтобы она распознавала все комбинации

До сих пор в этой книге мы обучали нейронные сети моделировать единственный обучающий набор (пару `input -> goal_pred`). Но теперь мы пытаемся построить нейронную сеть, которая подскажет, когда безопасно переходить улицу. Соответственно, мы должны обучить ее распознавать все комбинации. Как это сделать? Выполнить обучение на всех комбинациях сразу:

```
import numpy as np

weights = np.array([0.5,0.48,-0.7])
alpha = 0.1

streetlights = np.array( [[ 1, 0, 1 ],
                           [ 0, 1, 1 ],
                           [ 0, 0, 1 ],
                           [ 1, 1, 1 ],
                           [ 0, 1, 1 ],
                           [ 1, 0, 1 ] ] )

walk_vs_stop = np.array( [ 0, 1, 0, 1, 1, 0 ] )

input = streetlights[0]  ← [1,0,1]
goal_prediction = walk_vs_stop[0]  ← Содержит 0 (стоять)

for iteration in range(40):
    error_for_allLights = 0
    for row_index in range(len(walk_vs_stop)):
        input = streetlights[row_index]
        goal_prediction = walk_vs_stop[row_index]

        prediction = input.dot(weights)

        error = (goal_prediction - prediction) ** 2
        error_for_allLights += error

    delta = prediction - goal_prediction
    weights = weights - (alpha * (input * delta))
    print("Prediction:" + str(prediction))
    print("Error:" + str(error_for_allLights) + "\n")

    Error:2.6561231104
    Error:0.962870177672
    ...
    Error:0.000614343567483
    Error:0.000533736773285
```

## Полный, пакетный и стохастический градиентный спуск

**Стохастический градиентный спуск корректирует сразу все веса для одного примера**

Как оказывается, идея обучения на одном примере за раз — это разновидность градиентного спуска, который называется *стохастическим градиентным спуском* и является одним из немногих методов, позволяющих выполнять обучение сразу на всем наборе данных.

Как работает стохастический градиентный спуск? Как было показано в предыдущем примере, он выполняет прогноз и корректировку веса для каждого обучающего примера в отдельности. Иначе говоря, он берет первую комбинацию огней светофора и пытается спрогнозировать поведение пешеходов для нее, вычисляет `weight_delta` и корректирует веса. Затем переходит ко второй комбинации и так далее. Он многократно перебирает все данные из набора, пока не найдет комбинацию весов, которая хорошо прогнозирует все обучающие примеры.

**(Полный) градиентный спуск корректирует веса сразу для одного набора данных**

Как рассказывалось в главе 4, другим методом обучения на полном наборе данных является градиентный спуск (или *усредняющий/полный градиентный спуск*). Вместо корректировки весов по одному для каждого обучающего примера сеть вычисляет среднее значение `weight_delta` для всего набора данных и изменяет веса, только когда вычисляет полное среднее значение.

**Пакетный градиентный спуск корректирует веса после просмотра  $n$  примеров**

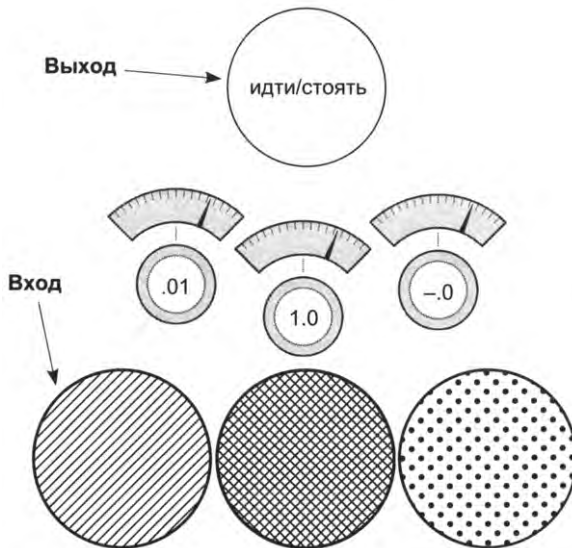
Кроме стохастического и полного градиентного спуска существует третья разновидность этого метода, занимающая промежуточное положение, которую мы рассмотрим позже. Пакетный градиентный спуск корректирует веса не после каждого примера или всего набора данных, а после просмотра *пакета* с указанным вами числом примеров (обычно от 8 до 256).

Мы обсудим этот метод позже, а пока лишь отмечу, что в предыдущем примере реализована нейронная сеть, которая обучается на всем наборе данных, корректируя веса после каждого примера.

## Нейронные сети изучают корреляцию

### Чему обучилась предыдущая нейронная сеть?

Мы только что закончили обучение однослойной нейронной сети, которая принимает комбинацию огней светофора и сообщает, насколько безопасно переходить дорогу на эту комбинацию. Теперь отвлечемся ненадолго и посмотрим на происходящее глазами нейронной сети. Нейронная сеть не знает, что обрабатывает сигналы светофора. Она лишь пытается определить, какой из входов (из трех возможных) коррелирует с выходом. Она правильно определила смысл средней секции, проанализировав окончательные позиции весов в сети.



Обратите внимание, что средний вес очень близок к 1, тогда как левый и правый веса очень близки к 0. Проще говоря, все эти сложные итеративные процессы обучения пришли к простому выводу: они *выявили корреляцию* между средним входом и выходом. Корреляция существует везде, где веса имеют высокие значения. И наоборот, было выявлено, что левый и правый входы (веса которых близки к нулю) *никак не влияют* на выходной результат.

Как сеть выявила корреляцию? В процессе градиентного спуска каждый обучающий пример оказывает *повышающее* или *понижающее* давление на веса. В целом на вес среднего входа оказывалось повышающее давление, а на другие веса — понижающее. Откуда взялось это давление? Почему оно оказалось разным для разных весов?

## Повышающее и понижающее давление

### Давление обусловлено данными

Каждый выход в отдельности пытается правильно спрогнозировать результат по входным данным. По большей части каждый выход игнорирует все другие узлы. Единственное, что *связывает* выходы, — общая мера ошибки. *Величина корректировки веса* — это не что иное, как произведение общей меры ошибки на соответствующий вход.

Почему именно так? Ключевым аспектом обучения нейронных сетей является выявление *причин ошибок*. То есть при наличии общей ошибки сеть должна определить, какие веса внесли в нее свой вклад (чтобы скорректировать их), а какие *нет* (чтобы оставить их в неприкосновенности).

| Обучающие данные |   |   |   | Давление на вес |   |   |   |
|------------------|---|---|---|-----------------|---|---|---|
| 1                | 0 | 1 | → | 0               | 0 | 0 | → |
| 0                | 1 | 1 | → | 1               | 0 | + | → |
| 0                | 0 | 1 | → | 0               | 0 | 0 | → |
| 1                | 1 | 1 | → | 1               | + | + | → |
| 0                | 1 | 1 | → | 1               | 0 | + | → |
| 1                | 0 | 1 | → | 0               | - | 0 | → |

Рассмотрим первый обучающий пример. Поскольку средний вход имеет значение 0, соответствующий ему вес полностью исключен из этого прогноза. Какое бы значение ни имел этот вес, он будет умножен на 0 (значение входа). То есть любая ошибка для этого обучающего примера (высокая или низкая) обусловлена только левым и правым весами.

Посмотрим, как формируется давление в этом первом обучающем примере. Если сеть должна вернуть прогноз 0, а два входа равны 1, это вызовет ошибку, из-за чего значения соответствующих весов уменьшатся *в сторону* 0.

Таблица «Давление на вес» на рисунке помогает увидеть влияние каждого обучающего примера на значение каждого веса. Знак «плюс» (+) показывает, что давление поднимает вес в сторону 1, а знак «минус» (–) показывает, что давление опускает вес в сторону 0. Нуль (0) показывает, что на данный вес не оказывается давления, потому что соответствующее входное значение равно 0, то есть этот вес не изменяется. Обратите внимание, что весу слева соответствуют два минуса и один плюс, то есть в целом этот вес уменьшается. Весу в середине соответствуют три плюса, то есть в целом он увеличивается.

| Обучающие данные |   |   |   | Давление на вес |   |   |   |
|------------------|---|---|---|-----------------|---|---|---|
| 1                | 0 | 1 | 0 | –               | 0 | – | 0 |
| 0                | 1 | 1 | 1 | 0               | + | + | 1 |
| 0                | 0 | 1 | 0 | 0               | 0 | – | 0 |
| 1                | 1 | 1 | 1 | +               | + | + | 1 |
| 0                | 1 | 1 | 1 | 0               | + | + | 1 |
| 1                | 0 | 1 | 0 | –               | 0 | – | 0 |

Каждый отдельно взятый вес стремится компенсировать ошибку. В первом обучающем примере наблюдается *несоответствие* между левым и правым входами и желаемым результатом, из-за чего соответствующие веса испытывают давление, толкающее их вниз.

Это явление наблюдается во всех шести обучающих примерах — наличие корреляции толкает веса вверх к 1, а ее отсутствие толкает веса вниз к 0. В итоге сеть обнаруживает, что корреляция между желаемым результатом и весом в середине является доминирующей прогностической силой (самый большой вес в средневзвешенных входных значениях), и это помогает ей обеспечить высокую точность прогнозирования.

### ВАЖНЫЙ ВЫВОД

Прогноз — это взвешенная сумма входных значений. Алгоритм обучения придает дополнительную значимость входам, значения которых коррелируют с выходами, оказывая повышающее давление на соответствующие веса, и уменьшает значимость входов, не коррелирующих с выходами, оказывая понижающее давление. Взвешенная сумма входов находит наилучшую корреляцию между входами и выходами, взвешивая не коррелирующие входы нулем.

Математик, сидящий в вас, может поморщиться. Повышающее и понижающее давления не являются точными математическими определениями и имеют множество пограничных случаев, когда эта логика не действует (об этом мы поговорим чуть ниже). Но позже вы поймете, что такая аналогия весьма ценная, потому что она позволяет на время забыть о сложностях градиентного спуска и просто помнить, что *процесс обучения поощряет корреляцию* увеличением веса (или, говоря более общими словами, *процесс обучения отыскивает корреляцию между двумя наборами данных*).

## Пограничный случай: переобучение

### Иногда корреляция возникает случайно

Рассмотрим еще раз первый пример в обучающих данных. Что получится, если весу слева присвоить начальное значение 0.5, а весу справа — значение -0.5? В результате они дадут прогноз 0. То есть сеть с этими весами будет иметь идеальную точность для первого примера. Но в действительности она не обучилась распознавать сигналы светофора (в реальном мире сеть с этими весами подстерегает неудача). Это явление известно как *переобучение*.

#### **САМЫЙ БОЛЬШОЙ НЕДОСТАТОК ГЛУБОКОГО ОБУЧЕНИЯ: ПЕРЕОБУЧЕНИЕ**

Ошибка является общей для всех весов. Если какая-то комбинация весов случайно создаст идеальную корреляцию между прогнозом и фактическим результатом в обучающем наборе (когда  $\text{error} == 0$ ) без увеличения веса входов с более высокой прогностической способностью, *нейронная сеть прекратит обучение*.

Если бы не другие обучающие примеры, этот роковой недостаток нанес бы непоправимый ущерб нейронной сети. Какое влияние оказывают другие обучающие примеры? Давайте посмотрим на второй пример. Он увеличит вес справа, оставив вес слева неизменным. Это нарушит равновесие, остановившее обучение в первом примере. Если обучение не будет ограничено только первым примером, остальные примеры помогут сети избежать остановки на подобных пограничных комбинациях, существующих для любого обучающего примера.

Это *очень важно*. Нейронные сети настолько гибкие, что могут находить много, много разных комбинаций весов, которые позволяют получить верный про-

гноз для подмножества обучающих данных. Если обучить сеть на первых двух примерах, она, скорее всего, прекратит обучение в точке, которая не позволит получить хороший прогноз для других обучающих примеров. По сути, она просто запомнит два обучающих примера, вместо того чтобы искать *корреляцию*, *обобщающую* любые возможные комбинации сигналов светофора.

Если обучить сеть на первых двух примерах, и она обнаружит только эти два пограничных случая, она не сможет сказать вам, можно ли переходить улицу, увидев комбинацию сигналов светофора, отсутствующую в обучающих данных.

### ВАЖНОЕ ЗАМЕЧАНИЕ

Самая большая сложность, с которой вы столкнетесь в глубоком обучении, — убедить свою нейронную сеть *обобщить* данные, а не просто *запомнить* их. Вы увидите подобное еще не раз.

## Пограничный случай: конфликт давлений

Иногда корреляция вступает в противоречие сама с собой

Взгляните на правый столбец в таблице «Давление на вес». Что вы видите?

Здесь одинаковое количество моментов давления вверх и вниз. Но сеть правильно опустила этот (правый) вес до 0, а это значит, что понижающее давление превысило повышающее. Как такое возможно?

| Обучающие данные |   |   |   |   | Давление на вес |   |   |   |   |
|------------------|---|---|---|---|-----------------|---|---|---|---|
| 1                | 0 | 1 | → | 0 | −               | 0 | − | → | 0 |
| 0                | 1 | 1 | → | 1 | 0               | + | + | → | 1 |
| 0                | 0 | 1 | → | 0 | 0               | 0 | − | → | 0 |
| 1                | 1 | 1 | → | 1 | +               | + | + | → | 1 |
| 0                | 1 | 1 | → | 1 | 0               | + | + | → | 1 |
| 1                | 0 | 1 | → | 0 | −               | 0 | − | → | 0 |



Веса слева и в середине обеспечивают достаточно мощный сигнал, чтобы обеспечить схождение. Вес слева падает до 0, а вес в середине поднимается до 1. По мере того как вес в середине растет все выше и выше, ошибка для положительных примеров уменьшается. Но с приближением весов к оптимальным значениям отсутствие корреляции для веса справа становится все более очевидным.

Рассмотрим крайний случай, когда веса слева и в середине изначально имеют идеальные значения 0 и 1 соответственно. Что в этом случае произойдет с сетью? Если вес справа будет иметь значение выше 0, сеть предскажет слишком высокий результат; а если ниже 0, сеть предскажет слишком низкий результат.

В ходе обучения другие узлы поглощают часть ошибки и, соответственно, поглощают часть корреляции. Они заставляют сеть генерировать прогноз с *умеренной* степенью корреляции, что уменьшает ошибку. Другие веса будут пытаться скорректировать свое значение, чтобы правильно предсказать то, что осталось.

В данном случае вес в середине дает достаточно согласованный сигнал, чтобы поглотить всю корреляцию (потому что между средним входом и выходом наблюдается прямая связь). В результате ошибка предсказания, равного 1, становится очень маленькой, а ошибка предсказания, равного 0, — большой, что толкает вес в середине вниз.

## Так случается не всегда

Нам, в некотором смысле, повезло. Если бы вес в середине не имел идеальной корреляции с результатом, сеть могла бы попытаться максимально заглушить вес справа. Позже вы познакомитесь с *регуляризацией* — приемом, который заставляет веса с противоречивым давлением опускаться в сторону 0.

Забегая вперед, отмечу главное достоинство регуляризации. Если вес испытывает одинаковое давление вверх и вниз, это не приведет ни к чему хорошему. Это вообще никак не поможет. Фактически, регуляризация стремится сохранить влияние только весов с действительно сильной корреляцией, влияние всех остальных весов должно быть сведено к минимуму, потому что они вносят помехи. Это похоже на естественный отбор и, как побочный эффект, приводит к ускорению обучения нейронной сети (за меньшее число итераций), потому что вес справа страдает проблемой одновременного наличия положительного и отрицательного давления.

В данном случае, поскольку вес справа не имеет выраженной корреляции с выходом, сеть немедленно начнет опускать его до 0. Без регуляризации (как мы делали это прежде) вы не узнаете, что входное значение справа бесполезно,

пока сеть не выявит закономерность, имеющую отношение к входным значениям слева и в середине. Подробнее об этом мы поговорим позже.

Если сеть ищет корреляцию между входными и выходными данными, тогда как она поступит, увидев следующий набор данных?

| Обучающие данные |   |   |   |   | Давление на вес |   |   |   |   |
|------------------|---|---|---|---|-----------------|---|---|---|---|
| 1                | 0 | 1 | → | 1 | +               | 0 | + | → | 1 |
| 0                | 1 | 1 | → | 1 | 0               | + | + | → | 1 |
| 0                | 0 | 1 | → | 0 | 0               | 0 | - | → | 0 |
| 1                | 1 | 1 | → | 0 | -               | - | - | → | 0 |

Здесь отсутствует корреляция между входными и выходными данными. Каждый вес испытывает одинаковое количество моментов давления вверх и вниз. *Этот набор данных станет настоящей проблемой для нейронной сети.*

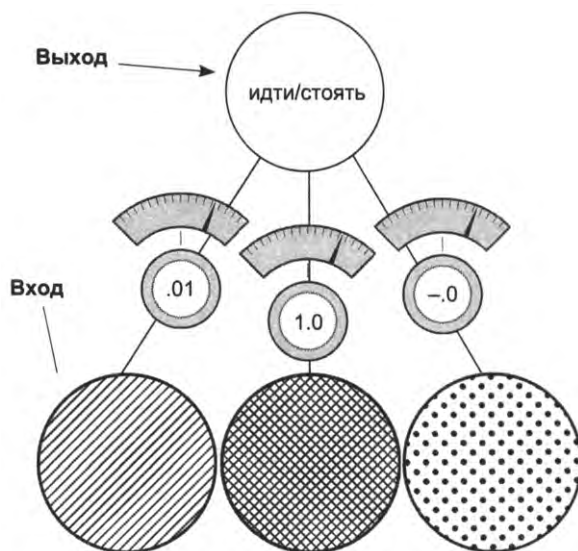
В предыдущих примерах у нас была возможность отсеять входные точки данных, испытывающие равное давление вверх и вниз, благодаря чему другие начинали оказывать более существенное влияние на положительные или отрицательные прогнозы, перетягивая сбалансированные узлы вверх или вниз. Но в данном случае все входы в равной степени подвержены давлению вверх и вниз. И что мы получаем в результате?

## Определение косвенной корреляции

Если в ваших данных отсутствует явная корреляция, сгенерируйте промежуточные данные, в которых такая корреляция имеется!

Выше я описал нейронную сеть как инструмент, который ищет корреляцию между входным и выходным наборами данных. Теперь я хочу немного уточнить это описание. В действительности нейронные сети ищут корреляцию между своими входным и выходным *слоями*.

Вы определяете отдельные строки из входных данных как значения входного слоя и пытаетесь обучить сеть так, чтобы ее выходной слой совпал с выходным набором данных. Как ни странно, но нейронная сеть ничего не знает о данных. Она просто ищет корреляцию между входным и выходным слоями.

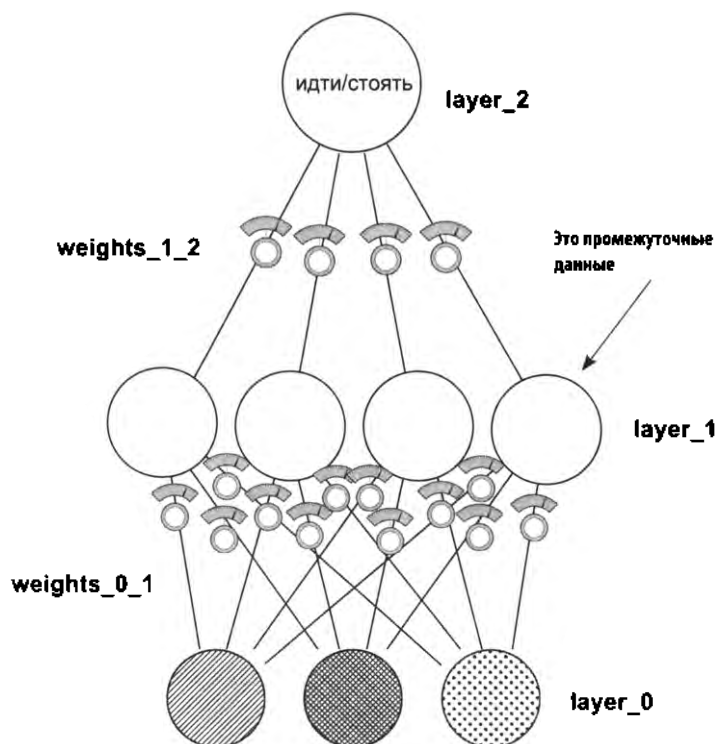


К сожалению, наш новый набор данных с комбинациями сигналов светофора *не имеет явной корреляции* между входами и выходами. Решение этой проблемы выглядит просто: использовать две такие сети. Первая создаст промежуточный набор данных, имеющий ограниченную корреляцию с выходными данными, а вторая использует эту ограниченную корреляцию для правильного прогнозирования результата.

Поскольку входной набор данных не коррелирует с выходным, мы используем входной набор для создания промежуточного набора, имеющего корреляцию с выходным набором. Этот прием чем-то напоминает подтасовку.

## Создание корреляции

На рисунке ниже изображена наша новая нейронная сеть. Фактически она является комбинацией двух нейронных сетей, наложенных друг на друга. Средний слой узлов (*layer\_1*) представляет *промежуточный набор данных*. Наша цель — обучить эту сеть так, чтобы даже в отсутствие явной корреляции между входным и выходным наборами данных (*layer\_0* и *layer\_2*) набор *layer\_1*, созданный из набора *layer\_0*, имел корреляцию с набором *layer\_2*.

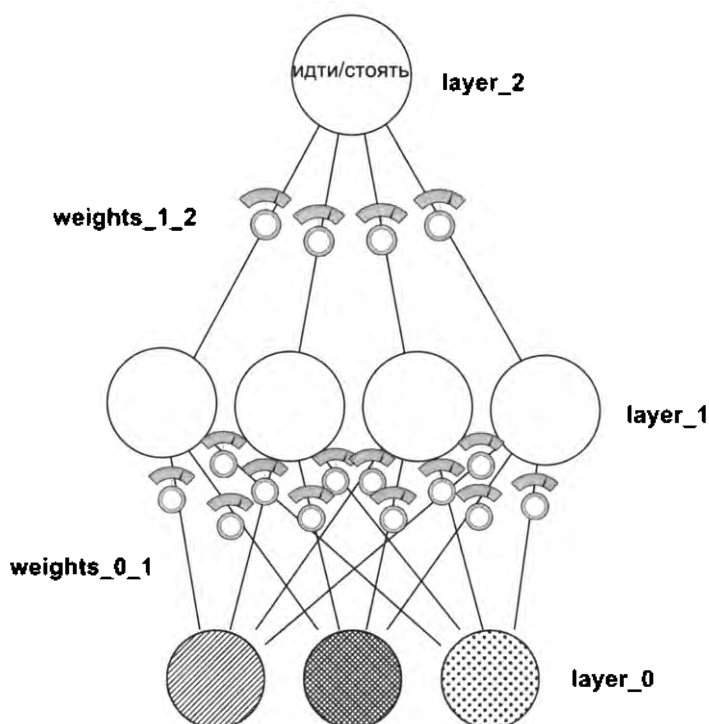


Обратите внимание, что эта сеть все еще остается функцией. Она имеет комбинацию весов, которые объединяются определенным способом. Кроме того, она способна использовать алгоритм градиентного спуска, потому что есть возможность вычислить вклад каждого веса в ошибку и скорректировать его для уменьшения ошибки. Созданием этой сети мы сейчас и займемся.

## Объединение нейронных сетей в стек: обзор

В главе 3 кратко упоминалась возможность комбинирования нейронных сетей. Посмотрим, как это делается

Архитектура, изображенная на следующем рисунке, вычисляет прогноз в точном соответствии с моими словами: «Объединение нейронных сетей в стек». Выход первой, нижней сети (преобразующей **layer\_0** в **layer\_1**) служит входом для второй, верхней сети (преобразующей **layer\_1** в **layer\_2**). Каждая сеть действует ровно так, как описывалось выше.



Поразмыслив немного над тем, как обучается эта сеть, вы обнаружите, что многое вам уже известно. Если оставить в стороне вопрос, как получаются веса нижней сети, и просто рассматривать их как обучающий набор данных, тогда становится очевидным, что верхняя половина нейронной сети (преобразующая `layer_1` в `layer_2`) ничем не отличается от сетей, которые мы обучали выше в этой главе. Мы можем использовать ровно ту же логику обучения.

Единственное, чего вы пока не знаете, — это как корректировать веса между `layer_0` и `layer_1`. Как в этом случае измеряется ошибка? Как рассказывалось в главе 5, хранимая/нормализованная ошибка называется разностью (*delta*). В данном случае мы должны выяснить, как узнать разность значений в `layer_1`, чтобы помочь `layer_2` получить точный прогноз.

## Обратное распространение: определение причин ошибок на расстоянии

### Взвешенная средняя ошибка

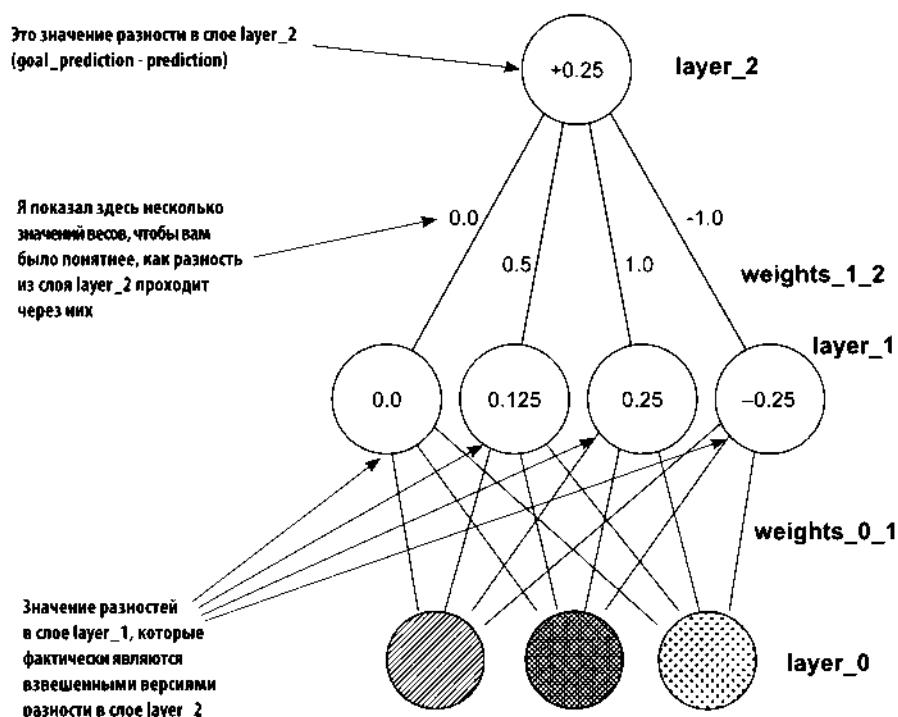
Что прогнозирует сеть, преобразующая `layer_1` в `layer_2`? Это взвешенное среднее значений в `layer_1`. Если `layer_2` превысит истинное значение на ве-

личину  $x$ , как узнать, какое из значений в `layer_1` обусловило ошибку? Ответ прост: наибольший вклад в ошибку внесли самые *высокие веса* (`weights_1_2`). Веса с *меньшими значениями* соответственно внесли меньший вклад.

Рассмотрим крайний случай. Допустим, вес слева между `layer_1` и `layer_2` равен нулю. Какая доля ошибки обусловлена этим узлом в `layer_1`? *Никакая*.

Все до смешного просто. Веса между `layer_1` и `layer_2` в точности описывают вклад каждого узла в `layer_1` в прогноз `layer_2`. Это также означает, что эти веса в точности описывают вклад каждого узла `layer_1` в ошибку `layer_2`.

И как можно использовать разность в `layer_2` для определения разности в `layer_1`? Ее нужно умножить на каждый соответствующий вес в `layer_1`. Это как логика прогнозирования, только наоборот. Этот процесс обратной передачи сигнала *delta* называется *обратным распространением*.



## Обратное распространение: как это работает?

### Взвешенное среднее разности

В нейронной сети, созданной в главе 5, переменная `delta` определяла направление и величину корректировки данного узла. Прием обратного распространения позволяет нам сказать: «Эй! Если вы хотите, чтобы значение этого узла стало на  $x$  больше, тогда каждый из предыдущих узлов следует увеличить/уменьшить на  $x * \text{weights\_1\_2}$ , потому что эти веса увеличивают прогноз в `weights\_1\_2` раз».

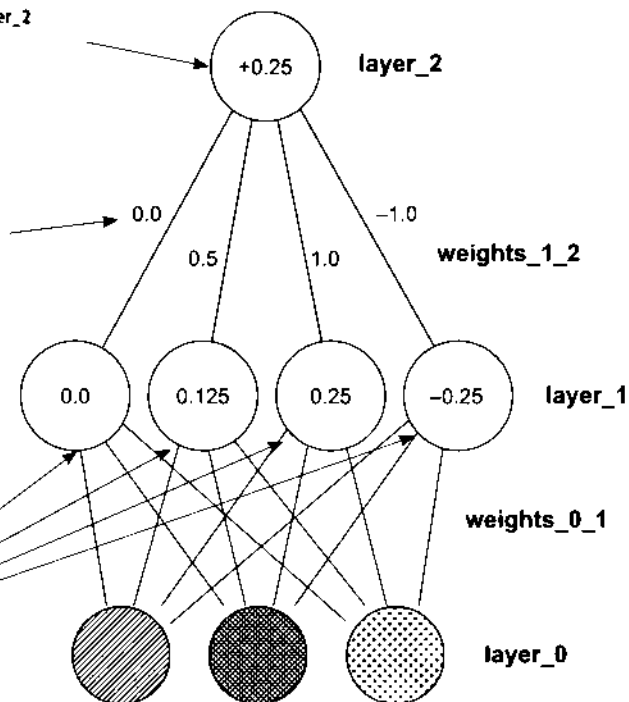
При применении в обратном направлении матрица `weights\_1\_2` усиливает ошибку на соответствующую величину. А поскольку ошибка усиливается, мы можем узнать, на какую величину вверх или вниз следует скорректировать каждый узел в `layer\_1`.

Узнав это, можно скорректировать каждый вес в матрице, как мы это уже делали раньше. Нужно умножить выходную разность на входное значение и скорректировать соответствующий вес (также можно применить масштабный альфа-коэффициент).

Это значение разности в слое `layer\_2`  
(`goal\_prediction - prediction`)

Я показал здесь несколько значений весов, чтобы вам было понятнее, как разность из слоя `layer\_2` проходит через них

Значение разностей в слое `layer\_1`, которые фактически являются взвешенными версиями разности в слое `layer\_2`



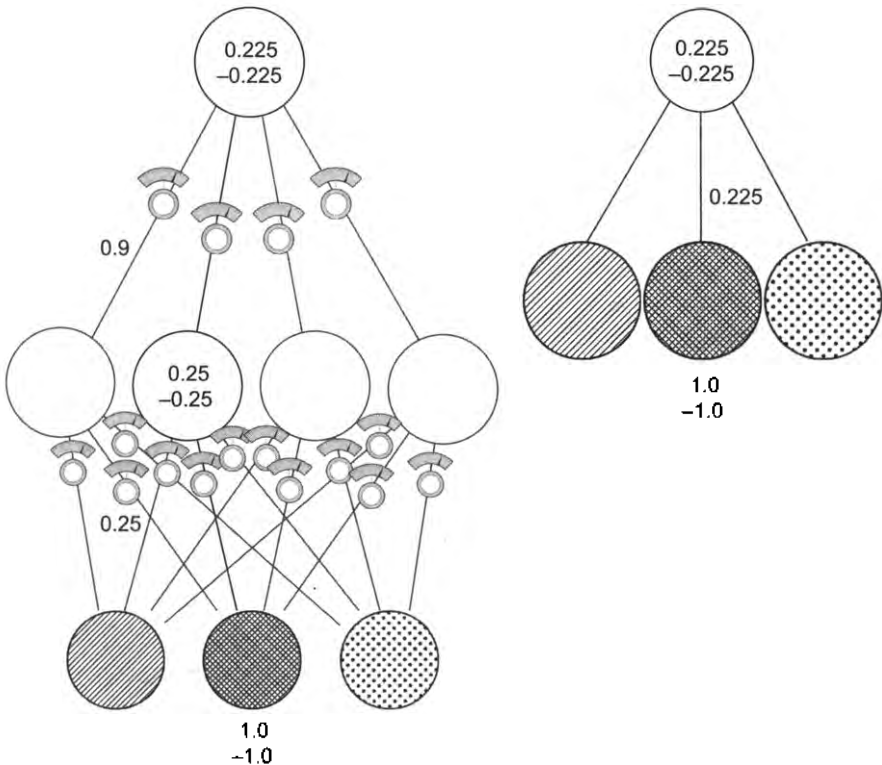
# Линейность и нелинейность

Это, пожалуй, самая сложная тема в книге.  
Не будем спешить здесь

Я хочу показать вам одно интересное явление. Как оказывается, нам не хватает еще кое-чего, чтобы получить законченный поезд из нейронных сетей. Рассмотрим эту проблему с двух точек зрения. Сначала я покажу, почему нейронная сеть не может обучаться без этого недостающего звена. То есть я покажу, почему нейронная сеть в текущем ее виде неработоспособна. Затем, когда мы добавим это звено, я покажу, как исправить проблему. А сейчас взгляните на следующие простые выражения:

$$\begin{array}{ll} 1 * 10 * 2 = 100 & 1 * 0.25 * 0.9 = 0.225 \\ 5 * 20 = 100 & 1 * 0.225 = 0.225 \end{array}$$

Отсюда следует вывод: любое выражение, состоящее из двух умножений, можно переписать в форме с одним умножением. Как оказывается, в этом нет ничего хорошего для нас. Взгляните на следующий рисунок:





Здесь изображены две сети, обучающиеся на двух примерах — один с входным значением 1.0 и другой со значением -1.0. Вывод: *для любой трехслойной сети существует двухслойная сеть с идентичным поведением*. Простое наложение двух нейронных сетей друг на друга (как мы сделали это выше) не дает никаких преимуществ. Последовательное вычисление двух взвешенных сумм — это всего лишь более дорогая версия вычисления одной взвешенной суммы.

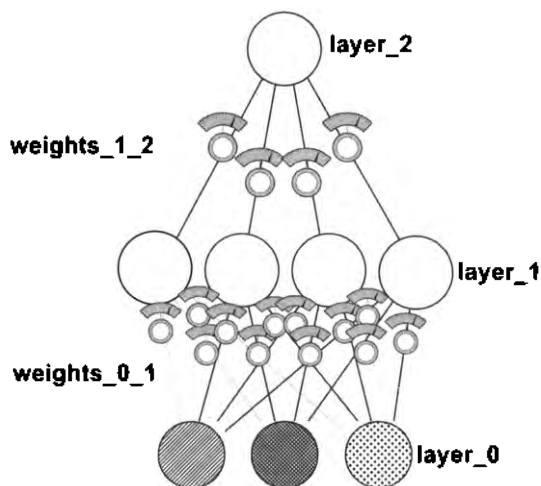
## Почему составная нейронная сеть не работает

Если попробовать обучить трехслойную нейронную сеть в текущем ее виде, она не сойдется

**Проблема:** для любых двух взвешенных сумм входов, вычисляемых последовательно, существует единственная взвешенная сумма, имеющая точно такой же результат. Все, что может трехслойная сеть, может и двухслойная.

Прежде чем устранить проблему, поговорим о слое в середине (`layer_1`). Прямо сейчас каждый узел (из четырех) имеет вес, определяемый каждым из входов. Поразмыслим об этом с точки зрения корреляции. Каждый узел в среднем уровне объясняет определенную долю корреляции с каждым входным узлом. Если вес между входным слоем и слоем в середине равен 1.0, значит, узел в среднем слое объясняет все 100 % изменений значения входного узла. Если входной узел увеличит значение на 0.3, узел в среднем слое последует за ним.

Если вес, связывающий два узла, равен 0.5, значит, узел в среднем слое объясняет только 50 % изменений значения входного узла.



Единственная возможность для узла в среднем слое нарушить корреляцию от одного конкретного входного узла — образовать дополнительную зависимость от другого входного узла. В такую сеть *не вносится ничего нового*. Для каждого узла в скрытом (среднем) слое организуется небольшая зависимость от других входных узлов.

Узлы среднего слоя не добавляют в поток данных никакой новой информации; они не имеют собственной корреляции. Они просто в большей или меньшей степени коррелируют с входными узлами.

Но чем может помочь средний слой, если в новом наборе данных, который мы сейчас рассматриваем, отсутствует корреляция между любыми входами и выходами? Он же просто перемешивает и без того бесполезные корреляции. Все немного не так, на самом деле нам нужно, чтобы средний слой мог выборочно устанавливать связи с входным слоем.

Нам нужно, чтобы узлы в среднем слое *иногда могли зависеть* от узлов во входном слое, а *иногда нет*. Это поможет им создать свои корреляции. Это даст среднему слою возможность избавиться от прямой зависимости узла  $x\%$  от одного входа, а узла  $y\%$  — от другого. Вместо этого узел  $x\%$  будет зависеть от конкретного входа, только когда пожелает этого, а в других случаях вообще не будет коррелировать с ним. Это называется *условной, или эпизодической, корреляцией*.

## Тайна эпизодической корреляции

### Отключение узла, когда значение оказывается ниже 0

Рассмотрим следующую простую ситуацию: когда значение узла падает ниже 0, его корреляция с входным узлом останется прежней, просто так получилось, что значение оказалось отрицательным. Но если отключить узел (присвоить ему 0), когда тот станет отрицательным, *его корреляция с любыми входными узлами станет нулевой*.

Что это значит? Теперь узел может выбирать, с каким другим входным узлом устанавливать корреляцию. Это позволит нам выразить, например, такое требование: «Образовать прямую корреляцию с левым входом, но только если правый выключен». И что это даст? А вот что: если вес для левого входа равен 1.0, а вес для правого входа имеет большое отрицательное значение, тогда включение обоих входов, левого и правого, может привести к тому, что узел, зависящий от них, всегда будет иметь значение 0. Но если оставить включенным только левый вход, узел будет получать значение только от левого входа.

В предыдущей сети такое невозможно. В ней узел среднего слоя либо постоянно связан с определенным входом, либо не связан вообще. Теперь связи могут быть условными. Теперь каждый узел может сам принимать решения.

**Решение:** отключая любой узел в среднем слое, когда тот принимает отрицательное значение, вы позволяете сети устанавливать эпизодические корреляции с разными входными узлами. Это невозможно в двухслойных нейронных сетях, но придает дополнительную мощность трехслойным сетям.

Логика «если узел становится отрицательным, присвоить ему 0» описывается забавным термином *нелинейность*. Нейронная сеть, не имеющая этой логики, называется *линейной*. Если этот прием не применяется, выходной слой формируется на основе тех же корреляций, что и в двухслойной сети. Он напрямую зависит от входного слоя, а значит, неспособен решить задачу о светофоре с новым набором данных.

Есть много способов организовать нелинейность. Но тот, что рассматривается здесь, является наиболее универсальным, а также самым простым. (Он называется *ReLU*.)

Хочу отметить, что во многих других книгах и курсах говорится, что последовательное умножение матриц является линейным преобразованием. Я считаю такое объяснение малопонятным. Оно также затрудняет понимание, что дает нелинейность и почему предпочтительнее выбрать тот или иной прием (об этом мы поговорим чуть позже). В них также говорится, что «без нелинейности произведение двух матриц может быть равно 1». Мое объяснение, пусть и не самое краткое, помогает понять, зачем нужна нелинейность.

## Короткий перерыв

**Предыдущий раздел мог показаться излишне абстрактным, и это совершенно нормально**

В предыдущих главах использовалась простая алгебра, поэтому все, о чем рассказывалось в них, было основано на простых базовых инструментах. Здесь же повествование строится на основе знаний, полученных ранее. Например, ранее вы узнали, что:

Можно вычислить отношение между ошибкой и любым из весов и определить, как изменение веса влияет на изменение ошибки. Затем эту информацию можно использовать для уменьшения ошибки до 0.

Это был *важный урок*. Но сейчас, выучив этот урок и выяснив, как все это работает, мы можем принять это утверждение на веру. Следующий важный урок мы получили в начале этой главы:

Корректировка весов для уменьшения ошибки с использованием серии обучающих примеров в общем случае сводится к поиску корреляции между входным и выходным слоями. Если корреляция отсутствует, ошибка никогда не достигнет 0.

А этот *урок еще важнее*. Фактически он означает, что предыдущий урок можно выбросить из головы. Он нам не нужен. Теперь мы сосредоточились на корреляции. Постарайтесь понять, что невозможно постоянно думать обо всем сразу. Просто поверьте в правильность каждого урока. Оказавшись перед более абстрактным обобщением каких-то уроков с мелкими деталями, просто отложите детали и сосредоточьтесь на понимании этого обобщения.

Это можно сравнить с подходом профессионального пловца, велосипедиста или другого спортсмена, которому необходим комплексный результат множества небольших уроков. Игрок в бейсбол, замахивающийся битой, прошел тысячи маленьких уроков, чтобы в итоге научиться идеально отбивать мяч. Но игрок не думает обо всех выученных тонкостях, когда выходит на площадку. Он действует практически подсознательно. То же самое верно в отношении изучения математических понятий.

Нейронные сети ищут корреляцию между входом и выходом и теперь вам не обязательно задумываться о том, как это происходит. Вы просто знаете, что это так. Теперь мы возьмем эту идею на вооружение, расслабимся и будем доверять всему, с чем уже познакомились.

## Ваша первая глубокая нейронная сеть

### Вот как можно получить прогноз

Следующий фрагмент инициализирует веса и реализует прямое распространение. Новый код в этом фрагменте выделен **жирным**.

```
import numpy as np

np.random.seed(1)

def relu(x):
    return (x > 0) * x
```

← Эта функция преобразует отрицательные числа в 0

```

alpha = 0.2
hidden_size = 4

streetlights = np.array( [[ 1, 0, 1 ],
                           [ 0, 1, 1 ],
                           [ 0, 0, 1 ],
                           [ 1, 1, 1 ] ] )

walk_vs_stop = np.array([[ 1, 1, 0, 0]]).T

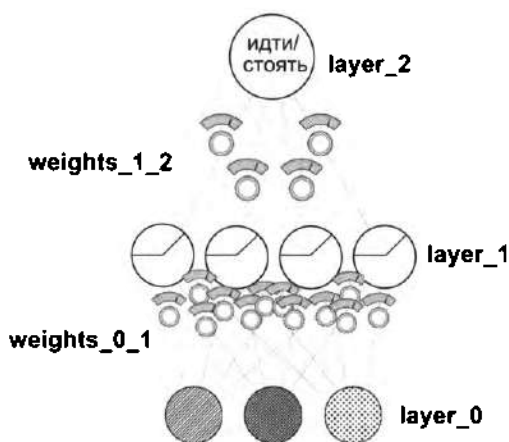
weights_0_1 = 2*np.random.random((3,hidden_size)) - 1
weights_1_2 = 2*np.random.random((hidden_size,1)) - 1

layer_0 = streetlights[0]
layer_1 = relu(np.dot(layer_0,weights_0_1))
layer_2 = np.dot(layer_1,weights_1_2)

```

Эта функция преобразует отрицательные числа в 0

Выход слоя `layer_1` пропускается через функцию `relu`, которая превращает отрицательные значения в 0. Он служит входом для следующего слоя, `layer_2`



Порядок действий этого фрагмента кода изображен на следующем рисунке. Входные данные поступают в слой `layer_0`. Посредством функции `dot` сигнал передается вверх через веса из слоя `layer_0` в слой `layer_1` (вычисляются взвешенные суммы для всех четырех узлов в слое `layer_1`). Затем взвешенные суммы из слоя `layer_1` передаются в функцию `relu`, которая преобразует отрицательные числа в 0. И далее окончательные взвешенные суммы попадают в последний слой `layer_2`.

## Обратное распространение в коде

Мы можем узнать величину вклада каждой взвешенной суммы в окончательную ошибку

В конце предыдущей главы я говорил, что очень важно запомнить код реализации двухслойной сети, чтобы вы могли быстро вспомнить его, когда я буду ссылаться на него при объяснении более сложных понятий. Теперь настал момент, когда это пригодится.

В следующем листинге представлен новый обучающий код, и очень важно, чтобы вы могли распознать в нем элементы, описанные в предыдущих главах. Если вы потеряете нить рассуждений, вернитесь к главе 5, запомните код, представленный там, и возвращайтесь сюда. Когда-нибудь это будет иметь большое значение.

```
import numpy as np

np.random.seed(1)

def relu(x):
    return (x > 0) * x  ← Возвращает x, если x > 0; иначе возвращает 0

def relu2deriv(output):
    return output > 0  ← Возвращает 1, если output > 0; иначе возвращает 0

alpha = 0.2
hidden_size = 4

weights_0_1 = 2*np.random.random((3,hidden_size)) - 1
weights_1_2 = 2*np.random.random((hidden_size,1)) - 1

for iteration in range(60):
    layer_2_error = 0
    for i in range(len(streetlights)):
        layer_0 = streetlights[i:i+1]
        layer_1 = relu(np.dot(layer_0,weights_0_1))
        layer_2 = np.dot(layer_1,weights_1_2)

        layer_2_error += np.sum((layer_2 - walk_vs_stop[i:i+1]) ** 2)

        layer_2_delta = (walk_vs_stop[i:i+1] - layer_2)
        layer_1_delta=layer_2_delta.dot(weights_1_2.T)*relu2deriv(layer_1)  ←

        weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
        weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)

    if(iteration % 10 == 9):
        print("Error:" + str(layer_2_error))
```

Эта строка вычисляет разность в слое layer\_1 с учетом разности в слое layer\_2, умножая layer\_2\_delta на соответствующие веса weights\_1\_2

Хотите верьте, хотите нет, но новые в этом листинге только три строки кода, выделенные жирным. Весь остальной код практически остался прежним, как на предыдущих страницах. Функция `relu2deriv` возвращает 1, когда `output > 0`; иначе она возвращает 0. Это *наклон (производная)* функции `relu`. Она служит важной цели, в чем вы убедитесь далее.

Как вы помните, главная наша цель — *выяснение причин ошибки*. То есть мы должны выяснить, какой вклад вносит каждый вес в окончательную ошибку. В первой (двухслойной) нейронной сети мы вычисляли переменную *delta*, которая сообщала нам, насколько выше или ниже должен быть прогноз. Теперь

взгляните на предыдущий код. Именно так мы вычисляем `layer_2_delta`. Ничего нового. (И снова, если вы забыли, как работает эта часть, вернитесь к главе 5.)

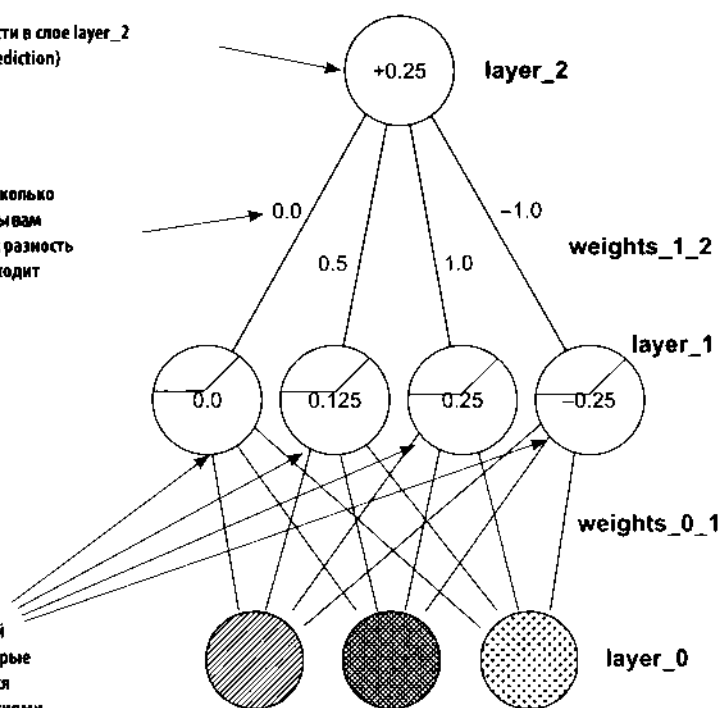
Теперь, когда известно, насколько вверх или вниз нужно скорректировать прогноз (`delta`), нам нужно выяснить, насколько вверх или вниз нужно скорректировать каждый узел в среднем слое (`layer_1`). Фактически это *промежуточные прогнозы*. Вычислив `delta` для `layer_1`, мы сможем повторить тот же процесс для вычисления приращений весов (умножить входные значения на разность `delta` и скорректировать соответствующие веса на полученные значения).

Но как вычислить разности для `layer_1`? Сначала сделаем очевидное: умножим разность `delta` на выходе на каждый вес, обусловивший ее. В результате мы узнаем вклад каждого веса в эту ошибку. Есть еще кое-что, что необходимо учесть. Если `relu` присвоила 0 выходному узлу слоя `layer_1`, значит, он не имеет вклада в ошибку. В такой ситуации разность `delta` для этого узла так же надо установить в 0. Это обеспечивает умножение каждого узла в слое `layer_1` на функцию `relu2deriv`, которая возвращает 1 или 0, в зависимости от значения узла в `layer_1`.

Это значение разности в слое `layer_2`  
(`goal_prediction - prediction`)

Я показал здесь несколько значений весов, чтобы вам было понятнее, как разность из слоя `layer_2` проходит через них

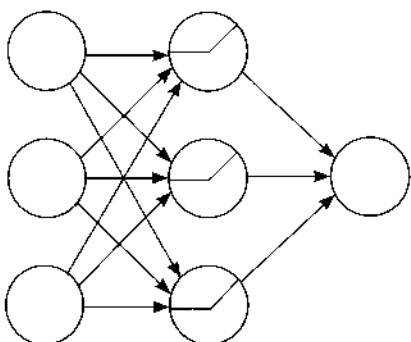
Значение разностей в слое `layer_1`, которые фактически являются взвешенными версиями разности в слое `layer_2`



# Одна итерация обратного распространения

## 1. Инициализация весов и данных сети

Входы    Скрытый слой    Прогноз



```
import numpy as np
np.random.seed(1)

def relu(x):
    return (x > 0) * x

def relu2deriv(output):
    return output>0

lights = np.array( [[ 1, 0, 1 ],
                    [ 0, 1, 1 ],
                    [ 0, 0, 1 ],
                    [ 1, 1, 1 ] ])

walk_stop = np.array([[ 1, 1, 0, 0]]).T

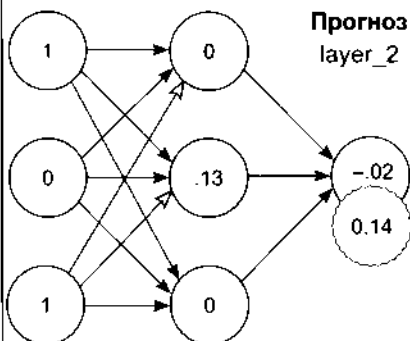
alpha = 0.2
hidden_size = 3

weights_0_1 = 2*np.random.random(\
    (3,hidden_size)) - 1
weights_1_2 = 2*np.random.random(\
    (hidden_size,1)) - 1
```

## 2. ПРОГНОЗ + СРАВНЕНИЕ: получение прогноза, вычисление ошибки и разности на выходе

Входы    Скрытый слой

layer\_0    layer\_1



```
layer_0 = lights[0:1]
layer_1 = np.dot(layer_0,weights_0_1)
layer_1 = relu(layer_1)
layer_2 = np.dot(layer_1,weights_1_2)

error = (layer_2-walk_stop[0:1])**2
layer_2_delta=(layer_2-walk_stop[0:1])
```



## 3. ОБУЧЕНИЕ: обратное распространение из layer\_2 в layer\_1

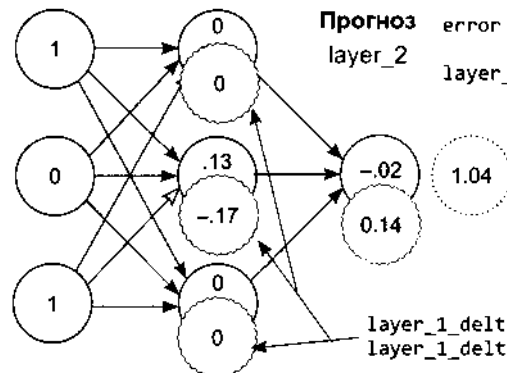
Входы Скрытый слой

layer\_0

layer\_1

Прогноз

layer\_2



```

layer_0 = lights[0:1]
layer_1 = np.dot(layer_0, weights_0_1)
layer_1 = relu(layer_1)
layer_2 = np.dot(layer_1, weights_1_2)

error = (layer_2 - walk_stop[0:1])**2
layer_2_delta = (layer_2 - walk_stop[0:1])

```

```

layer_1_delta = layer_2_delta.dot(weights_1_2.T)
layer_1_delta *= relu2deriv(layer_1)

```

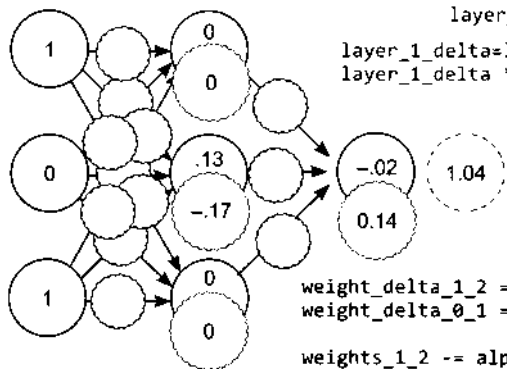
## 4. ОБУЧЕНИЕ: вычисление приращений и корректировка весов

Входы Скрытый слой Прогноз

layer\_0

layer\_1

layer\_2



```

layer_0 = lights[0:1]
layer_1 = np.dot(layer_0, weights_0_1)
layer_1 = relu(layer_1)
layer_2 = np.dot(layer_1, weights_1_2)
error = (layer_2 - walk_stop[0:1])**2
layer_2_delta = (layer_2 - walk_stop[0:1])

```

```

layer_1_delta = layer_2_delta.dot(weights_1_2.T)
layer_1_delta *= relu2deriv(layer_1)

```

```

weight_delta_1_2 = layer_1.T.dot(layer_2_delta)
weight_delta_0_1 = layer_0.T.dot(layer_1_delta)

weights_1_2 -= alpha * weight_delta_1_2
weights_0_1 -= alpha * weight_delta_0_1

```

Как видите, суть обратного распространения заключается в вычислении разностей  $\delta$  для промежуточных слоев, чтобы обеспечить возможность градиентного спуска. Для этого мы получаем средневзвешенную разность между

layer\_2 и layer\_1 (взвешенную весами между ними). Затем выключаем (устанавливаем в 0) узлы, не участвовавшие в прямом прогнозировании, то есть не внесшие вклада в ошибку.

## Объединяем все вместе

Вот пример готовой программы, которую вы можете запустить (результат ее работы приводится ниже)

```
import numpy as np

np.random.seed(1)

def relu(x):
    return (x > 0) * x  ← Возвращает x, если x > 0; иначе возвращает 0

def relu2deriv(output):
    return output > 0  ← Возвращает 1, если output > 0; иначе возвращает 0

streetlights = np.array( [[ 1, 0, 1 ],
                           [ 0, 1, 1 ],
                           [ 0, 0, 1 ],
                           [ 1, 1, 1 ] ] )

walk_vs_stop = np.array([[ 1, 1, 0, 0]]).T

alpha = 0.2
hidden_size = 4

weights_0_1 = 2*np.random.random((3,hidden_size)) - 1
weights_1_2 = 2*np.random.random((hidden_size,1)) - 1

for iteration in range(60):
    layer_2_error = 0
    for i in range(len(streetlights)):
        layer_0 = streetlights[i:i+1]
        layer_1 = relu(np.dot(layer_0,weights_0_1))
        layer_2 = np.dot(layer_1,weights_1_2)

        layer_2_error += np.sum((layer_2 - walk_vs_stop[i:i+1]) ** 2)

    layer_2_delta = (layer_2 - walk_vs_stop[i:i+1])
    layer_1_delta=layer_2_delta.dot(weights_1_2.T)*relu2deriv(layer_1)

    weights_1_2 -= alpha * layer_1.T.dot(layer_2_delta)
    weights_0_1 -= alpha * layer_0.T.dot(layer_1_delta)

if(iteration % 10 == 9):
    print("Error:" + str(layer_2_error))
```

```
Error:0.634231159844
Error:0.358384076763
Error:0.0830183113303
Error:0.0064670549571
Error:0.000329266900075
Error:1.50556226651e-05
```

## Почему глубокие сети важны для нас?

**Зачем генерировать «промежуточные наборы данных», имеющие корреляцию?**

Взгляните на изображение кошки ниже. Представьте теперь, что у меня есть комплект изображений с кошками и без кошек (и я подписал их соответственно). Если бы я захотел обучить нейросеть принимать пиксели изображений и определять наличие или отсутствие кошки на изображении, двухслойная сеть едва ли справилась бы с этой задачей.

Так же как в последнем наборе данных с наблюдениями за светофором, никакой отдельно взятый пиксел не коррелирует с фактом наличия кошки на изображении. Наличие кошки можно определить только по разным совокупностям пикселей.



В этом заключена суть глубокого обучения. Глубокое обучение основано на создании промежуточных слоев (наборов данных), в которых каждый узел определяет наличие или отсутствие разных комбинаций на входе.

В наборе изображений с кошками никакой отдельно взятый пиксел не коррелирует с присутствием кошки на фотографии. И в этом случае средний слой будет пытаться выявить разные комбинации пикселей, которые могут коррелировать с изображением кошки (такие, как ухо, глаз или шерсть кошки). Присутствие

множества комбинаций, говорящих в пользу присутствия кошки, даст последнему слою достаточно информации, чтобы правильно определить присутствие или отсутствие кошки на изображении.

Хотите верьте, хотите нет, но три слоя — это не предел. Вы можете добавить в нейронную сеть дополнительные новые слои. Некоторые нейронные сети насчитывают сотни слоев, каждый узел в которых играет свою роль в обнаружении разных комбинаций входных данных. В оставшейся части книги мы много времени уделим изучению разных явлений, имеющих место в этих слоях, пытаясь понять всю мощь глубоких нейронных сетей.

Заканчивая эту главу, я хочу обратиться к вам с той же просьбой, что и в конце главы 5: запомните последний код. Вы должны подробно разобрать каждую операцию, чтобы лучше понимать, о чем будет рассказываться в следующих главах. Остановитесь на время и не переворачивайте страницу, пока не сможете воспроизвести трехслойную сеть по памяти.

# 7

## Как изобразить нейронную сеть: в голове и на бумаге



### В этой главе

- ✓ Обобщение корреляции.
- ✓ Упрощенная визуализация.
- ✓ Интерпретация прогноза сети.
- ✓ Визуализация с использованием букв вместо картинок.
- ✓ Связывание переменных.
- ✓ Важность инструментов визуализации.

Числа могут рассказать много важного. Но только вы можете придать их рассказу ясность и убедительность.

*Стивен Фью (Stephen Few),  
новатор в области информационных технологий,  
преподаватель и консультант*

## Время упрощать

Невозможно постоянно помнить обо всем. Помочь в этом могут инструменты визуализации

Глава 6 закончилась весьма выразительным примером кода. Нейронная сеть содержала 35 строк невероятно емкого кода. Прочитав его, становится ясно, насколько много всего в нем воплощено; этот код включает более 100 страниц с описанием идей и понятий, которые в совокупности могут подсказать, безопасно ли перейти улицу по тому или иному сигналу светофора.

Я надеюсь, вы сможете запоминать и восстанавливать по памяти примеры и в следующих главах. По мере расширения примеров вы все реже будете запоминать конкретный код и все чаще — идеи, лежащие в его основе, и затем восстанавливать код по этим идеям.

Именно о таком эффективном запоминании идей я хочу поговорить в этой главе. Этот рассказ, даже при том, что он не описывает архитектуру или эксперимент, является, пожалуй, самой важной ценностью, которой я хочу поделиться с вами. Здесь я покажу, как сам обобщаю маленькие уроки, чтобы потом на основе этих обобщений конструировать и отлаживать новые архитектуры и использовать их для решения новых задач на основе новых наборов данных.

Начнем с обзора понятий, которые мы узнали к настоящему моменту

Эта книга началась с рассмотрения нескольких простых понятий, поверх которых потом накладывались слои абстракции. Сначала мы поговорили об идеях машинного обучения в целом. Затем посмотрели, как происходит обучение отдельных линейных узлов (или *нейронов*), за которыми последовали горизонтальные группы нейронов (слои) и вертикальные группы слоев (стеки). Попутно мы узнали, что процесс обучения фактически сводится к уменьшению ошибки, и познакомились с вычислительными методами, которые позволяют скорректировать каждый вес в сети с целью сдвинуть ошибку в направлении 0.

Далее мы обсудили, как нейронные сети выполняют поиск (а иногда и создают) корреляции между наборами данных на входе и выходе. Знакомство с этой последней идеей позволило нам подняться над деталями поведения

отдельных нейронов, потому что она обобщает все предыдущие уроки. Обобщение всех этих нейронов, градиентов, стеков, слоев и прочих тонкостей привело нас к единственной идее: нейронные сети обнаруживают и создают корреляцию.

Для дальнейшего изучения глубокого обучения намного важнее запомнить эту идею корреляции, чем более мелкие детали, описанные ранее. Иначе вам будет нелегко овладеть ошеломляющей сложностью нейронных сетей. Давайте дадим этой идее новое название: *обобщение корреляции*.

## Обобщение корреляции

Это ключ к уверенному овладению еще более сложными нейронными сетями

### ОБОБЩЕНИЕ КОРРЕЛЯЦИИ

Нейронные сети стремятся отыскать прямую и косвенную корреляцию между входным и выходным слоями, которые определяются входным и выходным наборами данных соответственно.

В общем и целом, так действуют все нейронные сети. Нейронная сеть — это всего лишь группа матриц, связанных слоями. А теперь выберем масштаб крупнее и посмотрим, что делает любая весовая матрица.

### ОБОБЩЕНИЕ ЛОКАЛЬНОЙ КОРРЕЛЯЦИИ

Любой данный набор весов оптимизируется в процессе обучения так, чтобы с его помощью из входных данных можно было получить то, что должно быть на выходе.

Когда имеется всего два слоя (входной и выходной), нейронная сеть легко может определить веса в весовой матрице, опираясь на выходной слой и набор данных, который должен получаться на выходе. Она ищет корреляцию между входным и выходным наборами данных, потому что они представлены входным и выходным слоями. Но в случае с большим числом слоев задача начинает обрстать нюансами, помните?

### ОБОБЩЕНИЕ ГЛОБАЛЬНОЙ КОРРЕЛЯЦИИ

Определить, какие значения должны получаться в предыдущем слое, можно по значениям, которые должны получаться в следующем слое, умножив выход следующего слоя на матрицу весов между слоями. Таким способом последующие слои могут сообщать предыдущим, какой сигнал им нужен, чтобы в итоге выявить корреляцию с выходом. Такое перекрестное взаимовлияние называют *обратным распространением*.

Глобальная корреляция сообщает каждому слою, каким он должен быть, а локальная корреляция оптимизирует веса локально. Когда нейрон в последнем слое обнаруживает, что его значение должно быть чуть больше, он сообщает всем нейронам в предыдущем слое: «Эй, там, в предыдущем слое, пришлите мне более высокий сигнал». Те, в свою очередь, сообщают нейронам в предшествующем им слое: «Эй! Пришлите нам более высокий сигнал». Это напоминает игру в телефон — в конце игры каждый слой знает, какие нейроны должны получить большее значение, а какие меньшее, и происходит обобщение локальной корреляции в виде соответствующей корректировки весов.

## Прежняя усложненная визуализация

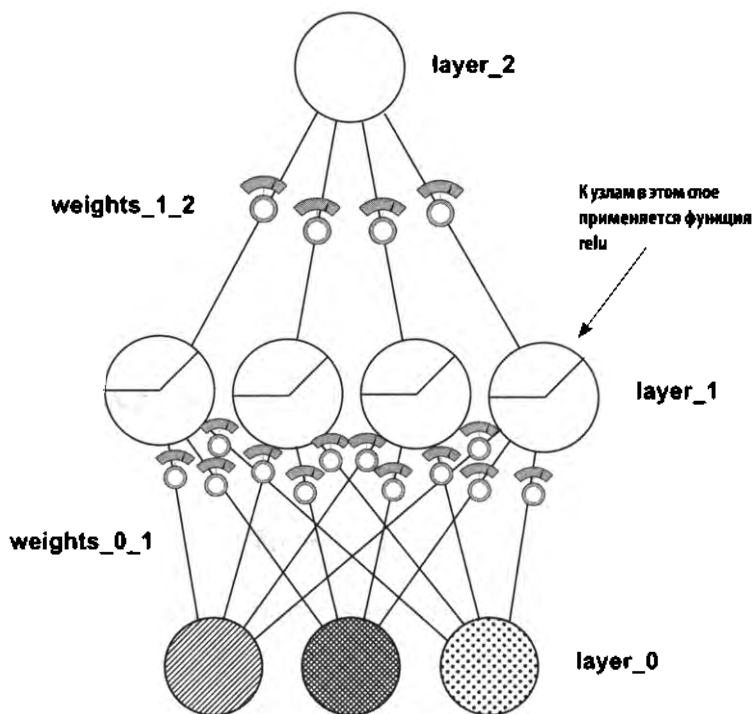
Упрощая мысленное представление, заодно упростим и визуализацию

Я полагаю, что в данный момент вы представляете себе нейронные сети, как на рисунке ниже (потому что именно так они и изображались в книге). Входной набор данных в слое `layer_0` связан посредством весовой матрицы (множество линий на рисунке) со слоем `layer_1` и так далее. Такое представление удобно было использовать, чтобы понять, как объединяются коллекции весов и слоев для достижения эффекта обучения.

Но теперь такое представление стало излишне подробным. Придя к идее обобщения корреляции, мы поняли, что можем больше не беспокоиться вопросом корректировки отдельных весов. Последующие слои уже знают, как взаимодействовать с предыдущими и сообщить им: «Эй, мне нужен более высокий сигнал», — или: «Эй, мне нужен более низкий сигнал». По правде говоря, вам вообще не нужно заботиться о значениях весов, достаточно знать, что они ведут себя должным образом, правильно фиксируя корреляцию обобщенным способом.



Чтобы отразить изменения, произошедшие в нашем представлении, изменим визуальное представление нейронных сетей на бумаге. Также сделаем еще кое-что, что обретет смысл позже. Как мы уже знаем, нейронная сеть — это серия весовых матриц. Кроме того, для представления каждого слоя мы создаем векторы.



На рисунке выше весовые матрицы — это линии, связывающие узлы, а векторы — это горизонтальные ряды узлов. Например, **weights\_1\_2** — это матрица, **weights\_0\_1** — это тоже матрица, а **layer\_1** — вектор.

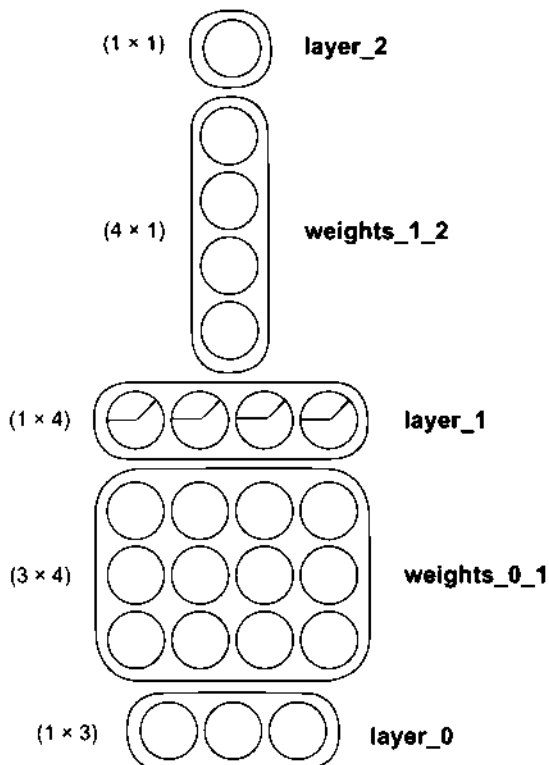
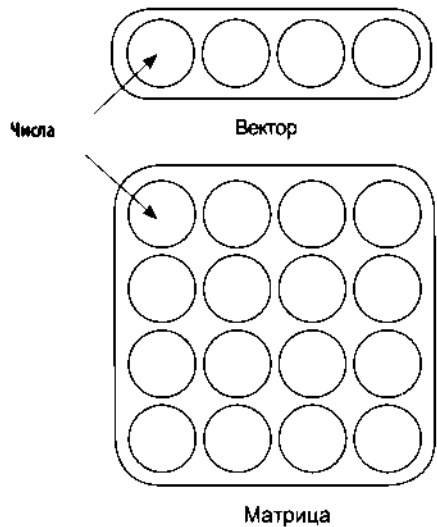
В последующих главах мы будем упорядочивать векторы и матрицы множеством разных способов, поэтому вместо такого детального представления со всеми узлами и связями между ними (которое будет трудно прочитать, если, к примеру, в слое **layer\_1** окажется 500 узлов) поразмышляем в обобщенных терминах. Представим их как векторы и матрицы произвольного размера.

## Упрощенная визуализация

Нейронные сети похожи на сооружения из блоков конструктора LEGO, где каждый блок — это вектор или матрица

Продвигаясь вперед, мы будем создавать нейронные сети с новыми архитектурами, используя детали LEGO. Самое замечательное в идее обобщения корреляции, что все ее детали (обратное распространение, градиентный спуск, альфа-коэффициент, прореживание, пакетный градиентный спуск и так далее) не зависят от конкретной конфигурации деталей LEGO. Независимо от того, как вы соедините последовательности матриц со слоями, нейронная сеть будет пытаться выявить закономерности в данных, корректируя веса между входным и выходным слоями.

Чтобы отразить это, мы будем конструировать все нейронные сети из элементов, изображенных справа. Горизонтальный ряд окружностей — это вектор, прямо-



угольный блок — это матрица, а окружности — отдельные веса и узлы. Обратите внимание, что прямоугольный блок можно интерпретировать как «вектор векторов» по горизонтали или вертикали.

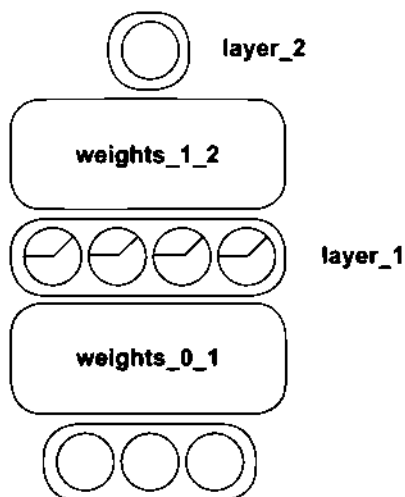
### ВАЖНОЕ ПРИМЕЧАНИЕ

Рисунок слева по-прежнему сообщает нам полную информацию о структуре нейронной сети. Здесь мы видим формы и размеры всех слоев и матриц. Так как теперь мы знакомы с идеей обобщения корреляции и всем, что с ней связано, детали, изображавшиеся на предыдущих рисунках, нам больше не нужны. Но мы еще не закончили: это представление можно упростить еще больше.

## Еще более упрощенная визуализация

### Размеры матриц определяются слоями

В предыдущем разделе вы могли заметить одну интересную закономерность. Размеры каждой матрицы (количество строк и столбцов) напрямую зависят от размеров предшествующего и последующего слоев. То есть визуальное представление можно упростить еще больше.



Взгляните на рисунок слева. Он все еще содержит всю информацию, необходимую для создания нейронной сети. По этому рисунку мы без труда можем определить, что **weights\_0\_1** — это матрица  $3 \times 4$ , потому что предыдущий слой (**layer\_0**) — это вектор с тремя элементами, а последующий слой (**layer\_1**) — вектор с четырьмя элементами. То есть матрица должна иметь такие размеры, чтобы связать каждый узел в слое **layer\_0** с каждым узлом в слое **layer\_1**, проще говоря, она должна иметь размеры  $3 \times 4$ .

Такой способ изображения позволяет нам рассуждать о нейронной сети, опираясь на идею обобщения корреляции. Эта нейронная сеть должна лишь выявить корреляцию между слоями **layer\_0** и **layer\_2** и скорректировать веса. Это будет сделано с использованием всех методов, которые рассматривались в этой книге до сих пор. Но разные комбинации весов и слоев между входным и выходным

слоями оказывают сильное влияние на успешное выявление корреляции нейронной сетью (и/или скорость выявления этой корреляции).

Конкретная комбинация слоев и весов нейронной сети называется ее *архитектурой*, и значительную долю оставшейся части этой книги мы посвятим обсуждению «за» и «против» разных архитектур. Согласно идее обобщения корреляции, нейронная сеть корректирует веса, стремясь выявить корреляцию между входным и выходным слоями, иногда даже искусственно формируя корреляцию в скрытых слоях. Разные архитектуры по-разному комбинируют сигналы, чтобы упростить выявление корреляции.

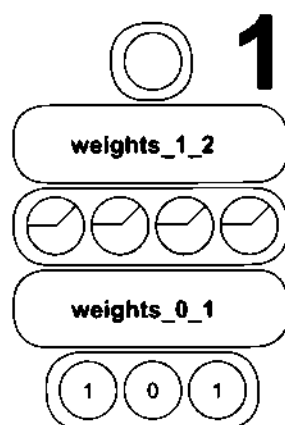
Удачные нейронные архитектуры комбинируют сигналы, стремясь упростить выявление корреляции. А более удачные архитектуры фильтруют шум, чтобы предотвратить переобучение.

Основная задача сферы исследования нейронных сетей заключается в поиске новых архитектур, способных быстрее выявлять корреляцию и лучше обобщать прежде не наблюдавшиеся данные. Значительную долю оставшейся части этой книги мы посвятим обсуждению таких новых архитектур.

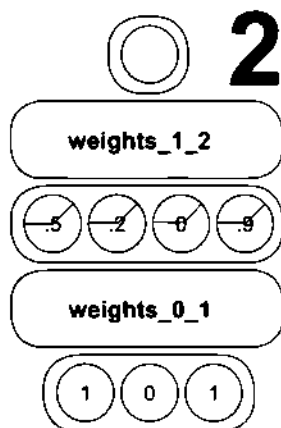
## Посмотрим, как эта сеть получает прогноз

Изобразим поток данных из примера со светофором, протекающий через систему

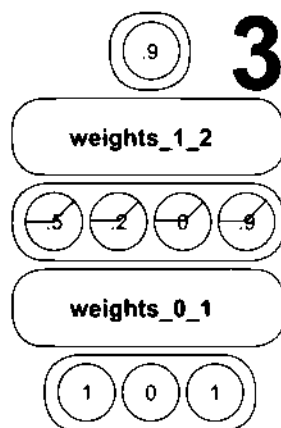
На рисунке 1 изображена передача единственной точки данных из наблюдений за работой светофора. Значения передаются в слой `layer_0`.



На рисунке 2 показан результат вычисления четырех взвешенных сумм в слое `layer_0`. Вычисление выполняется с использованием весовой матрицы `weights_0_1`. Как мы уже знаем, эта операция называется *векторно-матричным умножением*. Четыре полученных значения помещаются в четыре узла слоя `layer_1` после обработки функцией `relu` (преобразует отрицательные числа в 0). Чтобы было понятнее, поясню, что третий узел слева в слое `layer_1` должен был получить отрицательное значение, но функция `relu` заменила это значение нулем.



Как показано на рисунке 3, на заключительном этапе вычисляется средневзвешенное значение для слоя `layer_1`. Здесь снова выполняется операция векторно-матричного умножения. Она возвращает число 0.9, которое и является окончательным прогнозом.



## ВЕКТОРНО-МАТРИЧНОЕ УМНОЖЕНИЕ

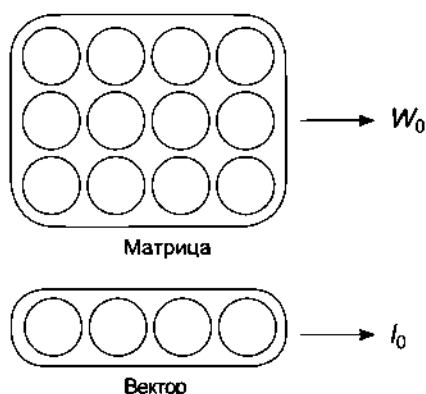
Векторно-матричное умножение вычисляет несколько *взвешенных сумм*, по числу элементов в векторе. Матрица должна иметь столько же строк, сколько элементов в векторе, чтобы, используя каждый столбец в матрице, можно было найти уникальную взвешенную сумму. То есть если матрица имеет четыре столбца, она генерирует четыре взвешенных суммы. Результат взвешивания зависит от значений в матрице.

## Визуализация с использованием букв вместо картинок

Все изображения и детальные объяснения на самом деле можно выразить языком алгебры

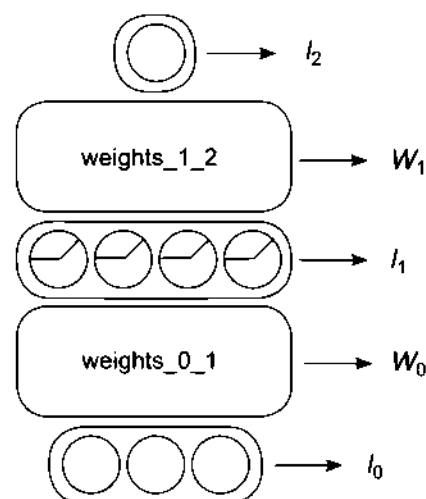
Для представления тех же матриц и векторов вместо картинок можно использовать буквы латинского алфавита.

Как представить *матрицу* на языке математики? Обозначьте ее заглавной буквой латинского алфавита. Для этого я выберу букву, которую проще запомнить, например  $W$  (от слова «weights» — веса). Нижний индекс 0 означает, что эта матрица, скорее всего, одна из нескольких весовых матриц. В данном случае в нашей сети две такие матрицы. Теоретически, я мог бы выбрать любую другую заглавную букву латинского алфавита. Нижний индекс 0 — это небольшой довесок, позволяющий мне обозначить и различать все имеющиеся весовые матрицы  $W$ . Как видите, запомнить такое представление намного проще.



Как представить *вектор* на языке математики? Обозначьте его строчной буквой латинского алфавита. Почему я выбрал букву  $l$ ? Да просто потому, что векторы представляют слои (*layers*) и мне показалось, что такое обозначение проще запомнить. Почему я решил назвать его  $l_0$  («эль-ноль»)? У меня есть несколько слоев, и я подумал, что проще пронумеровать их, чем выбирать новую букву для каждого. Имейте в виду, что мой выбор не является истиной в последней инстанции.

А как при таком представлении матриц и векторов на языке математики будут выглядеть все элементы сети? Справа на рисунке можно видеть отличный набор



имен переменных, соответствующих элементам нейронной сети. Но простое определение переменных не отражает взаимосвязей между ними. Давайте объединим переменные операциями векторно-матричного умножения.

## Связывание переменных

Буквы можно объединять с использованием функций и операций

Векторно-матричное умножение обозначается просто. Достаточно расположить сомножители рядом друг с другом. Например:

| На языке алгебры | Перевод                                             |
|------------------|-----------------------------------------------------|
| $l_0 W_0$        | Умножить вектор слоя $l_0$ на весовую матрицу $W_0$ |
| $l_1 W_1$        | Умножить вектор слоя $l_1$ на весовую матрицу $W_1$ |

В выражения можно добавлять произвольные функции, такие как `relu`, используя форму записи, напоминающую код на языке Python. Такие выражения выглядят просто и понятно.

| На языке алгебры             | Перевод                                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $l_1 = \text{relu}(l_0 W_0)$ | Чтобы получить вектор слоя $l_1$ , нужно умножить вектор слоя $l_0$ на весовую матрицу $W_0$ , а результат пропустить через функцию <code>relu</code> (преобразует отрицательные числа в 0) |
| $l_2 = l_1 W_1$              | Чтобы получить вектор слоя $l_2$ , нужно умножить вектор слоя $l_1$ на весовую матрицу $W_1$                                                                                                |

Обратите внимание, что определение слоя  $l_2$  на языке алгебры основано на слое  $l_1$ . Это означает, что *всю нейронную сеть* можно представить в виде единого выражения, связав все ее компоненты вместе.

| На языке алгебры                 | Перевод                                                                                                                                                                                 |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $l_2 = \text{relu}(l_0 W_0) W_1$ | То есть всю логику этапа прямого распространения можно выразить единственной формулой. Обратите внимание: эта формула предполагает, что все матрицы и векторы имеют совместимые размеры |

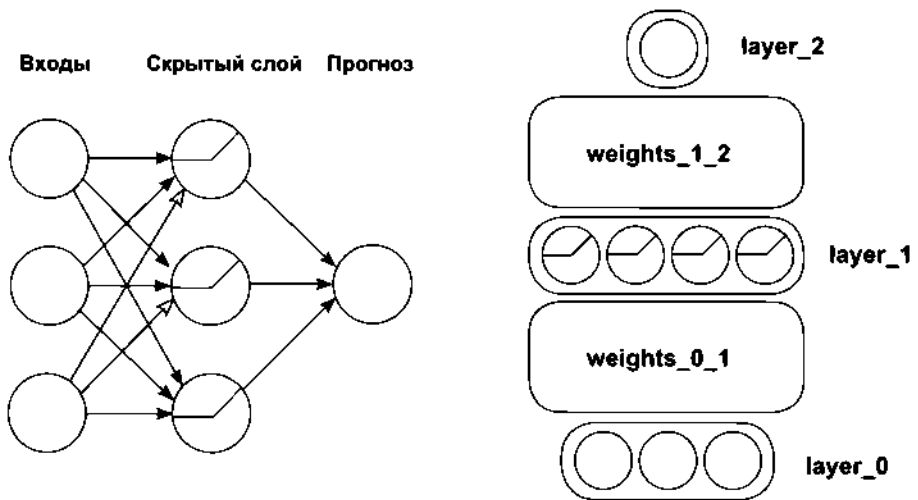
## Сравнение разных способов визуализации

Посмотрим и сравним визуальное представление, алгебраическую формулу и код на Python

Я не думаю, что на этой странице потребуются подробные пояснения. Потратьте минуту и рассмотрите представление этапа прямого распространения четырьмя разными способами. Я очень надеюсь, что вы по-настоящему прогнали прямое распространение и теперь способны понять архитектуру, рассматривая разные ее представления.

```
layer_2 = relu(layer_0.dot(weights_0_1)).dot(weights_1_2)
```

$$l_2 = \text{relu}(l_0 W_0) W_1$$



## Важность инструментов визуализации

Нам предстоит знакомство с новыми архитектурами

В следующих главах мы рассмотрим разные творческие подходы к комбинированию этих векторов и матриц. Доходчивость моих пояснений к разным архитектурам напрямую зависит от того, насколько вы освоили наш взаимно согласованный язык описания. Поэтому постарайтесь не оставлять эту главу, пока до конца не разберетесь, как прямое распространение манипулирует этими векторами и матрицами и как читаются разные формы их описания.



**ВАЖНОЕ ПРИМЕЧАНИЕ**

Удачные нейронные архитектуры комбинируют сигналы, стремясь упростить выявление корреляции. А более удачные архитектуры фильтруют шум, чтобы предотвратить переобучение.

Как отмечалось выше, нейронная архитектура определяет порядок распространения сигнала в сети. Выбор архитектуры напрямую влияет на способность сети обнаруживать корреляцию. Далее вы узнаете, что предпочтительнее создавать архитектуры, помогающие сети сосредоточиться на областях, где имеется значимая корреляция, и игнорировать области, по большей части содержащие шум.

Разные наборы данных и разные предметные области имеют разные характеристики. Например, для изображений характерно большее разнообразие сигналов и шума, чем для текстовых данных. Нейронные сети могут использоваться в разных областях, и для решения конкретных задач предпочтительнее использовать специализированные архитектуры, способные выявлять определенные виды корреляции. Поэтому в следующих нескольких главах мы посмотрим, как конструировать нейронные сети, предназначенные для выявления искомой корреляции. Увидимся там!

# 8

## Усиление сигнала и игнорирование шума: введение в регуляризацию и группировку



### В этой главе

- ✓ Переобучение.
- ✓ Прореживание.
- ✓ Пакетный градиентный спуск.

С четырьмя параметрами я смогу описать слона,  
а с пятью — заставить его махать хоботом.

*Джон фон Нейман (John von Neumann),  
математик, физик, информатик и энциклопедист*

## Трехслойная сеть для классификации набора данных MNIST

Вернемся к набору данных MNIST и попробуем реализовать классификацию рукописных цифр с помощью новой сети

В последних нескольких главах мы узнали, что нейронные сети моделируют корреляцию. Скрытые слои (например, средний слой в трехслойной сети) могут даже создавать промежуточную корреляцию (практически из ничего), чтобы помочь решить задачу. Но как узнать, что сеть создает хорошую корреляцию?

Обсуждая стохастический градиентный спуск с несколькими входами, мы провели эксперимент, зафиксировав один вес и предложив сети продолжить обучение. После обучения точки, соответствующие ошибке, оказались внизу всех трех графиков. Вы видели, как происходит корректировка весов для минимизации ошибки.

Когда мы зафиксировали вес, соответствующая ему ошибка все равно оказалась в нижней точке графика. По некоторой причине кривая графика сместилась так, что зафиксированный вес оказался оптимальным. Кроме того, если заморозить зафиксированный вес и провести дополнительное обучение, он не изменится. Почему? Да просто потому, что ошибка уже достигла 0. То есть с точки зрения сети дальнейшее обучение потеряло смысл.

Возникает вопрос: а что если вход, соответствующий зафиксированному весу, в действительности играет важную роль в предсказании победы бейсбольной команды? А что если сеть нашла путь к точному прогнозу исходов игр в обучающем наборе (потому что именно так и работают сети: они минимизируют ошибку), но почему-то забыла учесть ценный вход?

К сожалению, это явление — переобучение — очень часто встречается в мире нейронных сетей. Можно сказать, что это злейший враг нейронных сетей; и чем мощнее выразительная способность сети (чем больше слоев и весов), тем уязвимее такая сеть для явления переобучения. В исследованиях идет непрекращающаяся борьба, когда люди все время находят задачи, для решения которых нужны все более мощные слои, а потому бьются над задачей, стараясь избавиться от эффекта переобучения.

В этой главе мы познакомимся с основами *регуляризации* — ключевым средством в борьбе против переобучения нейронных сетей. С этой целью мы сразу возьмем самую мощную нейронную сеть из имеющихся у нас (трехслойную

сеть со скрытым слоем `relu`) и используем ее для решения задачи классификации рукописных цифр из набора данных MNIST.

А теперь давайте обучим эту сеть, как показано ниже. В итоге вы должны получить вывод, следующий сразу за листингом. Сеть научилась довольно точно классифицировать обучающие данные. Значит ли это, что мы можем отпраздновать такое событие?

```
import sys, numpy as np
from keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

images, labels = (x_train[0:1000].reshape(1000,28*28) \
                  255, y_train[0:1000])
one_hot_labels = np.zeros((len(labels),10))

for i,l in enumerate(labels):
    one_hot_labels[i][l] = 1
labels = one_hot_labels

test_images = x_test.reshape(len(x_test),28*28) / 255
test_labels = np.zeros((len(y_test),10))
for i,l in enumerate(y_test):
    test_labels[i][l] = 1

np.random.seed(1)
relu = lambda x:(x>=0) * x  ← Возвращает x, если x > 0; иначе возвращает 0
relu2deriv = lambda x: x>=0 ← Возвращает 1, если output > 0; иначе возвращает 0
alpha, iterations, hidden_size, pixels_per_image, num_labels = \
    (0.005, 350, 40, 784, 10)
weights_0_1 = 0.2*np.random.random((pixels_per_image,hidden_size)) - 0.1
weights_1_2 = 0.2*np.random.random((hidden_size,num_labels)) - 0.1

for j in range(iterations):
    error, correct_cnt = (0.0, 0)

    for i in range(len(images)):
        layer_0 = images[i:i+1]
        layer_1 = relu(np.dot(layer_0,weights_0_1))
        layer_2 = np.dot(layer_1,weights_1_2)
        error += np.sum((labels[i:i+1] - layer_2) ** 2)
        correct_cnt += int(np.argmax(layer_2) == \
                                   np.argmax(labels[i:i+1]))
        layer_2_delta = (labels[i:i+1] - layer_2)
        layer_1_delta = layer_2_delta.dot(weights_1_2.T)\
                           * relu2deriv(layer_1)
        weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
        weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)

    sys.stdout.write("\r"+ \
```

```

" I:"+str(j)+ \
" Error:" + str(error/float(len(images)))[0:5] +\
" Correct:" + str(correct_cnt/float(len(images)))

```

```

....
I:349 Error:0.108 Correct:1.0

```

## Это было просто

### Нейронная сеть научилась точно классифицировать цифры на всей 1000 изображений

В каком-то смысле это настоящая победа. Нейронная сеть проанализировала 1000 изображений и научилась правильно классифицировать все исходные изображения.

Как это получилось? Сеть последовательно проанализировала все изображения, для каждого сгенерировала прогноз и, опираясь на него, скорректировала каждый вес, чтобы в следующей итерации получить лучший результат. Если повторить анализ всех изображений достаточно большое число раз, в конце концов сеть достигнет состояния, когда научится правильно классифицировать все изображения.

А теперь попробуем ответить на каверзный вопрос: насколько хорошо сеть справится с изображениями, которых никогда не видела прежде? То есть насколько правильно она будет классифицировать изображения, не вошедшие в число той 1000 изображений, на которых производилось обучение? В наборе MNIST изображений гораздо больше 1000; давайте попробуем.

В блокноте Jupyter Notebook с предыдущим кодом имеются две переменные: `test_images` и `test_labels`. Если выполнить следующий код, он передаст нейронной сети эти изображения и сообщит, насколько точно они были классифицированы:

```

if(j % 10 == 0 or j == iterations-1):
    error, correct_cnt = (0.0, 0)

    for i in range(len(test_images)):

        layer_0 = test_images[i:i+1]
        layer_1 = relu(np.dot(layer_0,weights_0_1))
        layer_2 = np.dot(layer_1,weights_1_2)

        error += np.sum((test_labels[i:i+1] - layer_2) ** 2)

```

```

correct_cnt += int(np.argmax(layer_2) == \
                           np.argmax(test_labels[i:i+1]))
sys.stdout.write(" Test-Err:" + str(error/float(len(test_images)))[0:5] + \
                 " Test-Acc:" + str(correct_cnt/float(len(test_images))))
print()

```

**Error:0.653 Correct:0.7073**

Сеть показала ужасный результат! Точность классификации составила всего 70.7 %. Почему на новых контрольных изображениях получился такой плохой результат, ведь на обучающих данных мы добились 100 %-ной точности? Странно.

Это число 70.7 % называется *точностью проверки*. Это точность нейронной сети на данных, *не* участвовавших в ее обучении. Это число играет важную роль, потому что показывает, насколько хорошо сеть справится со своей задачей в реальном мире (где сеть будет получать только изображения, которых прежде не видела). Это важная оценка.

## Запоминание и обобщение

**Запомнить 1000 изображений проще,  
чем обобщить все изображения**

Посмотрим еще раз, как обучается нейронная сеть. Она корректирует каждый вес в каждой матрице, чтобы для *конкретных входных данных* давать *конкретный прогноз*. Возможно, более уместен был бы такой вопрос: «Мы обучили сеть на наборе, содержащем 1000 изображений, и она научилась точно классифицировать их, тогда почему ей вообще удастся хоть как-то классифицировать другие изображения?»

Как нетрудно догадаться, когда полностью обученная нейронная сеть получает новое изображение, она успешно классифицирует его, только если это новое изображение *почти идентично изображениям из обучающего набора*. Почему? Потому что нейронная сеть научилась преобразовывать входные данные в выходные только для *очень конкретной комбинации входных данных*. Если передать в сеть данные, которые не покажутся ей знакомыми, она вернет случайный прогноз.

Это делает конструирование и обучение нейронных сетей бессмысленным занятием. И действительно, какой толк от нейронной сети, способной распознавать только те данные, на которых она обучалась? Правильная классификация этих данных и без того известна. Нейронные сети могут приносить пользу, только

если они способны распознавать данные, классификация которых пока неизвестна.

Как оказывается, есть средство, помогающее бороться с этим эффектом. Ниже приводится листинг, демонстрирующий процесс обучения нейронной сети и результаты *тестирования во время обучения* (через каждые 10 итераций). Заметили что-нибудь интересное? Вы должны увидеть, как можно улучшить работу сети:

```
I:0 Train-Err:0.722 Train-Acc:0.537 Test-Err:0.601 Test-Acc:0.6488
I:10 Train-Err:0.312 Train-Acc:0.901 Test-Err:0.420 Test-Acc:0.8114
I:20 Train-Err:0.260 Train-Acc:0.93 Test-Err:0.414 Test-Acc:0.8111
I:30 Train-Err:0.232 Train-Acc:0.946 Test-Err:0.417 Test-Acc:0.8066
I:40 Train-Err:0.215 Train-Acc:0.956 Test-Err:0.426 Test-Acc:0.8019
I:50 Train-Err:0.204 Train-Acc:0.966 Test-Err:0.437 Test-Acc:0.7982
I:60 Train-Err:0.194 Train-Acc:0.967 Test-Err:0.448 Test-Acc:0.7921
I:70 Train-Err:0.186 Train-Acc:0.975 Test-Err:0.458 Test-Acc:0.7864
I:80 Train-Err:0.179 Train-Acc:0.979 Test-Err:0.466 Test-Acc:0.7817
I:90 Train-Err:0.172 Train-Acc:0.981 Test-Err:0.474 Test-Acc:0.7758
I:100 Train-Err:0.166 Train-Acc:0.984 Test-Err:0.482 Test-Acc:0.7706
I:110 Train-Err:0.161 Train-Acc:0.984 Test-Err:0.489 Test-Acc:0.7686
I:120 Train-Err:0.157 Train-Acc:0.986 Test-Err:0.496 Test-Acc:0.766
I:130 Train-Err:0.153 Train-Acc:0.99 Test-Err:0.502 Test-Acc:0.7622
I:140 Train-Err:0.149 Train-Acc:0.991 Test-Err:0.508 Test-Acc:0.758

I:210 Train-Err:0.127 Train-Acc:0.998 Test-Err:0.544 Test-Acc:0.7446
I:220 Train-Err:0.125 Train-Acc:0.998 Test-Err:0.552 Test-Acc:0.7416
I:230 Train-Err:0.123 Train-Acc:0.998 Test-Err:0.560 Test-Acc:0.7372
I:240 Train-Err:0.121 Train-Acc:0.998 Test-Err:0.569 Test-Acc:0.7344
I:250 Train-Err:0.120 Train-Acc:0.999 Test-Err:0.577 Test-Acc:0.7316
I:260 Train-Err:0.118 Train-Acc:0.999 Test-Err:0.585 Test-Acc:0.729
I:270 Train-Err:0.117 Train-Acc:0.999 Test-Err:0.593 Test-Acc:0.7259
I:280 Train-Err:0.115 Train-Acc:0.999 Test-Err:0.600 Test-Acc:0.723
I:290 Train-Err:0.114 Train-Acc:0.999 Test-Err:0.607 Test-Acc:0.7196
I:300 Train-Err:0.113 Train-Acc:0.999 Test-Err:0.614 Test-Acc:0.7183
I:310 Train-Err:0.112 Train-Acc:0.999 Test-Err:0.622 Test-Acc:0.7165
I:320 Train-Err:0.111 Train-Acc:0.999 Test-Err:0.629 Test-Acc:0.7133
I:330 Train-Err:0.110 Train-Acc:0.999 Test-Err:0.637 Test-Acc:0.7125
I:340 Train-Err:0.109 Train-Acc:1.0 Test-Err:0.645 Test-Acc:0.71
I:349 Train-Err:0.108 Train-Acc:1.0 Test-Err:0.653 Test-Acc:0.7073
```

## Переобучение нейронных сетей

Если нейронную сеть обучать слишком долго, точность прогноза может *ухудшиться*!

На протяжении первых 20 итераций точность *проверки* по какой-то причине продолжала расти, а потом, по мере дальнейшего обучения, начала медленно

уменьшаться (при этом точность на обучающих данных продолжала неуклонно расти). Это обычное явление в нейронных сетях. Позвольте мне объяснить это явление на примере аналогии.

Представьте, что вы создали заливочную форму для типичной столовой вилки, но решили использовать ее не для отливки новых вилок, а чтобы с ее помощью проверять — является ли тот или иной столовый прибор вилок. Если прибор укладывается в форму, вы приходите к выводу, что он является вилок, а если нет, тогда вы решаете, что это *не* вилка.

Допустим, что вы приступили к созданию заливочной формы, имея кусок сырой глины и большую связку с вилок с тремя зубцами, ложками и ножами. Вы вдавливате каждую из вилок в одно и то же место на глиняной заготовке, чтобы получить отпечаток, имеющий форму вилки. Вы снова и снова вдавливате все вилок несколько сотен раз. Дав глине высохнуть, вы начинаете экспериментировать и обнаруживаете, что ни ложки, ни ножи не укладываются в получившуюся форму, зато все вилок прекрасно вписываются в нее. Отлично! Вы создали форму, в которую укладываются только вилок.

А теперь представьте, что кто-то протянул вам вилок с четырьмя зубцами. Взглянув на свою форму, вы замечаете, что в ней отпечатан контур вилки с тремя зубцами. Вилка с четырьмя зубцами не укладывается в нее. Но почему? Ведь это тоже вилка.

Это объясняется тем, что в формировании заливочной формы не участвовали вилок с четырьмя зубцами — использовались только разные вилок с тремя зубцами. Глина *переобучилась* распознавать только те типы вилок, которые использовались при «обучении» формы.

Это то же явление, которое мы только что наблюдали в нейронной сети. Описанная аналогия намного точнее, чем вы думаете. Веса нейронной сети можно рассматривать как многомерную форму. В процессе обучения сеть создает *отпечаток*, повторяющий форму данных, обучаясь отличать один шаблон от другого. К сожалению, изображения в контрольном наборе данных *немного* отличаются от изображений в обучающем наборе. В результате многие примеры из контрольного набора не совпадают с отпечатком, созданным сетью.

Формально такие переобученные нейронные сети называют сетями, научившимися распознавать *шум*, а не принимать решения исключительно на основе *истинного сигнала*.



## Причины переобучения

### Что вызывает эффект переобучения нейронной сети?

Давайте немного видоизменим эксперимент с глиной. Снова представьте бесформенный кусок сырой глины. Что если вдавить в него только одну вилку? Если допустить, что глина имеет очень плотную консистенцию, отпечаток получится не таким четким, как в предыдущем эксперименте (когда каждая вилка вдавливалась много раз). То есть мы получим *лишь общий контур вилки*. В этот контур будут укладываться вилки и с тремя, и с четырьмя зубцами, потому что он очень нечеткий.

Но по мере вдавливания других вилок способность формы распознавать контрольные данные начнет ухудшаться, потому что в ней будет отпечатываться все больше деталей об обучающих данных. В итоге это приведет к тому, что в форму перестанут укладываться вилки, даже немного отличающиеся от тех, что использовались для формирования отпечатка.

Что же это за *детали* на изображениях, которые ухудшают точность классификации контрольных данных? Если следовать аналогии с вилками, — это число зубцов. На изображениях такие детали обычно называют *шумом*. Но на самом деле все намного сложнее. Взгляните на два следующих изображения с собаками.



Все отличительные детали на этих изображениях, кроме самого изображения «собаки», являются *шумом*. На рисунке слева шум — это подушка и задний фон. На рисунке справа черная заливка контура собаки тоже является разновидностью шума. Только контур позволяет нам сказать, что здесь изображена собака, а черная заливка ничего нам не говорит. На рисунке слева тело собаки имеет текстуру шерсти и характерный окрас, что может помочь классификатору правильно определить, что здесь изображена собака.

Как заставить нейронную сеть изучать только полезный *сигнал* (образ собаки) и игнорировать шум (все остальное, не учитываемое при классификации)? Одним из решений является *ранняя остановка*. Как оказывается, большая часть шума кроется в мелких деталях на изображениях, а большая часть полезного сигнала (объекты) сосредоточена в общей форме и иногда в цветовой палитре.

## Простейшая регуляризация: ранняя остановка

### Остановите обучение сети, когда начнет падать ее точность

Как заставить нейронную сеть игнорировать мелкие детали и фиксировать только общую информацию, присутствующую в данных (например, общую форму собаки или цифр в наборе MNIST)? Просто не обучайте сеть слишком долго, чтобы не дать сети возможности изучить эти детали.

В примере с формой в глине мы отпечатывали вилки по многу раз, чтобы получить идеальный отпечаток вилки с тремя зубцами. При этом первые попытки сформировали в глине лишь общий контур вилки. То же можно сказать в отношении нейронных сетей. То есть *ранняя остановка* является простейшей формой регуляризации, и в безвыходных ситуациях она может быть весьма эффективным решением.

Мы подошли к главной теме этой главы: *регуляризации*. Регуляризация — это подмножество методов, помогающих *обобщить* модель для распознавания новых данных (вместо простого запоминания обучающих примеров). Эти методы помогают нейронной сети изучить сигнал и игнорировать шум. В данном случае это набор инструментов для создания нейронных сетей, обладающих этими свойствами.

#### РЕГУЛЯРИЗАЦИЯ

Регуляризация — это подмножество методов, способствующих обобщению изучаемых моделей, часто за счет препятствования изучению мелких деталей.

Следующий вопрос, который может у вас появиться: как узнать, когда остановиться? Единственный верный способ — опробовать модель на данных,

не участвовавших в обучении. Обычно для этого используется второй набор данных, который называют *проверочным*. В некоторых случаях использование контрольного набора данных, чтобы узнать, когда остановиться, может привести к *переобучению на контрольных данных*. По этой причине не следует использовать контрольный набор данных для управления обучением. Всегда используйте для этого проверочный набор.

## Стандартный способ регуляризации: прореживание (дропаут)

**Суть метода: выключение (установка в 0) случайных нейронов в процессе обучения**

Этот метод регуляризации очень прост. Суть его заключается в том, чтобы в процессе обучения устанавливать в 0 случайные нейроны (и разности для соответствующих узлов на этапе обратного распространения, хотя технически этого не требуется). Это вынудит нейронную сеть обучаться только с использованием *случайного подмножества* нейронов.

Хотите верьте, хотите нет, но этот прием регуляризации считается одним из самых современных и используется в подавляющем большинстве сетей. Это простая и недорогая методология. Правда, понять, *как* она работает, довольно сложно.

### КАК РАБОТАЕТ ПРОРЕЖИВАНИЕ (УПРОЩЕННОЕ ОБЪЯСНЕНИЕ)

Прореживание заставляет большую сеть действовать подобно маленькой сети, обучая случайно выбираемые подразделы, а маленькие сети не подвержены переобучению.

Как оказывается, чем меньше нейронная сеть, тем меньше она подвержена эффекту переобучения. Почему? Да потому, что маленькие нейронные сети не обладают большой выразительной силой. Они не в состоянии фиксировать мелкие детали (шум), которые, как правило, являются причиной переобучения. Их объема хватает, только чтобы захватить лишь самые общие, очевидные и высокоуровневые закономерности.

Понятие *объема*, или *емкости*, достаточно важно, чтобы запомнить его. Представить его можно так. Помните аналогию с глиной? Представьте, что форма

для оттиска наполнена камнями размером с десятикопеечную монету. Сможете ли вы в этом случае получить четкий отпечаток вилки? Конечно нет. Эти камни похожи на веса в нейронной сети. Они образуют отпечаток данных, фиксируя только самые общие закономерности. Если у вас будет лишь несколько больших камней, вы не сможете с их помощью получить отпечаток с мелкими деталями. Каждый камень будет углубляться, отражая крупные детали вилки, более или менее *усредняя* ее форму (игнорируя мелкие выступы и углубления).

Теперь представьте форму с мелкозернистым песком. Она наполнена миллионами и миллионами мелких песчинок, которые способны отразить самые мелкие детали вилки. Именно это придает большим нейронным сетям выразительную силу, которая часто становится причиной эффекта переобучения.

Как совместить выразительную силу большой нейронной сети с устойчивостью к переобучению маленькой? Нужно взять большую сеть и отключить случайно выбранные узлы. Что происходит, когда в обучении участвует только малая часть большой нейронной сети? Она начинает действовать как маленькая нейронная сеть. Но, когда отключение происходит случайно и в обучении участвуют миллионы маленьких подсетей, вся сеть в целом сохраняет свою выразительную силу. Остроумно, правда?

## Как работает прореживание: в работе участвуют ансамбли

**Прореживание — это форма обучения множества сетей и их усреднения**

Важно помнить, что начальное состояние нейронных сетей всегда выбирается случайно. Почему это важно? Нейронные сети обучаются методом проб и ошибок, а это означает, что все они учатся немного по-разному. Они могут учиться одинаково эффективно, но никакие две нейронных сети не станут абсолютно одинаковыми (если не имели одинакового начального состояния, по случайности или преднамеренно).

Отсюда вытекает интересное свойство. Никакие две нейронных сети не обучаются одинаково. Обучение продолжается только до момента, когда сеть научится правильно распознавать каждое изображение в обучающем наборе, то есть до момента, когда выполнится условие `error == 0`, после чего обучение прекратится (даже если итерации будут продолжаться). Но так как каждая

нейронная сеть получает случайное начальное состояние и затем корректирует свои веса, стремясь добиться точного прогноза, каждая сеть неизбежно совершает разные ошибки и корректирует веса на разные значения. В результате мы приходим к основополагающей идее:

Даже при том, что большие, нерегуляризованные нейронные сети изучают шум, весьма маловероятно, что это будет *один и тот же шум*.

Почему они не приходят к одному и тому же шуму? Потому, что имеют случайное начальное состояние и прекращают обучаться, когда изучат достаточный объем шума, чтобы однозначно классифицировать все изображения в обучающем наборе. Чтобы обучиться, сеть MNIST должна найти лишь несколько случайных пикселей, которые коррелируют с классами на выходе. Из этого вытекает еще более важная идея:

Нейронные сети, даже при том, что имеют случайное начальное состояние, все еще начинают изучение данных с самых общих закономерностей, и только потом начинают изучать шум.

Из вышесказанного следует: если обучить 100 нейронных сетей (инициализированных случайным образом), каждая из них будет стремиться захватить разный шум, но одинаковый общий *сигнал*. То есть допуская ошибки, они часто допускают *разные* ошибки. Если позволить им принимать решение голосованием, их шумы нейтрализуют друг друга и останется только общее знание: *сигнал*.

## Прореживание в коде

Здесь показано, как использовать прием прореживания на практике

Добавим прореживание в скрытый слой модели классификации MNIST и в процессе обучения будем отключать 50 % случайно выбранных узлов. Возможно, вас удивит, что для этого достаточно изменить всего три строки в коде. Ниже приводится фрагмент из уже знакомой реализации нейронной сети с добавленной маской прореживания:

```

i = 0
layer_0 = images[i:i+1]
dropout_mask = np.random.randint(2,size=layer_1.shape)

layer_1 *= dropout_mask * 2
layer_2 = np.dot(layer_1, weights_1_2)

error += np.sum((labels[i:i+1] - layer_2) ** 2)

correct_cnt += int(np.argmax(layer_2) == \
                        np.argmax(labels[i:i+1]))

layer_2_delta = (labels[i:i+1] - layer_2)
layer_1_delta = layer_2_delta.dot(weights_1_2.T) \
                * relu2deriv(layer_1)

layer_1_delta *= dropout_mask

weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)

```

Чтобы реализовать прореживание в слое (в данном случае в слое `layer_1`), нужно умножить значения в `layer_1` на случайную матрицу из единиц и нулей. В результате получится эффект выключения случайных узлов в `layer_1` путем записи в них значения 0. Обратите внимание, что для получения `dropout_mask` используется так называемое *50 %-ное распределение Бернулли*, благодаря чему в 50 % случаев значение в `dropout_mask` равно 1 и в  $(1 - 50 \% = 50 \%)$  случаев равно 0.

Далее следует операция, которая может показаться странной. Слой `layer_1` умножается на 2. С какой целью это делается? Не забывайте, что `layer_2` будет вычислять взвешенную сумму на основе `layer_1`. Даже притом, что это взвешенная *сумма*, она все еще остается суммой значений в `layer_1`. Если отключить половину узлов в `layer_1`, сумма уменьшится наполовину. В результате слою `layer_2` пришлось бы внимательнее «прислушиваться» к слою `layer_1`, как это делает человек, наклоняясь к радиоприемнику, если громкость уменьшить слишком сильно. Но во время проверки, когда прореживание не выполняется, громкость восстановится до нормального уровня. Это нарушит способность слоя `layer_2` правильно слушать слой `layer_1`. Мы должны предотвратить это, умножив `layer_1` на  $(1/\text{доля выключенных узлов})$ . В данном случае мы получаем выражение  $1/0.5$ , которое равно 2. Благодаря этому «громкость» слоя `layer_1` будет одинакова при обучении и проверке, несмотря на прореживание.

```

import numpy, sys
np.random.seed(1)
def relu(x):
    return (x >= 0) * x  ← Возвращает x, если x > 0; иначе возвращает 0

def relu2deriv(output):
    return output >= 0  ← Возвращает 1, если output > 0; иначе возвращает 0

alpha, iterations, hidden_size = (0.005, 300, 100)
pixels_per_image, num_labels = (784, 10)

weights_0_1 = 0.2*np.random.random((pixels_per_image,hidden_size)) - 0.1
weights_1_2 = 0.2*np.random.random((hidden_size,num_labels)) - 0.1

for j in range(iterations):
    error, correct_cnt = (0.0,0)
    for i in range(len(images)):
        layer_0 = images[i:i+1]
        layer_1 = relu(np.dot(layer_0,weights_0_1))
        dropout_mask = np.random.randint(2, size=layer_1.shape)
        layer_1 *= dropout_mask * 2
        layer_2 = np.dot(layer_1,weights_1_2)

        error += np.sum((labels[i:i+1] - layer_2) ** 2)
        correct_cnt += int(np.argmax(layer_2) == \
                               np.argmax(labels[i:i+1]))
        layer_2_delta = (labels[i:i+1] - layer_2)
        layer_1_delta = layer_2_delta.dot(weights_1_2.T) * relu2deriv(layer_1)
        layer_1_delta *= dropout_mask

        weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
        weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)

    if(j%10 == 0):
        test_error = 0.0
        test_correct_cnt = 0

        for i in range(len(test_images)):
            layer_0 = test_images[i:i+1]
            layer_1 = relu(np.dot(layer_0,weights_0_1))
            layer_2 = np.dot(layer_1, weights_1_2)

            test_error += np.sum((test_labels[i:i+1] - layer_2) ** 2)
            test_correct_cnt += int(np.argmax(layer_2) == \
                                       np.argmax(test_labels[i:i+1]))

        sys.stdout.write("\n" + \
            "I:" + str(j) + \
            " Test-Err:" + str(test_error/ float(len(test_images)))[0:5] + \
            " Test-Acc:" + str(test_correct_cnt/ float(len(test_images)))+ \
            " Train-Err:" + str(error/ float(len(images)))[0:5] + \
            " Train-Acc:" + str(correct_cnt/ float(len(images))))

```

## Влияние прореживания на модель MNIST

Как было показано в примере выше, нейронная сеть (без прореживания) достигла точности 81.14 % на контрольных данных, после чего, к концу обучения, ее точность упала до 70.73 %. После добавления прореживания сеть ведет себя иначе:

```
I:0 Test-Err:0.641 Test-Acc:0.6333 Train-Err:0.891 Train-Acc:0.413
I:10 Test-Err:0.458 Test-Acc:0.787 Train-Err:0.472 Train-Acc:0.764
I:20 Test-Err:0.415 Test-Acc:0.8133 Train-Err:0.430 Train-Acc:0.809
I:30 Test-Err:0.421 Test-Acc:0.8114 Train-Err:0.415 Train-Acc:0.811
I:40 Test-Err:0.419 Test-Acc:0.8112 Train-Err:0.413 Train-Acc:0.827
I:50 Test-Err:0.409 Test-Acc:0.8133 Train-Err:0.392 Train-Acc:0.836
I:60 Test-Err:0.412 Test-Acc:0.8236 Train-Err:0.402 Train-Acc:0.836
I:70 Test-Err:0.412 Test-Acc:0.8033 Train-Err:0.383 Train-Acc:0.857
I:80 Test-Err:0.410 Test-Acc:0.8054 Train-Err:0.386 Train-Acc:0.854
I:90 Test-Err:0.411 Test-Acc:0.8144 Train-Err:0.376 Train-Acc:0.868
I:100 Test-Err:0.411 Test-Acc:0.7903 Train-Err:0.369 Train-Acc:0.864
I:110 Test-Err:0.411 Test-Acc:0.8003 Train-Err:0.371 Train-Acc:0.868
I:120 Test-Err:0.402 Test-Acc:0.8046 Train-Err:0.353 Train-Acc:0.857
I:130 Test-Err:0.408 Test-Acc:0.8091 Train-Err:0.352 Train-Acc:0.867
I:140 Test-Err:0.405 Test-Acc:0.8083 Train-Err:0.355 Train-Acc:0.885
I:150 Test-Err:0.404 Test-Acc:0.8107 Train-Err:0.342 Train-Acc:0.883
I:160 Test-Err:0.399 Test-Acc:0.8146 Train-Err:0.361 Train-Acc:0.876
I:170 Test-Err:0.404 Test-Acc:0.8074 Train-Err:0.344 Train-Acc:0.889
I:180 Test-Err:0.399 Test-Acc:0.807 Train-Err:0.333 Train-Acc:0.892
I:190 Test-Err:0.407 Test-Acc:0.8066 Train-Err:0.335 Train-Acc:0.898
I:200 Test-Err:0.405 Test-Acc:0.8036 Train-Err:0.347 Train-Acc:0.893
I:210 Test-Err:0.405 Test-Acc:0.8034 Train-Err:0.336 Train-Acc:0.894
I:220 Test-Err:0.402 Test-Acc:0.8067 Train-Err:0.325 Train-Acc:0.896
I:230 Test-Err:0.404 Test-Acc:0.8091 Train-Err:0.321 Train-Acc:0.894
I:240 Test-Err:0.415 Test-Acc:0.8091 Train-Err:0.332 Train-Acc:0.898
I:250 Test-Err:0.395 Test-Acc:0.8182 Train-Err:0.320 Train-Acc:0.899
I:260 Test-Err:0.390 Test-Acc:0.8204 Train-Err:0.321 Train-Acc:0.899
I:270 Test-Err:0.382 Test-Acc:0.8194 Train-Err:0.312 Train-Acc:0.906
I:280 Test-Err:0.396 Test-Acc:0.8208 Train-Err:0.317 Train-Acc:0.9
I:290 Test-Err:0.399 Test-Acc:0.8181 Train-Err:0.301 Train-Acc:0.908
```

Сеть не только достигает максимума с оценкой 82.36 %, но и прекрасно справляется с эффектом переобучения, завершая обучение с точностью 81.81 % на контрольных данных. Обратите внимание, что прореживание также замедляет обучение — рост показателя Training-Acc, который в предыдущем примере достиг 100 % и остановился.

Это явно указывает, что прореживание отсеивает шум и тормозит обучение. Это напоминает бег с тяжестями на ногах. Тренировка усложняется, но потом, сняв тяжести перед состязанием, вы бежите намного быстрее, потому что тренируетесь для преодоления чего-то более сложного.



## Пакетный градиентный спуск

Этот метод увеличивает скорость обучения и улучшает сходимость

Завершая эту главу, я хотел бы коротко рассказать еще об одном методе, с которым мы познакомились в одной из предыдущих глав: о пакетном стохастическом градиентном спуске. Я не буду углубляться в детали, потому что этот метод считается как нечто само собой разумеющееся при обучении нейронных сетей. Кроме того, этот простой метод не получил особого развития, даже в современных нейронных сетях.

Прежде мы передавали обучающие примеры в сеть по одному, корректируя веса после каждого из них. Теперь попробуем передавать сети сразу по 100 примеров, усредняя корректирующие значения для весов по всем 100 примерам. Результаты обучения/проверки показаны ниже, а код, реализующий этот метод, следует далее.

```
I:0 Test-Err:0.815 Test-Acc:0.3832 Train-Err:1.284 Train-Acc:0.165
I:10 Test-Err:0.568 Test-Acc:0.7173 Train-Err:0.591 Train-Acc:0.672
I:20 Test-Err:0.510 Test-Acc:0.7571 Train-Err:0.532 Train-Acc:0.729
I:30 Test-Err:0.485 Test-Acc:0.7793 Train-Err:0.498 Train-Acc:0.754
I:40 Test-Err:0.468 Test-Acc:0.7877 Train-Err:0.489 Train-Acc:0.749
I:50 Test-Err:0.458 Test-Acc:0.793 Train-Err:0.468 Train-Acc:0.775
I:60 Test-Err:0.452 Test-Acc:0.7995 Train-Err:0.452 Train-Acc:0.799
I:70 Test-Err:0.446 Test-Acc:0.803 Train-Err:0.453 Train-Acc:0.792
I:80 Test-Err:0.451 Test-Acc:0.7968 Train-Err:0.457 Train-Acc:0.786
I:90 Test-Err:0.447 Test-Acc:0.795 Train-Err:0.454 Train-Acc:0.799
I:100 Test-Err:0.448 Test-Acc:0.793 Train-Err:0.447 Train-Acc:0.796
I:110 Test-Err:0.441 Test-Acc:0.7943 Train-Err:0.426 Train-Acc:0.816
I:120 Test-Err:0.442 Test-Acc:0.7966 Train-Err:0.431 Train-Acc:0.813
I:130 Test-Err:0.441 Test-Acc:0.7906 Train-Err:0.434 Train-Acc:0.816
I:140 Test-Err:0.447 Test-Acc:0.7874 Train-Err:0.437 Train-Acc:0.822
I:150 Test-Err:0.443 Test-Acc:0.7899 Train-Err:0.414 Train-Acc:0.823
I:160 Test-Err:0.438 Test-Acc:0.797 Train-Err:0.427 Train-Acc:0.811
I:170 Test-Err:0.440 Test-Acc:0.7884 Train-Err:0.418 Train-Acc:0.828
I:180 Test-Err:0.436 Test-Acc:0.7935 Train-Err:0.407 Train-Acc:0.834
I:190 Test-Err:0.434 Test-Acc:0.7935 Train-Err:0.410 Train-Acc:0.831
I:200 Test-Err:0.435 Test-Acc:0.7972 Train-Err:0.416 Train-Acc:0.829
I:210 Test-Err:0.434 Test-Acc:0.7923 Train-Err:0.409 Train-Acc:0.83
I:220 Test-Err:0.433 Test-Acc:0.8032 Train-Err:0.396 Train-Acc:0.832
I:230 Test-Err:0.431 Test-Acc:0.8036 Train-Err:0.393 Train-Acc:0.853
I:240 Test-Err:0.430 Test-Acc:0.8047 Train-Err:0.397 Train-Acc:0.844
I:250 Test-Err:0.429 Test-Acc:0.8028 Train-Err:0.386 Train-Acc:0.843
I:260 Test-Err:0.431 Test-Acc:0.8038 Train-Err:0.394 Train-Acc:0.843
I:270 Test-Err:0.428 Test-Acc:0.8014 Train-Err:0.384 Train-Acc:0.845
I:280 Test-Err:0.430 Test-Acc:0.8067 Train-Err:0.401 Train-Acc:0.846
I:290 Test-Err:0.428 Test-Acc:0.7975 Train-Err:0.383 Train-Acc:0.851
```

Обратите внимание, что точность на контрольных данных изменяется более плавно, чем в предыдущих примерах. Этот эффект обусловлен усреднением корректирующих воздействий на весовые коэффициенты в процессе обучения. Как оказывается, отдельные обучающие примеры несут в себе очень много шума, в смысле корректирующих значений, которые они генерируют. То есть усреднение делает процесс обучения более плавным.

```
import numpy as np
np.random.seed(1)

def relu(x):
    return (x >= 0) * x  ← Возвращает x, если x > 0

def relu2deriv(output):
    return output >= 0  ← Возвращает 1, если output > 0

batch_size = 100
alpha, iterations = (0.001, 300)
pixels_per_image, num_labels, hidden_size = (784, 10, 100)

weights_0_1 = 0.2*np.random.random((pixels_per_image,hidden_size)) - 0.1
weights_1_2 = 0.2*np.random.random((hidden_size,num_labels)) - 0.1

for j in range(iterations):
    error, correct_cnt = (0.0, 0)
    for i in range(int(len(images) / batch_size)):
        batch_start, batch_end = ((i * batch_size),((i+1)*batch_size))

        layer_0 = images[batch_start:batch_end]
        layer_1 = relu(np.dot(layer_0,weights_0_1))
        dropout_mask = np.random.randint(2,size=layer_1.shape)
        layer_1 *= dropout_mask * 2
        layer_2 = np.dot(layer_1,weights_1_2)

        error += np.sum((labels[batch_start:batch_end] - layer_2) ** 2)
        for k in range(batch_size):
            correct_cnt += int(np.argmax(layer_2[k:k+1]) == \
                                     np.argmax(labels[batch_start+k:batch_start+k+1]))

        layer_2_delta = (labels[batch_start:batch_end]-layer_2) \
                        /batch_size
        layer_1_delta = layer_2_delta.dot(weights_1_2.T)* \
                        relu2deriv(layer_1)
        layer_1_delta *= dropout_mask

        weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
        weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)

    if(j%10 == 0):
        test_error = 0.0
```

```
test_correct_cnt = 0

for i in range(len(test_images)):
    layer_0 = test_images[i:i+1]
    layer_1 = relu(np.dot(layer_0, weights_0_1))
    layer_2 = np.dot(layer_1, weights_1_2)
```

Первое, что бросается в глаза при опробовании этого кода, — он выполняется намного быстрее. Это объясняется тем, что теперь каждый вызов функции `np.dot` вычисляет сразу 100 скалярных произведений. Процессоры устроены так, что скалярное произведение пакетами выполняется быстрее.

Однако это еще не все. Обратите внимание, что здесь значение коэффициента `alpha` в 20 раз больше, чем в предыдущих примерах. В данном случае мы смогли увеличить его по одной интересной причине. Представьте, что вы пытаетесь прибежать в нужную точку, ориентируясь по компасу с очень неустойчивой стрелкой. Вы берете азимут, пробегаете 2 мили и почти наверняка отклоняетесь в сторону. Но если вы возьмете азимут 100 раз, усредните показания компаса и пробежите 2 мили, то, скорее всего, окажетесь в нужной точке или отклонитесь совсем чуть-чуть.

В этом примере берется среднее зашумленного сигнала (средняя величина изменения веса, полученная по 100 обучающим примерам), поэтому есть возможность двигаться вперед более широкими шагами. Обычно размер пакета в пакетном градиентном спуске выбирается равным от 8 до 256. Как правило, исследователи выбирают числа произвольным образом, пока не найдут подходящую пару `batch_size/alpha`.

## Итоги

В этой главе мы рассмотрели два наиболее широко распространенных метода, увеличивающих точность и скорость обучения практически любой нейронной архитектуры. В следующих главах мы перейдем от наборов инструментов, применимых практически к любым нейронным сетям, к специализированным архитектурам, пригодным для моделирования особых явлений в данных.

# 9

## Моделирование случайности и нелинейности: функции активации



### В этой главе

- ✓ Что такое функция активации?
- ✓ Стандартные функции активации для скрытых слоев:
  - Sigmoid;
  - Tanh.
- ✓ Стандартные функции активации для выходного слоя:
  - Softmax.
- ✓ Инструкции по внедрению функций активации.

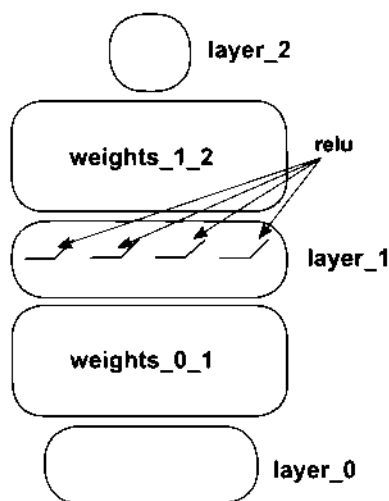
Я знаю, что дважды два — четыре, и был бы рад доказать это, если б смог, хотя должен признать, что, если бы каким-то образом я мог превратить дважды два в пять, это доставило бы мне намного больше удовольствия.

*Джордж Гордон Байрон (George Gordon Byron),  
из письма Аннабелле Милбенк (Annabella Milbanke),  
10 ноября 1813*

## Что такое функция активации?

Это функция, которая применяется к нейронам в слое в процессе прогнозирования

*Функция активации* — это функция, которая применяется к нейронам в слое в процессе прогнозирования. Это понятие может показаться очень знакомым, потому что мы уже использовали функцию активации с именем `relu` (показана здесь на рисунке, изображающем трехслойную сеть). Функция `relu` преобразует отрицательные значения в 0.



Проще говоря, функция активации — это любая функция, принимающая одно число и возвращающая другое. Во Вселенной существует бесконечное множество функций, но не все они могут использоваться как функции активации.

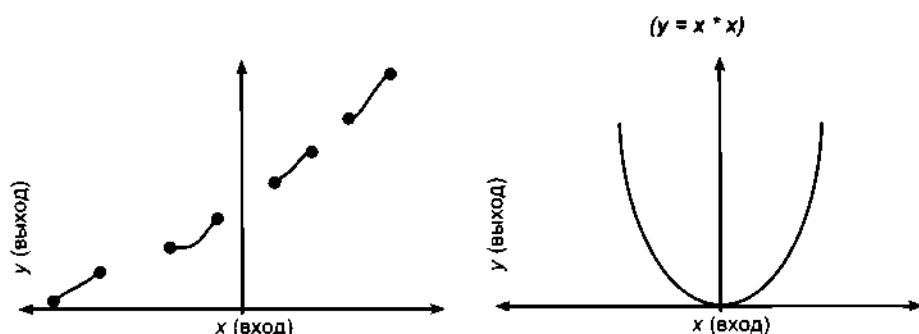
Есть целый ряд ограничений, которые делают функцию функцией активации. Использование функций, не соответствующих этим ограничениям, обычно не приводит ни к чему хорошему, как будет показано ниже.

**Ограничение 1: функция должна быть непрерывной и бесконечной на всей области определения**

Первое ограничение, которому должна соответствовать функция, чтобы иметь право называться функцией активации: она должна возвращать число для

любого входного значения. Иначе говоря, не должно быть такого входного числа, получив которое функция не смогла бы вернуть результат.

Возможно, это излишне, но посмотрите на график функции слева на рисунке ниже (состоящий из четырех отрезков). Заметили, что на нем не каждому значению  $X$  соответствует некоторое значение  $Y$ ? Функция определена только на четырех участках. Эта функция совершенно не годится на роль функции активации. Однако функция на графике справа непрерывна и бесконечна на всей области определения. Нет такого входного числа ( $X$ ), для которого функция не смогла бы вычислить соответствующее выходное значение.

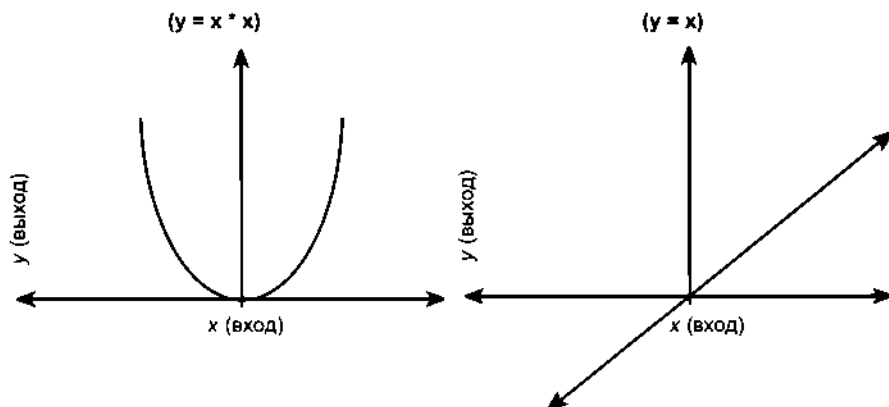


## Ограничение 2: хорошие функции активации монотонны и никогда не меняют направления

Второе ограничение требует, чтобы ни для каких двух входных значений не получался одинаковый результат. Функция никогда не должна менять направления. Иначе говоря, значения функции должны всегда возрастать или убывать.

В качестве примера рассмотрим две следующие функции. Эти функции обе отвечают на вопрос: «Какое значение  $Y$  соответствует заданному значению  $X$ ?» Но функция слева ( $y = x * x$ ) не является идеальной функцией активации, потому что она не всегда только возрастает или только убывает.

Почему я так решил? Все просто: обратите внимание, что во многих случаях двум разным значениям  $X$  соответствует одно и то же значение  $Y$  (это верно для всех значений  $X$ , отличных от 0). Однако функция справа всегда возрастает! Вы не найдете двух значений  $X$ , для которых эта функция давала бы одно и то же значение  $Y$ :



Технически данное требование не является обязательным. В отличие от функций, которые не для каждого входного значения имеют выходное значение (их называют разрывными), немонотонные функции можно оптимизировать. Но имейте в виду, что они могут отображать множество входных значений в одно и то же выходное значение.

В процессе обучения нейронная сеть пытается найти правильную комбинацию весов, дающую правильный ответ. Задача может усложниться, если правильных ответов окажется несколько. При наличии нескольких путей, ведущих к правильному ответу, сеть будет иметь на выбор несколько правильных комбинаций весов.

Оптимист может сказать: «Но это же замечательно! Мы наверняка придем к правильному ответу, если их будет несколько!» Пессимист, напротив, может усомниться: «Это ужасно! Теперь мы не знаем, куда двигаться, чтобы уменьшить ошибку, потому что можно пойти в любом направлении и теоретически добиться успеха».

К сожалению, явление, обнаруженное пессимистом, более важно. Чтобы лучше понять суть, я советую изучить различия между выпуклой и невыпуклой оптимизациями; во многих университетах (и онлайн-классах) этим вопросам посвящены целые курсы.

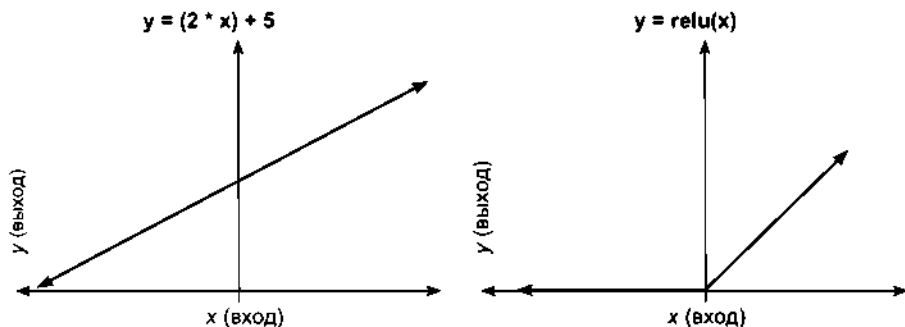
### Ограничение 3: хорошие функции активации нелинейны (имеют точки перегиба)

Третье ограничение возвращает нас в главу 6. Помните рассказ об *эпизодической корреляции*? Чтобы создать ее, нужно позволить нейронам выборочно

коррелировать с входными нейронами, чтобы можно было уменьшить влияние сильного отрицательного сигнала от одного входа на корреляцию нейрона с любыми другими входами (принудительно сбросив значение нейрона в 0, как это делает функция `relu`).

Как оказывается, этому явлению способствует *любая изгибающаяся (нелинейная) функция*. Функции, выглядящие на графике как прямые линии, напротив, масштабируют входящее среднее взвешенное. Масштабирование (умножение на константу, например на 2) не влияет на величину корреляции нейрона с его входами. Оно просто заставляет коллективную корреляцию звучать тише или громче. Хорошая функция активации не позволяет одному весу «забывать» корреляцию нейрона с другими весами. Фактически, нам необходима *выборочная* корреляция. Нам *действительно* нужно, чтобы входной сигнал, попадающий в нейрон с функцией активации, мог увеличивать или уменьшать корреляцию нейрона с другими входными сигналами. Все нелинейные функции обладают такой способностью (в разной степени, как будет показано далее).

Например, функция слева на следующем рисунке является линейной, а функция справа — нелинейной и, соответственно, лучшей функцией активации (с некоторыми исключениями, которые мы обсудим позже).



**Ограничение 4: хорошие функции активации (и их производные) должны иметь низкую вычислительную сложность**

С этим ограничением все просто. Функция активации будет вызываться очень много раз (иногда несколько миллиардов раз), поэтому нежелательно, чтобы она выполнялась слишком медленно. Многие современные функции активации обрели популярность просто потому, что легко вычисляются (отличным примером может служить функция `relu`).



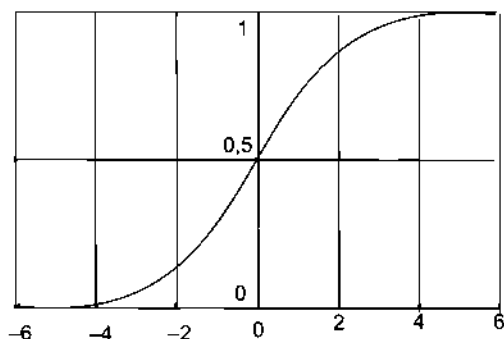
## Стандартные функции активации для скрытых слоев

Какие функции из их бесконечного разнообразия используются наиболее часто?

Даже с учетом перечисленных ограничений очевидно, что существует бесконечное (или безграничное?) количество функций, которые можно использовать в роли функций активации. За последние несколько лет был достигнут большой прогресс в создании функций активации. Но пока подавляющее большинство потребностей в активации удовлетворяется довольно узким кругом функций, и в большинстве случаев эти функции почти не подвергались усовершенствованиям.

**sigmoid** — одна из основных функций активации

Замечательной особенностью функции **sigmoid** является ее способность плавно сжимать бесконечный диапазон входных значений в диапазон от 0 до 1. Часто это позволяет интерпретировать выход любого отдельно взятого нейрона как вероятность. Благодаря этому свойству многие используют эту нелинейную функцию в скрытых и в выходных слоях.



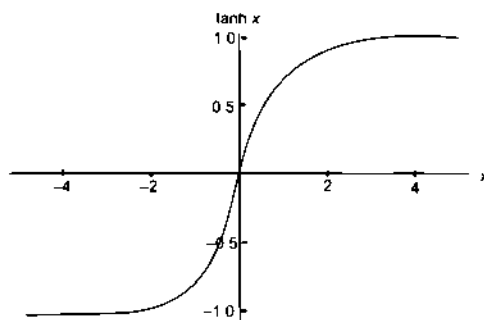
(Взято из «Википедии»)

**tanh** лучше подходит для скрытых слоев, чем **sigmoid**

Функция **tanh** обладает не менее замечательными свойствами. Помните, как мы моделировали выборочную корреляцию? Так вот, функция **sigmoid** дает

разные степени положительной корреляции. И это замечательно. Функция  $\tanh$  похожа на  $\text{sigmoid}$ , но изменяется в диапазоне от  $-1$  до  $1$ !

Это означает, что она может также усиливать *отрицательную корреляцию*. В этом мало пользы для выходного слоя (за исключением случая, когда на выходе должны получаться значения в диапазоне от  $-1$  до  $1$ ), но эта способность усиливать отрицательную корреляцию с успехом может использоваться в скрытых слоях; во многих решениях по своей эффективности в скрытых слоях  $\tanh$  превосходит  $\text{sigmoid}$ .



(Взято из Wolfram Alpha)

## Стандартные функции активации для выходного слоя

Что значит «лучше» зависит от желаемого результата

Как оказывается, функции активации, хорошо подходящие для использования в скрытых слоях, могут сильно отличаться от функций активации, которые лучше работают в выходном слое, особенно в задачах классификации. Вообще говоря, есть три разновидности выходных слоев.

### Разновидность 1: прогноз — простые числовые значения (без функции активации)

Это, пожалуй, самая простая, но наименее распространенная разновидность выходных слоев. Иногда требуется обучить нейронную сеть преобразовывать одну матрицу чисел в другую, где диапазон выходных значений (разность между наименьшим и наибольшим значением) не является диапазоном вероятностей.

Примером может служить прогноз средней температуры воздуха в Колорадо по температуре воздуха в соседних штатах.

Главное здесь обеспечить такую нелинейность в выходном слое, чтобы можно было получить достоверный прогноз. В такой ситуации функции `sigmoid` и `tanh` будут неуместны, потому что втискивают прогноз в диапазон значений от 0 до 1 (здесь требуется предсказать температуру, а не какое-то абстрактное значение между 0 и 1). Если бы я взялся сконструировать такую сеть, я бы вообще не использовал функции активации в выходном слое.

## Разновидность 2: прогноз — независимые ответы да/нет (`sigmoid`)

Часто бывает нужно получить на выходе одной нейронной сети несколько бинарных прогнозов. Мы видели такую сеть в разделе «Обучение методом градиентного спуска с несколькими входами и выходами», в главе 5, которая по входным данным прогнозировала победу, вероятность травм и моральный дух команды (ликование или огорчение).

Кстати, когда нейронная сеть имеет скрытые слои, получение нескольких прогнозов может быть даже выгодно. Часто, обучаясь предсказывать что-то одно, сеть может с выгодой использовать это знание для предсказания чего-то другого. Например, если сеть научится точно прогнозировать выигрыш команды, тот же скрытый слой наверняка пригодится ей для прогнозирования морального состояния игроков. Без такого дополнительного сигнала предсказать моральное состояние будет сложнее. Эта зависимость может быть более или менее сильной в разных задачах, но помнить о ней полезно всегда.

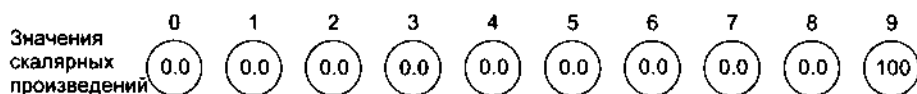
В таких случаях лучше всего использовать функцию активации `sigmoid`, потому что она моделирует вероятности для каждого узла независимо.

## Разновидность 3: прогноз — выбор одного варианта из нескольких (`softmax`)

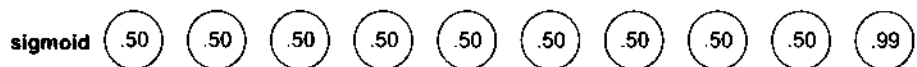
Самый распространенный вариант использования нейронных сетей — прогнозирование выбора одной метки из нескольких. Например, классификатор цифр из набора MNIST предсказывает (классифицирует) изображенную цифру. Мы заранее знаем, что на изображении может быть только одна цифра. Мы можем обучить сеть с функцией активации `sigmoid` и объявить, что наибольшая вероятность на выходе соответствует наиболее вероятной цифре. Это вполне разумное решение. Но намного лучше использовать функцию актива-

ции, моделирующую идею «чем более вероятна какая-то одна метка, тем менее вероятны все остальные».

Что даст нам такое решение? Посмотрите, как корректируются веса. Допустим, что классификатор цифр из набора MNIST должен предсказать, что изображение представляет цифру 9. Также предположим, что взвешенные суммы, поступающие в заключительный слой (до применения функции активации), имеют следующие значения:



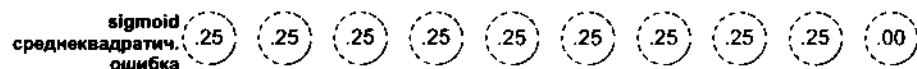
Данные, попадающие на вход последнего слоя, предсказывают 0 для всех цифр, кроме 9, и 100 для 9. Такой прогноз можно назвать идеальным. Посмотрим, что получится, если пропустить эти значения через функцию активации `sigmoid`:



Как ни странно, прогноз сети выглядит менее уверенным: цифра 9 все еще имеет самую высокую оценку вероятности, но, похоже, сеть посчитала, что с 50 %-ной вероятностью это может быть любая другая цифра. Чушь! Функция `softmax`, напротив, интерпретирует входные данные совершенно иначе:



Так намного лучше. Сеть не только посчитала цифру 9 наиболее вероятной, но даже не сомневается, что это изображение не может представлять никакую другую цифру. Такое поведение функции `sigmoid` может показаться теоретическим недостатком, но оно имеет далекоидущие последствия для обратного распространения. Посмотрим, как вычисляется среднеквадратическая ошибка для вывода, возвращаемого функцией `sigmoid`. Теоретически сеть дает почти идеальный прогноз, верно? Определенно, величина ошибки в обратном распространении будет небольшой. Но это неверно для случая с функцией `sigmoid`:



Посмотрите на эти ошибки! Все вместе они предполагают существенную корректировку весов, даже притом, что прогноз выглядит идеальным. Чем это объясняется? Чтобы с функцией `sigmoid` получить нулевую ошибку, недостаточно иметь наибольшее положительное значение в правильном выходе; нужно еще предсказать 0 во всех остальных. Если `softmax` спрашивает: «Какая цифра точнее соответствует входным значениям?», — то функция `sigmoid` говорит: «Проверьте еще раз, что это именно цифра 9, и данное изображение не имеет ничего общего с другими цифрами MNIST».

## Главная проблема: входные данные могут быть схожи между собой

Разные цифры имеют схожие черты. И хорошо, если сеть поверит в это

Цифры в наборе MNIST не являются совершенно отличными друг от друга: они имеют похожие участки. Типичная двойка имеет довольно много общего с типичной тройкой.

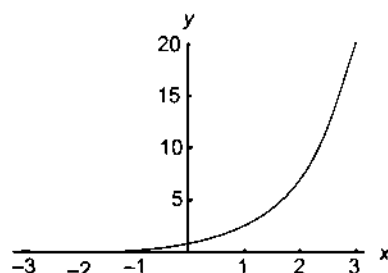
Почему это важно? Потому что схожие входные данные порождают схожий результат. Если взять несколько чисел и умножить их на матрицу, то в случае сходства этих начальных чисел вы получите схожие результаты.



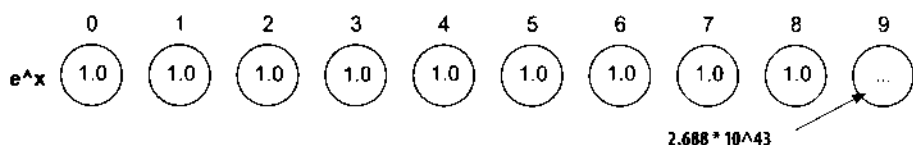
Взгляните на цифры 2 и 3, изображенные здесь. Если передать в сеть изображение с цифрой 2 и та по ошибке отдаст некоторую долю вероятности метке 3, будет ли это расценено как значительная ошибка и ответит ли сеть существенной корректировкой весов? Сеть оштрафует веса, которые распознали какую-то другую цифру, и поощрит веса, распознавшие характерные особенности 2. Например, она оштрафует веса, распознавшие в 2 верхнее закругление. Почему? Потому что такое же закругление имеется в цифре 3. Обучение с функцией `sigmoid` оштрафует сеть за попытку предсказать 2 по



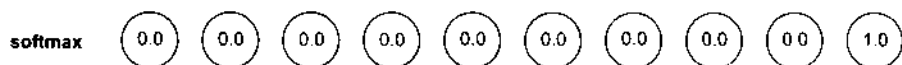
Чтобы вычислить значение функции **softmax** для всего слоя, сначала увеличим каждое значение экспоненциально. Для каждого значения  $x$  найдем  $e$  в степени  $x$  ( $e$  — это особое число  $\sim 2.71828\dots$ ). График функции  $e^x$  показан ниже.



Обратите внимание, что каждое предсказание при этом превращается в положительное число, потому что возведение в отрицательную степень дает положительное число меньше 1, а возведение в большую степень дает очень большое положительное число. (Если вам приходилось слышать об экспоненциальном росте, вероятнее всего, речь шла об этой или похожей функции.)



Проще говоря, все 0 превращаются в 1 (потому что 1 — это ордината точки пересечения кривой  $e^x$  с осью  $y$ ), а 100 превращается в гигантское число (2 с 43 нулями). Если бы имелись какие-то отрицательные числа, они превратились бы в положительные значения между 0 и 1. Следующий шаг — суммирование всех узлов в слое и деление каждого выходного значения на эту сумму. В результате этого все выходные значения, кроме соответствующего цифре 9, превратятся в 0.



Благодаря функции **softmax** чем выше будет одно из предсказанных значений, тем ниже будут все остальные. Она усиливает так называемую *резкость затухания* и побуждает сеть спрогнозировать один выход с очень высокой вероятностью.

Для регулировки агрессивности этой функции можно использовать другие числа взамен  $\epsilon$ . Чем меньше число, тем более пологим будет затухание, чем больше число, тем круче будет затухание. Однако многие предпочитают придерживаться числа  $\epsilon$ .

## Инструкции по внедрению функций активации

### Как внедрить выбранную функцию активации в тот или иной слой?

Теперь, рассмотрев широкий спектр функций активации и выяснив, чем они могут быть полезны в скрытых и выходных слоях нейронных сетей, поговорим о том, как правильно внедрить их в нейронную сеть. Выше мы уже видели пример внедрения нелинейности в нашу первую глубокую сеть: там мы добавили функцию активации `relu` в скрытый слой. Внедрение функции в поток прямого распространения не вызвало у нас никаких сложностей — мы просто применили функцию `relu` к каждому входному значению слоя `layer_1`:

```
layer_0 = images[i:i+1]
layer_1 = relu(np.dot(layer_0, weights_0_1))
layer_2 = np.dot(layer_1, weights_1_2)
```

Давайте кое-что уточним. Под *входными значениями слоя* здесь подразумеваются значения, полученные до применения функции активации. В данном случае на вход слоя `layer_1` подается `np.dot(layer_0, weights_0_1)`. Не путайте эти входные значения со значениями в предыдущем слое `layer_0`.

Внедрение функции активации в поток прямого распространения выполняется относительно просто. Но правильная компенсация влияния функции активации на этапе обратного распространения — несколько более сложная задача.

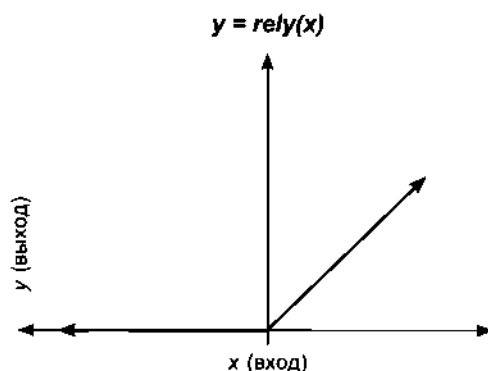
В главе 6 мы выполнили интересную операцию, создав переменную `layer_1_delta`. Всякий раз, когда `relu` преобразовывала значение в слое `layer_1` в 0, мы также умножали соответствующую разность `delta` на 0. Тогда мы аргументировали это так: «Поскольку значение в слое `layer_1` равно 0, оно не влияло на прогноз, а значит не должно влиять и на соответствующий вес, потому что не вносило вклада в ошибку». Это частный случай более общего свойства. Рассмотрим форму функции `relu`.

Производная (наклон) функции `relu` для положительных чисел равна 1, а для отрицательных 0. Изменение входного значения этой функции (на небольшую



величину) произведет эффект 1:1, если внесло положительный вклад в прогноз, и эффект 0:1 (то есть нет эффекта), если внесло отрицательный вклад в прогноз. Эта производная определяет, насколько изменение входного значения повлияет на выходное.

Цель разности `delta` на этом этапе — сообщить предыдущим слоям, что в следующий раз те должны уменьшить или увеличить входное значение данного слоя, поэтому разность `delta` играет очень важную роль. Она изменяет разность `delta`, распространяемую в обратном направлении от следующего слоя к предыдущему, чтобы учесть вклад данного узла в ошибку.



То есть чтобы получить `layer_1_delta` в процессе обратного распространения, нужно умножить разность `delta`, передаваемую обратно из слоя `layer_2` (`layer_2_delta.dot(weights_1_2.T)`), на производную `relu` в точке, *предсказанной в процессе прямого распространения*. Для одних разностей `delta` производная будет равна 1 (положительные числа), а для других 0 (отрицательные числа):

```
error += np.sum((labels[i:i+1] - layer_2) ** 2)

correct_cnt += int(np.argmax(layer_2) == \
                    np.argmax(labels[i:i+1]))

layer_2_delta = (labels[i:i+1] - layer_2)
layer_1_delta = layer_2_delta.dot(weights_1_2.T) \
                * relu2deriv(layer_1)

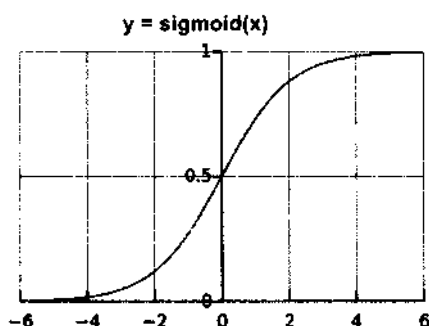
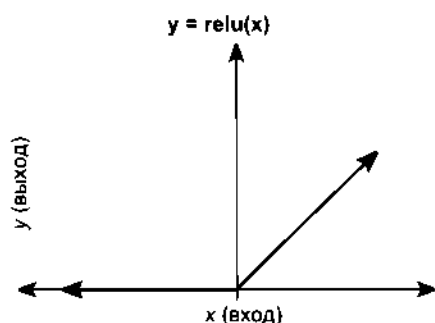
weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)

def relu(x):
```

```
return (x >= 0) * x ← Возвращает x, если x > 0; иначе возвращает 0
```

```
def relu2deriv(output):
    return output >= 0 ← Возвращает 1, если output > 0; иначе возвращает 0
```

`relu2deriv` — специальная функция, которая принимает значения, возвращаемые функцией `relu`, и вычисляет производную `relu` в этой точке (это делается для всех значений в выходном векторе). Теперь встает вопрос: как реализовать аналогичные корректировки для других функций активации, отличных от `relu`? Рассмотрим функции `relu` и `sigmoid`:

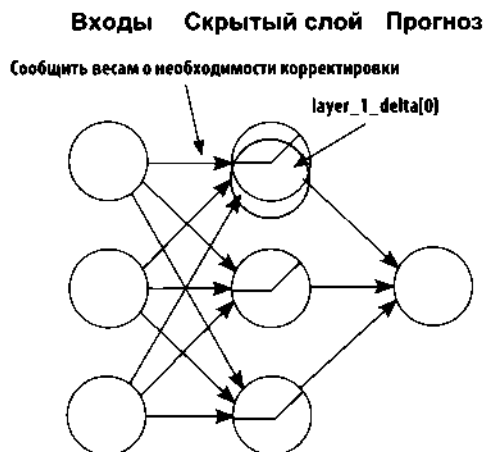


Напомню еще раз, что производная (наклон) кривой определяет, насколько *незначительное* изменение входных данных повлияет на выходные. Мы должны изменить входящую разность `delta` (из следующего слоя), чтобы учесть влияние коррекции веса перед этим узлом. Напомню, что конечной целью является корректировка весов для уменьшения ошибки. Этот шаг побуждает сеть оставить веса без изменения, если их корректировка почти или совсем никак не скажется на прогнозе, а достигается это умножением на производную. Ситуация с функцией `sigmoid` немного иная.

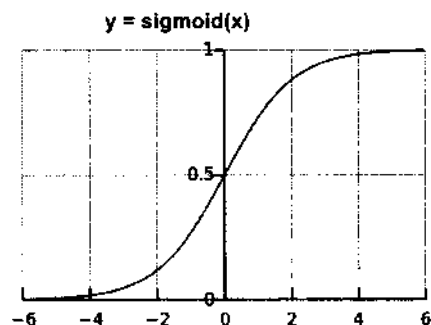
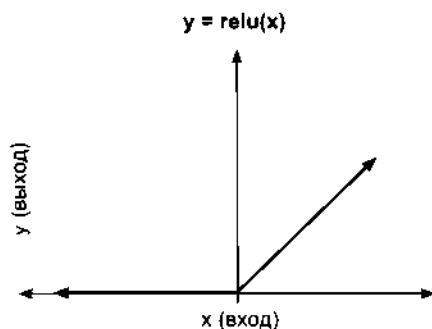
## Умножение разности на производную

Чтобы получить `layer_delta`, умножьте распространяемую обратно разность на производную слоя

`layer_1_delta[0]` определяет, насколько выше или ниже должно быть значение первого узла в скрытом слое `layer_1`, чтобы уменьшить ошибку сети (для данного обучающего примера). В отсутствие нелинейности это значение равно средневзвешенной разности `delta` слоя `layer_2`.



Но конечная цель разности **delta** для нейрона состоит в том, чтобы сообщить весам о необходимости корректировки. Если корректировка не даст желаемого эффекта, они (как группа) должны остаться неизменными. Это очевидно при использовании функции **relu**, которая может возвращать всего два значения: включено или выключено. В случае с функцией **sigmoid** ситуация немного сложнее.



Рассмотрим единственный нейрон с функцией активации **sigmoid**. Чувствительность **sigmoid** к изменению входного значения постепенно растет при его приближении к 0 (с любой стороны). Но для очень больших по величине положительных или отрицательных значений производная близка к 0. Соответственно, когда входное значение оказывается слишком большим по величине положительным или отрицательным числом, небольшие изменения входящих весов вносят меньший вклад в ошибку нейрона на данном обучающем примере. Вообще говоря, многие узлы скрытого слоя не влияют на точность распознавания цифры 2 (например, они могут использоваться только при распознава-

нии цифры 8). Мы не должны оказывать большого влияния на их веса, чтобы не уменьшить их ценность при распознавании других примеров.

С другой стороны, такая нелинейность создает эффект *жесткости*. Веса, ранее многократно корректировавшиеся в одном направлении (при анализе похожих обучающих примеров), надежно предсказывают высокое или низкое значение. В этом случае нелинейность помогает уменьшить влияние случайных ошибок в обучающих примерах и предотвращает искажение весов, которые перед этим многократно усиливались.

## Преобразование выхода в наклон (производную)

Большинство функций активации может преобразовывать их выход в наклон. (Выигрыш в эффективности!)

Теперь, зная как внедрить функцию активации в слой, чтобы повлиять на вычисление разности *delta* в этом слое, поговорим об эффективности. Внедрение функции активации порождает необходимость в новой операции — вычислении ее производной.

Для большинства функций активации (в том числе для всех популярных) используется метод вычисления производной, который может удивить тех из вас, кто знаком с исчислением. Вместо вычисления производной в определенной точке кривой обычным способом большинство популярных функций активации позволяет вычислять производную с использованием *выхода* слоя (полученного при прямом распространении). Этот подход стал стандартной практикой вычисления производных в нейронных сетях, и он очень удобен.

В следующей небольшой таблице перечислены функции, которые вы уже видели, вместе с их производными. Здесь *input* — это вектор NumPy (соответствует входу слоя); *output* — прогноз (выход) слоя; *deriv* — производная вектора производных активации, соответствующих производным активации в каждом узле; *true* — вектор истинных значений (обычно содержит 1 в позиции, соответствующей правильной метке, и 0 во всех остальных).

| Функция | Прямое распространение                                        | Обратно распространяемая разность                 |
|---------|---------------------------------------------------------------|---------------------------------------------------|
| relu    | ones_and_zeros = (input > 0)<br>output = input*ones_and_zeros | mask = output > 0<br>deriv = output * mask        |
| sigmoid | output = 1/(1 + np.exp(-input))                               | deriv = output*(1-output)                         |
| tanh    | output = np.tanh(input)                                       | deriv = 1 - (output**2)                           |
| softmax | temp = np.exp(input)<br>output /= np.sum(temp)                | temp = (output - true)<br>output = temp/len(true) |

Обратите внимание, что вычисление `delta` для `softmax` — особый случай, потому что эта функция активации используется только в последнем слое. Мы могли бы продолжить углубляться в детали происходящего, но для этого недостаточно места в книге. Поэтому я предлагаю двинуться дальше и внедрить некоторые из лучших функций активации в сеть классификации MNIST.

## Усовершенствование сети MNIST

### Усовершенствуем сеть MNIST, применив новые знания

Теоретически `tanh` является лучшей функцией активации для скрытого слоя, а `softmax` — лучшей функцией активации для выходного слоя. Судя по результатам проверки, они позволяют добиться лучших результатов. Но, как всегда, все не так просто, как кажется.

Мне пришлось внести пару корректировок, чтобы правильно настроить сеть на работу с этими функциями активации. Для `tanh` мне пришлось уменьшить стандартное отклонение входящих весов. Напомню, что мы инициализируем веса случайными числами. Вызов `np.random.random` создает матрицу со случайными значениями из диапазона между 0 и 1. Умножая эти числа на 0.2 и вычитая 0.1, мы переводим их в диапазон между  $-0.1$  и  $0.1$ . Этот прием позволяет получить отличные результаты при использовании `relu`, но плохо подходит для `tanh`. Функция `tanh` дает лучшие результаты при более узком диапазоне исходных значений весов, поэтому я уместил их в диапазон между  $-0.01$  и  $0.01$ .

Я также удалил вычисление ошибки, потому что мы пока не готовы к этому. Технически функцию активации `softmax` лучше использовать в паре с функцией ошибки, которая называется *перекрестной энтропией*. Эта сеть правильно вычисляет `layer_2_delta` для этой меры ошибки, но, поскольку мы еще не обсудили преимущества использования функции ошибки, я удалил соответствующие строки из листинга.

Наконец, так же как при внесении любых других изменений в нейронную сеть, я пересмотрел настройку альфа-коэффициента. Я выяснил, что для достижения лучшего результата за 300 итераций необходимо значительно увеличить альфа-коэффициент. Как и ожидалось, сеть достигла на этапе проверки более высокой точности в 87 %.

```
import numpy as np, sys
np.random.seed(1)

from keras.datasets import mnist
```

```

(x_train, y_train), (x_test, y_test) = mnist.load_data()

images, labels = (x_train[0:1000].reshape(1000,28*28)\
                  / 255, y_train[0:1000])

one_hot_labels = np.zeros((len(labels),10))
for i,l in enumerate(labels):
    one_hot_labels[i][l] = 1
labels = one_hot_labels

test_images = x_test.reshape(len(x_test),28*28) / 255
test_labels = np.zeros((len(y_test),10))
for i,l in enumerate(y_test):
    test_labels[i][l] = 1

def tanh(x):
    return np.tanh(x)
def tanh2deriv(output):
    return 1 - (output ** 2)
def softmax(x):
    temp = np.exp(x)
    return temp / np.sum(temp, axis=1, keepdims=True)

alpha, iterations, hidden_size = (2, 300, 100)
pixels_per_image, num_labels = (784, 10)
batch_size = 100

weights_0_1 = 0.02*np.random.random((pixels_per_image,hidden_size))-0.01
weights_1_2 = 0.2*np.random.random((hidden_size,num_labels)) - 0.1

for j in range(iterations):
    correct_cnt = 0
    for i in range(int(len(images) / batch_size)):
        batch_start, batch_end=((i * batch_size),((i+1)*batch_size))
        layer_0 = images[batch_start:batch_end]
        layer_1 = tanh(np.dot(layer_0,weights_0_1))
        dropout_mask = np.random.randint(2,size=layer_1.shape)
        layer_1 *= dropout_mask * 2
        layer_2 = softmax(np.dot(layer_1,weights_1_2))

        for k in range(batch_size):
            correct_cnt += int(np.argmax(layer_2[k:k+1]) == \
                                     np.argmax(labels[batch_start+k:batch_start+k+1]))
        layer_2_delta = (labels[batch_start:batch_end]-layer_2)\
                        / (batch_size * layer_2.shape[0])
        layer_1_delta = layer_2_delta.dot(weights_1_2.T) \
                        * tanh2deriv(layer_1)
        layer_1_delta *= dropout_mask

        weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
        weights_0_1 += alpha * layer_0.T.dot(layer_1_delta)
    test_correct_cnt = 0

    for i in range(len(test_images)):

```

```

layer_0 = test_images[i:i+1]
layer_1 = tanh(np.dot(layer_0,weights_0_1))
layer_2 = np.dot(layer_1,weights_1_2)
test_correct_cnt += int(np.argmax(layer_2) == \
                             np.argmax(test_labels[i:i+1]))

if(j % 10 == 0):
    sys.stdout.write("\n" + "I:" + str(j) + \
        " Test-Acc:" + str(test_correct_cnt/float(len(test_images))) + \
        " Train-Acc:" + str(correct_cnt/float(len(images))))

I:0 Test-Acc:0.394 Train-Acc:0.156      I:150 Test-Acc:0.8555 Train-Acc:0.914
I:10 Test-Acc:0.6867 Train-Acc:0.723   I:160 Test-Acc:0.8577 Train-Acc:0.925
I:20 Test-Acc:0.7025 Train-Acc:0.732   I:170 Test-Acc:0.8596 Train-Acc:0.918
I:30 Test-Acc:0.734 Train-Acc:0.763     I:180 Test-Acc:0.8619 Train-Acc:0.933
I:40 Test-Acc:0.7663 Train-Acc:0.794   I:190 Test-Acc:0.863 Train-Acc:0.933
I:50 Test-Acc:0.7913 Train-Acc:0.819   I:200 Test-Acc:0.8642 Train-Acc:0.926
I:60 Test-Acc:0.8102 Train-Acc:0.849   I:210 Test-Acc:0.8653 Train-Acc:0.931
I:70 Test-Acc:0.8228 Train-Acc:0.864   I:220 Test-Acc:0.8668 Train-Acc:0.93
I:80 Test-Acc:0.831 Train-Acc:0.867    I:230 Test-Acc:0.8672 Train-Acc:0.937
I:90 Test-Acc:0.8364 Train-Acc:0.885   I:240 Test-Acc:0.8681 Train-Acc:0.938
I:100 Test-Acc:0.8407 Train-Acc:0.88   I:250 Test-Acc:0.8687 Train-Acc:0.937
I:110 Test-Acc:0.845 Train-Acc:0.891   I:260 Test-Acc:0.8684 Train-Acc:0.945
I:120 Test-Acc:0.8481 Train-Acc:0.90   I:270 Test-Acc:0.8703 Train-Acc:0.951
I:130 Test-Acc:0.8505 Train-Acc:0.90   I:280 Test-Acc:0.8699 Train-Acc:0.949
I:140 Test-Acc:0.8526 Train-Acc:0.90   I:290 Test-Acc:0.8701 Train-Acc:0.94

```

# 10

## Края и углы нейронного обучения: введение в сверточные нейронные сети



### В этой главе

- ✓ Повторное использование весов в нескольких местах.
- ✓ Сверточный слой.

Операция объединения, используемая в сверточных нейронных сетях, — это большая ошибка, а тот факт, что она помогает добиться лучших результатов — катастрофа.

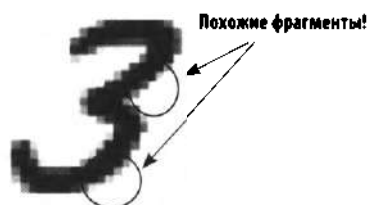
*Джеффри Хинтон (Geoffrey Hinton),  
из статьи «Ask Me Anything» на Reddit*



## Повторное использование весов в нескольких местах

Используйте те же веса, чтобы определить  
одну и ту же особенность в нескольких местах!

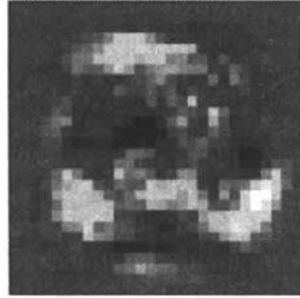
Самая большая проблема нейронных сетей — переобучение, когда нейронная сеть запоминает набор исходных данных вместо выявления полезных абстракций, которые можно обобщить на другие похожие данные. Иначе говоря, нейронная сеть учится предсказывать на основе шума в наборе данных, игнорируя основополагающий сигнал (вспомните аналогию с оттиском вилки в глине).



Переобучение часто обусловлено наличием большего числа параметров, чем необходимо для изучения закономерностей в конкретном наборе данных. В этом случае сеть имеет так много параметров, что может запомнить каждую деталь в обучающем наборе (нейронная сеть: «Ага! Я снова вижу изображение с номером 363. А я запомнила, что это изображение цифры 2»), вместо того чтобы выявить обобщенную абстракцию (нейронная сеть: «Хм-м, здесь я вижу дугу сверху, завитушку внизу слева и хвостик внизу справа; должно быть, это цифра 2»). Когда нейронная сеть имеет слишком много параметров и мало обучающих примеров, переобучения трудно избежать.

Мы подробно рассмотрели эту тему в главе 8, когда знакомились с регуляризацией как средством борьбы с переобучением. Но регуляризация — не единственный (и далеко не лучший) способ противостоять переобучению.

Как уже отмечалось, склонность к переобучению определяется отношением между количеством весов в модели и количеством точек данных, на основе которых происходит обучение весов. Следовательно, существует более эффективный метод борьбы с переобучением. Всегда, когда возможно, желательно использовать то, что неопределенно называют *структурой*.



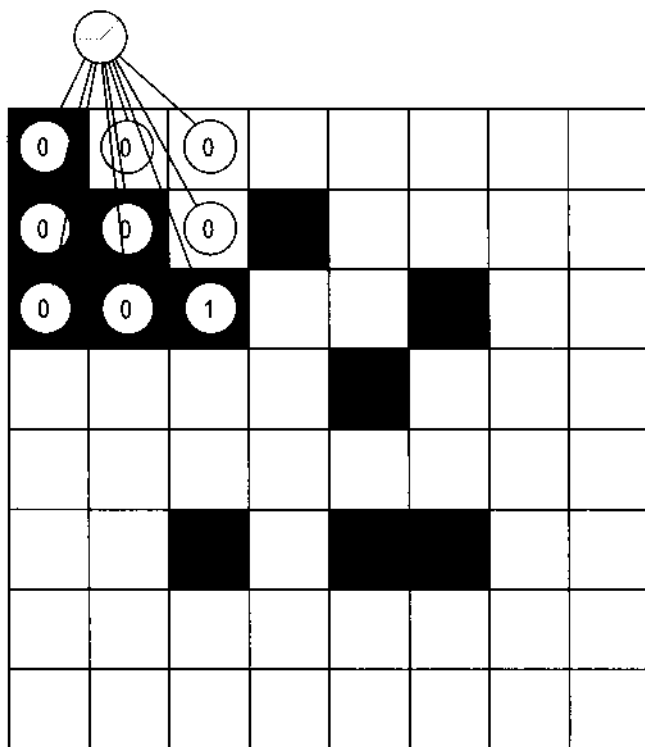
Под структурой здесь понимается повторное использование отдельных весов для нескольких целей, когда ожидается, что одна и та же закономерность будет проявляться в нескольких местах. Как вы увидите далее, этот подход способствует ослаблению эффекта переобучения и увеличению точности моделей за счет уменьшения отношения количества весов к количеству точек данных.

Но если уменьшение количества параметров обычно делает модель менее выразительной (менее способной к выявлению закономерностей), то при грамотной организации повторного использования весов модель может остаться такой же выразительной и при этом более устойчивой к переобучению. Как ни удивительно, но этот метод также способствует уменьшению моделей (так как требуется хранить меньше параметров). Наиболее известной и широко используемой структурой в нейронных сетях является *свертка*, а когда она используется как слой, ее называют *сверточным слоем*.

## Сверточный слой

Вместо одного большого слоя в каждой позиции повторно используется множество очень маленьких линейных слоев

Основная идея сверточного слоя состоит в том, чтобы вместо одного большого, плотного, линейного слоя, связывающего каждый вход с каждым выходом, использовать в каждой позиции на входе множество очень маленьких линейных слоев, обычно имеющих не более 25 входов и один выход. Каждый такой мини-слой называется *сверточным ядром*, но в действительности это всего лишь дочерние линейные слои с небольшим числом входов и одним выходом.



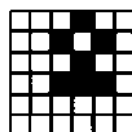
На этом рисунке показано единственное сверточное ядро размером  $3 \times 3$ . Оно сгенерирует прогноз для текущего местоположения, передвинется на один пиксел вправо, снова сгенерирует прогноз, опять сместится вправо на один пиксел и так далее. Просканировав все изображение по ширине, оно сместится на один пиксел вниз и повторит процесс сканирования справа налево. Эти действия будут повторяться, пока не будут просканированы все возможные местоположения внутри изображения. Результатом станет меньший квадрат с предсказаниями, который послужит входом для следующего слоя. Обычно сверточные слои имеют много ядер.

Внизу справа на рисунке изображены четыре разных сверточных ядра, сканирующих одно и то же изображение  $8 \times 8$  цифры 2. Каждое ядро дает в результате матрицу прогноза  $6 \times 6$ . Результатом всего сверточного слоя с четырьмя ядрами  $3 \times 3$  являются четыре матрицы прогноза  $6 \times 6$ . Мы можем сложить эти матрицы поэлементно (объединение суммированием), взять поэлементное среднее (объединение с усреднением) или выбрать максимальные значения из соответствующих элементов (объединение выбором максимального).

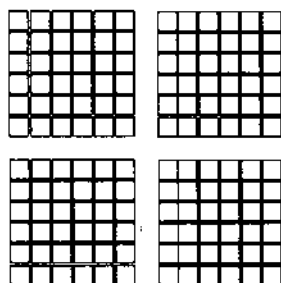
Последний способ пользуется наибольшей популярностью: для каждой позиции из результатов всех четырех ядер выбираются максимальные значения и копируются в окончательную матрицу  $6 \times 6$ , изображенную справа вверху на рисунке. Эта (и только эта) окончательная матрица передается следующему уровню.

Отметим несколько важных аспектов на этом рисунке. Во-первых, ядро внизу справа передает дальше 1, только если сфокусировано на сегменте горизонтальной линии. Ядро внизу слева передает дальше 1, только если сфокусировано на диагональной линии, направленной вправо и вверх. То есть ядро внизу справа не обнаруживает никаких закономерностей, выявлять которые оно обучено.

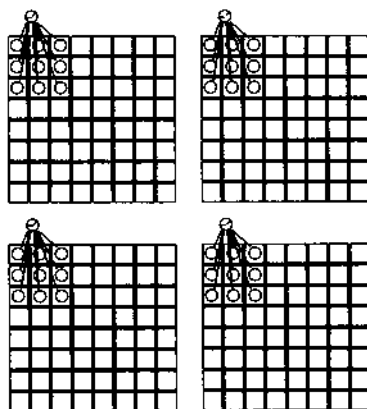
Важно понимать, что этот прием позволяет каждому ядру изучить определенную закономерность и затем обнаруживать ее везде, где она присутствует в изображении. Один небольшой набор весов может обучаться на намного более обширном наборе обучающих примеров, потому что даже при неизменности исходного набора данных каждое ядро просматривает множество сегментов данных, из-за чего меняется отношение количества весов к количеству точек данных, на которых обучаются эти веса. Это оказывает большое влияние на сеть, резко ухудшая ее способность переобучаться на обучающих данных и улучшая ее способность к обобщению.



Выборкой максимальных значений из результатов всех ядер формируется значимое представление, которое передается следующему слою



Результаты каждого из четырех ядер для каждой позиции



Четыре сверточных ядра, сканирующих одно и то же изображение 2

## Простая реализация в NumPy

Просто вспомните, что речь идет о линейных мини-слоях, и вы уже знаете все, что нужно

Начнем с прямого распространения. Следующий метод демонстрирует, как выбрать подобласть в пакете изображений с помощью NumPy. Обратите внимание, что он выбирает одну и ту же подобласть во всем пакете:

```
def get_image_section(layer, row_from, row_to, col_from, col_to):
    sub_section = layer[:, row_from:row_to, col_from:col_to]
    return sub_section.reshape(-1, 1, row_to - row_from, col_to - col_from)
```

Теперь посмотрим, как этот метод используется. Так как подобласть выбирается в пакете входных изображений, мы должны вызвать его несколько раз (для каждой позиции в изображении). Ниже показан соответствующий цикл `for`:

```
layer_0 = images[batch_start:batch_end]
layer_0 = layer_0.reshape(layer_0.shape[0], 28, 28)
layer_0.shape

sects = list()
for row_start in range(layer_0.shape[1] - kernel_rows):
    for col_start in range(layer_0.shape[2] - kernel_cols):
        sect = get_image_section(layer_0,
                                row_start,
                                row_start + kernel_rows,
                                col_start,
                                col_start + kernel_cols)
        sects.append(sect)

expanded_input = np.concatenate(sects, axis=1)
es = expanded_input.shape
flattened_input = expanded_input.reshape(es[0] * es[1], -1)
```

Здесь `layer_0` — это пакет изображений  $28 \times 28$ . Цикл `for` последовательно выбирает подобласти (`kernel_rows`  $\times$  `kernel_cols`) в изображениях и помещает их в список `sects`. Далее этот список объединяется и переформируется особым образом.

Будем считать (пока), что каждая подобласть — это отдельное изображение. Таким образом, для 8 изображений в пакете и 100 подобластей в каждом мы получим 800 изображений меньшего размера. Передача их в прямом направлении через линейный слой с одним выходным нейроном равносильна получению прогноза из этого линейного слоя для каждой подобласти в каждом пакете (приостановитесь и убедитесь, что понимаете, о чем речь).

Если, напротив, передать изображения в линейный слой с  $n$  выходными нейронами, на выходе мы получим тот же результат, что и при использовании  $n$  линейных слоев (ядер) в каждой позиции в изображении. Мы выбрали такой путь, потому что в этом случае код получается более простым и быстрым:

```
kernels = np.random.random((kernel_rows*kernel_cols,num_kernels))
...
kernel_output = flattened_input.dot(kernels)
```

В следующем листинге показана полная реализация на основе NumPy:

```
import numpy as np, sys
np.random.seed(1)

from keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

images, labels = (x_train[0:1000].reshape(1000,28*28) / 255,
                  y_train[0:1000])

one_hot_labels = np.zeros((len(labels),10))
for i,l in enumerate(labels):
    one_hot_labels[i][l] = 1
labels = one_hot_labels

test_images = x_test.reshape(len(x_test),28*28) / 255
test_labels = np.zeros((len(y_test),10))
for i,l in enumerate(y_test):
    test_labels[i][l] = 1

def tanh(x):
    return np.tanh(x)

def tanh2deriv(output):
    return 1 - (output ** 2)

def softmax(x):
    temp = np.exp(x)
    return temp / np.sum(temp, axis=1, keepdims=True)

alpha, iterations = (2, 300)
pixels_per_image, num_labels = (784, 10)
batch_size = 128

input_rows = 28
input_cols = 28

kernel_rows = 3
kernel_cols = 3
num_kernels = 16
```

```

hidden_size = ((input_rows - kernel_rows) *
               (input_cols - kernel_cols)) * num_kernels

kernels = 0.02*np.random.random((kernel_rows*kernel_cols,
                                num_kernels))-0.01

weights_1_2 = 0.2*np.random.random((hidden_size,
                                    num_labels)) - 0.1

def get_image_section(layer,row_from, row_to, col_from, col_to):
    section = layer[:,row_from:row_to,col_from:col_to]
    return section.reshape(-1,1,row_to-row_from, col_to-col_from)

for j in range(iterations):
    correct_cnt = 0
    for i in range(int(len(images) / batch_size)):
        batch_start, batch_end=((i * batch_size),((i+1)*batch_size))
        layer_0 = images[batch_start:batch_end]
        layer_0 = layer_0.reshape(layer_0.shape[0],28,28)
        layer_0.shape

        sects = list()
        for row_start in range(layer_0.shape[1]-kernel_rows):
            for col_start in range(layer_0.shape[2] - kernel_cols):
                sect = get_image_section(layer_0,
                                        row_start,
                                        row_start+kernel_rows,
                                        col_start,
                                        col_start+kernel_cols)
                sects.append(sect)

        expanded_input = np.concatenate(sects,axis=1)
        es = expanded_input.shape
        flattened_input = expanded_input.reshape(es[0]*es[1],-1)

        kernel_output = flattened_input.dot(kernels)
        layer_1 = tanh(kernel_output.reshape(es[0],-1))
        dropout_mask = np.random.randint(2,size=layer_1.shape)
        layer_1 *= dropout_mask * 2
        layer_2 = softmax(np.dot(layer_1,weights_1_2))

        for k in range(batch_size):
            labelset = labels[batch_start+k:batch_start+k+1]
            _inc = int(np.argmax(layer_2[k:k+1]) ==
                      np.argmax(labelset))
            correct_cnt += _inc

        layer_2_delta = (labels[batch_start:batch_end]-layer_2)\
            / (batch_size * layer_2.shape[0])
        layer_1_delta = layer_2_delta.dot(weights_1_2.T) * \
            tanh2deriv(layer_1)
        layer_1_delta *= dropout_mask

```

```

weights_1_2 += alpha * layer_1.T.dot(layer_2_delta)
l1d_reshape = layer_1_delta.reshape(kernel_output.shape)
k_update = flattened_input.T.dot(l1d_reshape)
kernels -= alpha * k_update

test_correct_cnt = 0

for i in range(len(test_images)):

    layer_0 = test_images[i:i+1]
    layer_0 = layer_0.reshape(layer_0.shape[0],28,28)
    layer_0.shape

    sects = list()
    for row_start in range(layer_0.shape[1]-kernel_rows):
        for col_start in range(layer_0.shape[2] - kernel_cols):
            sect = get_image_section(layer_0,
                                     row_start,
                                     row_start+kernel_rows,
                                     col_start,
                                     col_start+kernel_cols)

            sects.append(sect)

    expanded_input = np.concatenate(sects,axis=1)
    es = expanded_input.shape
    flattened_input = expanded_input.reshape(es[0]*es[1],-1)

    kernel_output = flattened_input.dot(kernels)
    layer_1 = tanh(kernel_output.reshape(es[0],-1))
    layer_2 = np.dot(layer_1,weights_1_2)

    test_correct_cnt += int(np.argmax(layer_2) ==
                               np.argmax(test_labels[i:i+1]))

if(j % 1 == 0):
    sys.stdout.write("\n" + \
        "I:" + str(j) + \
        " Test-Acc:"+str(test_correct_cnt/float(len(test_images)))+\
        " Train-Acc:" + str(correct_cnt/float(len(images))))

I:0 Test-Acc:0.0288 Train-Acc:0.055
I:1 Test-Acc:0.0273 Train-Acc:0.037
I:2 Test-Acc:0.028 Train-Acc:0.037
I:3 Test-Acc:0.0292 Train-Acc:0.04
I:4 Test-Acc:0.0339 Train-Acc:0.046
I:5 Test-Acc:0.0478 Train-Acc:0.068
I:6 Test-Acc:0.076 Train-Acc:0.083
I:7 Test-Acc:0.1316 Train-Acc:0.096
I:8 Test-Acc:0.2137 Train-Acc:0.127
....
I:297 Test-Acc:0.8774 Train-Acc:0.816
I:298 Test-Acc:0.8774 Train-Acc:0.804
I:299 Test-Acc:0.8774 Train-Acc:0.814

```



Как видите, замена первого слоя в сети из главы 9 сверточным слоем дала еще несколько процентов к точности прогноза. Выход сверточного слоя (`kernel_output`) сам является серией двумерных изображений (выход каждого ядра в каждой позиции входных изображений).

В большинстве случаев используется сразу несколько сверточных слоев, накладываемых друг на друга так, что каждый следующий сверточный слой интерпретирует результат предыдущего как входное изображение. (Не стесняйтесь использовать этот прием в своих проектах; он поможет добиться более высокой точности.)

Возможность комбинирования сверточных слоев является одним из основных достижений, которые сделали возможным создание очень глубоких нейронных сетей (и, как следствие, способствовали популяризации словосочетания *глубокое обучение*). Невозможно преувеличить значимость этого изобретения для индустрии; без него мы могли бы до сих пор пребывать в зиме искусственного интеллекта.

## Итоги

### Повторное использование весов является самым важным нововведением в глубоком обучении

Сверточные нейронные сети — это намного более общее направление развития, чем можно представить. Идея повторного использования весов для повышения точности чрезвычайно важна и основывается на простом и понятном фундаменте. Попробуйте перечислить, что нужно выяснить, чтобы определить, что на изображении присутствует кошка. Во-первых, определить палитру цветов, затем линии и контуры, мелкие формы и, наконец, комбинации признаков, характерных для кошки. Точно так же и нейронные сети должны изучить эти мелкие признаки (такие, как линии и контуры), а за изучение линий и контуров отвечают веса.

Но если для анализа разных частей изображения используются разные веса, каждая группа весов должна независимо изучить, что такое линия. Почему? Все просто: если одна группа весов, просматривающая свою часть изображения, узнает, что такое линия, она не сможет поделиться этой информацией с другой группой весов, потому что та находится в другой части сети.

Свертки позволяют воспользоваться преимуществом совместного обучения. Иногда бывает нужно использовать ту же идею или знания в нескольких

местах; и в таком случае следует попробовать использовать в этих местах те же веса. Это подводит нас к самой главной идее этой книги. Если вы чего-то не знаете, научитесь этому.

### **ТРИК СО СТРУКТУРОЙ**

Когда нейронная сеть должна распознавать одну и ту же закономерность в нескольких местах, попробуйте использовать в этих местах одни и те же веса. Благодаря этому веса получатся более интеллектуальными, потому что получают больше образцов для изучения, и более обобщенными.

Многие из крупнейших разработок в области глубокого обучения, полученных за последние пять лет (или раньше), являются повторением этой идеи. Свертки, рекуррентные нейронные сети (Recurrent Neural Network, RNN), векторные пространства слов и недавно предложенные капсульные сети (capsule networks) можно рассматривать как дальнейшее развитие этой идеи. Когда известно, что сети понадобится выявлять одни и те же закономерности в нескольких местах, заставьте ее использовать одинаковые веса в этих местах. Я уверен, что эта идея приведет еще к многим открытиям в глубоком обучении, потому что пока сложно представить новые абстрактные идеи, которые нейронные сети могли бы многократно использовать в своей архитектуре.

# 11

## Нейронные сети, понимающие человеческий язык: король – мужчина + женщина == ?



### В этой главе

- ✓ Обработка естественного языка (NLP).
- ✓ Обработка естественного языка с учителем.
- ✓ Выявление корреляций между словами во входных данных.
- ✓ Введение в слой с векторным представлением.
- ✓ Нейронная архитектура.
- ✓ Сравнение векторных представлений слов.
- ✓ Подстановка пропущенных слов.
- ✓ Смысл определяется потерями.
- ✓ Словесные аналогии.

Человек медлителен, неаккуратен, но блестящий мыслитель; компьютеры быстры, точны, но не умеют думать.

Джон Пфайффер (John Pfeiffer), из книги «Fortune». 1961

## Что значит понимать человеческий язык?

### Какие предсказания можно делать в отношении языка?

До сих пор мы использовали нейронные сети для моделирования изображений. Но их можно использовать для исследования гораздо более широкого спектра данных. Исследование новых наборов данных также поможет нам узнать много нового о нейронных сетях в целом, потому что разные наборы данных часто требуют применения разных способов обучения нейронных сетей из-за разных проблем, скрытых в них.



Мы начнем эту главу со знакомства с гораздо более старой областью, которая перекрывается областью глубокого обучения: *обработкой естественного языка* (natural language processing, NLP). Эта область посвящена исключительно автоматизированному изучению человеческого языка (ранее не использовавшему методы глубокого обучения). И на протяжении главы обсудим основы применения глубокого обучения в этой сфере.

## Обработка естественного языка (NLP)

Обработка естественного языка делится на коллекцию задач или проблем

Лучший способ познакомиться с NLP, как мне кажется, — рассмотреть некоторые задачи, которые сообщество исследователей NLP стремится решить. Вот несколько видов задач классификации, типичных для NLP:

- ❑ Определение *начала и конца слов по символам* в документе.
- ❑ Определение *начала и конца предложений по словам* в документе.
- ❑ Определение *части речи* каждого слова в предложении.
- ❑ Определение *начала и конца словосочетаний по словам* в предложении.
- ❑ Определение *начала и конца именованных сущностей* (имен, людей, географических названий) по словам в предложении.
- ❑ Определение *человека/места/предмета*, к которому относятся местоимения, по предложениям в документе.
- ❑ Определение *эмоциональной окраски предложения по словам* в нем.

В общем случае задачи NLP преследуют три цели: классифицировать область текста (например, определить часть речи, эмоциональную окраску или именованную сущность); связать несколько областей в тексте (например, выяснить кореферентность, то есть действительно ли два упоминания объекта реального мира относятся к одному и тому же объекту, которым может быть человек, географический объект или другая именованная сущность); или попытаться восполнить отсутствующую информацию (пропущенное слово) по контексту.

Возможно, вы заметили, что машинное обучение и NLP тесно переплетены между собой. До недавнего времени большинство современных алгоритмов NLP было представлено продвинутыми вероятностными непараметрическими моделями (не имеющими отношения к глубокому обучению). Но появление и популяризация двух основных нейронных алгоритмов — нейронного векторного представления слов и рекуррентных нейронных сетей (recurrent neural network, RNN) — способствовали проникновению глубокого обучения в область NLP.

В этой главе мы реализуем алгоритм векторного представления слов и посмотрим, как он способствует увеличению точности алгоритмов NLP. В следующей

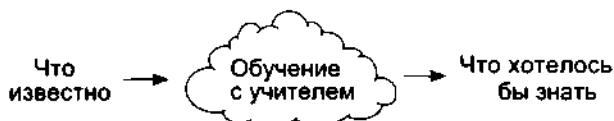
главе мы создадим рекуррентную нейронную сеть и посмотрим, почему она настолько эффективна в предсказании последовательностей.

Стоит также отметить ключевую роль, которую NLP (во многом благодаря использованию приемов глубокого обучения) играет в развитии искусственного интеллекта (ИИ). Целью развития ИИ является создание машин, которые могут думать и взаимодействовать с окружающим миром подобно человеку (и не только). NLP играет особую роль в этом начинании, поскольку естественный язык является основой логики сознания и общения человека. То есть методы, которые позволяют машинам понимать язык, образуют основу человекоподобной логики: основу мысли.

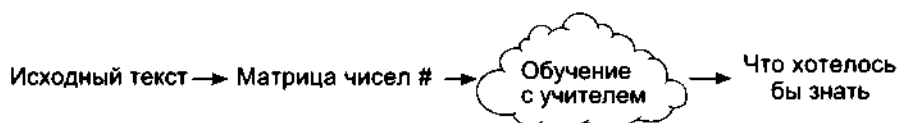
## Обработка естественного языка с учителем

### Слова на входе, прогноз на выходе

Вспомним рисунок из главы 2. Обучение с учителем — это метод преобразования «известных знаний» в то, что «хотелось бы знать». До сих пор «известные знания» были представлены в той или иной числовой форме. Но в NLP роль входных данных играет текст. Как его обрабатывать?



Поскольку нейронные сети отображают числа на входе в числа на выходе, первым делом нужно преобразовать текст в числовую форму. Примерно так же, как мы преобразовывали данные наблюдений за светофором, мы должны преобразовать фактические данные (в данном случае текст) в *матрицу*, которую можно передать в нейронную сеть. Как оказывается, порядок такого преобразования чрезвычайно важен!



Как правильно преобразовать текст в числа? Чтобы ответить на этот вопрос, нужно немного поразмышлять над проблемой. Как вы помните, нейронные сети

ищут корреляцию между входным и выходным слоями. То есть мы должны преобразовать текст в числа так, чтобы корреляция между входом и выходом была *наиболее очевидной* для сети. Это позволит ускорить обучение и улучшить обобщение.

Чтобы узнать, какой способ представления входных данных делает корреляцию между входом и выходом наиболее очевидной для сети, нужно знать, как выглядят входной/выходной наборы данных. Для исследования этого вопроса попробуем решить задачу *классификации темы*.

## Набор данных IMDB с обзорами фильмов

По тексту обзора можно определить, является ли он положительным или отрицательным

Набор данных IMDB с обзорами фильмов — это коллекция пар «обзор → рейтинг», которые обычно имеют следующий вид (это придуманный пример, а не реальный обзор из коллекции IMDB):

«Ужасный фильм! Сюжет слабый, актеры неубедительны,  
и я рассыпал попкорн на рубашку».

Рейтинг: 1 (звезда)

Всего в наборе примерно 50 000 таких пар. Входные обзоры обычно состоят из нескольких предложений, а выходные рейтинги определяются числом от 1 до 5 звезд. Люди считают его *набором данных с признаком эмоциональной окраски*, потому что звезды явно указывают на общее отношение к фильму. Но совершенно понятно, что этот набор данных с признаком эмоциональной окраски может сильно отличаться от других подобных наборов, таких как обзоры товаров или отзывы пациентов больницы.

Мы должны обучить нейронную сеть, чтобы она могла по входному тексту точно предсказать выходной рейтинг. Но прежде нужно решить, как превратить входные и выходные наборы данных в матрицы. Обратите внимание, что выходной набор данных — это число, что облегчает нам задачу. Нам остается только преобразовать диапазон между 1 и 5 в диапазон между 0 и 1, чтобы использовать бинарную функцию *softmax*. Это все, что нужно сделать с выходом. Пример, как это реализуется, я покажу на следующей странице.

Однако входные данные немного сложнее. Для начала рассмотрим исходные данные. Это обычный список символов, что создает несколько проблем: входные данные не только являются текстом, но еще имеют *переменную длину*.

До сих пор наши нейронные сети принимали входные данные фиксированного размера. Мы должны как-то преодолеть эту проблему.

Итак, мы не можем использовать входные данные в первоначальном виде. Следующий вопрос, который встает перед нами: «В каком виде следует представить входные данные, чтобы можно было определить корреляцию между ними и выходом?» Такое представление должно хорошо коррелировать с выходными данными. Прежде всего, я не думаю, что какие-либо символы (в списке символов) будут иметь какую-либо корреляцию с эмоциональной окраской. Символы не годятся, мы должны подыскать что-то другое.

Слова — достаточно ли это хорошее представление? Некоторые слова определенно будут иметь некоторую корреляцию. Готов поспорить, что слова *ужасный* и *неубедительны* имеют явную отрицательную корреляцию с рейтингом. Под *отрицательной* я имею в виду, что чем чаще эти слова повторяются в обзоре, тем ниже рейтинг фильма.

Пожалуй, что слова являются более универсальным представлением! Возможно, что сами слова (даже вне контекста) будут иметь значительную корреляцию с эмоциями. Исследуем эту тему дальше.

## Выявление корреляции слов во входных данных

### Мешок слов: предсказание эмоциональной окраски по словарю обзора

Чтобы выяснить наличие корреляции между лексикой обзора в наборе IMDB и его рейтингом, сделаем следующий шаг: создадим входную матрицу, представляющую словарь обзора фильма.

В таких случаях обычно создается матрица, в которой каждая строка (вектор) соответствует одному обзору фильма, а каждый столбец определяет, содержит ли обзор определенное слово. Чтобы создать вектор из текста обзора, вычислим словарь обзора и затем запишем 1 в каждый столбец для этого обзора, соответствующий имеющемуся в нем слову, и 0 в остальные столбцы. Насколько велики эти векторы? Если словарь насчитывает 2000 слов и в каждом векторе нужно отметить присутствие/отсутствие каждого слова, тогда каждый вектор будет иметь 2000 измерений.

Эта форма хранения, называемая *прямым кодированием* (one-hot encoding), является наиболее распространенным форматом представления бинарных данных (описывающих присутствие или отсутствие на входе точки данных из



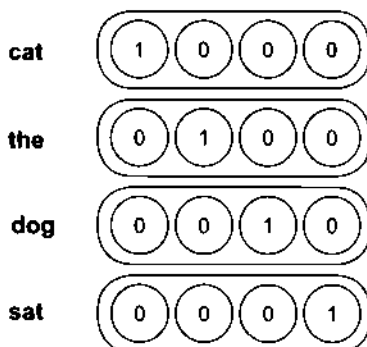
числа всех возможных точек данных в словаре). Если представить, что словарь содержит всего четыре слова, тогда данные в формате прямого кодирования будут выглядеть, как показано на следующем рисунке.

```
import numpy as np
```

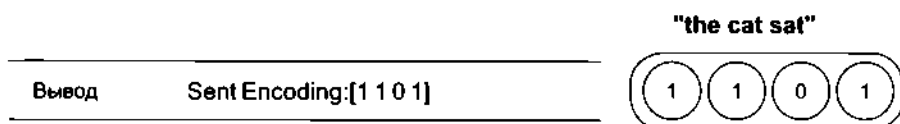
```
onehots = {}
onehots['cat'] = np.array([1,0,0,0])
onehots['the'] = np.array([0,1,0,0])
onehots['dog'] = np.array([0,0,1,0])
onehots['sat'] = np.array([0,0,0,1])
```

```
sentence = ['the', 'cat', 'sat']
x = word2hot[sentence[0]] + \
    word2hot[sentence[1]] + \
    word2hot[sentence[2]]
```

```
print("Sent Encoding:" + str(x))
```



Как видите, для каждого слова в словаре создается вектор. Это дает возможность использовать простое сложение векторов для получения векторного представления подмножества словаря (например, подмножества словаря, соответствующего предложению).



Обратите внимание, что при создании векторного представления для нескольких слов (например, «the cat sat») мы можем по-разному обрабатывать повторяющиеся слова. Например, для словосочетания «cat cat cat» можно трижды сложить вектор для слова «cat» с самим собой (и получить [3, 0, 0, 0]) или игнорировать повторения, оставив только одно слово «cat» (и получить [1, 0, 0, 0]). Последний способ обычно подходит лучше для обработки естественного языка.

## Прогнозирование обзоров фильмов

С помощью стратегии кодирования и предыдущей сети можно предсказать эмоциональную окраску обзора

Используя только что описанную стратегию, можно создать вектор для каждого слова в наборе данных с обзорами и использовать предыдущую двухслойную

сеть для определения эмоциональной окраски этих обзоров. Я покажу вам программный код, но настоятельно советую опробовать его, воспроизведя по памяти. Откройте новый блокнот Jupyter Notebook, загрузите набор данных, создайте векторы методом прямого кодирования и затем создайте нейронную сеть для предсказания рейтингов (положительных или отрицательных) фильмов по их обзорам.

Вот как я реализовал этап предварительной обработки:

```
import sys

f = open('reviews.txt')
raw_reviews = f.readlines()
f.close()

f = open('labels.txt')
raw_labels = f.readlines()
f.close()

tokens = list(map(lambda x:set(x.split(" ")),raw_reviews))

vocab = set()
for sent in tokens:
    for word in sent:
        if(len(word)>0):
            vocab.add(word)
vocab = list(vocab)

word2index = {}
for i,word in enumerate(vocab):
    word2index[word]=i

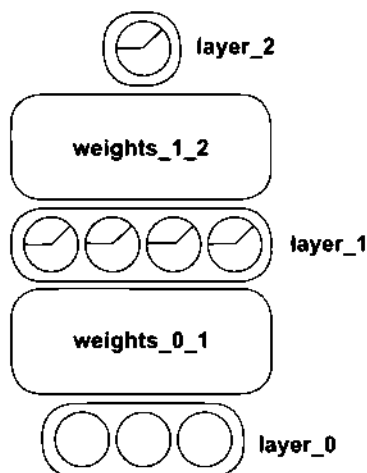
input_dataset = list()
for sent in tokens:
    sent_indices = list()
    for word in sent:
        try:
            sent_indices.append(word2index[word])
        except:
            ""
    input_dataset.append(list(set(sent_indices)))

target_dataset = list()
for label in raw_labels:
    if label == 'positive\n':
        target_dataset.append(1)
    else:
        target_dataset.append(0)
```

## Введение в слой с векторным представлением

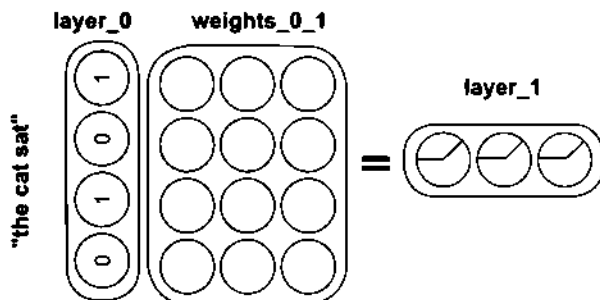
### Еще один трюк, помогающий ускорить сеть

Справа изображена диаграмма предыдущей нейронной сети, которую теперь мы используем для определения эмоциональной окраски. Но прежде я хотел бы остановиться на названиях слоев. Первый слой — это слой исходных данных (*layer\_0*). За ним следует так называемый *линейный слой* (*weights\_0\_1*). Далее располагаются: слой *relu* (*layer\_1*), еще один линейный слой (*weights\_1\_2*) и затем выходной слой, представляющий прогноз. Как оказывается, путь к слою *layer\_1* можно немного сократить, заменив первый линейный слой (*weights\_0\_1*) слоем векторного представления.

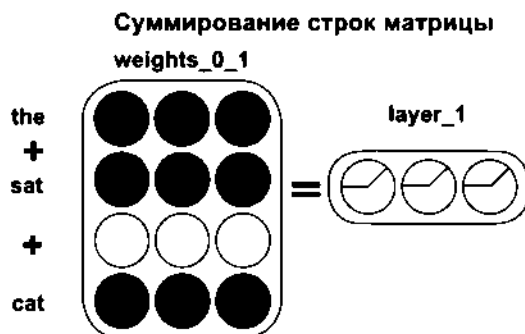


Умножение вектора из нулей и единиц на матрицу математически эквивалентно суммированию нескольких строк в матрице. То есть намного эффективнее выбрать соответствующие строки из *weights\_0\_1* и суммировать их, чем выполнять полноценную операцию векторно-матричного умножения. Учитывая, что словарь насчитывает порядка 70 000 слов, большая часть операций векторно-матричного умножения будет напрасно тратить время, умножая нули во входном векторе на разные строки в матрице перед их суммированием. Выбирать строки из матрицы, соответствующие имеющимся словам, и суммировать их — намного эффективнее.

### Умножение вектора, полученного методом прямого кодирования, на матрицу



Этот процесс выбора строк и их суммирование (или усреднение) определяет первый линейный слой (`weights_0_1`) как слой векторного представления. Конструктивно они идентичны (слой `layer_1` нисколько не изменился, какой бы метод прямого распространения ни использовался). Единственное отличие — суммирование небольшого числа строк ускоряет вычисления.



После выполнения предыдущего кода запустите этот код

```
import numpy as np
np.random.seed(1)

def sigmoid(x):
    return 1/(1 + np.exp(-x))

alpha, iterations = (0.01, 2)
hidden_size = 100

weights_0_1 = 0.2*np.random.random((len(vocab),hidden_size)) - 0.1
weights_1_2 = 0.2*np.random.random((hidden_size,1)) - 0.1

correct,total = (0,0)
for iter in range(iterations):
    for i in range(len(input_dataset)-1000):
        x,y = (input_dataset[i],target_dataset[i])
        layer_1 = sigmoid(np.sum(weights_0_1[x],axis=0))
        layer_2 = sigmoid(np.dot(layer_1,weights_1_2))
        layer_2_delta = layer_2 - y
        layer_1_delta = layer_2_delta.dot(weights_1_2.T)
        weights_0_1[x] -= layer_1_delta * alpha
        weights_1_2 -= np.outer(layer_1,layer_2_delta) * alpha
    if(np.abs(layer_2_delta) < 0.5):
```

← Обучение на первых 24 000 образах

← Векторное представление + sigmoid

← Линейный слой + softmax

← Разность между прогнозом и истинной

← Обратное распространение

```

        correct += 1
    total += 1
    if(i % 10 == 9):
        progress = str(i/float(len(input_dataset)))
        sys.stdout.write('\rIter:'+str(iter)\
            + ' Progress:'+progress[2:4]\
            + '.'+progress[4:6]\
            + '% Training Accuracy:'\
            + str(correct/float(total)) + '%')

    print()

correct,total = (0,0)
for i in range(len(input_dataset)-1000,len(input_dataset)):

    x = input_dataset[i]
    y = target_dataset[i]

    layer_1 = sigmoid(np.sum(weights_0_1[x],axis=0))
    layer_2 = sigmoid(np.dot(layer_1,weights_1_2))

    if(np.abs(layer_2 - y) < 0.5):
        correct += 1
    total += 1

print("Test Accuracy:" + str(correct / float(total)))

```

## Интерпретация результата

### Чему обучилась нейронная сеть?

Вот вывод нейронной сети, анализирующей обзоры фильмов. С одной стороны, это то же самое обобщение корреляции, которое мы уже обсуждали:

```

Iter:0 Progress:95.99% Training Accuracy:0.832%
Iter:1 Progress:95.99% Training Accuracy:0.8663333333333333%
Test Accuracy:0.849

```

Нейронная сеть искала корреляцию между входными и выходными точками данных. Но эти точки данных имеют знакомые нам характеристики (особенно языковые). Кроме того, очень интересно посмотреть, какие языковые закономерности были выявлены в результате обобщения корреляции, и еще интереснее — какие остались не выявленными. В конце концов, сам факт обнаружения корреляции

**Метка положительный/  
отрицательный**

(Нейронная сеть)

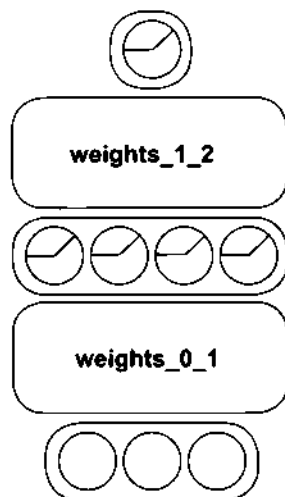
**Словарь обзора**

лянии между входным и выходным наборами данных еще не означает, что сеть смогла выучить все полезные закономерности языка.

Кроме того, понимание разницы между тем, что сеть может изучить (в текущей конфигурации), и тем, что она должна изучить для правильного понимания естественного языка, поможет направить мысли в нужное русло. Именно так поступают исследователи, находящиеся на переднем крае науки, и так же поступим мы.

И что узнает сеть о языке обзоров фильмов? Для начала посмотрим, что мы передали сети. Как показано на рисунке справа, мы передаем на вход сети словарь каждого обзора и предлагаем ей спрогнозировать одну из меток («положительный» или «отрицательный»). Учитывая, что согласно правилу обобщения корреляции сеть будет искать корреляцию между входным и выходным наборами данных, мы ожидаем, что она как минимум выявит слова, которые сами по себе имеют положительную или отрицательную корреляцию.

Это естественным образом вытекает из правила обобщения корреляции. Мы сообщаем сети о наличии или отсутствии слова, а она, обобщая корреляцию, найдет прямую связь между присутствием/отсутствием и каждой из двух меток. Но это еще не все.



## Нейронная архитектура

**Как выбор архитектуры влияет на то, что будет изучено сетью?**

Мы только что узнали, что самый простой вид информации, которую извлекает нейронная сеть, — это прямая корреляция между входным и целевым наборами данных. Это весьма характерно для нейронного интеллекта. (Если бы сеть не обнаруживала прямой корреляции между входными и выходными данными, это могло бы означать, что с ней что-то не так.) Для выявления более сложных закономерностей, чем прямая корреляция, нужны более сложные архитектуры, и данный пример не исключение.

Для выявления прямой корреляции достаточно простой двухслойной сети, включающей единственную весовую матрицу, которая непосредственно свя-

зывает входной слой с выходным. Но мы использовали сеть со скрытым слоем. Возникает вопрос: что дает этот скрытый слой?

По сути, скрытые слои осуществляют объединение точек данных из предыдущих слоев в  $n$  групп (где  $n$  — число нейронов в скрытом слое). Каждый скрытый нейрон принимает точку данных и отвечает на вопрос: «Она принадлежит моей группе?» В процессе обучения скрытый слой ищет полезные способы группировки своих входных данных. Что это за полезные способы группировки?

Способ группировки входных данных полезен, если отвечает двум критериям. Во-первых, он должен способствовать предсказанию выходной метки. Если некоторый способ группировки бесполезен для получения верного предсказания на выходе, *обобщение корреляции никогда не приведет сеть к выявлению группы*. Это очень важный аспект. Основной задачей нейронной сети является поиск закономерностей (или какого-то другого искусственного сигнала, способного помочь получить верный прогноз) в обучающих данных, поэтому она находит такие способы группировки, которые полезны для решаемой задачи (например, для предсказания рейтинга фильма по его обзору). Мы вернемся к этому вопросу чуть позже.

Во-вторых, чтобы стать полезным, способ группировки должен обнаруживать в данных искомую закономерность. Плохой способ группировки просто запоминает данные. Хороший — выявляет закономерности, полезные с лингвистической точки зрения. Например, в анализе эмоциональной окраски обзора фильма понимание разницы между «ужасно» и «не ужасно» является очень мощным способом группировки. Было бы хорошо иметь нейрон, *выключающийся* при встрече со словом «ужасно» и *включающийся* при встрече с «не ужасно». Он мог бы стать хорошим прогнозным признаком для следующего слоя. Но, так как на вход нейронной сети подается словарь обзора, фраза «отлично, не ужасно» создаст в слое `layer_1` точно такое же значение, как и фраза «ужасно, не отлично». По этой причине сети едва ли удастся создать скрытый нейрон, понимающий отрицание.

Проверка появления различий в слое в зависимости от некоторой закономерности — лучший первый шаг, чтобы узнать, способна ли архитектура обнаружить эту закономерность с использованием обобщения корреляции. Если вы сможете создать два образца входных данных, с присутствующим и отсутствующим шаблоном, для которых получится идентичный скрытый слой, значит, сеть едва ли сумеет обнаружить этот шаблон.

Как вы только что узнали, скрытый слой фактически группирует данные из предыдущего слоя. На низком уровне каждый нейрон классифицирует точку данных как соответствующую или не соответствующую его группе. На высоком

уровне две точки данных (два обзора фильмов) похожи, если принадлежат множеству одних и тех же групп. Наконец, два входа (слова) похожи, если имеют схожие веса, связывающие их с разными скрытыми нейронами (то есть имеют одинаковые меры сходства с каждой группой). Учитывая вышесказанное, какие изменения мы должны наблюдать в весах слов, поступающих в скрытый слой в предыдущей сети?

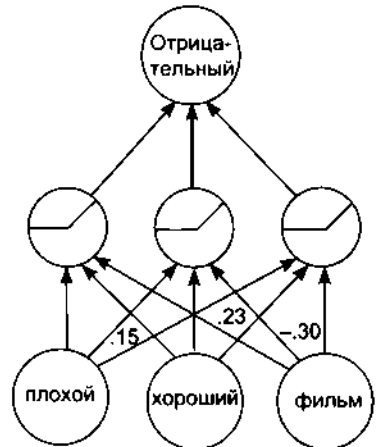
### Что мы должны увидеть в весах, связывающих слова со скрытыми нейронами?

Подсказка: слова, обладающие похожими прогнозными признаками, должны соответствовать похожим группам (сочетаниям скрытых нейронов). Что это означает с точки зрения весов, связывающих каждое слово с каждым скрытым нейроном?

Вот ответ на вопрос. Слова, коррелирующие с похожими метками («положительный» или «отрицательный»), будут иметь похожие веса, связывающие их с разными скрытыми нейронами. Это связано с тем, что нейронная сеть учится объединять их с похожими скрытыми нейронами, чтобы последний слой (`weights_1_2`) мог сделать правильный прогноз, положительный или отрицательный.

Чтобы увидеть это явление, можно взять слова с явной положительной и отрицательной окраской и попробовать найти другие слова с похожими значениями весов. Иначе говоря, вы можете взять каждое слово и посмотреть, какие другие слова имеют похожие значения весов, связывающие их с каждым скрытым нейроном (с каждой группой).

Слова, принадлежащие похожим группам, будут иметь похожую прогнозную силу для меток «положительный» или «отрицательный». Кроме того, слова, принадлежащие похожим группам и имеющие похожие значения весов, будут иметь похожий смысл. В более общем смысле нейрон имеет похожий смысл с другими нейронами в том же слое тогда и только тогда, когда имеет похожие значения весов, связывающих его со следующим и/или предыдущим слоем.



Три веса слова «хороший», связывающих его векторное представление, отражают степень принадлежности слова «хороший» каждой из групп (скрытым нейронам). Слова со схожей прогнозностью имеют похожие векторные представления (значения весов).



## Сравнение векторных представлений слов

### Как визуализировать сходство весов?

Чтобы получить список весов для каждого входного слова, связывающих его с разными скрытыми нейронами, достаточно выбрать соответствующую строку в матрице `weights_0_1`. Каждый элемент строки представляет вес, связывающий это слово с конкретным скрытым нейроном. То есть чтобы найти слова, наиболее похожие на данное слово, нужно сравнить вектор весов каждого слова (строку в матрице) с вектором весов целевого слова. Для сравнения можно использовать метрику, которая называется *евклидовым расстоянием*, как показано в следующем листинге:

```
from collections import Counter
import math

def similar(target='beautiful'):
    target_index = word2index[target]
    scores = Counter()
    for word, index in word2index.items():
        raw_difference = weights_0_1[index] - (weights_0_1[target_index])
        squared_difference = raw_difference * raw_difference
        scores[word] = -math.sqrt(sum(squared_difference))

    return scores.most_common(10)
```

Используя этот прием, легко найти наиболее похожее слово (нейрон) в данной сети:

```
print(similar('beautiful'))          print(similar('terrible'))

[('beautiful', -0.0),                [('terrible', -0.0),
 ('atmosphere', -0.70542101298),      ('dull', -0.760788602671491),
 ('heart', -0.7339429768542354),      ('lacks', -0.76706470275372),
 ('tight', -0.7470388145765346),      ('boring', -0.7682894961694),
 ('fascinating', -0.7549291974),      ('disappointing', -0.768657),
 ('expecting', -0.759886970744),      ('annoying', -0.78786389931),
 ('beautifully', -0.7603669338),      ('poor', -0.825784172378292),
 ('awesome', -0.76647368382398),      ('horrible', -0.83154121717),
 ('masterpiece', -0.7708280057),      ('laughable', -0.8340279599),
 ('outstanding', -0.7740642167)]      ('badly', -0.84165373783678)]
```

Как и следовало ожидать, наиболее похожим для каждого слова является само это слово, а затем следуют слова с той же полезностью, что и целевое слово. Кроме того, что также ожидаемо, поскольку сеть возвращает только две метки

(«положительный» или «отрицательный»), входные слова группируются согласно их положительной или отрицательной эмоциональной окраске.

Это стандартное проявление обобщения корреляции. Сеть стремится создать похожие представления (значения в слое `layer_1`) на основе прогнозируемой метки, чтобы научиться предсказывать правильную метку. В данном случае веса, поступающие в слой `layer_1`, группируются согласно метке на выходе.

Ключевой вывод: важно понимать, что это — проявление эффекта обобщения корреляции. Он заключается в том, что сеть пытается последовательно убедить скрытые нейроны быть похожими на метки, которые она должна предсказать.

## В чем заключается смысл нейрона?

Смысл целиком и полностью зависит от прогнозируемых целевых меток

Обратите внимание, что смысл разных слов не всегда точно определяет, как их можно сгруппировать. Например, на слово «beautiful» (прекрасный) больше других подходит слово «atmosphere» (атмосфера). Это весьма показательно. С точки зрения определения положительной или отрицательной эмоциональной окраски отзыва к фильму эти слова имеют почти идентичный смысл. Но в реальном мире их смысл сильно отличается (например, первое является прилагательным, а второе — существительным).

```
print(similar('beautiful'))
```

```
print(similar('terrible'))
```

```
[('beautiful', -0.0),
 ('atmosphere', -0.70542101298),
 ('heart', -0.7339429768542354),
 ('tight', -0.7470388145765346),
 ('fascinating', -0.7549291974),
 ('expecting', -0.759886970744),
 ('beautifully', -0.7603669338),
 ('awesome', -0.76647368382398),
 ('masterpiece', -0.7708280057),
 ('outstanding', -0.7740642167)]
```

```
[('terrible', -0.0),
 ('dull', -0.760788602671491),
 ('lacks', -0.76706470275372),
 ('boring', -0.7682894961694),
 ('disappointing', -0.768657),
 ('annoying', -0.78786389931),
 ('poor', -0.825784172378292),
 ('horrible', -0.83154121717),
 ('laughable', -0.8340279599),
 ('badly', -0.84165373783678)]
```

Понимание этого факта очень важно. Смысл (нейрона) в сети определяется целевыми метками. Все, что имеется внутри сети, увязывается с контекстом механизмом обобщения корреляции с целью получить правильный прогноз.

Несмотря на то что вы и я много знаем об этих словах, нейронная сеть полностью игнорирует всю информацию, не имеющую отношения к решаемой задаче.

Как можно заставить сеть изучить больше нюансов (в данном случае больше нюансов о словах)? Если дать ей входные и выходные данные, требующие более тонкого понимания языка, у нее появится повод изучить более тонкие интерпретации разных слов.

Что еще можно поручить предсказать нейронной сети, чтобы она изучила более интересные значения весов для слов? Такой задачей, вынуждающей сеть изучить более интересные значения весов слов, может служить задача подстановки недостающих слов, которой мы и займемся далее. Почему именно она? Во-первых, потому, что существует почти неисчерпаемый источник обучающих данных (интернет), а это значит, что существует бесконечный сигнал, который может использовать нейронная сеть для получения более детальной информации о словах. Кроме того, способность точно подбирать недостающие слова требует некоторого понимания контекста реального мира.

Например, в следующем предложении какое слово вы бы подставили на место пропуска, «наковальня» или «шерсть»? Давайте посмотрим, сможет ли нейронная сеть справиться с этой задачей.

*У Маши был маленький ягненок с ??? , белой как снег.*

## Подстановка пропущенных слов

### Изучение более тонких значений слов при наличии богатого обучающего сигнала

В этом примере используется та же нейронная сеть, что и прежде, но с некоторыми небольшими изменениями. Во-первых, вместо предсказания единственной метки по обзору фильма мы возьмем каждую фразу (из пяти слов), удалим одно слово и попытаемся научить сеть определять по остатку фразы, какое слово было удалено. Во-вторых, мы используем трюк с названием *отрицательное семплирование* (negative sampling), чтобы немного ускорить обучение сети.

Имейте в виду, что для предсказания пропущенного слова нужно иметь по одной метке для каждого возможного варианта. Это может потребовать определить несколько тысяч меток и стать причиной медленного обучения сети. Чтобы преодолеть этот недостаток, будем случайным образом игнорировать

большинство меток в каждом шаге процесса прямого распространения (например, притворимся, что их просто не существует). Этот метод может показаться слишком грубой аппроксимацией, но он хорошо зарекомендовал себя на практике. Далее приводится код для этого примера, реализующий предварительную обработку:

```
import sys, random, math
from collections import Counter
import numpy as np

np.random.seed(1)
random.seed(1)
f = open('reviews.txt')
raw_reviews = f.readlines()
f.close()

tokens = list(map(lambda x:(x.split(" ")),raw_reviews))
wordcnt = Counter()
for sent in tokens:
    for word in sent:
        wordcnt[word] += 1
vocab = list(set(map(lambda x:x[0],wordcnt.most_common()))))

word2index = {}
for i,word in enumerate(vocab):
    word2index[word]=i

concatenated = list()
input_dataset = list()
for sent in tokens:
    sent_indices = list()
    for word in sent:
        try:
            sent_indices.append(word2index[word])
            concatenated.append(word2index[word])
        except:
            ""
    input_dataset.append(sent_indices)
concatenated = np.array(concatenated)

random.shuffle(input_dataset)
alpha, iterations = (0.05, 2)
hidden_size, window, negative = (50, 2, 5)

weights_0_1 = (np.random.rand(len(vocab),hidden_size) - 0.5) * 0.2
weights_1_2 = np.random.rand(len(vocab),hidden_size)*0

layer_2_target = np.zeros(negative+1)
layer_2_target[0] = 1
```

```

def similar(target='beautiful'):
    target_index = word2index[target]

    scores = Counter()
    for word, index in word2index.items():
        raw_difference = weights_0_1[index] - (weights_0_1[target_index])
        squared_difference = raw_difference * raw_difference
        scores[word] = -math.sqrt(sum(squared_difference))
    return scores.most_common(10)

def sigmoid(x):
    return 1/(1 + np.exp(-x))

for rev_i, review in enumerate(input_dataset * iterations):
    for target_i in range(len(review)):

        target_samples = [review[target_i]] + list(concatenated\
            [(np.random.rand(negative)*len(concatenated)).astype('int').tolist()])

        left_context = review[max(0, target_i-window):target_i]
        right_context = review[target_i+1:min(len(review), target_i+window)]

        layer_1 = np.mean(weights_0_1[left_context+right_context], axis=0)
        layer_2 = sigmoid(layer_1.dot(weights_1_2[target_samples].T))
        layer_2_delta = layer_2 - layer_2_target
        layer_1_delta = layer_2_delta.dot(weights_1_2[target_samples])

        weights_0_1[left_context+right_context] -= layer_1_delta * alpha
        weights_1_2[target_samples] -= np.outer(layer_2_delta, layer_1)*alpha

    if(rev_i % 250 == 0):
        sys.stdout.write('\rProgress: '+str(rev_i/float(len(input_dataset)
            *iterations)) + " " + str(similar('terrible')))
        sys.stdout.write('\rProgress: '+str(rev_i/float(len(input_dataset)
            *iterations)))

print(similar('terrible'))

Progress:0.99998 [('terrible', -0.0), ('horrible', -2.846300248788519),
('brilliant', -3.039932544396419), ('pathetic', -3.4868595532695967),
('superb', -3.6092947961276645), ('phenomenal', -3.660172529098085),
('masterful', -3.6856112636664564), ('marvelous', -3.9306620801551664),

```

Прогнозировать только случайное подмножество,  
потому что прогнозирование всего словаря  
обойдется слишком дорого

## Смысл определяется потерями

Как можно заметить, эта новая нейронная сеть группирует векторные представления слов немного иначе. Если раньше слова группировались согласно их степени соответствия положительной или отрицательной оценке,

то теперь они группируются согласно вероятности занять место отсутствующего слова в фразе (иногда независимо от получающейся эмоциональной окраски).

#### Предсказание эмоциональной окраски

```
print(similar('terrible'))
```

```
[('terrible', -0.0),
 ('dull', -0.760788602671491),
 ('lacks', -0.76706470275372),
 ('boring', -0.7682894961694),
 ('disappointing', -0.768657),
 ('annoying', -0.78786389931),
 ('poor', -0.825784172378292),
 ('horrible', -0.83154121717),
 ('laughable', -0.8340279599),
 ('badly', -0.84165373783678)]
```

```
print(similar('beautiful'))
```

```
[('beautiful', -0.0),
 ('atmosphere', -0.70542101298),
 ('heart', -0.7339429768542354),
 ('tight', -0.7470388145765346),
 ('fascinating', -0.7549291974),
 ('expecting', -0.759886970744),
 ('beautifully', -0.7603669338),
 ('awesome', -0.76647368382398),
 ('masterpiece', -0.7708280057),
 ('outstanding', -0.7740642167)]
```

#### Подстановка пропущенного слова

```
print(similar('terrible'))
```

```
[('terrible', -0.0),
 ('horrible', -2.79600898781),
 ('brilliant', -3.3336178881),
 ('pathetic', -3.49393193646),
 ('phenomenal', -3.773268963),
 ('masterful', -3.8376122586),
 ('superb', -3.9043150978490),
 ('bad', -3.9141673639585237),
 ('marvelous', -4.0470804427),
 ('dire', -4.178749691835959)]
```

```
print(similar('beautiful'))
```

```
[('beautiful', -0.0),
 ('lovely', -3.0145597243116),
 ('creepy', -3.1975363066322),
 ('fantastic', -3.2551041418),
 ('glamorous', -3.3050812101),
 ('spooky', -3.4881261617587),
 ('cute', -3.592955888181448),
 ('nightmarish', -3.60063813),
 ('heartwarming', -3.6348147),
 ('phenomenal', -3.645669007)]
```

Из этого следует, что хотя сеть обучалась на одном и том же наборе данных и имеет похожую архитектуру (три слоя, перекрестная энтропия, функция активации *sigmoid*), мы можем влиять на смысловую нагрузку весов, приобретаемую в процессе обучения, изменяя ожидаемый результат на выходе. Несмотря на то что сеть просматривает одну и ту же информацию, ее можно переориентировать на другой результат, выбирая исходные и целевые значения. Назовем этот процесс выбора желаемого результата *определением цели обучения*.

Управление входными/целевыми значениями — не единственный способ определить цель обучения. Можно изменить способ измерения ошибки в сети, размеры и типы слоев, а также применяемые методы регуляризации. В глубоком обучении все эти приемы объединяются одним общим названием: *функция потерь*.

## Нейронные сети не изучают данные; они минимизируют функцию потерь

В главе 4 мы узнали, что обучение заключается в корректировке каждого веса в нейронной сети, чтобы уменьшить ошибку до 0. В этом разделе я объясню то же самое с другой точки зрения, выбрав ошибку так, чтобы заставить нейронную сеть выявить интересующие нас закономерности. Вспомним следующие выводы, сделанные в главе 4.

### ЗОЛОТОЕ ПРАВИЛО ОБУЧЕНИЯ

Корректировать каждый вес в правильном направлении и на правильную величину, чтобы в конечном счете уменьшить ошибку до 0

### СЕКРЕТ

Для любых input и goal\_pred точное отношение между error и weight определяется комбинацией формул прогнозирования и вычисления ошибки

$$\text{error} = ((0.5 * \text{weight}) - 0.8) ** 2$$

Возможно, вы помните эту формулу для нейронной сети с одним весом. В той сети мы вычисляли ошибку, сначала выполняя прямое распространение ( $0.5 * \text{weight}$ ) и сравнивая с целевым значением (0.8). Попробуем рассматривать это действие не как состоящее из двух разных шагов (прямое распространение и вычисление ошибки), а как единую формулу (включая прямое распространение) вычисления ошибки. В таком контексте мы сможем увидеть истинную причину различий в объединении векторных представлений слов. Несмотря на сходство архитектуры и наборов данных, сети существенно отличаются функцией ошибки, что вызвало группировку слов по-разному.

### Предсказание эмоциональной окраски

```
print(similar('terrible'))
```

```
[('terrible', -0.0),
 ('dull', -0.760788602671491),
 ('lacks', -0.76706470275372),
 ('boring', -0.7682894961694),
 ('disappointing', -0.768657),
 ('annoying', -0.78786389931),
 ('poor', -0.825784172378292),
 ('horrible', -0.83154121717),
 ('laughable', -0.8340279599),
 ('badly', -0.84165373783678)]
```

### Подстановка пропущенного слова

```
print(similar('terrible'))
```

```
[('terrible', -0.0),
 ('horrible', -2.79600898781),
 ('brilliant', -3.3336178881),
 ('pathetic', -3.49393193646),
 ('phenomenal', -3.773268963),
 ('masterful', -3.8376122586),
 ('superb', -3.9043150978490),
 ('bad', -3.9141673639585237),
 ('marvelous', -4.0470804427),
 ('dire', -4.178749691835959)]
```

## Выбор функции потерь определяет смысл знаний, приобретаемых нейронной сетью

Формально *функция ошибки* называется *функцией потерь*, или *целевой функцией* (все три названия являются взаимозаменяемыми). Приняв, что обучение сводится к минимизации функции потерь (включающей также прямое распространение), мы получаем более широкий взгляд на обучение нейронных сетей. Две нейронные сети могут иметь идентичные начальные веса, обучаться на идентичных наборах данных, но выявлять совершенно разные закономерности из-за выбора разных функций потерь. В примере выше с обзорами фильмов две нейронные сети имели разные функции потерь, потому что мы выбрали разные целевые значения (оценка «положительный»/«отрицательный» и выбор слова для заполнения пробела).

Разные типы архитектур и слоев, методы регуляризации, наборы данных и функции активации в действительности не дают настолько существенных различий, как может показаться. Все они представляют разные детали для конструирования функции потерь. Если сеть обучается недостаточно хорошо, решение может заключаться в любой из этих категорий.

Например, если появляется эффект переобучения сети, можно усовершенствовать функцию потерь, задействовав нелинейную функцию активации, слои меньшего размера, менее глубокие архитектуры, большие наборы данных или более агрессивные методы регуляризации. Все эти варианты будут иметь принципиально похожее влияние на функцию потерь и на поведение сети. Все они взаимодействуют друг с другом, и со временем вы узнаете, как изменение одного влияет на работу другого; а пока просто запомните, что обучение заключается в конструировании функции потерь и последующей ее минимизации.

Всякий раз, когда требуется, чтобы нейронная сеть выявила закономерность, нужно лишь отразить это в функции потерь. Когда у нас был только один вес, функция потерь была простой, как вы помните:

$$\text{error} = ((0.5 * \text{weight}) - 0.8) ** 2$$

Но с появлением большого количества сложных слоев функция потерь становится все сложнее (и это нормально). Просто запомните, что если что-то пошло не так, решение заключается в функции потерь, которая включает прямое распространение и вычисление значения ошибки (например, среднеквадратичной ошибки или перекрестной энтропии).



## Король – мужчина + женщина $\sim$ королева

### Словесные аналогии — интересное следствие ранее построенной сети

Прежде чем закончить эту главу, обсудим еще одно из самых известных свойств векторных представлений слов (векторов слов, подобных тем, что мы только что создали). Задача подстановки отсутствующего слова создает векторные представления, обладающие интересной особенностью, которая называется *словесной аналогией*, позволяющей выполнять простые алгебраические операции над векторами слов.

Например, обучив предыдущую сеть на достаточно большом корпусе, вы сможете взять вектор, соответствующий слову *король*, вычесть из него вектор слова *мужчина*, прибавить вектор слова *женщина* и попробовать отыскать наиболее похожий вектор (отличный от используемых в запросе). Во многих случаях похожему вектору будет соответствовать слово *королева*. Похожий эффект наблюдается даже в сети, обученной подстановке слов на обзорах фильмов.

```
def analogy(positive=['terrible','good'],negative=['bad']):
    norms = np.sum(weights_0_1 * weights_0_1,axis=1)
    norms.resize(norms.shape[0],1)

    normed_weights = weights_0_1 * norms

    query_vect = np.zeros(len(weights_0_1[0]))
    for word in positive:
        query_vect += normed_weights[word2index[word]]
    for word in negative:
        query_vect -= normed_weights[word2index[word]]

    scores = Counter()
    for word,index in word2index.items():
        raw_difference = weights_0_1[index] - query_vect
        squared_difference = raw_difference * raw_difference
        scores[word] = -math.sqrt(sum(squared_difference))

    return scores.most_common(10)[1:]
```

**terrible – bad + good ~ =**

analogy(['terrible', 'good'], ['bad'])

```
[('superb', -223.3926217861),
 ('terrific', -223.690648739),
 ('decent', -223.7045545791),
 ('fine', -223.9233021831882),
 ('worth', -224.03031703075),
 ('perfect', -224.125194533),
 ('brilliant', -224.2138041),
 ('nice', -224.244182032763),
 ('great', -224.29115420564)]
```

**elizabeth – she + he ~ =**

analogy(['elizabeth', 'he'], ['she'])

```
[('christopher', -192.7003),
 ('it', -193.3250398279812),
 ('him', -193.459063887477),
 ('this', -193.59240614759),
 ('william', -193.63049856),
 ('mr', -193.6426152274126),
 ('bruce', -193.6689279548),
 ('fred', -193.69940566948),
 ('there', -193.7189421836)]
```

## Словесные аналогии

### Линейное ожатие существующего свойства данных

Открытие этого свойства вызвало настоящий фурор, потому что этой технологии можно найти множество разных применений. Это удивительное свойство само по себе, и оно обусловило появление целой индустрии, основанной на создании векторных представлений разного рода. Но с тех пор исследования словесных аналогий не особенно продвинулись вперед, и в настоящее время основное внимание при работе с естественным языком уделяется рекуррентным архитектурам (с которыми мы познакомимся в главе 12).

И все же чрезвычайно важно хорошо понимать, что происходит с векторными представлениями слов в результате выбора функции потерь. Вы уже знаете, что выбор функции потерь может влиять на группировку слов, но явление словесной аналогии — это нечто иное. Какие другие функции потерь приводят к проявлению этого эффекта?

Возможно, будет легче представить, как работают словесные аналогии, если рассмотреть векторные представления слов, имеющие всего два измерения.

```
king = [0.6 , 0.1]
man = [0.5 , 0.0]
woman = [0.0 , 0.8]
queen = [0.1 , 1.0]
```

```
king - man = [0.1 , 0.1]
queen - woman = [0.1 , 0.2]
```



Сходство между «king»/«man» (король/мужчина) и «queen»/«woman» (королева/женщина) представляет определенную пользу для конечного прогноза. Почему? Разность между векторами «king» и «man» дает в результате вектор *королевской власти*. В одной группе присутствует множество слов мужского и женского рода, и есть еще одна группа, описывающая причастность к королевской власти.

Это можно отследить до выбранной функции потерь. Когда во фразе появляется слово «король», определенным образом изменяется вероятность появления других слов в этой же фразе: увеличивается вероятность появления слов мужского рода и слов, имеющих отношение к королевской власти. Появление слова «королева» увеличивает вероятность появления слов женского рода и слов, имеющих отношение к королевской власти (как группы). То есть из-за того, что слова оказывают подобное влияние на вероятность результата, они оказываются включенными в похожие комбинации групп.

Если говорить совсем упрощенно, «король» описывается мужским и королевским измерениями в скрытом слое, тогда как «королева» — женским и королевским измерениями. Если взять вектор, соответствующий слову «король», вычесть из него измерение, определяющее принадлежность к мужскому роду, и прибавить измерение принадлежности к женскому роду, получится вектор, близкий к слову «королева». Самый важный вывод, который можно сделать: все вышесказанное в большей степени относится к особенностям языка, чем к глубокому обучению. Любое линейное сжатие подобных взаимозависимых статистик будет давать аналогичные результаты.

## Итоги

### Вы многое узнали о векторном представлении слов и влиянии функции потерь на обучение

В этой главе мы познакомились с фундаментальными принципами использования нейронных сетей для анализа естественного языка. Мы начали с обзора основных проблем обработки естественного языка, а затем изучили вопрос моделирования языка в нейронных сетях с использованием векторных представлений слов. Мы также узнали, как выбор функции потерь может влиять на свойства, выявляемые с помощью векторных представлений. В заключение мы обсудили самое необычное, пожалуй, из нейронных явлений в этой области: словесные аналогии.

Как в других главах, я призываю вас воспроизвести все примеры в этой главе по памяти. Может показаться, что эта глава стоит особняком, но навыки создания и настройки функции потерь неоценимы. Они пригодятся, когда вы займетесь решением более сложных задач в следующих главах. Удачи!

# 12

## Нейронные сети, которые пишут как Шекспир: рекуррентные слои для данных переменной длины



### В этой главе

- ✓ Проблема произвольной длины.
- ✓ Удивительная мощь усредненных векторов слов.
- ✓ Ограничение векторов в модели «мешок слов».
- ✓ Объединение векторных представлений слов с использованием единичной матрицы.
- ✓ Определение переходных матриц.
- ✓ Создание векторов предложений.
- ✓ Прямое распространение на Python.
- ✓ Прямое и обратное распространение с данными произвольной длины.
- ✓ Корректировка весов с данными произвольной длины.

Есть что-то таинственное в рекуррентных нейронных сетях.

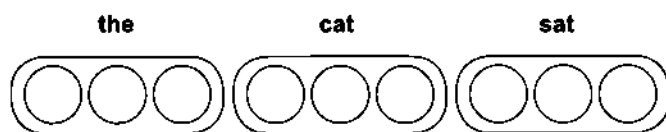
Андрей Карпати (Andrej Karpathy).  
«The Unreasonable Effectiveness of Recurrent  
Neural Networks», <http://mng.bz/VqPW>

## Проблема произвольной длины

### Моделирование в нейронных сетях последовательностей данных произвольной длины

Эта глава и глава 11 тесно взаимосвязаны, поэтому, прежде чем продолжить чтение, я призываю вас убедиться, что вы освоили все понятия и приемы, представленные в главе 11. Там мы познакомились с особенностями обработки естественного языка, в том числе как изменить функцию потерь, чтобы отразить в весах нейронной сети определенные закономерности. Мы также познакомились с векторными представлениями слов и узнали, как они могут представлять смысловое сходство с векторными представлениями других слов. В этой главе мы продолжим исследования в том же направлении и перейдем от векторных представлений, передающих смысл единственного слова, к векторным представлениям, передающим смысл фраз и предложений переменной длины.

Для начала познакомимся с проблемой, стоящей перед нами. Как вы поступите, если понадобится создать вектор, передающий информацию обо всей последовательности слов в предложении или фразе, подобно тому как векторное представление передает информацию об одном слове? Начнем с самого простого варианта. Теоретически, если объединить все векторные представления, мы получим вектор, содержащий информацию обо всей последовательности слов.



Но такой подход оставляет желать лучшего, потому что для разных предложений будут создаваться векторы разной длины, из-за чего прямое сравнение векторов станет невозможным. Взгляните на второе предложение:



Теоретически эти два предложения очень похожи и сравнение их векторов должно показать высокую степень сходства. Но так как «the cat sat» (кошка си-

дит) дает более короткий вектор, нам придется выбирать, какую часть вектора «the cat sat still» (кошка сидит неподвижно) использовать для сравнения. Если сравнивать начиная слева, векторы покажутся идентичными (при этом будут проигнорированы отличия предложения «the cat sat still»). А если сравнивать начиная справа, векторы покажутся совершенно разными, несмотря на то что три четверти слов в них совпадают и следуют в том же порядке. Даже притом, что этот наивный подход иногда позволяет получить хороший результат, он далек от идеала с точки зрения представления смысла предложения некоторым практичным способом (способом, позволяющим выполнить сравнение с другими векторами).

## Действительно ли сравнение имеет значение?

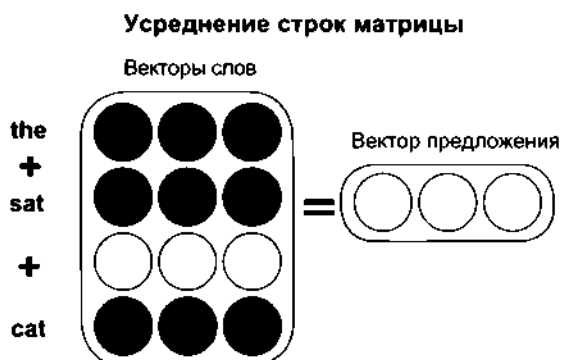
### Почему нас должна волновать возможность сравнения векторов двух предложений?

Возможность сравнения двух векторов очень важна, потому что позволяет судить о том, что видит нейронная сеть. Даже не имея возможности прочесть два вектора, вы сможете сказать, насколько они похожи или отличаются (с помощью функции из главы 11). Если способ создания векторов предложений не отражает сходства, имеющегося между ними, тогда сети трудно будет понять, когда предложения похожи, а когда отличаются. Только при этом она должна работать с векторами!

Продолжая изучать и оценивать разные способы получения векторов предложений, я хочу, чтобы вы помнили, для чего мы это делаем. Мы пытаемся рассматривать задачу с точки зрения нейронной сети. Мы спрашиваем: «Будет ли найдена корреляция между векторами похожих предложений и желаемой меткой на выходе, или для двух почти идентичных предложений будут сгенерированы настолько разные векторы, что между ними и меткой не будет найдено никакой корреляции?» Нам нужны такие векторы предложений, которые позволили бы прогнозировать смысл этих предложений, то есть для похожих предложений должны получаться похожие векторы.

Предыдущий способ создания векторов предложений (объединение) имел проблемы из-за неопределенности способа выбора, с какого края их рассматривать, поэтому рассмотрим другой простой подход. А что если взять все вектора слов в предложении и усреднить их? Это сразу же избавляет нас от проблемы сравнения векторов разной длины, потому что векторы всех предложений будут иметь одинаковую длину!

В таком случае предложения «the cat sat» и «the cat sat still» получают похожие векторы из-за сходства слов, составляющих их. Более того, вполне вероятно, что предложения «a dog walked» (собака идет) и «the cat sat» (кошка сидит) тоже получают похожие векторы из-за сходства слов, даже при том, что сами слова в этих векторах не совпадают.



Как оказывается, усреднение векторных представлений слов оказывается удивительно эффективным приемом создания векторных представлений предложений. Он не идеален (как вы увидите далее), но отлично справляется с задачей передачи сложных отношений между словами. Прежде чем двинуться дальше, будет очень полезно взять векторные представления слов из главы 11 и поэкспериментировать с их усреднением.

## Удивительная мощь усредненных векторов слов

### Это удивительно мощный инструмент нейронного прогнозирования

В предыдущем разделе я предложил второй способ создания векторов, способных передавать смысл последовательностей слов. Этот метод заключается в усреднении векторов слов, входящих в предложение, и позволяет получать новые векторы предложений, обладающие некоторыми желаемыми свойствами.

В этом разделе мы поэкспериментируем с векторами предложений, полученными из векторных представлений слов в предыдущей главе. Для этого возьмем код из главы 11, извлечем векторные представления слов из корпуса IMDB и проведем несколько экспериментов с усредненными векторами предложений. Код выполняет ту же нормализацию при сравнении векторных представлений



слов. Но на этот раз он предварительно нормализует все векторные представления слов, помещая их в матрицу `normed_weights`. Затем определяет функцию `make_sent_vect` и использует ее для преобразования каждого обзора (списка слов) в векторное представление методом усреднения. Результат сохраняется в матрицу `reviews2vectors`.

Далее определяется функция, которая запрашивает обзоры, наиболее похожие на заданный, выполняя скалярное произведение между вектором обзора на входе и векторами всех обзоров в корпусе. Это скалярное произведение — та же самая метрика сходства, которую мы обсуждали в главе 4, когда учились получать прогноз по нескольким входам.

```
import numpy as np
norms = np.sum(weights_0_1 * weights_0_1,axis=1)
norms.resize(norms.shape[0],1)
normed_weights = weights_0_1 * norms

def make_sent_vect(words):
    indices = list(map(lambda x:word2index[x],\
        filter(lambda x:x in word2index,words)))
    return np.mean(normed_weights[indices],axis=0)

reviews2vectors = list()
for review in tokens:  ◀— Лексемизированные обзоры
    reviews2vectors.append(make_sent_vect(review))
reviews2vectors = np.array(reviews2vectors)

def most_similar_reviews(review):
    v = make_sent_vect(review)
    scores = Counter()
    for i,val in enumerate(reviews2vectors.dot(v)):
        scores[i] = val
    most_similar = list()

    for idx,score in scores.most_common(3):
        most_similar.append(raw_reviews[idx][0:40])
    return most_similar
most_similar_reviews(['boring','awful'])

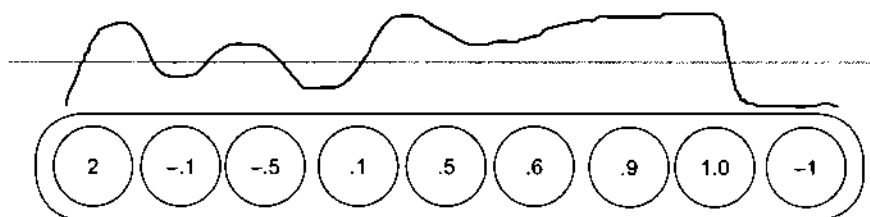
['I am amazed at how boring this film',
 'This is truly one of the worst dep',
 'It just seemed to go on and on and.]
```

Самое удивительное, пожалуй, что в ответ на запрос с двумя словами, «boring» (скучный) и «awful» (ужасный), мы получили три отрицательных отзыва. Похоже, что в этих векторах содержится интересная статистическая информация, объединяющая положительные и отрицательные отзывы в разные группы.

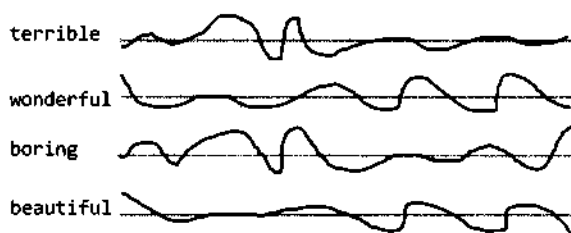
## Как векторные представления хранят информацию?

При усреднении векторных представлений остается усредненная форма

Чтобы понять суть происходящего, нужно немного отвлечься и дать мыслям устояться в течение некоторого времени, потому что для многих из вас это непривычный взгляд на вещи. Итак, давайте остановимся и попробуем представить вектор слова в виде *волнистой линии*, как показано ниже:



Представим вектор не как список чисел, а как линию, то вздымающуюся вверх, то опускающуюся вниз, согласно высоким и низким значениям в разных элементах вектора. Если выбрать несколько слов из корпуса, их можно изобразить, как показано ниже:



Посмотрим, насколько похожи друг на друга разные слова. Обратите внимание, что каждому вектору соответствует линия уникальной формы. Но линии, соответствующие словам «terrible» (ужасный) и «boring» (скучный), имеют похожую форму. Аналогично линии для слов «beautiful» (замечательный) и «wonderful» (прекрасный) тоже имеют похожую форму, но отличаются от линий других слов. Если попробовать выполнить группировку по сходству линий, слова с похожим смыслом окажутся в одной группе. Что особенно важно, отдельные фрагменты этих извилистых линий тоже имеют определенный смысл.

Например, для слов с отрицательным оттенком характерно небольшое понижение с последующим всплеском вверх, примерно на расстоянии 40 % слева. Если нарисовать линии, соответствующие другим словам, этот всплеск продолжал бы отличать отрицательные слова. В этом нет ничего удивительного, он просто означает «отрицательность», и если бы мы обучили сеть, он, скорее всего, обнаружился бы где-то еще. Всплеск указывает на отрицательный оттенок только потому, что он есть у всех отрицательных слов!

В процессе обучения эти формы моделируются таким образом, что разные изгибы в разных местах передают определенный смысл (как говорилось в главе 11). При усреднении кривых слов в предложении наиболее доминирующие изгибы не исчезают, а шум, создаваемый каждым конкретным словом, усредняется.

## Как нейронная сеть использует векторные представления?

**Нейронные сети определяют кривые, которые коррелируют с целевой меткой**

Мы познакомились с новой интерпретацией векторных представлений слов — в виде волнистых линий с характерными особенностями (изгибами). Мы также узнали, что в процессе обучения в этих кривых обнаруживается все больше отличительных черт, помогающих достижению поставленной цели. Слова с похожим значением, так или иначе, часто имеют кривые похожей формы: комбинации высоких и низких значений весов. В этом разделе мы посмотрим, как происходит обобщение корреляции при получении таких кривых на входе данных. Как слой обрабатывает эти кривые?

Вообще говоря, нейронная сеть обрабатывает векторные представления точно так же, как, например, данные с результатами наблюдения за светодифракцией, о которых рассказывалось в первых главах книги. В скрытом слое она отыскивает корреляцию между разными изгибами и целевой меткой. Такое возможно благодаря тому, что слова, обладающие некоторой сходной чертой, имеют кривые с похожими изгибами. В какой-то момент в процессе обучения нейронная сеть начинает различать уникальные особенности форм разных слов, что помогает ей группировать слова (по схожим изгибам) и приходит к точному прогнозу. Это всего лишь другой взгляд на выводы, сделанные в конце главы 11. Но продолжим.

В этой главе мы выясним смысл объединения векторных представлений слов в векторное представление предложения. В каких случаях может пригодиться

такое объединенное векторное представление? Мы уже выяснили, что в результате усреднения векторных слов в предложении получается вектор, отражающий усредненные характеристики отдельных слов. Если в предложении присутствует много положительных слов, окончательный вектор будет иметь положительные черты (прочий шум от слов обычно при этом компенсируется). Но обратите внимание, что этот подход склонен сглаживать обобщенную кривую: при достаточно большом количестве слов в результате усреднения их кривых может получиться прямая линия.

Это подводит нас к первому недостатку данного приема: при попытке сохранить информацию из последовательности произвольной длины (предложения) в векторе фиксированной длины вектор предложения (как среднее по множеству слов) может быть усреднен (если его длины окажется недостаточно для сохранения большого объема информации) до прямой линии (вектора со значениями элементов, близкими к 0).

Проще говоря, этот процесс сохранения информации характеризуется слишком быстрым затуханием. Если попытаться сохранить в одном векторе слишком много слов, вся информация может быть потеряна. С другой стороны, предложения обычно содержат не очень много слов; и если предложение содержит повторяющиеся закономерности, их векторы могут сослужить неплохую службу благодаря сохранению доминирующих закономерностей векторов слов (как, например, всплеск, отвечающий за отрицательную окраску, в предыдущем разделе).

## Ограничение векторов в модели «мешок слов»

При усреднении векторных представлений слов их порядок следования становится неактуальным

Самая большая проблема усреднения векторных представлений заключается в отсутствии понятия порядка следования. Например, рассмотрим два предложения: «Yankees defeat Red Sox» («Янки» выиграли у «Ред Сокс») и «Red Sox defeat Yankees» («Ред Сокс» выиграли у «Янки»). Применяв прием усреднения, описанный выше, вы получите два идентичных вектора, но предложения имеют совершенно противоположный смысл! Кроме того, этот прием игнорирует синтаксис и грамматику, поэтому для предложения «Sox Red Yankees defeat»<sup>1</sup> будет получено точно такое же векторное представление.

<sup>1</sup> «Сокс Ред Янки выиграли у» — предложение одинаково бессмысленное, что на английском, что на русском языке. — *Примеч. пер.*

Прием суммирования или усреднения векторных представлений слов до векторного представления фразы или предложения известен как классическая модель «мешка слов», потому что при таком объединении слов в «мешок» их порядок не сохраняется и не учитывается. Главное ограничение состоит в том, что можно взять любое предложение, переставить слова местами и получить в результате тот же самый вектор предложения, независимо от порядка следования слов (потому что сложение ассоциативно:  $a + b = b + a$ ).

Истинная цель этой главы — научиться генерировать векторы предложений так, чтобы *учитывался* порядок следования слов. Нам нужен такой способ создания векторов, чтобы при перемешивании слов в предложении получались разные векторы. Еще более важно найти *способ обучения, для которого порядок имеет значение* (то есть способ, изменяющий вектор при изменении порядка слов). При таком подходе с представлением порядка слов нейронная сеть может попытаться решить лингвистическую задачу и даже уловить суть порядка следования слов. Я использую лингвистику лишь как пример, но эти утверждения можно обобщить на любые последовательности. Просто лингвистика — хотя и сложная, но широко известная область.

Один из самых известных и успешных способов создания векторов из последовательностей (таких, как предложения) основан на использовании *рекуррентных нейронных сетей* (recurrent neural network, RNN). Чтобы показать, как работает этот способ, сначала рассмотрим несколько расточительный подход к усреднению векторных представлений слов с использованием так называемой *единичной матрицы*. Единичная матрица — это просто квадратная матрица (число строк == числу столбцов) произвольного размера с единицами на главной диагонали (соединяющей верхний левый и правый нижний углы) и с нулями во всех остальных элементах, как в примерах ниже.

```
[1,0]
```

```
[0,1]
```

```
[1,0,0]
```

```
[0,1,0]
```

```
[0,0,1]
```

```
[1,0,0,0]
```

```
[0,1,0,0]
```

```
[0,0,1,0]
```

```
[0,0,0,1]
```

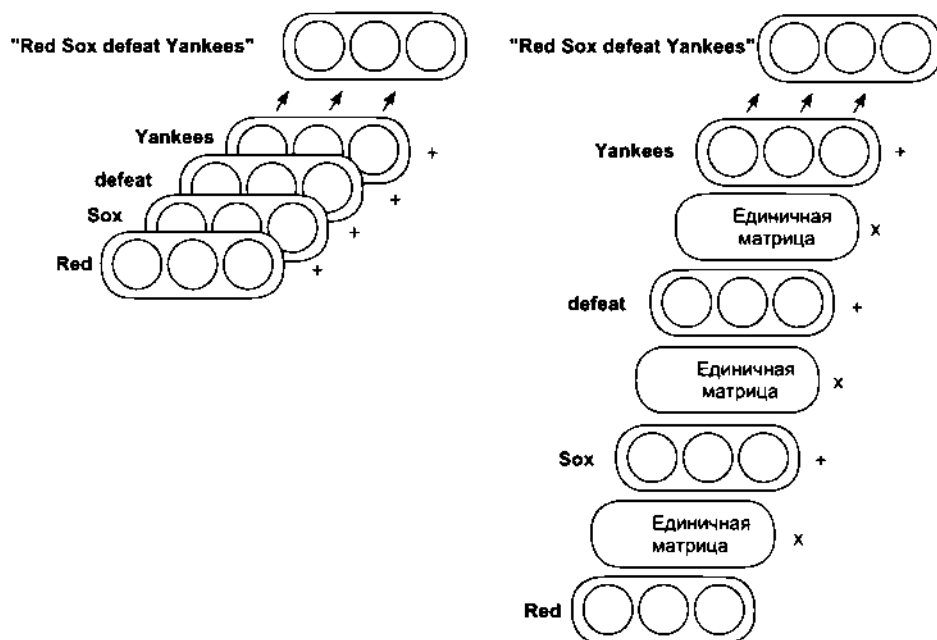
Все три матрицы являются единичными и обладают одним свойством: при умножении *любого* вектора на эту матрицу в результате получается исходный

вектор. Если умножить вектор [3, 5] на верхнюю единичную матрицу, в результате получится вектор [3, 5].

## Объединение векторных представлений слов с использованием единичной матрицы

Реализуем ту же логику другим способом

Может показаться, что применение единичных матриц не имеет смысла. В чем смысл использовать умножение вектора на единичную матрицу, если в результате получается все тот же вектор? В данном случае мы используем ее в роли учебного пособия, чтобы показать, как реализовать более сложный способ объединения векторных представлений слов, чтобы заставить нейронную сеть учитывать порядок слов при создании векторного представления предложения. Исследуем другой способ объединения векторных представлений.



Это стандартный способ объединения векторных представлений нескольких слов для получения векторного представления предложения (деление на число слов дает усредненное векторное представление предложения). Пример на ри-

сунке на предыдущей странице добавляет промежуточный шаг *между* каждым сложением: умножение вектора на единичную матрицу.

Вектор для слова «Red» умножается на единичную матрицу, затем суммируется с вектором слова «Sox», после чего результат вновь умножается на единичную матрицу и складывается с вектором слова «defeat», и так далее до конца предложения. Обратите внимание: так как умножение вектора на единичную матрицу дает в результате тот же вектор, процесс справа дает *то же векторное представление предложения*, что и процесс слева.

Это несколько расточительный подход к вычислениям, но наберитесь терпения, скоро все изменится. Здесь важно отметить, что если вместо единичной матрицы использовать любую другую матрицу, изменение порядка следования слов приведет к другому результату. Теперь посмотрим, как это решение выглядит на языке Python.

## Матрицы, которые ничего не меняют

Реализуем вычисление векторного представления предложения с использованием единичной матрицы на Python

В этом разделе мы посмотрим, как использовать единичные матрицы на Python и, в конечном счете, реализовать новый способ получения векторных представлений предложений, описанный в предыдущем разделе (доказав, что в результате получаются идентичные векторы).

```
import numpy as np

a = np.array([1,2,3])
b = np.array([0.1,0.2,0.3])
c = np.array([-1,-0.5,0])
d = np.array([0,0,0])
```

```
identity = np.eye(3)
print(identity)
```

---

```
[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
```

---

```
print(a.dot(identity))
print(b.dot(identity))
print(c.dot(identity))
print(d.dot(identity))
```

---

```
{ 1.  2.  3.}
[ 0.1  0.2  0.3]
[-1.  -0.5  0. ]
[ 0.   0.   0. ]
```

В этом листинге мы сначала инициализируем четыре вектора (a, b, c и d) с тремя элементами в каждом, а также единичную матрицу с тремя строками и тремя столбцами (единичные матрицы всегда квадратные). Обратите внимание, что единичная матрица характеризуется наличием единиц в элементах на диагонали, соединяющей верхний левый и правый нижний угол (в линейной алгебре эта диагональ называется *главной диагональю*). Любая квадратная матрица с единицами на главной диагонали и нулями в остальных элементах является единичной.

Затем мы умножаем каждый вектор на единичную матрицу (используя функцию `dot` из библиотеки NumPy). Как видите, в результате этого процесса получаются новые векторы, идентичные исходным.

Так как умножение вектора на единичную матрицу возвращает тот же начальный вектор, включение его в процесс вычисления векторного представления предложения выглядит тривиально просто:

```
this = np.array([2,4,6])
movie = np.array([10,10,10])
rocks = np.array([1,1,1])

print(this + movie + rocks)
print((this.dot(identity) + movie).dot(identity) + rocks)

[13 15 17]
[ 13.  15.  17.]
```

Оба способа получения векторного представления предложения дают в результате один и тот же вектор. Такое возможно только потому, что единичная матрица — это особый вид матриц. Но что получится, если вместо единичной использовать другую матрицу? На самом деле единичная матрица — *единственная*, которая гарантирует получение того же самого вектора, который умножался на нее. Никакие другие матрицы таких гарантий не дают.



## Определение переходных матриц

Что если изменить единичную матрицу, чтобы уменьшить потери?

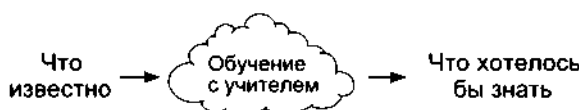
Перед началом вспомним нашу цель: научиться генерировать векторные представления предложений, которые можно группировать так, чтобы по ним можно было находить близкие по смыслу предложения. Эти векторные представления, в частности, должны учитывать порядок следования слов.

Выше мы пытались суммировать векторные представления слов. Но при таком подходе для предложения «Red Sox defeat Yankees» («Ред Сокс» выиграли у «Янки») он дает точно такой же вектор, как для предложения «Yankees defeat Red Sox» («Янки» выиграли у «Ред Сокс»), даже притом, что эти предложения имеют противоположный смысл. Поэтому нам нужен такой способ, который генерировал бы для этих предложений *разные* векторные представления (и при этом все еще осмысленные).

Теоретически, если использовать прием с единичной матрицей, но вместо этой единичной матрицы использовать любую другую матрицу, векторные представления предложений будут получаться разными в зависимости от порядка слов.

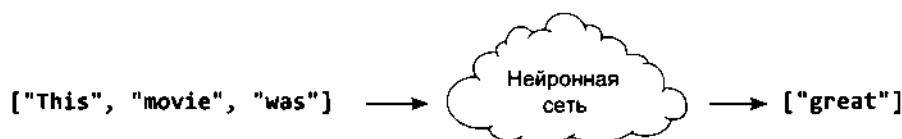
Возникает очевидный вопрос: какую матрицу использовать вместо единичной? Ответов на этот вопрос бесчисленное множество. Но в глубоком обучении есть свой стандартный ответ: обучить матрицу так же, как любую другую матрицу в нейронной сети! Хорошо, мы должны просто обучить эту матрицу. Но как?

Чтобы обучить нейронную сеть чему-то, нужно иметь некоторую задачу, на которой она будет обучаться. В данном случае задача должна потребовать от сети сгенерировать значимые векторные представления предложений, исследовав векторы слов и возможные модификации единичных матриц. Как такую задачу реализовать?



Аналогичную цель мы преследовали, когда решали проблему получения значимых векторных представлений слов (задача подстановки недостающих слов).

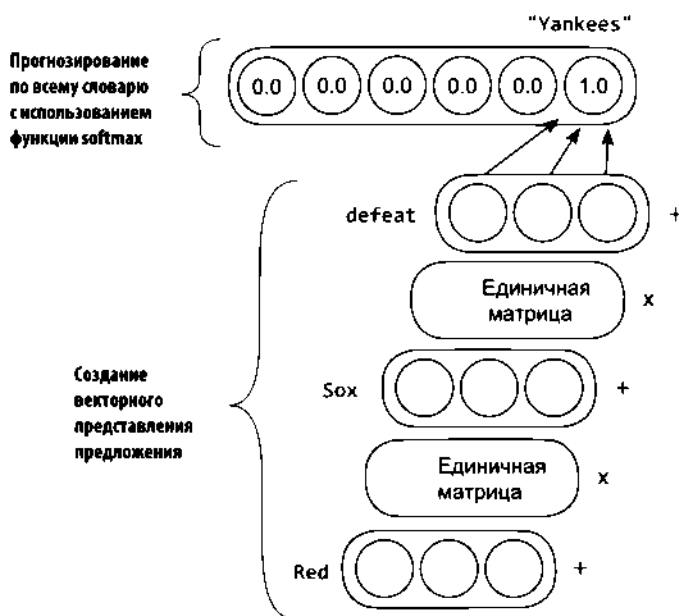
Попробуем решить очень похожую задачу: обучить нейронную сеть по списку слов предсказывать следующее слово.



## Обучение созданию векторов предложений

Создайте вектор предложения, получите прогноз  
и измените вектор предложения, меняя его составляющие

Я не хочу, чтобы в следующем нашем эксперименте вы думали о сети так же, как думали о предыдущих нейронных сетях. Вместо этого думайте о создании векторного представления предложения для прогнозирования следующего слова и последующем изменении соответствующих частей, участвовавших в формировании этого вектора, для получения более точного прогноза. Поскольку речь идет о прогнозировании следующего слова, векторное представление предложения будет состоять из частей, уже виденных до сих пор. Наша новая нейронная сеть будет выглядеть, как показано на следующем рисунке.



Она состоит из двух сегментов: первый создает векторное представление предложения, а второй использует это представление для предсказания следующего слова. На вход этой сети передается предложение «Red Sox defeat», а прогнозируемое слово — «Yankees».

Блоки между векторами слов я подписал «Единичная матрица». Эта матрица будет единичной *только в первый момент*. В процессе обучения эти матрицы будут корректироваться на этапе обратного распространения, чтобы помочь сети получить более точный прогноз (так же, как остальные весовые коэффициенты в сети).

Благодаря этому сеть будет учиться *внедрять в векторное представление больше информации, чем простую сумму векторов слов*. Позволив матрицам (первоначально единичным) изменяться (и переставать быть единичными), мы даем нейронной сети возможность обучиться созданию векторных представлений, которые меняются при изменении порядка следования слов. Но это изменение единичных матриц происходит не произвольно. Сеть будет учиться учитывать порядок слов так, чтобы *максимально точно предсказать следующее слово*.

Мы также наложим ограничение на *переходные матрицы* (матрицы, которые первоначально были единичными), потребовав, чтобы все эти матрицы были одной и той же матрицей. Проще говоря, матрица, используемая на этапе перехода «Red» -> «Sox», будет повторно использована на этапе «Sox» -> «defeat». Все, чему сеть обучится при анализе первого перехода, будет повторно использовано в следующих переходах, благодаря этому она будет изучать только логику, полезную для прогнозирования.

## Прямое распространение на Python

Остановимся на этой идее и посмотрим, как реализовать простое прямое распространение

Теперь, имея концептуальное представление о том, что мы должны создать, рассмотрим упрощенную версию на Python. Сначала настроим веса (здесь я использовал ограниченный словарь из девяти слов):

```
import numpy as np

def softmax(x_):
    x = np.atleast_2d(x_)
```

```

temp = np.exp(x)
return temp / np.sum(temp, axis=1, keepdims=True)

word_vectors = {}
word_vectors['yankees'] = np.array([[0.,0.,0.]])
word_vectors['bears'] = np.array([[0.,0.,0.]])
word_vectors['braves'] = np.array([[0.,0.,0.]])
word_vectors['red'] = np.array([[0.,0.,0.]])
word_vectors['sox'] = np.array([[0.,0.,0.]])
word_vectors['lose'] = np.array([[0.,0.,0.]])
word_vectors['defeat'] = np.array([[0.,0.,0.]])
word_vectors['beat'] = np.array([[0.,0.,0.]])
word_vectors['tie'] = np.array([[0.,0.,0.]])

sent2output = np.random.rand(3,len(word_vectors))
identity = np.eye(3)

```

Векторные представления слов

Векторное представление предложения для вывода классифицирующих весов

Переходные веса

Этот код генерирует три набора весов. Он создает словарь Python с векторными представлениями слов, единичную (переходную) матрицу и классифицирующий слой. Роль классифицирующего слоя здесь играет матрица весов `sent2output`, прогнозирующая следующее слово по заданному вектору предложения из трех слов. С этими инструментами реализация прямого распространения выглядит тривиально просто. Вот как работает прямое распространение для случая «red sox defeat» -> «yankees»:

```

layer_0 = word_vectors['red']
layer_1 = layer_0.dot(identity) + word_vectors['sox']
layer_2 = layer_1.dot(identity) + word_vectors['defeat']

pred = softmax(layer_2.dot(sent2output))
print(pred)
[[ 0.11111111  0.11111111  0.11111111  0.11111111  0.11111111  0.11111111
  0.11111111  0.11111111  0.11111111]]

```

Создание векторного представления предложения

Прогнозирование по всему словарю

## Как добавить сюда обратное распространение?

Этот шаг может показаться сложнее, но все необходимое вы уже знаете

Вы только что видели, как реализовать прямое распространение в этой сети. В первый момент может быть не понятно, как реализовать обратное распространение. На самом деле все просто. Многое вы уже видели выше:

Обычная нейронная сеть  
(главы 1–5)

Немного странная добавка

```

layer_0 = word_vecs['red']
layer_1 = layer_0.dot(identity) + word_vecs['sox']
layer_2 = layer_1.dot(identity) + word_vecs['defeat']

pred = softmax(layer_2.dot(sent2output))
print(pred)

```

| Снова обычная нейронная  
сеть (глава 9)

Опираясь на знания, полученные в предыдущих главах, вычисление потерь и реализация обратного распространения не должны вызывать вопросов, пока вы не доберетесь до градиентов `layer_2_delta` в слое `layer_2`. В этой точке у многих из вас может появиться вопрос: в каком направлении выполнять обратное распространение? Градиенты могут возвращаться обратно в слой `layer_1` через умножение на единичную матрицу `identity` или в `word_vecs['defeat']`.

Складывая два вектора на этапе прямого распространения, мы распространяем градиент в обе стороны сложения. Когда генерируем `layer_2_delta`, мы выполняем обратное распространение дважды: через единичную матрицу, чтобы получить `layer_1_delta`, и в `word_vecs['defeat']`:

```

y = np.array([1,0,0,0,0,0,0,0])

```

← Вектор для слова «yankees»,  
полученный прямым кодированием

```

pred_delta = pred - y
layer_2_delta = pred_delta.dot(sent2output.T)
defeat_delta = layer_2_delta * 1
layer_1_delta = layer_2_delta.dot(identity.T)
sox_delta = layer_1_delta * 1
layer_0_delta = layer_1_delta.dot(identity.T)
alpha = 0.01
word_vecs['red'] -= layer_0_delta * alpha
word_vecs['sox'] -= sox_delta * alpha
word_vecs['defeat'] -= defeat_delta * alpha
identity -= np.outer(layer_0, layer_1_delta) * alpha
identity -= np.outer(layer_1, layer_2_delta) * alpha
sent2output -= np.outer(layer_2, pred_delta) * alpha

```

← Можно игнорировать «1», как в главе 11

← И снова можно игнорировать «1»

## Обучим ее!

У нас есть все необходимое; обучим сеть на ограниченном корпусе

Чтобы понять, что происходит, сначала обучим новую сеть на маленьком наборе данных под названием Babi. Это искусственно созданный корпус из вопросов и ответов, предназначенный для обучения машин отвечать на простые

вопросы об окружающей среде. Мы не будем использовать его для оценки качества обучения (пока), эта простая задача лишь поможет вам лучше увидеть, как на ее выполнение повлияет обучение единичной матрицы. Сначала загрузите набор данных Babi, выполнив следующие команды bash:

```
wget http://www.thespermwhale.com/jaseweston/babi/tasks_1-20_v1-1.tar.gz
tar -xvf tasks_1-20_v1-1.tar.gz
```

С помощью следующего кода на Python откройте этот набор данных и подготовьте его к обучению сети:

```
import sys, random, math
from collections import Counter
import numpy as np

f = open('tasksv11/en/qa1_single-supporting-fact_train.txt', 'r')
raw = f.readlines()
f.close()

tokens = list()
for line in raw[0:1000]:
    tokens.append(line.lower().replace("\n", "").split(" ")[1:])

print (tokens[0:3])

[['Mary', 'moved', 'to', 'the', 'bathroom'],
 ['John', 'went', 'to', 'the', 'hallway'],
 ['Where', 'is', 'Mary', 'bathroom'],
```

Как видите, этот набор содержит простые предложения и вопросы (без знаков препинания). За каждым вопросом следует правильный ответ. При использовании в контексте тестирования нейронная сеть читает предложения и отвечает на вопросы (правильно или неправильно), опираясь на информацию в недавно прочитанных предложениях.

Сейчас мы попробуем научить сеть правильно завершать предложения по одному или нескольким словам. Попутно вы увидите, насколько важную роль играет обучение рекуррентной (первоначально единичной) матрицы.

## Подготовка

**Перед созданием матриц нужно определить количество параметров**

Так же как в примере нейронной сети, предсказывающей векторные представления слов, сначала нужно создать несколько счетчиков, списков и вспомога-

тельных функций, которые будут использоваться в процессе прогнозирования, сравнения и обучения. Эти вспомогательные функции и объекты показаны ниже и должны выглядеть для вас знакомыми:

```

vocab = set()
for sent in tokens:
    for word in sent:
        vocab.add(word)

vocab = list(vocab)

word2index = {}
for i, word in enumerate(vocab):
    word2index[word] = i

def words2indices(sentence):
    idx = list()
    for word in sentence:
        idx.append(word2index[word])
    return idx

def softmax(x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum(axis=0)

```

В листинге слева создается простой список слов из словаря, а также словарь, позволяющий определять индексы слов в словаре. Эти индексы будут использоваться для выбора номеров строк и столбцов в матрицах векторных представлений и прогнозирования, соответствующих словам. В листинге справа определяются функция для преобразования списка слов в список индексов и функция активации `softmax` для предсказания следующего слова.

Код в следующем листинге инициализирует начальное значение случайной последовательности (чтобы получить непротиворечивые результаты) и устанавливает размер векторного представления равным 10. Далее создаются матрица векторных представлений слов, рекуррентная матрица и начальное векторное представление предложения `start`. Начальное векторное представление предложения соответствует пустой фразе, именно так начинается моделирование предложений в нейронных сетях. Наконец, создается весовая матрица `decoder` (по аналогии с матрицей векторных представлений) и вспомогательная матрица `one_hot`:

```

np.random.seed(1)
embed_size = 10

embed = (np.random.rand(len(vocab), embed_size) - 0.5) * 0.1  ← Вложения слов

recurrent = np.eye(embed_size)  ← Рекуррентная матрица (первоначально единичная)

start = np.zeros(embed_size)  ← Векторное представление для пустого предложения

decoder = (np.random.rand(embed_size, len(vocab)) - 0.5) * 0.1  ←

one_hot = np.eye(len(vocab))  ← Матрица поиска выходных весов (для функции потерь)

```

Выходные веса для прогнозирования  
векторного представления предложения

## Прямое распространение с данными произвольной длины

Прямое распространение выполняется так же, как раньше

Следующий код реализует логику прямого распространения и предсказания следующего слова. Обратите внимание, что хотя конструктивно она может показаться незнакомой, эта логика следует той же процедуре, что использовалась прежде для сложения векторных представлений слов с использованием единичной матрицы. Здесь вместо единичной матрицы используется матрица с именем `recurrent`, инициализированная нулями (и она будет корректироваться в процессе обучения).

Кроме того, вместо предсказания только по последнему слову мы вычисляем предсказание (`layer['pred']`) на каждом шаге, опираясь на векторное представление, сгенерированное из предыдущих слов. Это эффективнее, чем выполнять прямое распространение с начала фразы каждый раз, когда требуется предсказать новое слово.

```
def predict(sent):
    layers = list()
    layer = {}
    layer['hidden'] = start
    layers.append(layer)

    loss = 0

    preds = list()  # ← Для прогноза
    for target_i in range(len(sent)):

        layer = {}

        layer['pred'] = softmax(layers[-1]['hidden'].dot(decoder))  # ←
        # Попытка предсказать следующее слово

        loss += -np.log(layer['pred'][sent[target_i]])

        layer['hidden'] = layers[-1]['hidden'].dot(recurrent) + \
            embed[sent[target_i]]  # ←
        # Сгенерировать следующее состояние скрытого слоя

        layers.append(layer)

    return layers, loss
```

В этом фрагменте кода нет ничего нового, но в нем есть кое-что, что я хотел бы отметить особо, прежде чем двинуться дальше. Список с именем `layers` — это новый способ организации прямого распространения.





```

if(layer_idx == len(layers)-1):
    layer['hidden_delta'] = new_hidden_delta
else:
    layer['hidden_delta'] = new_hidden_delta + \
        layers[layer_idx+1]['hidden_delta']\
            .dot(recurrent.transpose())
else: # если это первый слой
    layer['hidden_delta'] = layers[layer_idx+1]['hidden_delta']\
        .dot(recurrent.transpose())

```

Если это последний слой, не добавлять  
последующий, потому что его не существует

Прежде чем перейти к следующему разделу, внимательно прочитайте этот код и попробуйте объяснить своему другу (или хотя бы самому себе), как он действует. В этом фрагменте нет ничего нового, но в первый момент его конструкция может показаться незнакомой. Уделите время и свяжите этот код с каждой строкой в примере «Red Sox defeat Yankees». После этого вы будете готовы перейти к следующему разделу и заняться коррекцией весов с использованием градиентов.

## Корректировка весов с данными произвольной длины

Корректировка весов выполняется так же, как раньше

В логике корректировки весов, так же как в логике прямого и обратного распространения, нет ничего нового. Но я покажу это после объяснения, чтобы вы могли сосредоточиться на инженерной задаче, когда вы (надеюсь) грокнете (ха!) сложную теорию.

```

for iter in range(30000):  ← Прямое распространение
    alpha = 0.001
    sent = words2indices(tokens[iter%len(tokens)])[1:]

    layers, loss = predict(sent)

    for layer_idx in reversed(range(len(layers))):  ← Обратное распространение
        layer = layers[layer_idx]
        target = sent[layer_idx-1]

        if(layer_idx > 0):
            layer['output_delta'] = layer['pred'] - one_hot[target]
            new_hidden_delta = layer['output_delta']\
                .dot(decoder.transpose())

```

```

if(layer_idx == len(layers)-1):
    layer['hidden_delta'] = new_hidden_delta
else:
    layer['hidden_delta'] = new_hidden_delta + \
        layers[layer_idx+1]['hidden_delta']\
        .dot(recurrent.transpose())
else:
    layer['hidden_delta'] = layers[layer_idx+1]['hidden_delta']\
        .dot(recurrent.transpose())

start -= layers[0]['hidden_delta'] * alpha / float(len(sent))
for layer_idx, layer in enumerate(layers[1:]):
    decoder -= np.outer(layers[layer_idx]['hidden'],\
        layer['output_delta']) * alpha / float(len(sent))

    embed_idx = sent[layer_idx]
    embed[embed_idx] -= layers[layer_idx]['hidden_delta'] * \
        alpha / float(len(sent))
    recurrent -= np.outer(layers[layer_idx]['hidden'],\
        layer['hidden_delta']) * alpha / float(len(sent))

if(iter % 1000 == 0):
    print("Perplexity:" + str(np.exp(loss/len(sent))))

```

← Если это последний слой, не добавлять последующий, потому что его не существует

← Обновления весов

## Запуск и анализ результатов

### Перплексия как метрика качества модели

Настал момент истины: что произойдет, если запустить этот код? Когда я попробовал это сделать, то получил устойчивый тренд уменьшения значения метрики с названием *перплексия*<sup>1</sup> (perplexity). Технически, перплексия — это результат извлечения логарифма из вероятности получения правильной метки (слова), последующего отрицания и вычисления экспоненты ( $e^x$ ).

Теоретически она представляет разность между двумя распределениями вероятностей. В данном случае идеальным считается такое распределение вероятностей, когда правильное слово получает 100 %-ную вероятность, а все остальные 0 %.

Перплексия получает большое значение, когда два распределения вероятностей не совпадают, и низкое (близкое к 1), когда совпадают. То есть последовательное уменьшение перплексии, как и любой другой функции потерь, используе-

<sup>1</sup> Мера несоответствия, или «удивленности», используемая для оценки моделей языка в вычислительной лингвистике. — *Примеч. пер.*

мой в стохастическом градиентном спуске, это хороший знак! Это означает, что сеть учится предсказывать вероятности, соответствующие данным.

```
Perplexity:82.09227500075585
Perplexity:81.87615610433569
Perplexity:81.53705034457951
....
Perplexity:4.132556753967558
Perplexity:4.071667181580819
Perplexity:4.0167814473718435
```

Но эта метрика ничего не говорит о том, что происходит с весами. В течение многих лет перплексия подвергалась критике (особенно в сообществе, занимающемся проблемами лингвистического моделирования) за неумеренное ее использование в качестве метрики. Давайте рассмотрим внимательнее процесс прогнозирования:

```
sent_index = 4

l,_ = predict(words2indices(tokens[sent_index]))

print(tokens[sent_index])

for i,each_layer in enumerate(l[1:-1]):
    input = tokens[sent_index][i]
    true = tokens[sent_index][i+1]
    pred = vocab[each_layer['pred']].argmax()
    print("Prev Input:" + input + (' ' * (12 - len(input))) + \
          "True:" + true + (" " * (15 - len(true))) + "Pred:" + pred)
```

Этот код получает предложение и сообщает, какое слово вероятнее всего будет следующим. Это помогает получить представление о характеристиках, приобретаемых моделью. Что она делает правильно? Какие ошибки допускает? Ответы на эти вопросы вы найдете в следующем разделе.

## Обзор прогнозов может помочь понять происходящее

Узнать, какие закономерности и в каком порядке выявляет нейронная сеть в процессе обучения, можно, посмотрев прогнозы, которые она возвращает. После 100 итераций обучения вывод выглядит так:

```
['sandra', 'moved', 'to', 'the', 'garden.']
Prev Input:sandra      True:moved             Pred:is
Prev Input:moved       True:to                Pred:kitchen
Prev Input:to          True:the               Pred:bedroom
Prev Input:the         True:garden.           Pred:office
```

В начальный момент нейронные сети больше склонны возвращать случайные прогнозы. В данном случае нейронная сеть, по всей видимости, отдает предпочтение словам, имевшимся в случайном начальном состоянии. Продолжим обучение:

```
['sandra', 'moved', 'to', 'the', 'garden.']
Prev Input:sandra      True:moved      Pred:the
Prev Input:moved       True:to         Pred:the
Prev Input:to          True:the        Pred:the
Prev Input:the         True:garden.    Pred:the
```

Через 10 000 итераций обучения нейронная сеть выбирает наиболее распространенное слово («the») и предсказывает его на каждом шаге. Эта ошибка очень часто встречается в рекуррентных нейронных сетях. Требуется выполнить большое количество итераций обучения, чтобы сеть смогла выявить мелкие детали в сильно искаженном наборе данных.

```
['sandra', 'moved', 'to', 'the', 'garden.']
Prev Input:sandra      True:moved      Pred:is
Prev Input:moved       True:to         Pred:to
Prev Input:to          True:the        Pred:the
Prev Input:the         True:garden.    Pred:bedroom.
```

Ошибки действительно очень интересные. Увидев одно только слово «sandra» (имя Sandra — Сандра), сеть предсказывает следующее слово «is» (есть, является), которое, хотя и не совпадает со словом «moved» (вышла), является не самой плохой догадкой. Она выбрала не тот глагол. Далее, обратите внимание, что сеть правильно выбрала слова «to» (в) и «the» (определенный артикль), что ничуть не удивительно, потому что это самые распространенные слова в наборе и, вероятно, сеть научилась правильно прогнозировать сочетание «to the», встретив ее много раз после глагола «moved». Последняя ошибка тоже довольно интересна, вместо слова «garden» (сад) сеть выбрала «bedroom» (спальня).

Важно отметить, что эта нейронная сеть не идеально справилась с этой задачей. В конце концов, если бы я дал вам слова «sandra moved to the» (Сандра вышла в), смогли бы вы сами правильно угадать следующее слово? Для решения этой задачи нужно больше контекста, но тот факт, что она, как мне кажется, в принципе неразрешима, помогает понять, почему сеть терпит неудачу.

## Итоги

### Рекуррентные нейронные сети способны дать прогноз на основе данных произвольной длины

В этой главе мы узнали, как создавать векторные представления для последовательностей произвольной длины. В последнем упражнении мы научили линейную рекуррентную сеть предсказывать следующее слово по фразе. Для этого нам пришлось научиться создавать векторные представления фиксированного размера, которые точно соответствовали бы строкам переменной длины.

Это последнее упражнение вызывает законный вопрос: как нейронной сети удастся вместить переменный объем информации в вектор фиксированной длины? На самом деле векторы предложений кодируют не всю информацию, имеющуюся в предложении. Фактически векторы в рекуррентных нейронных сетях определяют не только то, что запоминается, но и что забывается. В задаче предсказания следующего слова большинство рекуррентных нейронных сетей в процессе обучения выясняют, что важную роль играют лишь несколько последних слов<sup>1</sup>, и учатся забывать (то есть не запоминать в своих векторах уникальные закономерности) слова, расположенные в начале фразы.

Но отметьте, что в процессе создания этих векторных представлений не используются нелинейные функции активации. Как вы думаете, какие ограничения накладывает такой подход? В следующей главе мы рассмотрим этот и другие вопросы и используем нелинейные функции активации и логические элементы для формирования нейронной сети, которая называется *сетью с долгой краткосрочной памятью* (long short-term memory, LSTM). Но прежде проверьте, сможете ли вы по памяти воспроизвести код, реализующий линейную рекуррентную нейронную сеть. Динамика и поток управления в таких сетях могут показаться немного пугающими, и сложность их реализации немного выше. Поэтому, прежде чем двинуться дальше, убедитесь, что усвоили все, о чем рассказывалось в этой главе.

А теперь перейдем к сетям LSTM!

<sup>1</sup> Смотрите, например, статью «Frustratingly Short Attention Spans in Neural Language Modeling» Данилюка (Michał Daniluk) и его коллег (представлена на международной конференции ICLR 2017), <https://arxiv.org/abs/1702.04521>.

# 13 Введение в автоматическую оптимизацию: создание фреймворка глубокого обучения



## В этой главе

- ✓ Что такое фреймворк глубокого обучения?
- ✓ Введение в тензоры.
- ✓ Введение в автоматическое вычисление градиента (autograd).
- ✓ Как работает сложение в обратном распространении?
- ✓ Как научиться пользоваться фреймворком.
- ✓ Нелинейные слои.
- ✓ Слой с векторным представлением.
- ✓ Слой с перекрестной энтропией.
- ✓ Рекуррентный слой.

А то, что одни основаны на углероде, а другие на кремнии, несущественно. И те и другие в равной степени достойны уважения.

Артур Чарльз Кларк (Arthur C. Clarke),  
2010: *Odyssey Two* (1982)

## Что такое фреймворк глубокого обучения?

**Хорошие инструменты уменьшают количество ошибок, ускоряют разработку и увеличивают скорость выполнения**

Если вы много читали о глубоком обучении, то наверняка сталкивались с такими известными фреймворками, как PyTorch, TensorFlow, Theano (недавно был объявлен устаревшим), Keras, Lasagne и DyNet. В последние несколько лет фреймворки развивались очень быстро, и, несмотря на то что все эти фреймворки распространяются бесплатно и с открытым исходным кодом, в каждом из них присутствует дух состязания и товарищества.

До сих пор я избегал обсуждения фреймворков, потому что, прежде всего, для вас крайне важно было понять, что происходит за кулисами, реализовав алгоритмы вручную (с использованием только библиотеки NumPy). Но теперь мы начнем пользоваться такими фреймворками, потому что сети, которые мы собираемся обучать, — сети с долгой краткосрочной памятью (LSTM) — очень сложны, и код, реализующий их с использованием NumPy, сложно читать, использовать и отлаживать (градиенты в этом коде встречаются повсеместно).

Именно эту сложность призваны устранить фреймворки глубокого обучения. Фреймворк глубокого обучения может существенно снизить сложность кода (а также уменьшить количество ошибок и повысить скорость разработки) и увеличить скорость его выполнения, особенно если для обучения нейронной сети использовать графический процессор (GPU), что может ускорить процесс в 10–100 раз. По этим причинам фреймворки используются в сообществе исследователей почти повсеместно, и понимание особенностей их работы пригодится вам в вашей карьере пользователя и исследователя глубокого обучения.

Но мы не будем ограничивать себя рамками какого-то конкретного фреймворка, потому что это помешает вам узнать, как работают все эти сложные модели (такие, как LSTM). Вместо этого мы создадим свой легковесный фреймворк, следуя последним тенденциям в разработке фреймворков. Следуя этим путем, вы будете точно знать, что делают фреймворки, когда с их помощью создаются сложные архитектуры. Кроме того, попытка самостоятельно создать свой небольшой фреймворк поможет вам плавно перейти к использованию настоящих фреймворков глубокого обучения, потому что вы уже будете знать принципы организации программного интерфейса (API) и его функциональные возможности. Мне это упражнение очень пригодилось, а знания, полученные при создании собственного фреймворка, оказались как нельзя кстати при отладке проблемных моделей.

Как фреймворк упрощает код? Если говорить абстрактно, он избавляет от необходимости снова и снова писать один и тот же код. А конкретно, наиболее



удобной особенностью фреймворка глубокого обучения является поддержка автоматического обратного распространения и автоматической оптимизации. Это позволяет писать только код прямого распространения, а фреймворк автоматически позаботится об обратном распространении и коррекции весов. Большинство современных фреймворков упрощают даже код, реализующий прямое распространение, предлагая высокоуровневые интерфейсы для определения типичных слоев и функций потерь.

## Введение в тензоры

Тензоры — это абстрактная форма векторов и матриц

До этого момента в качестве основных структур мы использовали векторы и матрицы. Напомню, что матрица — это список векторов, а вектор — список скаляров (отдельных чисел). *Тензор* — это абстрактная форма представления вложенных списков чисел. Вектор — это одномерный тензор. Матрица — двумерный тензор, а структуры с большим числом измерений называются *n*-мерными тензорами. Поэтому начнем создание нового фреймворка глубокого обучения с определения базового типа, который назовем `Tensor`:

```
import numpy as np

class Tensor (object):

    def __init__(self, data):
        self.data = np.array(data)

    def __add__(self, other):
        return Tensor(self.data + other.data)

    def __repr__(self):
        return str(self.data.__repr__())

    def __str__(self):
        return str(self.data.__str__())

x = Tensor([1,2,3,4,5])
print(x)

[1 2 3 4 5]

y = x + x
print(y)

[2 4 6 8 10]
```

Это первая версия нашей базовой структуры данных. Обратите внимание, что всю числовую информацию она хранит в массиве NumPy (`self.data`) и поддерживает единственную тензорную операцию (сложение). Добавить дополнительные операции совсем несложно, достаточно добавить в класс `Tensor` дополнительные функции с соответствующей функциональностью.

## Введение в автоматическое вычисление градиента (autograd)

Прежде мы выполняли обратное распространение вручную. Теперь сделаем его автоматическим!

В главе 4 мы познакомились с производными. С тех пор мы вручную вычисляли эти производные в каждой новой нейронной сети. Напомню, что достигается это обратным перемещением через нейронную сеть: сначала вычисляется градиент на выходе сети, затем этот результат используется для вычисления производной в предыдущем компоненте, и так далее, пока для всех весов в архитектуре не будут определены правильные градиенты. Эту логику вычисления градиентов тоже можно добавить в класс тензора. Ниже показано, что я имел в виду. Новый код выделен **жирным**:

```
import numpy as np

class Tensor (object):

    def __init__(self, data, creators=None, creation_op=None):
        self.data = np.array(data)
        self.creation_op = creation_op
        self.creators = creators
        self.grad = None

    def backward(self, grad):
        self.grad = grad

        if(self.creation_op == "add"):
            self.creators[0].backward(grad)
            self.creators[1].backward(grad)

    def __add__(self, other):
        return Tensor(self.data + other.data,
                      creators=[self,other],
                      creation_op="add")

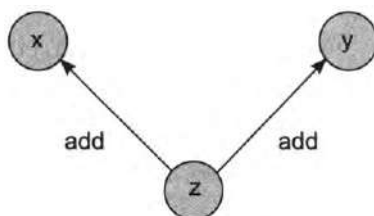
    def __repr__(self):
        return str(self.data.__repr__())
```

```
def __str__(self):
    return str(self.data.__str__())

x = Tensor([1,2,3,4,5])
y = Tensor([2,2,2,2,2])

z = x + y
z.backward(Tensor(np.array([1,1,1,1,1])))
```

Этот метод вводит два новшества. Во-первых, каждый тензор получает два новых атрибута. `creators` — это список любых тензоров, использовавшихся для создания текущего тензора (по умолчанию имеет значение `None`). То есть если тензор `z` получается сложением двух других тензоров, `x` и `y`, атрибут `creators` тензора `z` будет содержать тензоры `x` и `y`. `creation_op` — сопутствующий атрибут, который хранит операции, использовавшиеся в процессе создания данного тензора. То есть инструкция `z = x + y` создаст *вычислительный граф* с тремя узлами (`x`, `y` и `z`) и двумя ребрами (`z -> x` и `z -> y`). Каждое ребро при этом подписано операцией из `creation_op`, то есть `add`. Этот граф поможет организовать рекурсивное обратное распространение градиентов.



Первым новшеством в этой реализации является автоматическое создание графа при выполнении каждой математической операции. Если взять `z` и выполнить еще одну операцию, граф будет продолжен в новой переменной, ссылающейся на `z`.

Второе новшество в этой версии класса `Tensor` — возможность использовать граф для вычисления градиентов. Если вызвать метод `z.backward()`, он передаст градиент для `x` и `y` с учетом функции, с помощью которой создавался тензор `z` (`add`). Как показано в примере выше, мы передаем вектор градиентов (`np.array([1,1,1,1,1])`) в `z`, а тот применяет его к своим родителям. Как вы наверняка помните из главы 4, обратное распространение через сложение означает применение сложения при обратном распространении. В данном случае у нас есть только один градиент для добавления в `x` и `y`, поэтому мы копируем его из `z` в `x` и `y`:

```
print(x.grad)
print(y.grad)
print(z.creators)
print(z.creation_op)

[1 1 1 1 1]
[1 1 1 1 1]
[array([1, 2, 3, 4, 5]), array([2, 2, 2, 2, 2])]
add
```

Самой замечательной особенностью этой формы автоматического вычисления градиента является то, что она работает рекурсивно — каждый вектор вызывает метод `.backward()` всех своих родителей из списка `self.creators`:

```
a = Tensor([1,2,3,4,5])
b = Tensor([2,2,2,2,2])
c = Tensor([5,4,3,2,1])
d = Tensor([-1,-2,-3,-4,-5])
e = a + b
f = c + d
g = e + f
g.backward(Tensor(np.array([1,1,1,1,1])))
print(a.grad)
```

**Вывод:**  
[1 1 1 1 1]

## Контрольная точка

**Тензор — это просто другая форма представления всего, что мы узнали к данному моменту**

Прежде чем двинуться дальше, хочу заметить: даже притом, что кто-то из вас с трудом представляет распространение градиентов по графовой структуре, в этом нет ничего нового по сравнению с тем, что вы уже знаете. В предыдущей главе, рассказывающей о рекуррентных нейронных сетях, мы реализовали прямое распространение, а затем обратное по активациям (виртуальному графу).

Просто мы явно не обозначали узлы и ребра графовой структуры, а работали со списком слоев (словарей) и вручную определяли правильный порядок операций прямого и обратного распространения. Теперь мы создаем удобный интерфейс, чтобы избавить себя от необходимости писать много кода. Этот интерфейс позволяет рекурсивно выполнять обратное распространение и не писать для этого сложный код.

Эта глава только кажется немного теоретической. На самом деле в ней описываются инженерные приемы, широко используемые в практике обучения

нейронных сетей. В частности, граф, который конструируется в процессе прямого распространения, называется *динамическим вычислительным графом*, потому что строится «на лету», в ходе прямого распространения. Этот тип автоматического вычисления градиента (autograd) реализован в новейших фреймворках глубокого обучения, таких как DyNet и PyTorch. Более старые фреймворки, такие как Theano и TensorFlow, реализуют так называемый *статический вычислительный граф*, структура которого определяется до начала прямого распространения.

В общем случае динамические вычислительные графы проще в реализации и с ними легче экспериментировать, а статические вычислительные графы имеют более высокую производительность благодаря оптимизациям, скрытым за кулисами. Но обратите внимание, что динамические и статические фреймворки в последнее время движутся друг к другу, предлагая возможность компиляции динамических вычислительных графов в статические (для увеличения скорости выполнения) или конструирования статических графов динамически (для упрощения экспериментов). В какой-то момент в будущем мы, вероятно, получим и то и другое. Основное отличие заключается лишь в том, когда конструируется граф — до прямого распространения или во время него. В этой книге мы будем придерживаться динамического подхода.

Главная цель этой главы — помочь вам подготовиться к использованию глубокого обучения в реальном мире, где 10 % (или меньше) времени тратится на обдумывание новых идей, а 90 % — на выяснение, как лучше реализовать фреймворк, чтобы сделать его удобнее. Иногда отладка таких фреймворков может превращаться в чрезвычайно сложную задачу, потому что большинство ошибок не возбуждает исключений и не влечет вывода трассировки стека. Большинство ошибок скрыты в коде и мешают сетям обучаться (даже когда кажется, что сети чему-то обучаются).

А теперь погрузимся в эту главу. Вы будете рады, что сделали это, когда в два часа ночи выявите ошибку оптимизации, которая мешала новейшему алгоритму получить точную оценку.

## Тензоры, используемые многократно

Простейшая реализация автоматического вычисления градиента имеет досадную ошибку. Устраним ее!

Текущая версия Tensor поддерживает только однократное обратное распространение в переменную. Но иногда один и тот же тензор может использоваться

в процессе прямого распространения многократно (например, веса нейронной сети), поэтому разные части графа будут осуществлять обратное распространение градиентов в один и тот же тензор. Но в настоящее время код неправильно вычисляет градиенты для обратного распространения в переменную, которая используется несколько раз (является родителем нескольких потомков). Вот что я имею в виду:

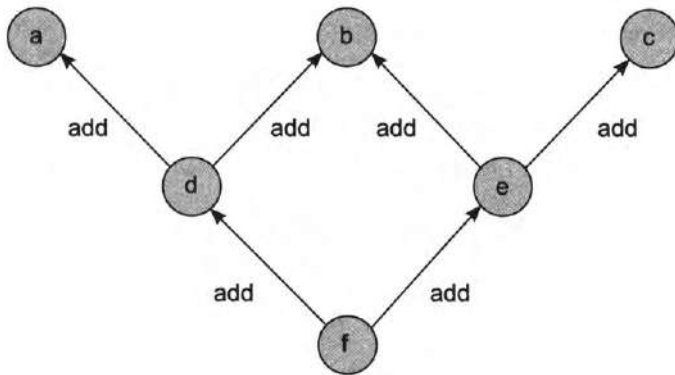
```
a = Tensor([1,2,3,4,5])
b = Tensor([2,2,2,2,2])
c = Tensor([5,4,3,2,1])

d = a + b
e = b + c
f = d + e
f.backward(Tensor(np.array([1,1,1,1,1])))

print(b.grad.data == np.array([2,2,2,2,2]))

array([False, False, False, False, False])
```

В этом примере переменная *b* используется дважды в процессе создания *f*. То есть ее градиент должен быть суммой двух производных:  $[2, 2, 2, 2, 2]$ . Ниже показан граф, созданный этой цепочкой операций. Обратите внимание, что на *b* указывают две стрелки: то есть градиент для этой переменной должен быть суммой градиентов, пришедших из *e* и *d*.



Но текущая реализация `Tensor` просто перезаписывает каждую производную предыдущей. Сначала *d* применяет свой градиент, а затем он перезаписывается градиентом из *e*. Мы должны изменить способ записи градиентов.

## Добавление поддержки тензоров многократного использования в реализацию autograd

Добавление одной новой функции и изменение трех имеющихся

Это изменение класса `Tensor` добавляет две новые возможности. Прежде всего, возможность накапливания градиентов, чтобы в случае многократного использования переменной она получала градиенты от всех своих потомков:

```
import numpy as np
class Tensor (object):

    def __init__(self, data,
                  autograd=False,
                  creators=None,
                  creation_op=None,
                  id=None):

        self.data = np.array(data)
        self.creators = creators
        self.creation_op = creation_op
        self.grad = None
        self.autograd = autograd
        self.children = {}
        if(id is None):
            id = np.random.randint(0,100000)
        self.id = id

        if(creators is not None):
            for c in creators:
                if(self.id not in c.children):
                    c.children[self.id] = 1
                else:
                    c.children[self.id] += 1

    def all_children_grads_accounted_for(self):
        for id,cnt in self.children.items():
            if(cnt != 0):
                return False
        return True

    def backward(self, grad=None, grad_origin=None):
        if(self.autograd):
            if(grad_origin is not None):
                if(self.children[grad_origin.id] == 0):
                    raise Exception("cannot backprop more than once")
                else:
                    self.children[grad_origin.id] -= 1
```

Скорректировать число потомков  
данного тензора

Проверить, получил ли тензор  
градиенты от всех потомков

Проверка возможности  
обратного распространения  
или ожидания градиента,  
в последнем случае нужно  
уменьшить счетчик

```

if(self.grad is None):
    self.grad = grad
else:
    self.grad += grad

if(self.creators is not None and
    (self.all_children_grads_accounted_for() or
     grad_origin is None)):
    if(self.creation_op == "add"):
        self.creators[0].backward(self.grad, self)
        self.creators[1].backward(self.grad, self)

def __add__(self, other):
    if(self.autograd and other.autograd):
        return Tensor(self.data + other.data,
                       autograd=True,
                       creators=[self, other],
                       creation_op="add")
    return Tensor(self.data + other.data)

def __repr__(self):
    return str(self.data.__repr__())

def __str__(self):
    return str(self.data.__str__())

a = Tensor([1,2,3,4,5], autograd=True)
b = Tensor([2,2,2,2,2], autograd=True)
c = Tensor([5,4,3,2,1], autograd=True)

d = a + b
e = b + c
f = d + e

f.backward(Tensor(np.array([1,1,1,1,1])))

print(b.grad.data == np.array([2,2,2,2,2]))

[ True True True True True]

```

Накопление градиентов  
от нескольких потомков

Фактическое  
начало обратного  
распространения

Дополнительно мы создали счетчик `self.children`, подсчитывающий количество градиентов, полученных от каждого потомка в процессе обратного распространения. Так мы исключаем возможность случайного повторного обратного распространения от одного и того же потомка (в этом случае возбуждается исключение).

Второе новшество — функция с говорящим именем `all_children_grads_accounted_for()`. Эта функция проверяет, были ли получены градиенты от всех потомков в графе. Обычно, когда метод `.backward()` вызывается промежуточной переменной в графе, она тут же вызывает метод `.backward()` своего



родителя. Но так как некоторые переменные получают свои градиенты от нескольких родителей, каждая должна отложить вызов `.backward()` своего родителя, пока не получит свой окончательный градиент.

Как отмечалось выше, ничего из этого не является чем-то новым с точки зрения теории глубокого обучения — все это типичные инженерные сложности, с которыми приходится сталкиваться при создании фреймворков глубокого обучения. Более того, это те же сложности, с которыми вы будете сталкиваться при отладке нейронных сетей даже с использованием стандартных фреймворков. Прежде чем двинуться дальше, изучите этот код и поэкспериментируйте с ним, чтобы лучше понимать, как он работает. Попробуйте убирать разные фрагменты и посмотрите, к каким последствиям это будет приводить. Попробуйте дважды вызвать `.backprop()`.

## Как работает сложение в обратном распространении?

Давайте на примере абстракции посмотрим, как добавить поддержку дополнительных функций создания тензоров

Мы достигли самого интересного этапа в разработке фреймворка. Теперь мы можем добавить поддержку произвольных операций, реализовав соответствующие функции в классе `Tensor` и вычисление их производных в методе `.backward()`. Операцию сложения в нашем фреймворке выполняет следующий метод:

```
def __add__(self, other):
    if(self.autograd and other.autograd):
        return Tensor(self.data + other.data,
                      autograd=True,
                      creators=[self, other],
                      creation_op="add")
    return Tensor(self.data + other.data)
```

А так выполняется обратное распространение градиента через функцию сложения в методе `.backward()`:

```
if(self.creation_op == "add"):
    self.creators[0].backward(self.grad, self)
    self.creators[1].backward(self.grad, self)
```

Обратите внимание, что сложение нигде в классе не обрабатывается. Логика обратного распространения абстрагирована, поэтому все необходимое для

поддержки сложения определяется в этих двух местах. Также обратите внимание, что логика обратного распространения вызывает `.backward()` дважды, по одному разу для каждой переменной, участвовавшей в сложении. То есть логика по умолчанию обратного распространения заключается в обратном распространении через каждую переменную в графе. Но иногда обратное распространение требуется пропустить, например, если переменная исключена из вычисления градиента (`self.autograd == False`). Эта проверка выполняется в методе `.backward()`:

```
def backward(self, grad=None, grad_origin=None):
    if(self.autograd):
        if(grad_origin is not None):
            if(self.children[grad_origin.id] == 0):
                raise Exception("cannot backprop more than once")
        ...
```

Несмотря на то что логика обратного распространения для операции сложения применяется ко всем переменным, внесшим свой вклад, обратное распространение не будет выполнено, если не записать значение `True` в свойство `.autograd` переменной (`self.creators[0]` или `self.creators[1]` соответственно). Также обратите внимание, что тензор, создаваемый методом `__add__()` (и для которого потом будет вызываться метод `backward()`), получит `self.autograd == True`, только если `self.autograd == other.autograd == True`.

## Добавление поддержки отрицания

Возьмем за основу поддержку сложения и реализуем поддержку отрицания

Теперь, взяв за основу поддержку операции сложения, можно скопировать несколько строк из нее, внести небольшие изменения и добавить в автоматическое вычисление градиентов поддержку операции отрицания. Попробуем сделать это. Изменения в функции `__add__` выделены **жирным**:

```
def __neg__(self):
    if(self.autograd):
        return Tensor(self.data * -1,
                       autograd=True,
                       creators=[self],
                       creation_op="neg")
    return Tensor(self.data * -1)
```

Большая часть кода не изменилась. Операции отрицания не нужны дополнительные параметры, поэтому мы убрали параметр «other» в нескольких местах. Теперь рассмотрим логику, которую нужно добавить в метод `.backward()`. Код, отличающийся от поддержки операции сложения в логике обратного распространения, выделен **жирным**:

```
if(self.creation_op == "neg"):
    self.creators[0].backward(self.grad.__neg__())
```

Поскольку в операции отрицания `__neg__` участвует только один родитель, метод `.backward()` должен вызываться только один раз. (Если вы забыли, как правильно вычислять градиенты для обратного распространения, прочитайте еще раз главы 4, 5 и 6.) Теперь проверим новый код:

```
a = Tensor([1,2,3,4,5], autograd=True)
b = Tensor([2,2,2,2,2], autograd=True)
c = Tensor([5,4,3,2,1], autograd=True)

d = a + (-b)
e = (-b) + c
f = d + e

f.backward(Tensor(np.array([1,1,1,1,1])))

print(b.grad.data == np.array([-2,-2,-2,-2,-2]))

[ True True True True True]
```

Когда в прямом распространении используется `-b` вместо `b`, градиенты в обратном распространении также должны менять знак. Кроме того, для этого не нужно ничего менять в системе обратного распространения в целом. Мы можем добавлять новые операции по мере необходимости. Сделаем это прямо сейчас!

## Добавление поддержки других операций

Вычитание, умножение, суммирование, расширение, транспонирование и матричное умножение

Действуя по аналогии со сложением и отрицанием, добавим в логику прямого и обратного распространения поддержку еще нескольких операций:

```
def __sub__(self, other):
    if(self.autograd and other.autograd):
        return Tensor(self.data - other.data,
```

```
        autograd=True,
        creators=[self,other],
        creation_op="sub")
    return Tensor(self.data - other.data)

def __mul__(self, other):
    if(self.autograd and other.autograd):
        return Tensor(self.data * other.data,
            autograd=True,
            creators=[self,other],
            creation_op="mul")
    return Tensor(self.data * other.data)

def sum(self, dim):
    if(self.autograd):
        return Tensor(self.data.sum(dim),
            autograd=True,
            creators=[self],
            creation_op="sum_"+str(dim))
    return Tensor(self.data.sum(dim))

def expand(self, dim,copies):

    trans_cmd = list(range(0,len(self.data.shape)))
    trans_cmd.insert(dim,len(self.data.shape))
    new_shape = list(self.data.shape) + [copies]
    new_data = self.data.repeat(copies).reshape(new_shape)
    new_data = new_data.transpose(trans_cmd)

    if(self.autograd):
        return Tensor(new_data,
            autograd=True,
            creators=[self],
            creation_op="expand_"+str(dim))
    return Tensor(new_data)

def transpose(self):
    if(self.autograd):
        return Tensor(self.data.transpose(),
            autograd=True,
            creators=[self],
            creation_op="transpose")
    return Tensor(self.data.transpose())

def mm(self, x):
    if(self.autograd):
        return Tensor(self.data.dot(x.data),
            autograd=True,
            creators=[self,x],
            creation_op="mm")
    return Tensor(self.data.dot(x.data))
```

Выше мы обсудили производные для всех этих функций, однако `sum` и `expand` могут показаться незнакомыми, потому что они получили новые имена. Функция `sum` выполняет сложение элементов тензора по измерениям; иными словами, для матрицы  $2 \times 3$  с именем `x`:

```
x = Tensor(np.array([[1,2,3],
                     [4,5,6]]))
```

функция `.sum(dim)` выполнит суммирование ее элементов по измерениям: `x.sum(0)` вернет матрицу  $1 \times 3$  (вектор с тремя элементами), а `x.sum(1)` вернет матрицу  $2 \times 1$  (вектор с двумя элементами):

```
x.sum(0) → array([5, 7, 9]) x.sum(1) → array([ 6, 15])
```

Функция `expand` используется для обратного распространения операции `.sum()`. Она копирует данные по измерению. Для той же матрицы `x` копирование по первому измерению даст две копии тензора:

```
array([[[[1, 2, 3],
          [4, 5, 6]],
        [[1, 2, 3],
          [4, 5, 6]],
        [[1, 2, 3],
          [4, 5, 6]],
        [[1, 2, 3],
          [4, 5, 6]]]])
x.expand(dim=0, copies=4) ----->
```

То есть если `.sum()` удаляет размерность (матрицу  $2 \times 3$  превращает в вектор длиной 2 или 3), то `expand` добавляет измерение. Матрица  $2 \times 3$  превращается в матрицу  $4 \times 2 \times 3$ . Ее можно считать списком из четырех тензоров размером  $2 \times 3$  каждый. Но если выполнить расширение по последнему измерению, скопировано будет только последнее измерение, то есть каждый элемент оригинального тензора превратится в список:

```
array([[[[1, 1, 1, 1],
          [2, 2, 2, 2],
          [3, 3, 3, 3]],
        [[4, 4, 4, 4],
          [5, 5, 5, 5],
          [6, 6, 6, 6]]]])
x.expand(dim=2, copies=4) ----->
```

Таким образом, если новый тензор создается применением операции `.sum(dim=1)` к тензору с четырьмя элементами в этом измерении, тогда на

этапе обратного распространения к градиенту нужно применить операцию `.expand(dim=1, copies=4)`.

Теперь можно добавить соответствующую логику в метод `.backward()`:

```
if(self.creation_op == "sub"):
    new = Tensor(self.grad.data)
    self.creators[0].backward(new, self)
    new = Tensor(self.grad.__neg__().data)
    self.creators[1].backward(, self)

if(self.creation_op == "mul"):
    new = self.grad * self.creators[1]
    self.creators[0].backward(new, self)
    new = self.grad * self.creators[0]
    self.creators[1].backward(new, self)

if(self.creation_op == "mm"):
    act = self.creators[0]  ← Обычно слой активации
    weights = self.creators[1]  ← Обычно весовая матрица
    new = self.grad.mm(weights.transpose())
    act.backward(new)
    new = self.grad.transpose().mm(act).transpose()
    weights.backward(new)

if(self.creation_op == "transpose"):
    self.creators[0].backward(self.grad.transpose())

if("sum" in self.creation_op):
    dim = int(self.creation_op.split("_")[1])
    ds = self.creators[0].data.shape[dim]
    self.creators[0].backward(self.grad.expand(dim,ds))

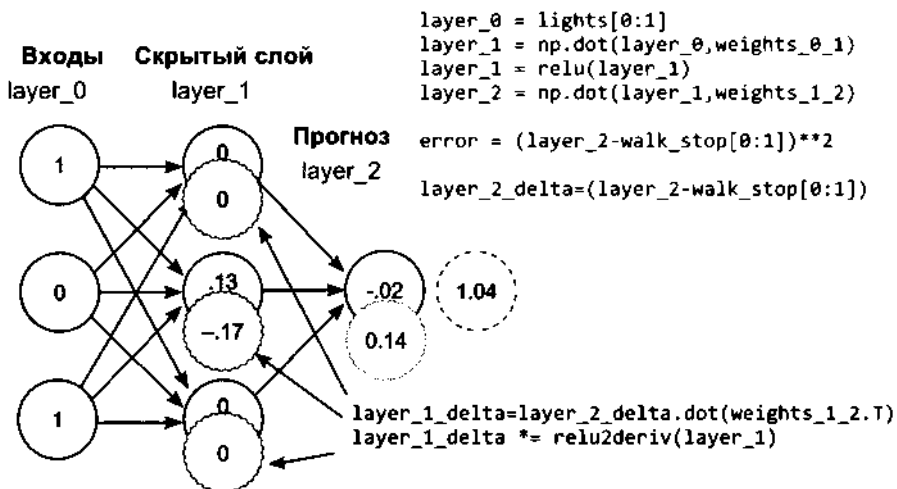
if("expand" in self.creation_op):
    dim = int(self.creation_op.split("_")[1])
    self.creators[0].backward(self.grad.sum(dim))
```

Если эта реализация вызывает у вас сомнения, вернитесь к главе 6 и посмотрите еще раз, как мы реализовали обратное распространение там. В той главе приводятся рисунки, отражающие каждый шаг обратного распространения, часть из которых я повторно привел здесь.

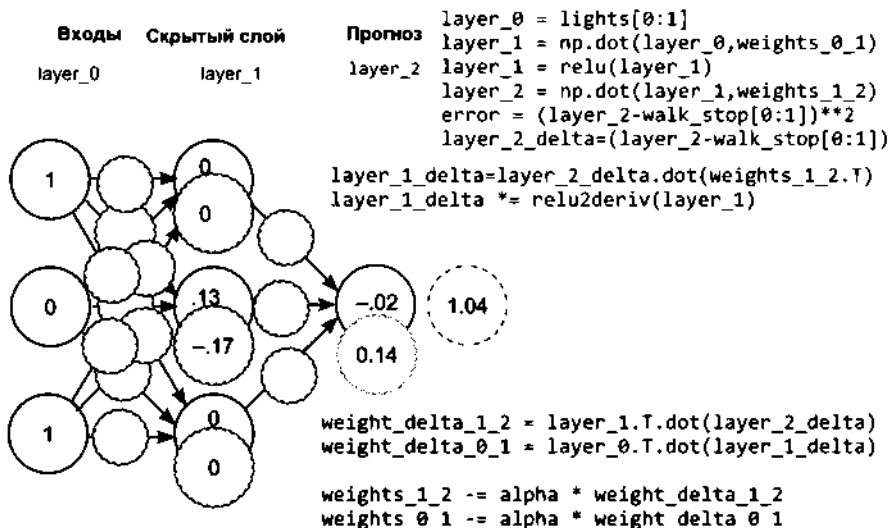
В конце сети вычисляются градиенты, после чего сигнал ошибки начинает *распространяться через сеть в обратном направлении* вызовом функций, соответствующих функциям, которые использовались для *прямого распространения* активаций. Если последней была операция умножения матриц (а это так и есть), в обратном распространении производится умножение матрицы (`dot`) на транспонированную матрицу.

На следующем рисунке это происходит в строке `layer_1_delta=layer_2_delta.dot(weights_1_2.T)`. В предыдущем коде это происходит в инструкции `if(self.creation_op == "mm")` (соответствующие строки выделены жирным). Здесь мы выполняем те же операции, что и прежде (в порядке, обратном порядку прямого распространения), но код организован гораздо лучше.

### 3. ОБУЧЕНИЕ: обратное распространение из layer\_2 в layer\_1



### 4. ОБУЧЕНИЕ: вычисление приращений и корректировка весов



## Использование autograd в обучении нейронной сети

Нам больше не нужно писать логику обратного распространения!

Все, что мы проделали выше, может показаться довольно сложной инженерной задачей, но наши усилия быстро окупятся. Теперь, когда понадобится обучить нейронную сеть, нам не придется писать логику обратного распространения! Для сравнения вот пример нейронной сети с обратным распространением, выполняемым вручную:

```
import numpy
np.random.seed(0)

data = np.array([[0,0],[0,1],[1,0],[1,1]])
target = np.array([[0],[1],[0],[1]])

weights_0_1 = np.random.rand(2,3)
weights_1_2 = np.random.rand(3,1)

for i in range(10):

    layer_1 = data.dot(weights_0_1)  ← Прогноз
    layer_2 = layer_1.dot(weights_1_2)

    diff = (layer_2 - target)  ← Сравнение
    sqdiff = (diff * diff)
    loss = sqdiff.sum()  ← Потеря как среднеквадратическая ошибка

    layer_1_grad = diff.dot(weights_1_2.transpose())  ←
    weight_1_2_update = layer_1.transpose().dot(diff)  ← Обучение; эта строка
    weight_0_1_update = data.transpose().dot(layer_1_grad)  ← реализует обратное
                                                                распространение

    weights_1_2 -= weight_1_2_update * 0.1
    weights_0_1 -= weight_0_1_update * 0.1
    print(loss[0])
```

0.4520108746468352  
0.33267400101121475  
0.25307308516725036  
0.1969566997160743  
0.15559900212801492  
0.12410658864910949  
0.09958132129923322  
0.08019781265417164  
0.06473333002675746  
0.05232281719234398



Мы должны так реализовать прямое распространение, чтобы `layer_1`, `layer_2` и `diff` существовали как переменные, потому что они потребуются позже. Затем мы должны распространить каждый градиент в обратном направлении до соответствующей матрицы весов и скорректировать веса.

```
import numpy
np.random.seed(0)

data = Tensor(np.array([[0,0],[0,1],[1,0],[1,1]]), autograd=True)
target = Tensor(np.array([[0],[1],[0],[1]]), autograd=True)

w = list()
w.append(Tensor(np.random.rand(2,3), autograd=True))
w.append(Tensor(np.random.rand(3,1), autograd=True))

for i in range(10):

    pred = data.mm(w[0]).mm(w[1])  ← Прогноз

    loss = ((pred - target)*(pred - target)).sum(0)  ← Сравнение

    loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение

    for w_ in w:
        w_.data -= w_.grad.data * 0.1
        w_.grad.data *= 0

    print(loss)
```

Но с новой системой автоматического вычисления градиента `autograd` код становится намного проще. Нам больше не нужны временные переменные (потому что вся необходимая информация сохраняется в динамическом вычислительном графе) и нет необходимости вручную определять логику обратного распространения (потому что она уже реализована в методе `.backward()`). Наш фреймворк не только удобен в обращении, но также снижает вероятность появления ошибок в коде обратного распространения!

```
[0.58128304]
[0.48988149]
[0.41375111]
[0.34489412]
[0.28210124]
[0.2254484]
[0.17538853]
[0.1324231]
[0.09682769]
[0.06849361]
```

Прежде чем двинуться дальше, хочу особо отметить стиль этой новой реализации. Обратите внимание, что все параметры я поместил в список, который можно перебирать при корректировке весов. Это превосходит следующую далее возможность. При использовании системы autograd реализация градиентного спуска сводится к тривиальному циклу `for` (который вы можете видеть в конце). Давайте реализуем механизм градиентного спуска в виде отдельного класса.

## Добавление автоматической оптимизации

### Реализуем оптимизатор стохастического градиентного спуска

На первый взгляд создание оптимизатора стохастического градиентного спуска может показаться сложной задачей, но в действительности для этого нам понадобится всего лишь скопировать некоторые фрагменты из предыдущего примера и добавить кое-какой код, написанный в старом добром объектно-ориентированном стиле:

```
class SGD(object):

    def __init__(self, parameters, alpha=0.1):
        self.parameters = parameters
        self.alpha = alpha

    def zero(self):
        for p in self.parameters:
            p.grad.data *= 0

    def step(self, zero=True):

        for p in self.parameters:

            p.data -= p.grad.data * self.alpha

            if(zero):
                p.grad.data *= 0
```

Благодаря этому предыдущая нейронная сеть упростится еще больше, как показано ниже, но результаты ее работы от этого не изменятся:

```
import numpy
np.random.seed(0)

data = Tensor(np.array([[0,0],[0,1],[1,0],[1,1]]), autograd=True)
target = Tensor(np.array([[0],[1],[0],[1]]), autograd=True)

w = list()
```

```

w.append(Tensor(np.random.rand(2,3), autograd=True))
w.append(Tensor(np.random.rand(3,1), autograd=True))

optim = SGD(parameters=w, alpha=0.1)

for i in range(10):

    pred = data.mm(w[0]).mm(w[1])  ← Прогноз

    loss = ((pred - target)*(pred - target)).sum(0)  ← Сравнение

    loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение
    optim.step()

```

## Добавление поддержки слоев разных типов

### Знакомые типы слоев в Keras или PyTorch

На данный момент мы реализовали самые сложные части нашего нового фреймворка глубокого обучения. Дальнейшие наши действия будут связаны в основном с добавлением новых методов в класс `Tensor` и созданием вспомогательных классов и функций. Вероятно, самая распространенная абстракция в большинстве фреймворков — абстракция слоя. Это коллекция процедур, часто используемых в прямом распространении, упакованных в простой программный интерфейс с неким методом `.forward()` для их использования. Вот пример простого линейного слоя:

```

class Layer(object):

    def __init__(self):
        self.parameters = list()

    def get_parameters(self):
        return self.parameters

class Linear(Layer):

    def __init__(self, n_inputs, n_outputs):
        super().__init__()
        W = np.random.randn(n_inputs, n_outputs)*np.sqrt(2.0/(n_inputs))
        self.weight = Tensor(W, autograd=True)
        self.bias = Tensor(np.zeros(n_outputs), autograd=True)

        self.parameters.append(self.weight)
        self.parameters.append(self.bias)

    def forward(self, input):
        return input.mm(self.weight)+self.bias.expand(0,len(input.data))

```

Здесь нет ничего особенно нового. Весовые коэффициенты организованы в класс (в который я также добавил матрицу смещений весов, потому что это настоящий линейный слой). Слой инициализируется как единое целое, благодаря чему матрицы весов и смещений получают правильные размеры и всегда используется правильная логика прямого распространения.

Отметьте также, что я определил абстрактный класс `Layer`, имеющий единственный метод чтения (`getter`). Его можно использовать для определения более сложных типов слоев (например, слоев, содержащих другие слои). Вам останется только переопределить метод `get_parameters()`, чтобы организовать передачу нужных тензоров в оптимизатор (такой, как класс `SGD`, созданный в предыдущем разделе).

## Слои, содержащие другие слои

### Слои могут также содержать другие слои

Наибольшей популярностью в практике глубокого обучения пользуются последовательные слои, которые осуществляют прямое распространение через список слоев, когда выход одного слоя передается на вход следующего:

```
class Sequential(Layer):
    def __init__(self, layers=list()):
        super().__init__()

        self.layers = layers

    def add(self, layer):
        self.layers.append(layer)

    def forward(self, input):
        for layer in self.layers:
            input = layer.forward(input)
        return input

    def get_parameters(self):
        params = list()
        for l in self.layers:
            params += l.get_parameters()
        return params

data = Tensor(np.array([[0,0],[0,1],[1,0],[1,1]]), autograd=True)
target = Tensor(np.array([[0],[1],[0],[1]]), autograd=True)

model = Sequential([Linear(2,3), Linear(3,1)])
```

```

optim = SGD(parameters=model.get_parameters(), alpha=0.05)

for i in range(10):

    pred = model.forward(data)  ← Прогноз

    loss = ((pred - target)*(pred - target)).sum(0)  ← Сравнение

    loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение
    optim.step()
    print(loss)

```

## Слои с функцией потерь

### Некоторые слои не имеют весов

Также можно определить слои, выход которых является функцией от входов. Наиболее популярной, пожалуй, разновидностью таких слоев являются слои с функцией потерь, такой как среднеквадратическая ошибка:

```

class MSELoss(Layer):

    def __init__(self):
        super().__init__()

    def forward(self, pred, target):
        return ((pred - target)*(pred - target)).sum(0)

import numpy
np.random.seed(0)

data = Tensor(np.array([[0,0],[0,1],[1,0],[1,1]]), autograd=True)
target = Tensor(np.array([[0],[1],[0],[1]]), autograd=True)

model = Sequential([Linear(2,3), Linear(3,1)])
criterion = MSELoss()

optim = SGD(parameters=model.get_parameters(), alpha=0.05)

for i in range(10):

    pred = model.forward(data)  ← Прогноз

    loss = criterion.forward(pred, target)  ← Сравнение

    loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение
    optim.step()
    print(loss)

```

[2.33428272]

```
[0.06743796]
...
[0.01153118]
[0.00889602]
```

Отмечу еще раз, что здесь по сути нет ничего нового. Последние несколько примеров реализуют одни и те же вычисления. Просто наша реализация `autograd` автоматически осуществляет обратное распространение, а этапы прямого распространения упакованы в удобные классы, обеспечивающие выполнение всех необходимых действий в правильном порядке.

## Как научиться пользоваться фреймворком

Упрощенно, любой фреймворк — это `autograd` + список предварительно созданных слоев и оптимизаторов

Используя базовую систему автоматического вычисления градиентов `autograd`, которая позволяет объединять слои с произвольной функциональностью, можно (довольно быстро) написать множество новых типов слоев. По правде говоря, именно в этом заключается главное достоинство современных фреймворков — они избавляют от необходимости вручную определять каждую математическую операцию на этапах прямого и обратного распространения. Использование фреймворков значительно сокращает время перехода от идеи к экспериментам и уменьшает количество ошибок в вашем коде.

Рассматривая фреймворки как простую комбинацию системы `autograd` с длинным списком слоев и оптимизаторов, вы сможете быстро научиться пользоваться ими. Я верю, что, прочитав эту главу, вы сможете быстро освоить любой фреймворк, однако отмечу, что наибольшее сходство с представленным здесь API имеет фреймворк `PyTorch`. В любом случае, найдите время и ознакомьтесь со списками слоев и оптимизаторов в следующих крупных фреймворках:

- ❑ <https://pytorch.org/docs/stable/nn.html>
- ❑ <https://keras.io/layers/about-keras-layers>
- ❑ [https://www.tensorflow.org/api\\_docs/python/tf/layers](https://www.tensorflow.org/api_docs/python/tf/layers)

Как правило, чтобы освоить новый фреймворк, нужно найти простейший пример кода, исследовать его и познакомиться с программным интерфейсом системы автоматического вычисления градиентов, а затем, меняя код, провести интересные для вас эксперименты.

И еще, прежде чем продолжить, я хочу добавить в `Tensor.backward()` поддержку дополнительной операции, которая позволит не передавать градиент из единиц в первый вызов `.backward()`. Строго говоря, в этом нет большой необходимости, но это очень удобно:

```
def backward(self, grad=None, grad_origin=None):
    if(self.autograd):

        if(grad is None):
            grad = Tensor(np.ones_like(self.data))
```

## Нелинейные слои

Добавим в `Tensor` нелинейные функции активации и затем определим несколько новых типов слоев

В следующей главе нам понадобятся функции `.sigmoid()` и `.tanh()`, поэтому добавим их в класс `Tensor`. Вы уже достаточно давно знакомы с производными обеих этих функций, поэтому легко разберетесь в следующем коде:

```
def sigmoid(self):
    if(self.autograd):
        return Tensor(1 / (1 + np.exp(-self.data)),
                      autograd=True,
                      creators=[self],
                      creation_op="sigmoid")
    return Tensor(1 / (1 + np.exp(-self.data)))

def tanh(self):
    if(self.autograd):
        return Tensor(np.tanh(self.data),
                      autograd=True,
                      creators=[self],
                      creation_op="tanh")
    return Tensor(np.tanh(self.data))
```

Ниже приводится логика обратного распространения, которую нужно добавить в метод `Tensor.backward()`:

```
if(self.creation_op == "sigmoid"):
    ones = Tensor(np.ones_like(self.grad.data))
    self.creators[0].backward(self.grad * (self * (ones - self)))

if(self.creation_op == "tanh"):
    ones = Tensor(np.ones_like(self.grad.data))
    self.creators[0].backward(self.grad * (ones - (self * self)))
```

Надеюсь, этот код не показался вам необычным. Попробуйте сами добавить еще пару нелинейных функций, например `HardTanh` и `relu`.

```
class Tanh(Layer):
    def __init__(self):
        super().__init__()

    def forward(self, input):
        return input.tanh()

class Sigmoid(Layer):
    def __init__(self):
        super().__init__()

    def forward(self, input):
        return input.sigmoid()
```

Опробуем наши новые функции. Новый код выделен **жирным**:

```
import numpy
np.random.seed(0)

data = Tensor(np.array([[0,0],[0,1],[1,0],[1,1]]), autograd=True)
target = Tensor(np.array([[0],[1],[0],[1]]), autograd=True)

model = Sequential([Linear(2,3), Tanh(), Linear(3,1), Sigmoid()])
criterion = MSELoss()

optim = SGD(parameters=model.get_parameters(), alpha=1)

for i in range(10):

    pred = model.forward(data)  ← Прогноз

    loss = criterion.forward(pred, target)  ← Сравнение

    loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение
    optim.step()
    print(loss)

[1.06372865]
[0.75148144]
[0.57384259]
[0.39574294]
[0.2482279]
[0.15515294]
[0.10423398]
[0.07571169]
[0.05837623]
[0.04700013]
```

Как видите, достаточно поместить новые слои `Tanh()` и `Sigmoid()` во входные параметры `Sequential()`, и нейронная сеть точно будет знать, как их использовать. Очень просто!

В предыдущей главе мы познакомились с рекуррентными нейронными сетями. В частности, мы обучили модель предсказывать следующее слово по несколь-



ким предшествующим словам. Прежде чем закончить эту главу, я хочу, чтобы вы попробовали перенести этот код на новую основу. Для этого понадобится создать три новых типа слоев: слой векторного представления, который изучает векторные представления слов, рекуррентный слой, который может обучаться моделированию входных последовательностей, и слой `softmax`, способный предсказать вероятности меток.

## Слой с векторным представлением

### Слой векторного представления преобразует индексы в функции активации

В главе 11 мы познакомились с векторными представлениями слов — векторами, представляющими слова, которые можно передать на вход нейронной сети. Для словаря с 200 словами будет иметься 200 векторных представлений. Это правило задает организацию слоя с векторным представлением. Во-первых, слой должен инициализировать список (правильной длины) векторных представлений слов (правильного размера):

```
class Embedding(Layer):
```

```
    def __init__(self, vocab_size, dim):
        super().__init__()
```

```
        self.vocab_size = vocab_size
        self.dim = dim
```

```
        weight = np.random.rand(vocab_size, dim) * 0.5 / dim
```

Такой способ инициализации  
соответствует соглашениям  
для алгоритма word2vec

Пока все идет хорошо. Матрица содержит по одной строке (вектору) для каждого слова из словаря. Как теперь начать прямое распространение? Прямое распространение всегда начинается с вопроса: как будут кодироваться входные данные? В случае с векторными представлениями слов мы не можем передать сами слова, потому что слова не позволяют определить, какие строки из матрицы `self.weight` должны участвовать в прямом распространении. Вместо слов, как вы наверняка помните из главы 11, мы должны передать индексы слов. К счастью, NumPy поддерживает эту операцию:

```
identity = np.eye(5)
print(identity)
```

```
----->
```

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

```

print(identity[np.array([[1,2,3,4],
                        [2,3,4,0]])])

```

```

[[[0. 1. 0. 0. 0.]
  [0. 0. 1. 0. 0.]
  [0. 0. 0. 1. 0.]
  [0. 0. 0. 0. 1.]]

 [[0. 0. 1. 0. 0.]
  [0. 0. 0. 1. 0.]
  [0. 0. 0. 0. 1.]
  [1. 0. 0. 0. 0.]]

```

Обратите внимание, что если передать библиотеке NumPy матрицу с целыми числами, она вернет ту же матрицу, но заменит каждое число в исходной матрице соответствующей строкой. Так, двумерная матрица индексов превратится в трехмерную матрицу векторных представлений (строк). Это потрясающе!

## Добавление индексирования в autograd

Прежде чем создать слой с векторным представлением, нужно добавить в autograd поддержку индексирования

Для поддержки новой стратегии векторного представления (предполагающей передачу слов в виде матрицы индексов) индексирование, с которым мы познакомились в предыдущем разделе, должно поддерживаться системой autograd. Это довольно простая идея. Мы должны гарантировать на этапе обратного распространения размещение градиентов в тех же строках, полученных в результате индексирования на этапе прямого распространения. Для этого нужно сохранить исходные индексы, чтобы на этапе обратного распространения мы смогли поместить каждый градиент в нужное место, воспользовавшись простым циклом for:

```

def index_select(self, indices):

    if(self.autograd):
        new = Tensor(self.data[indices.data],
                      autograd=True,
                      creators=[self],
                      creation_op="index_select")
        new.index_select_indices = indices
        return new
    return Tensor(self.data[indices.data])

```

Во-первых, используем трюк, с которым мы познакомились в предыдущем разделе, для выбора правильных строк:

```

if(self.creation_op == "index_select"):
    new_grad = np.zeros_like(self.creators[0].data)
    indices_ = self.index_select_indices.data.flatten()
    grad_ = grad.data.reshape(len(indices_), -1)
    for i in range(len(indices_)):
        new_grad[indices_[i]] += grad_[i]
    self.creators[0].backward(Tensor(new_grad))

```

Затем в `backward()` инициализируем новый градиент правильного размера (соответствующего размеру исходной матрицы, подвергавшейся индексированию). Во-вторых, преобразуем индексы в плоский вектор, чтобы получить возможность выполнить итерации по ним. В-третьих, свернем `grad_` в простой список строк. (Важно отметить, что при этом индексы в `indices_` и список векторов в `grad_` будут иметь соответствующий порядок.) Затем выполним обход всех индексов, добавим их в правильные строки в новой матрице градиентов и передадим ее обратно в `self.creators[0]`. Как показано ниже, каждая строка в `grad_[i]` правильно обновляется (в данном случае добавляется вектор из единиц), в соответствии с тем, сколько раз использовался данный индекс. Индексы 2 и 3 обновились дважды (выделены **жирным**):

```

x = Tensor(np.eye(5), autograd=True)
x.index_select(Tensor([[1,2,3],
                       [2,3,4]])).backward()
print(x.grad)

```

```

[[0. 0. 0. 0. 0.]
 [1. 1. 1. 1. 1.]
 [2. 2. 2. 2. 2.]
 [2. 2. 2. 2. 2.]
 [1. 1. 1. 1. 1.]]

```

## Слой с векторным представлением (повтор)

Теперь можно выполнить прямое распространение, вызвав новый метод `.index_select()`

Чтобы выполнить прямое распространение, вызовем `.index_select()`, а об остальном позаботится механизм `autograd`:

```
class Embedding(Layer):
```

```
    def __init__(self, vocab_size, dim):
        super().__init__()
```

```
        self.vocab_size = vocab_size
        self.dim = dim
```

```
        weight = np.random.rand(vocab_size, dim) - 0.5 / dim
        self.weight = Tensor((weight, autograd=True))
```

Такой способ инициализации  
соответствует соглашениям  
для алгоритма word2vec

```

self.parameters.append(self.weight)

def forward(self, input):
    return self.weight.index_select(input)

data = Tensor(np.array([1,2,1,2]), autograd=True)
target = Tensor(np.array([[0],[1],[0],[1]]), autograd=True)

embed = Embedding(5,3)
model = Sequential([embed, Tanh(), Linear(3,1), Sigmoid()])
criterion = MSELoss()

optim = SGD(parameters=model.get_parameters(), alpha=0.5)

for i in range(10):

    pred = model.forward(data)  ← Прогноз

    loss = criterion.forward(pred, target)  ← Сравнение

    loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение
    optim.step()
    print(loss)
[0.98874126]
[0.6658868]
[0.45639889]
...
[0.08731868]
[0.07387834]

```

Реализовав эту нейронную сеть, мы выявили корреляцию между входными индексами 1 и 2 и прогнозами 0 и 1. Теоретически индексы 1 и 2 могут соответствовать словам (или некоторым другим входным объектам), и в заключительном примере мы выявим это соответствие. Этот пример должен был показать, как действует слой с векторным представлением.

## Слой с перекрестной энтропией

Добавим перекрестную энтропию в `autograd` и создадим слой

Надеюсь, что к данному моменту вы получили более или менее полное представление о том, как создаются новые типы слоев. Перекрестная энтропия — стандартный механизм, и мы много раз встречались с ним в этой книге. Так как мы уже видели примеры создания нескольких новых типов слоев, я просто приведу код без комментариев. Прежде чем скопировать этот код, попробуйте сначала сами написать его.

```

def cross_entropy(self, target_indices):

    temp = np.exp(self.data)
    softmax_output = temp / np.sum(temp,
                                     axis=len(self.data.shape)-1,
                                     keepdims=True)

    t = target_indices.data.flatten()
    p = softmax_output.reshape(len(t),-1)
    target_dist = np.eye(p.shape[1])[t]
    loss = -(np.log(p) * (target_dist)).sum(1).mean()

    if(self.autograd):
        out = Tensor(loss,
                      autograd=True,
                      creators=[self],
                      creation_op="cross_entropy")
        out.softmax_output = softmax_output
        out.target_dist = target_dist
        return out

    return Tensor(loss)

    if(self.creation_op == "cross_entropy"):
        dx = self.softmax_output - self.target_dist
        self.creators[0].backward(Tensor(dx))

class CrossEntropyLoss(object):

    def __init__(self):
        super().__init__()

    def forward(self, input, target):
        return input.cross_entropy(target)

import numpy
np.random.seed(0)

# исходные индексы
data = Tensor(np.array([1,2,1,2]), autograd=True)

# целевые индексы
target = Tensor(np.array([0,1,0,1]), autograd=True)

model = Sequential([Embedding(3,3), Tanh(), Linear(3,4)])
criterion = CrossEntropyLoss()

optim = SGD(parameters=model.get_parameters(), alpha=0.1)

for i in range(10):

    pred = model.forward(data)  ← Прогноз

```

```

loss = criterion.forward(pred, target)  ← Сравнение

loss.backward(Tensor(np.ones_like(loss.data)))  ← Обучение
optim.step()
print(loss)

```

```

1.3885032434928422
0.9558181509266037
0.6823083585795604
0.5095259967493119
0.39574491472895856
0.31752527285348264
0.2617222861964216
0.22061283923954234
0.18946427334830068
0.16527389263866668

```

Добавив ту же логику перекрестной энтропии, что использовалась в нескольких предыдущих нейронных сетях, мы получили новую функцию потерь. Одна из отличительных черт этой реализации состоит в том, что теперь вычисление `softmax` и потеря производится в одном классе. Это широко распространенное соглашение в нейронных сетях. Почти все фреймворки работают именно так. Если вы пожелаете завершить разработку сети и обучить ее с использованием перекрестной энтропии, можете исключить `softmax` из этапа прямого распространения и вызвать класс перекрестной энтропии, который автоматически применит `softmax` как часть функции потерь.

Главной причиной такого объединения является производительность. Намного быстрее вместе вычислить градиент `softmax` и отрицательный логарифм подобия в функции перекрестной энтропии, чем распространять их вперед и назад по отдельности в двух разных модулях. Это один из приемов оптимизации при работе с градиентами.

## Рекуррентный слой

Объединив несколько слоев, можно выявлять закономерности во временных рядах

В качестве последнего упражнения в этой главе создадим еще один слой, состоящий из нескольких слоев других типов. Цель этого слоя — решить задачу, которую мы реализовали в конце предыдущей главы. Это будет *рекуррентный слой*. Мы сконструируем его из трех линейных слоев, а его метод `.forward()` будет принимать выходные данные предыдущего скрытого состояния и обучающие данные:

```

class RNNCell(Layer):

    def __init__(self, n_inputs, n_hidden, n_output, activation='sigmoid'):
        super().__init__()

        self.n_inputs = n_inputs
        self.n_hidden = n_hidden
        self.n_output = n_output

        if(activation == 'sigmoid'):
            self.activation = Sigmoid()
        elif(activation == 'tanh'):
            self.activation = Tanh()
        else:
            raise Exception("Non-linearity not found")

        self.w_ih = Linear(n_inputs, n_hidden)
        self.w_hh = Linear(n_hidden, n_hidden)
        self.w_ho = Linear(n_hidden, n_output)

        self.parameters += self.w_ih.get_parameters()
        self.parameters += self.w_hh.get_parameters()
        self.parameters += self.w_ho.get_parameters()

    def forward(self, input, hidden):
        from_prev_hidden = self.w_hh.forward(hidden)
        combined = self.w_ih.forward(input) + from_prev_hidden
        new_hidden = self.activation.forward(combined)
        output = self.w_ho.forward(new_hidden)
        return output, new_hidden

    def init_hidden(self, batch_size=1):
        return Tensor(np.zeros((batch_size, self.n_hidden)), autograd=True)

```

В этой главе я не буду повторно рассказывать о рекуррентных нейронных сетях, но хочу напомнить некоторые аспекты, которые должны быть вам знакомы. Рекуррентная нейронная сеть имеет вектор состояния, который передается из предыдущей итерации обучения в следующую. В данном случае это переменная `hidden`, которая одновременно является входным параметром и выходным значением функции `forward`. Рекуррентная нейронная сеть имеет также несколько разных весовых матриц: одна отображает входные векторы в скрытые (обработка входных данных), другая отображает скрытые векторы в скрытые векторы (коррекция каждого скрытого вектора на основе предыдущего состояния) и третья, необязательная, отображает скрытый слой в выходной, генерируя прогноз на основе скрытых векторов. Наша реализация `RNNCell` включает все три матрицы. Слой `self.w_ih` отображает входной слой в скрытый, `self.w_hh` — скрытый слой в скрытый и `self.w_ho` — скрытый слой в выходной. Обратите внимание на размерности всех трех матриц. Оба размера — `n_input` матрицы

`self.w_ih` и `n_output` матрицы `self.w_ho` — определяются размером словаря. Все остальные размеры определяются параметром `n_hidden`.

Наконец, входной параметр `activation` определяет нелинейную функцию для применения к скрытым векторам в каждой итерации. Я выбрал две (*Sigmoid* и *Tanh*), но вообще для выбора существует множество других вариантов. А теперь обучим сеть:

```
import sys, random, math
from collections import Counter
import numpy as np

f = open('tasksv11/en/qa1_single-supporting-fact_train.txt', 'r')
raw = f.readlines()
f.close()

tokens = list()
for line in raw[0:1000]:
    tokens.append(line.lower().replace("\n", "").split(" ")[1:])

new_tokens = list()
for line in tokens:
    new_tokens.append(['-'] * (6 - len(line)) + line)
tokens = new_tokens

vocab = set()
for sent in tokens:
    for word in sent:
        vocab.add(word)

vocab = list(vocab)

word2index = {}
for i, word in enumerate(vocab):
    word2index[word] = i

def words2indices(sentence):
    idx = list()
    for word in sentence:
        idx.append(word2index[word])
    return idx

indices = list()
for line in tokens:
    idx = list()
    for w in line:
        idx.append(word2index[w])
    indices.append(idx)

data = np.array(indices)
```



## Мы можем научить сеть решать задачу, которую уже решили в предыдущей главе

Теперь можно инициализировать рекуррентный слой входными векторными представлениями и обучить сеть решению той же задачи, что была решена в предыдущей главе. Обратите внимание, что эта сеть немного сложнее (она имеет один дополнительный слой), несмотря на то что код получился проще благодаря нашему маленькому фреймворку.

```
embed = Embedding(vocab_size=len(vocab),dim=16)
model = RNNCell(n_inputs=16, n_hidden=16, n_output=len(vocab))

criterion = CrossEntropyLoss()
params = model.get_parameters() + embed.get_parameters()
optim = SGD(parameters=params, alpha=0.05)
```

Здесь сначала определяются входные векторные представления, а затем инициализируется рекуррентная ячейка. (Обратите внимание, что единичный рекуррентный слой принято называть *ячейкой*. Если создать другой слой, объединяющий произвольное число ячеек, он будет называться рекуррентной нейронной сетью и принимать дополнительный входной параметр `n_layers`.)

```
for iter in range(1000):
    batch_size = 100
    total_loss = 0

    hidden = model.init_hidden(batch_size=batch_size)

    for t in range(5):
        input = Tensor(data[0:batch_size,t], autograd=True)
        rnn_input = embed.forward(input=input)
        output, hidden = model.forward(input=rnn_input, hidden=hidden)

    target = Tensor(data[0:batch_size,t+1], autograd=True)
    loss = criterion.forward(output, target)
    loss.backward()
    optim.step()
    total_loss += loss.data
    if(iter % 200 == 0):
        p_correct = (target.data == np.argmax(output.data,axis=1)).mean()
        print_loss = total_loss / (len(data)/batch_size)
        print("Loss:",print_loss,"% Correct:",p_correct)

Loss: 0.47631100976371393 % Correct: 0.01
Loss: 0.17189538896184856 % Correct: 0.28
Loss: 0.1460940222788725 % Correct: 0.37
Loss: 0.13845863915406884 % Correct: 0.37
Loss: 0.135574472565278 % Correct: 0.37
```

```

batch_size = 1
hidden = model.init_hidden(batch_size=batch_size)
for t in range(5):
    input = Tensor(data[0:batch_size,t], autograd=True)
    rnn_input = embed.forward(input=input)
    output, hidden = model.forward(input=rnn_input, hidden=hidden)

target = Tensor(data[0:batch_size,t+1], autograd=True)
loss = criterion.forward(output, target)

ctx = ""
for idx in data[0:batch_size][0][0:-1]:
    ctx += vocab[idx] + " "
print("Context:", ctx)
print("Pred:", vocab[output.data.argmax()])

Context: - mary moved to the
Pred: office.

```

Как видите, нейронная сеть, обученная на 100 первых примерах из обучающего набора данных, достигла точности прогнозирования около 37 % (почти идеально для этой задачи). Она предсказывает, куда может выйти Mary, почти так же, как сеть в конце главы 12.

## Итоги

### Фреймворки — эффективные и удобные абстракции логики прямого и обратного распространения

Надеюсь, что упражнения в этой главе помогли вам понять, насколько удобными могут быть фреймворки. Они дают возможность быстро писать легко читаемый код, который быстро выполняется (за счет встроенных оптимизаций), и допускать намного меньше ошибок. Что еще более важно, эта глава подготовила вас к использованию и расширению стандартных фреймворков, таких как PyTorch и TensorFlow. Независимо от того, отлаживаете ли вы существующие типы слоев или проектируете собственные, навыки, полученные здесь, являются, пожалуй, самыми важными из приобретенных вами в этой книге, потому что они связывают абстрактную теорию глубокого обучения из предыдущих глав с инструментами, которые вы будете использовать для реализации своих моделей в будущем.

Фреймворк, созданный здесь, больше всего напоминает PyTorch, и я настоятельно рекомендую заняться его изучением, когда вы закончите читать книгу.

# 14 Обучаем сеть писать как Шекспир: долгая краткосрочная память



## В этой главе

- ✓ Моделирование языка символов.
- ✓ Усеченное обратное распространение.
- ✓ Затухающие и взрывные градиенты.
- ✓ Упрощенный пример обратного распространения в RNN.
- ✓ Ячейки долгой краткосрочной памяти (LSTM).

Слабый разум смертным дан!<sup>1</sup>

Уильям Шекспир, «Сон в летнюю ночь»

<sup>1</sup> В переводе М. Лозинского.

## Моделирование языка символов

### Используем RNN для решения более сложной задачи

В конце глав 12 и 13 мы обучили простые рекуррентные нейронные сети (recurrent neural network, RNN) простой задаче прогнозирования последовательностей. Но брали для обучения очень маленький набор фраз, искусственно созданных с использованием некоторых правил.

В этой главе мы попробуем осуществить моделирование языка на более сложном наборе данных: на произведении Шекспира. Но вместо прогнозирования следующего слова по предыдущим (как в предшествующей главе) модель будет учиться предсказывать символы. Она будет учиться предсказывать следующий символ по предыдущим. Вот что я имею в виду:

```
import sys, random, math
from collections import Counter
import numpy as np
import sys

np.random.seed(0)

f = open('shakespeare.txt', 'r')
raw = f.read()  # Можно получить по адресу http://karpathy.github.io/2015/05/21/rnn-effectiveness/
f.close()

vocab = list(set(raw))
word2index = {}
for i, word in enumerate(vocab):
    word2index[word] = i
indices = np.array(list(map(lambda x: word2index[x], raw)))
```

Если в главах 12 и 13 словарь состоял из слов, встречающихся в наборе данных, то теперь словарь состоит из символов. Поэтому набор данных точно так же преобразуется в список индексов, но только соответствующих символам, а не словам. В коде выше — NumPy-массив `indices`:

```
embed = Embedding(vocab_size=len(vocab), dim=512)
model = RNNCell(n_inputs=512, n_hidden=512, n_output=len(vocab))

criterion = CrossEntropyLoss()
optim = SGD(parameters=model.get_parameters() + embed.get_parameters(),
            alpha=0.05)
```

Этот код должен выглядеть знакомым. Он инициализирует векторные представления с размерностью 8 и рекуррентную нейронную сеть с размерностью

скрытого состояния 512. Выходные веса инициализируются нулями (это не является обязательным требованием, но я обнаружил, что в этом случае сеть работает немного лучше). В конце инициализируется оптимизатор стохастического градиентного спуска с перекрестной энтропией в качестве функции потерь.

## Необходимо усеченное обратное распространение

Выполнять обратное распространение через 100 000 символов очень непрактично

Один из самых сложных аспектов при чтении кода RNN — анализ логики объединения входных данных в пакеты. В предыдущей (более простой) нейронной сети для этого использовался вложенный цикл `for`, как показано ниже (выделен **жирным**):

```
for iter in range(1000):
    batch_size = 100
    total_loss = 0

    hidden = model.init_hidden(batch_size=batch_size)

    for t in range(5):
        input = Tensor(data[0:batch_size,t], autograd=True)
        rnn_input = embed.forward(input=input)
        output, hidden = model.forward(input=rnn_input, hidden=hidden)

    target = Tensor(data[0:batch_size,t+1], autograd=True)
    loss = criterion.forward(output, target)
    loss.backward()
    optim.step()
    total_loss += loss.data
    if(iter % 200 == 0):
        p_correct = (target.data == np.argmax(output.data,axis=1)).mean()
        print_loss = total_loss / (len(data)/batch_size)
        print("Loss:",print_loss,"% Correct:",p_correct)
```

Вы можете спросить: «Почему пять итераций?» Дело в том, что в предыдущем наборе данных не было ни одного предложения длиннее шести слов. Поэтому сеть читала пять первых слов и пыталась предсказать шестое. Еще более важным является этап обратного распространения. Вспомним пример простой сети прямого распространения для классификации изображений рукописных цифр из набора MNIST: градиенты всегда распространялись в обратном направлении через всю сеть, верно? Обратное распространение продолжалось вплоть до са-

мого начала. Это позволило сети скорректировать каждый вес, чтобы попытаться научиться давать правильный прогноз с учетом всего входного примера.

Обсуждаемая рекуррентная сеть ничем не отличается. Мы двигаем вперед пять входных примеров и затем, когда позднее происходит вызов `loss.backward()`, градиенты распространяются в обратном направлении через всю сеть до входных данных. Это было вполне осуществимо, потому что за один прием передавалось небольшое количество входных точек данных. Но в наборе *Shakespeare* содержится 100 000 символов! Это слишком много, чтобы выполнять обратное распространение для каждого прогноза. И как быть?

Не выполнять обратного распространения в полном объеме! Выполнить обратное распространение на определенное число шагов и остановиться. Этот прием называется *усеченным обратным распространением* и широко используется в практике глубокого обучения. Количество шагов обратного распространения в этом случае становится еще одним настраиваемым параметром (как размер пакета или альфа-коэффициент).

## Усеченное обратное распространение

Технически этот прием снижает теоретический максимум нейронной сети

Недостатком использования усеченного обратного распространения является сокращение расстояния, которое нейронная сеть может запомнить. Фактически, остановка обратного распространения градиентов, например, после пятого шага, означает, что нейронная сеть не сможет запомнить события, имевшие место более пяти шагов тому назад.

Строго говоря, этот отрицательный эффект несколько сложнее. В скрытом слое RNN по случайности может остаться информация из более отдаленного прошлого, чем в пяти шагах, но нейронная сеть не сможет использовать градиенты, чтобы потребовать от модели сохранить информацию за шесть последних шагов, чтобы помочь получить текущий прогноз. Нейронная сеть не сможет научиться делать прогнозы, опираясь на входной сигнал, отстоящий далее чем на пять шагов в прошлом (если обратное распространение ограничено пятью шагами). В практике лингвистического моделирования переменной, определяющей границу усечения, обычно дают имя `bptt` и присваивают значение от 16 до 64:

```
batch_size = 32
bptt = 16
n_batches = int((indices.shape[0] / (batch_size)))
```

Другой недостаток усеченного обратного распространения — усложнение логики формирования пакетов. Чтобы использовать усеченное обратное распространение, мы должны сделать вид, что у нас не один большой набор данных, а много маленьких наборов, размер каждого из которых равен `bptt`. То есть мы должны соответствующим образом сгруппировать наборы данных:

```
trimmed_indices = indices[:n_batches*batch_size]
batched_indices = trimmed_indices.reshape(batch_size, n_batches)
batched_indices = batched_indices.transpose()

input_batched_indices = batched_indices[0:-1]
target_batched_indices = batched_indices[1:]

n_bptt = int(((n_batches-1) / bptt))
input_batches = input_batched_indices[:n_bptt*bptt]
input_batches = input_batches.reshape(n_bptt,bptt,batch_size)
target_batches = target_batched_indices[:n_bptt*bptt]
target_batches = target_batches.reshape(n_bptt, bptt, batch_size)
```

Здесь выполняется множество важных операций. Верхняя строка усекает набор данных до размера, кратного произведению `batch_size` и `n_batches`. Это делается с целью привести его к прямоугольной форме перед группировкой в тензоры (также можно дополнить набор данных нулями). Вторая и третья строки изменяют форму набора данных так, чтобы каждый столбец представлял сегмент начального массива индексов. Я покажу это, как если бы переменная `batch_size` имела значение 8 (для удобочитаемости):

```
print(raw[0:5])
print(indices[0:5])
```

---

```
'That,'
array([ 9, 14, 2, 10, 57])
```

Это первые пять символов из набора Shakespeare. Они образуют строку «That,». Ниже приводятся первые пять строк в `batched_indices`, полученных в результате преобразования:

```
print(batched_indices[0:5])
```

---

```
array([[ 9, 43, 21, 10, 10, 23, 57, 46],
       [14, 44, 39, 21, 43, 14, 1, 10],
       [ 2, 41, 39, 54, 37, 21, 26, 57],
       [10, 39, 57, 48, 21, 54, 38, 43],
       [57, 39, 43, 1, 10, 21, 21, 33]])
```

Я выделил первый столбец жирным. Вы заметили, что индексы в первом столбце соответствуют фразе «That,»? Это стандартный способ организации данных. Число столбцов в этом примере соответствует значению 8 в переменной `batch_size`. Далее этот тензор используется для конструирования списка наборов данных, каждый размером `bptt`.

Здесь можно видеть, как конструируются входные и целевые данные. Обратите внимание, что целевые индексы — это входные индексы со смещением на одну строку (то есть сеть обучается предсказывать следующий символ). Отмечу еще раз, что для формирования этого листинга я присвоил переменной `batch_size` значение 8 для удобочитаемости, но в действительности она будет иметь значение 32.

```
print(input_batches[0][0:5])

print(target_batches[0][0:5])

array([[ 9, 43, 21, 10, 10, 23, 57, 46],
       [14, 44, 39, 21, 43, 14,  1, 10],
       [ 2, 41, 39, 54, 37, 21, 26, 57],
       [10, 39, 57, 48, 21, 54, 38, 43],
       [57, 39, 43,  1, 10, 21, 21, 33]])

array([[14, 44, 39, 21, 43, 14,  1, 10],
       [ 2, 41, 39, 54, 37, 21, 26, 57],
       [10, 39, 57, 48, 21, 54, 38, 43],
       [57, 39, 43,  1, 10, 21, 21, 33],
       [43, 43, 41, 60, 52, 12, 54,  1]])
```

Не волнуйтесь, если что-то осталось для вас непонятным. Это не имеет отношения к теории глубокого обучения; это лишь одна из сложностей настройки RNN, с которыми вы будете сталкиваться время от времени. Я просто решил потратить пару страниц на эти объяснения.

## Посмотрим, как выполнять итерации, используя усеченное обратное распространение

Следующий код демонстрирует практическую реализацию усеченного обратного распространения. Обратите внимание, насколько она похожа на логику итераций из главы 13. Единственное существенное отличие — `batch_loss` генерируется на каждом шаге; и после каждых `bptt` шагов выполняется обратное распространение и корректировка весов. Затем чтение данных продолжается, как если бы ничего не произошло (и с использованием того же скрытого состояния, что и прежде, которое сбрасывается только при смене эпохи):



```

def train(iterations=100):
    for iter in range(iterations):
        total_loss = 0
        n_loss = 0

        hidden = model.init_hidden(batch_size=batch_size)
        for batch_i in range(len(input_batches)):

            hidden = Tensor(hidden.data, autograd=True)
            loss = None
            losses = list()
            for t in range(bptt):
                input = Tensor(input_batches[batch_i][t], autograd=True)
                rnn_input = embed.forward(input=input)
                output, hidden = model.forward(input=rnn_input,
                                                hidden=hidden)
                target = Tensor(target_batches[batch_i][t], autograd=True)
                batch_loss = criterion.forward(output, target)
                losses.append(batch_loss)
                if(t == 0):
                    loss = batch_loss
                else:
                    loss = loss + batch_loss
            for loss in losses:
                ""
            loss.backward()
            optim.step()
            total_loss += loss.data
            log = "\r Iter:" + str(iter)
            log += " - Batch " + str(batch_i+1) + "/" + str(len(input_batches))
            log += " - Loss:" + str(np.exp(total_loss / (batch_i+1)))
            if(batch_i == 0):
                log += " - " + generate_sample(70, '\n').replace("\n", " ")
            if(batch_i % 10 == 0 or batch_i-1 == len(input_batches)):
                sys.stdout.write(log)
            optim.alpha *= 0.99
            print()

train()

```

```

Iter:0 - Batch 191/195 - Loss:148.00388828554404
Iter:1 - Batch 191/195 - Loss:20.588816924127116 mhnethet tttttt t t t
....
Iter:99 - Batch 61/195 - Loss:1.0533843281265225 I af the mands your

```

## Образец вывода

Взяв прогнозы, генерируемые моделью, можно уподобиться Шекспиру!

Следующий код использует подмножество обучающей логики для получения прогнозов с помощью модели. Он сохраняет прогнозы в строке и возвращает их строковую версию. Полученный образец выглядит вполне шекспировским и даже включает символы, имитирующие диалоги:

```
def generate_sample(n=30, init_char=' '):
    s = ""
    hidden = model.init_hidden(batch_size=1)
    input = Tensor(np.array([word2index[init_char]]))
    for i in range(n):
        rnn_input = embed.forward(input)
        output, hidden = model.forward(input=rnn_input, hidden=hidden)
        output.data *= 10
        temp_dist = output.softmax()
        temp_dist /= temp_dist.sum()

        m = (temp_dist > np.random.rand()).argmax()
        c = vocab[m]
        input = Tensor(np.array([m]))
        s += c
    return s

print(generate_sample(n=2000, init_char='\n'))
```

← Температура для отбора образцов, чем больше, тем выше вероятность отбора

← Образцы из прогноза

---

I war ded abdons would.<sup>1</sup>

CHENRO:

Why, speed no virth to her,

Plirt, goth Plish love,

Befion

hath if be fe woulds is feally your hir, the confectife to the nightion

As rent Ron my hath iom

the worse, my goth Plish love,

Befion

Ass untrucerty of my fernight this we namn?

ANG, makes:

That's bond confect fe comes not commonour would be forch the conflill

As poing from your jus eep of m look o perves, the worse, my goth

Should be good lorges ever word

---

<sup>1</sup> Все результаты, возвращаемые функцией `generate_sample` в этой главе, — это бессмысленный текст, последовательности символов, не являющиеся словами. Он лишь имеет структуру, похожую на сонеты Шекспира. — *Примеч. ред.*

DESS:

Where exbinder: if not conffilll, the confectife to the nightion  
As co move, sir, this we namn?

ANG VINE PAET:

There was courter hower how, my goth Plish lo res

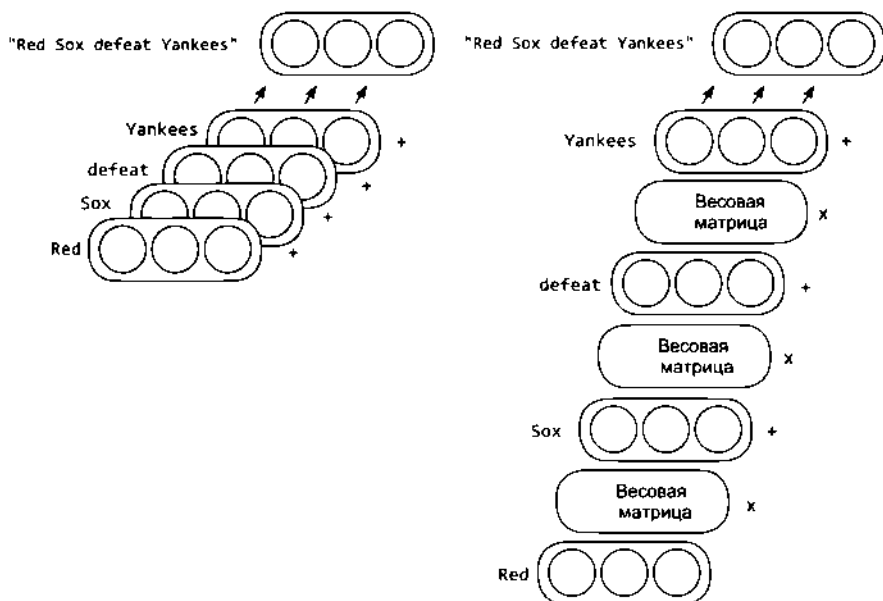
Toures

ever wo formall, have abon, with a good lorges ever word.

## Затухающие и взрывные градиенты

Простые рекуррентные нейронные сети страдают проблемой затухающих и взрывных градиентов

Это изображение вы уже видели, когда мы только начинали изучать рекуррентные нейронные сети. Тогда мы искали возможность комбинировать векторные представления слов так, чтобы их порядок имел значение. Мы добились этого, обучив матрицу, которая на следующем шаге преобразует каждое векторное представление. Прямое распространение при этом превратилось в двухэтапный процесс: сначала бралось векторное представление первого слова (слова «Red» на следующем рисунке), умножалось на весовую матрицу и к результату прибавлялось векторное представление следующего слова («Sox»). Затем полученный вектор мы вновь умножали на ту же весовую матрицу и прибавляли к результату векторное представление следующего слова. Этот процесс повторялся до окончания всей последовательности слов.



Как вы уже знаете, в процесс коррекции внутреннего состояния была добавлена дополнительная нелинейность. В результате прямое распространение превратилось в трехэтапный процесс: умножение предыдущего скрытого состояния на весовую матрицу, прибавление векторного представления следующего слова и применение нелинейной функции активации.

Обратите внимание, что эта нелинейность вносит важный вклад в стабильность сети. Независимо от длины последовательности слов значения в скрытом состоянии (которые теоретически могут расти с каждым шагом) принудительно приводятся к диапазону значений нелинейной функции (от 0 до 1 в случае функции `sigmoid`). Но обратное распространение происходит немного иначе, чем прямое, и не обладает этим замечательным свойством. Нередко обратное распространение приводит к чрезмерно большим или чрезмерно маленьким значениям. Появление больших значений вызывает расхождение (появление большого количества значений, не являющихся числами — `Not-a-Number` [NaN]), а маленькие числа препятствуют обучению сети. Рассмотрим поближе, как протекает обратное распространение в рекуррентной нейронной сети.

## Упрощенный пример обратного распространения в RNN

Сконструируем простой пример, чтобы воочию увидеть затухающие и взрывные градиенты

Ниже показан цикл обратного распространения в рекуррентной нейронной сети с функциями активации `sigmoid` и `relu`. Обратите внимание, как градиенты становятся очень маленькими/большими для `sigmoid/relu` соответственно. Увеличение обусловлено матричным умножением, а уменьшение объясняется плоской производной в хвостах функции `sigmoid` (это характерно для многих нелинейных функций активации).

```
(sigmoid,relu)=(lambda x:1/(1+np.exp(-x)), lambda x:(x>0).astype(float)*x)
weights = np.array([[1,4],[4,1]])
activation = sigmoid(np.array([1,0.01]))
```

```
print("Sigmoid Activations")
activations = list()
for iter in range(10):
    activation = sigmoid(activation.dot(weights))
    activations.append(activation)
    print(activation)
print("\nSigmoid Gradients")
gradient = np.ones_like(activation)
for activation in reversed(activations):
```

Производная функции `sigmoid` способствует чрезмерному уменьшению градиентов, когда ее значение близко к 0 или 1 (хвосты)

```

gradient = (activation * (1 - activation) * gradient)
gradient = gradient.dot(weights.transpose())
print(gradient)

print("Activations")
activations = list()
for iter in range(10):
    activation = relu(activation.dot(weights))
    activations.append(activation)
    print(activation)
print("\nGradients")
gradient = np.ones_like(activation)
for activation in reversed(activations):
    gradient = ((activation > 0) * gradient).dot(weights.transpose())
    print(gradient)

```

Матричное умножение вызывает взрывной рост градиентов, который не компенсируется нелинейной функцией активации (такой, как sigmoid)

## Sigmoid Activations

```

[0.93940638  0.96852968]
[0.9919462   0.99121735]
[0.99301385  0.99302901]

```

...

```

[0.99307291  0.99307291]

```

## Sigmoid Gradients

```

[0.03439552  0.03439552]
[0.00118305  0.00118305]

```

...

```

[4.06916726e-05 4.06916726e-05]

```

## Relu Activations

```

[23.71814585 23.98025559]
[119.63916823 118.852839 ]
[595.05052421 597.40951192]

```

...

```

[46583049.71437107 46577890.60826711]

```

## Relu Gradients

```

[5. 5.]
[25. 25.]

```

...

```

[9765625. 9765625.]

```

## Ячейки долгой краткосрочной памяти (LSTM)

LSTM — это стандартная модель, помогающая противодействовать затуханию и взрывному росту градиентов

В предыдущем разделе вы узнали, как возникает эффект затухания или взрывного роста градиентов при изменении скрытых состояний в рекуррентной нейронной сети. Проблема заключается в использовании комбинации матричного умножения и нелинейной функции активации для формирования скрытого состояния. Модель LSTM предлагает удивительно простое решение.

### ТРЮК С УПРАВЛЯЕМЫМ КОПИРОВАНИЕМ

LSTM создает следующее скрытое состояние, копируя предыдущее, а затем добавляет или удаляет информацию по мере необходимости. Для добавления и удаления информации LSTM использует механизмы, которые называют *вентилями*, или *фильтрами* (gate).

```
def forward(self, input, hidden):
    from_prev_hidden = self.w_hh.forward(hidden)
    combined = self.w_ih.forward(input) + from_prev_hidden
    new_hidden = self.activation.forward(combined)
    output = self.w_ho.forward(new_hidden)
    return output, new_hidden
```

Предыдущий фрагмент представляет логику прямого распространения в ячейке RNN. Ниже приводится новая логика прямого распространения в ячейке LSTM. Ячейка LSTM имеет два вектора со скрытым состоянием: *h* (от англ. *hidden* — скрытый) и *cell*.

В данном случае нас интересует *cell*. Обратите внимание, как он изменяется. Каждое новое значение является суммой предыдущего значения и приращения *u*, взвешенных весами *f* и *i*. Здесь *f* — это «забывающий» («forget») ventиль (фильтр). Если этот вес получит значение 0, новая ячейка «забудет» то, что видела прежде. Если *i* получит значение 1, приращение *u* будет полностью добавлено в новую ячейку. Переменная *o* — это выходной ventиль (фильтр), который определяет, какая доля состояния ячейки попадет в прогноз. Например, если все значения в *o* равны нулю, тогда строка `self.w_ho.forward(h)` вернет прогноз, полностью игнорируя состояние ячейки.

```
def forward(self, input, hidden):
    prev_hidden, prev_cell = (hidden[0], hidden[1])

    f = (self.xf.forward(input) + self.hf.forward(prev_hidden)).sigmoid()
    i = (self.xi.forward(input) + self.hi.forward(prev_hidden)).sigmoid()
    o = (self.xo.forward(input) + self.ho.forward(prev_hidden)).sigmoid()
    u = (self.xc.forward(input) + self.hc.forward(prev_hidden)).tanh()
    cell = (f * prev_cell) + (i * u)
    h = o * cell.tanh()
    output = self.w_ho.forward(h)
    return output, (h, cell)
```

## Аналогия, помогающая понять идею вентиляй LSTM

Семантически ventили LSTM похожи на операции чтения/записи с памятью

А теперь посмотрим, что мы получили! Мы получили три ventиля — *f*, *i*, *o* — и вектор приращений ячейки *u*; их можно представить как механизмы управления забыванием (*forget*), вводом (*input*), выводом (*output*) и изменением (*update*) соответственно. Они действуют вместе и гарантируют, что для кор-

ректировки информации, хранящейся в  $c$ , не потребуется применять матричное умножение или нелинейную функцию активации. Иначе говоря, избавляют от необходимости вызывать `nonlinearity(c)` или `c.dot(weights)`.

Такой подход позволяет модели LSTM сохранять информацию на протяжении временной последовательности, не беспокоясь о затухании или взрывном росте градиентов. Каждый шаг заключается в копировании (если  $f$  имеет ненулевое значение) и прибавлении приращения (если  $i$  имеет ненулевое значение). Скрытое значение  $h$  — это замаскированная версия ячейки, используемая для получения прогноза.

Обратите также внимание, что все три вентиля формируются совершенно одинаково. Они имеют свои весовые матрицы, но каждый зависит от входных значений и скрытого состояния, пропущенных через функцию `sigmoid`. Именно эта нелинейная функция `sigmoid` делает их полезными в качестве вентилях, преобразуя в диапазон от 0 до 1:

```
f = (self.xf.forward(input) + self.hf.forward(prev_hidden)).sigmoid()
i = (self.xi.forward(input) + self.hi.forward(prev_hidden)).sigmoid()
o = (self.xo.forward(input) + self.ho.forward(prev_hidden)).sigmoid()
```

И последнее критическое замечание в отношении вектора  $h$ . Очевидно, что он все еще подвержен эффекту затухания и взрывного роста градиентов, потому что фактически используется так же, как в простой рекуррентной нейронной сети. Во-первых, поскольку вектор  $h$  всегда создается из комбинации векторов, которые сжимаются с помощью `tanh` и `sigmoid`, эффект взрывного роста градиентов на самом деле отсутствует — проявляется только эффект затухания. Но в итоге в этом нет ничего страшного, потому что  $h$  зависит от ячейки  $c$ , которая может переносить информацию на дальние расстояния: ту информацию, которую затухающие градиенты не способны переносить. То есть вся перспективная информация транспортируется с помощью  $c$ , а  $h$  — это всего лишь локальная интерпретация  $c$ , удобная для получения прогноза на выходе и активации вентилях на следующем шаге. Проще говоря, способность  $c$  переносить информацию на большие расстояния нивелирует неспособность  $h$  к тому же самому.

## Слой долгой краткосрочной памяти

Для реализации LSTM можно использовать систему `autograd`

```
class LSTMCell(Layer):
    def __init__(self, n_inputs, n_hidden, n_output):
        super().__init__()
```

```
self.n_inputs = n_inputs
self.n_hidden = n_hidden
self.n_output = n_output

self.xf = Linear(n_inputs, n_hidden)
self.xi = Linear(n_inputs, n_hidden)
self.xo = Linear(n_inputs, n_hidden)
self.xc = Linear(n_inputs, n_hidden)
self.hf = Linear(n_hidden, n_hidden, bias=False)
self.hi = Linear(n_hidden, n_hidden, bias=False)
self.ho = Linear(n_hidden, n_hidden, bias=False)
self.hc = Linear(n_hidden, n_hidden, bias=False)

self.w_ho = Linear(n_hidden, n_output, bias=False)

self.parameters += self.xf.get_parameters()
self.parameters += self.xi.get_parameters()
self.parameters += self.xo.get_parameters()
self.parameters += self.xc.get_parameters()
self.parameters += self.hf.get_parameters()
self.parameters += self.hi.get_parameters()
self.parameters += self.ho.get_parameters()
self.parameters += self.hc.get_parameters()

self.parameters += self.w_ho.get_parameters()

def forward(self, input, hidden):

    prev_hidden = hidden[0]
    prev_cell = hidden[1]

    f=(self.xf.forward(input)+self.hf.forward(prev_hidden)).sigmoid()
    i=(self.xi.forward(input)+self.hi.forward(prev_hidden)).sigmoid()
    o=(self.xo.forward(input)+self.ho.forward(prev_hidden)).sigmoid()
    g = (self.xc.forward(input) +self.hc.forward(prev_hidden)).tanh()
    c = (f * prev_cell) + (i * g)
    h = o * c.tanh()

    output = self.w_ho.forward(h)
    return output, (h, c)

def init_hidden(self, batch_size=1):
    h = Tensor(np.zeros((batch_size, self.n_hidden)), autograd=True)
    c = Tensor(np.zeros((batch_size, self.n_hidden)), autograd=True)
    h.data[:,0] += 1
    c.data[:,0] += 1

return (h, c)
```



## Усовершенствование модели языка символов

### Заменяем простую ячейку RNN новой ячейкой LSTM

Выше в этой главе мы обучили модель языка символов генерировать тексты, схожие с произведениями Шекспира. Теперь тому же самому обучим модель, основанную на ячейке LSTM. К счастью, фреймворк, созданный в предыдущей главе, значительно упрощает эту задачу (полный код можно найти на сайте издательства: [www.manning.com/books/grokking-deep-learning](http://www.manning.com/books/grokking-deep-learning) или в репозитории GitHub: <https://github.com/iamtrask/grokking-deep-learning>). Вот новый код, выполняющий подготовительные операции. Все отличия от версии с простой ячейкой RNN выделены **жирным**. Обратите внимание, что в подготовке нейронной сети почти ничего не изменилось:

```
import sys, random, math
from collections import Counter
import numpy as np
import sys

np.random.seed(0)

f = open('shakespeare.txt', 'r')
raw = f.read()
f.close()

vocab = list(set(raw))
word2index = {}
for i, word in enumerate(vocab):
    word2index[word] = i
indices = np.array(list(map(lambda x: word2index[x], raw)))

embed = Embedding(vocab_size=len(vocab), dim=512)
model = LSTMCell(n_inputs=512, n_hidden=512, n_output=len(vocab))
model.w_ho.weight.data *= 0  ← Это немного поможет в обучении

criterion = CrossEntropyLoss()
optim = SGD(parameters=model.get_parameters() + embed.get_parameters(),
             alpha=0.05)

batch_size = 16
bptt = 25
n_batches = int((indices.shape[0] / (batch_size)))

trimmed_indices = indices[:n_batches*batch_size]
batched_indices = trimmed_indices.reshape(batch_size, n_batches)
batched_indices = batched_indices.transpose()

input_batched_indices = batched_indices[0:-1]
target_batched_indices = batched_indices[1:]
```

```

n_bptt = int(((n_batches-1) / bptt))
input_batches = input_batched_indices[:n_bptt*bptt]
input_batches = input_batches.reshape(n_bptt,bptt,batch_size)
target_batches = target_batched_indices[:n_bptt*bptt]
target_batches = target_batches.reshape(n_bptt, bptt, batch_size)
min_loss = 1000

```

## Обучение LSTM-модели языка символов

### Логика обучения тоже мало изменилась

Единственное, что действительно необходимо изменить в реализации обучения простой рекуррентной сети, — это логику усеченного обратного распространения, потому что в каждом шаге участвуют два скрытых вектора вместо одного. Но это относительно небольшое изменение (выделено **жирным**). Также, чтобы упростить обучение, я замедлил скорость уменьшения коэффициента  $\alpha$  и расширил журналирование процесса:

```

for iter in range(iterations):
    total_loss, n_loss = (0, 0)

    hidden = model.init_hidden(batch_size=batch_size)
    batches_to_train = len(input_batches)

    for batch_i in range(batches_to_train):

        hidden = (Tensor(hidden[0].data, autograd=True),
                  Tensor(hidden[1].data, autograd=True))
        losses = list()

        for t in range(bptt):
            input = Tensor(input_batches[batch_i][t], autograd=True)
            rnn_input = embed.forward(input=input)
            output, hidden = model.forward(input=rnn_input, hidden=hidden)

            target = Tensor(target_batches[batch_i][t], autograd=True)
            batch_loss = criterion.forward(output, target)

            if(t == 0):
                losses.append(batch_loss)
            else:
                losses.append(batch_loss + losses[-1])
        loss = losses[-1]

        loss.backward()
        optim.step()

        total_loss += loss.data / bptt
        epoch_loss = np.exp(total_loss / (batch_i+1))
        if(epoch_loss < min_loss):

```

```

        min_loss = epoch_loss
        print()
        log = "\r Iter:" + str(iter)
        log += " - Alpha:" + str(optim.alpha)[0:5]
        log += " - Batch "+str(batch_i+1)+"/"+str(len(input_batches))
        log += " - Min Loss:" + str(min_loss)[0:5]
        log += " - Loss:" + str(epoch_loss)
        if(batch_i == 0):
            s = generate_sample(n=70, init_char='T').replace("\n", " ")
            log += " - " + s
        sys.stdout.write(log)
    optim.alpha *= 0.99

```

## Настройка LSTM-модели языка символов

Я потратил два дня на настройку этой модели  
и обучил ее за одну ночь

Вот часть вывода, полученного при обучении описываемой модели. Обратите внимание, что обучение заняло очень много времени (из-за *очень большого* числа параметров). Кроме того, мне пришлось много раз повторить процесс обучения, чтобы подобрать оптимальные параметры (скорость обучения, размер пакета и др.). Окончательная модель обучалась в течение ночи (8 часов). В общем случае, чем дольше длится обучение, тем лучше будут результаты.

---

```

I:0 - Alpha:0.05 - Batch 1/249 - Min Loss:62.00 - Loss:62.00 - eeeeeeeeee
...
I:7 - Alpha:0.04 - Batch 140/249 - Min Loss:10.5 - Loss:10.7 - heres, and
...
I:91 - Alpha:0.016 - Batch 176/249 - Min Loss:9.900 - Loss:11.9757225699

```

---

```

def generate_sample(n=30, init_char=' '):
    s = ""
    hidden = model.init_hidden(batch_size=1)
    input = Tensor(np.array([word2index[init_char]]))
    for i in range(n):
        rnn_input = embed.forward(input)
        output, hidden = model.forward(input=rnn_input, hidden=hidden)
        output.data *= 15
        temp_dist = output.softmax()
        temp_dist /= temp_dist.sum()

        m = output.data.argmax()
        c = vocab[m]
        input = Tensor(np.array([m]))
        s += c
    return s

```

Выбрать в качестве прогноза символы  
с максимальной вероятностью

```
print(generate_sample(n=500, init_char='\n'))
```

---

Intestay thee.

SIR:

It thou my thar the sentastar the see the see:

Imentary take the subloud I

Stall my thentaring fook the senternight pead me, the gakentlenternot  
they day them.

KENNOR:

I stay the see talk :

Non the seady!

Sustar thou shour in the suble the see the senternow the antently the see  
the seaventlace peake,

I sentlentony my thent:

I the sentastar thamy this not thame.

## Итоги

### Модели LSTM невероятно мощные

Не следует легкомысленно воспринимать результаты обучения модели LSTM языку Шекспира. Язык — это невероятно сложное для изучения статистическое распределение, и тот факт, что модели LSTM справляются с этим довольно неплохо (на момент написания этих строк они с большим отрывом опережали все остальные модели), до сих пор вызывает удивление у меня (и у других). Варианты этой модели совсем недавно начали и продолжают использоваться для решения широкого круга задач и, вне всяких сомнений, еще долгое время будут использоваться наряду с векторными представлениями слов и сверточными слоями.

# 15 Глубокое обучение на конфиденциальных данных: введение в федеративное обучение



## В этой главе

- ✓ Проблема конфиденциальности в глубоком обучении.
- ✓ Федеративное обучение.
- ✓ Обучение определению спама.
- ✓ Взламываем федеративную модель.
- ✓ Безопасное агрегирование.
- ✓ Гомоморфное шифрование.
- ✓ Федеративное обучение с гомоморфным шифрованием.

Друзья не шпионят, истинная дружба означает и умение не вторгаться во внутреннюю жизнь друга.

Стивен Кинг. «Сердца в Атлантиде»<sup>1</sup> (1999)

<sup>1</sup> Кинг С. Сердца в Атлантиде. М.: АСТ, 2012. — Примеч. пер.

## Проблема конфиденциальности в глубоком обучении

Глубокое обучение (и использование соответствующих инструментов) часто предполагает наличие доступа к обучающим данным

Как вы уже знаете, глубокое обучение, являясь подразделом машинного обучения, основано на изучении данных. Но часто изучаемые данные являются глубоко личными. Многие модели анализируют частную информацию, рассказывающую о жизни людей то, что иным способом трудно было бы узнать. Говоря иными словами, модель может изучить тысячи жизней, чтобы помочь вам понять свою.

Основным ресурсом в глубоком обучении являются обучающие данные (естественные или синтетические). Без этих данных глубокое обучение невозможно; а поскольку самые ценные модели часто используют наборы личных данных, глубокое обучение нередко становится причиной, почему компании стремятся собрать такие данные. Они нужны им для использования в конкретных сферах.

В 2017 году компания Google опубликовала очень интересную статью<sup>1</sup> и пост в блоге, которые внесли значительный вклад в обсуждение этой темы. В Google предположили, что для обучения моделей не нужен централизованный набор данных. Компанией было предложено рассмотреть вопрос: что если вместо сбора данных в одном месте, попытаться перенести модель в данные? Этот новый и увлекательный раздел машинного обучения получил название *федеративное обучение* (federated learning), и именно о нем рассказывается в этой главе.

Что если вместо сбора всего корпуса в одном месте мы имели бы возможность перенести модель в место, где генерируются данные?

Эта простая перестановка имеет чрезвычайно большое значение. Во-первых, это означает, что для участия в цепочке глубокого обучения людям не нужно отправлять свои данные кому бы то ни было. Ценные модели в здравоохранении, управлении личными активами и других чувствительных областях можно обучать без необходимости раскрывать личную информацию. Теоретически, люди могут сохранить контроль над единственной копией своих личных данных (по крайней мере, в отношении глубокого обучения).

<sup>1</sup> <https://ai.googleblog.com/2017/04/federated-learning-collaborative.html>.

Эта методика также будет оказывать огромное влияние на конкурентную среду глубокого обучения в корпоративной конкуренции и предпринимательстве. Крупные предприятия, которые не могли (или не имели права) распространять данные о своих клиентах, теперь смогут извлекать дополнительную прибыль из них. Есть некоторые предметные области, где конфиденциальность и нормативные ограничения на использование данных препятствовали прогрессу. Здравоохранение — один из примеров таких областей, где данные часто оказываются закрытыми, что затрудняет исследования.

## Федеративное обучение

### Необязательно иметь доступ к набору данных, чтобы использовать его для обучения

Идея федеративного обучения зародилась из того, что многие данные, содержащие полезную информацию для решения задач (например, для диагностики онкологических заболеваний с использованием МРТ), трудно получить в количествах, достаточных для обучения мощной модели глубокого обучения. Кроме полезной информации, необходимой для обучения модели, наборы данных содержат также другие сведения, не имеющие отношения к решаемой задаче, но их раскрытие кому-либо потенциально может нанести вред.

Федеративное обучение — это методика заключения модели в защищенную среду и ее обучение без перемещения данных куда-либо. Рассмотрим пример.

```
import numpy as np
from collections import Counter
import random
import sys
import codecs
np.random.seed(12345)
with codecs.open('spam.txt', "r", encoding='utf-8', errors='ignore') as f:
    raw = f.readlines()

vocab, spam, ham = (set(["<unk>"]), list(), list())
for row in raw:
    spam.append(set(row[:-2].split(" ")))
    for word in spam[-1]:
        vocab.add(word)

with codecs.open('ham.txt', "r", encoding='utf-8', errors='ignore') as f:
    raw = f.readlines()

for row in raw:
```

Данные можно получить по адресу  
<http://www2.aueb.gr/users/ion/data/enron-spam/>

```

ham.append(set(row[:-2].split(" ")))
for word in ham[-1]:
    vocab.add(word)

vocab, w2i = (list(vocab), {})
for i,w in enumerate(vocab):
    w2i[w] = i

def to_indices(input, l=500):
    indices = list()
    for line in input:
        if(len(line) < l):
            line = list(line) + ["<unk>"] * (l - len(line))
            idxs = list()
            for word in line:
                idxs.append(w2i[word])
            indices.append(idxs)
    return indices

```

## Обучаем выявлять спам

Допустим, нам нужно обучить модель определять спам по электронным письмам людей

В данном случае мы говорим о классификации электронной почты. Нашу первую модель мы обучим на общедоступном наборе данных с названием Enron. Это огромный корпус электронных писем, опубликованных в ходе слушаний по делу компании Enron (теперь это стандартный аналитический корпус электронной почты). Интересный факт: я был знаком с людьми, которым по роду своей деятельности приходилось читать/комментировать этот набор данных, и они отмечают, что люди посылали друг другу в этих письмах самую разную информацию (часто очень личную). Но так как этот корпус был обнародован в ходе судебных разбирательств, в настоящее время его можно использовать без ограничений.

Код в предыдущем и в этом разделе реализует только подготовительные операции. Файлы с входными данными (`ham.txt` и `spam.txt`) доступны на веб-странице книги: [www.manning.com/books/grokking-deep-learning](http://www.manning.com/books/grokking-deep-learning) и в репозитории GitHub: <https://github.com/iamtrask/Grokking-Deep-Learning>. Мы должны предварительно обработать его, чтобы подготовить его для передачи в класс `Embedding` из главы 13, где мы создали свой фреймворк глубокого обучения. Как и прежде, все слова в этом корпусе преобразуются в списки индексов. Кроме того, мы приводим все письма к одинаковой длине в 500 слов, либо обрезаая их, либо дополняя лексемами `<unk>`. Благодаря этому мы получаем набор данных прямоугольной формы.



```

spam_idx = to_indices(spam)
ham_idx = to_indices(ham)

train_spam_idx = spam_idx[0:-1000]
train_ham_idx = ham_idx[0:-1000]

test_spam_idx = spam_idx[-1000:]
test_ham_idx = ham_idx[-1000:]

train_data = list()
train_target = list()

test_data = list()
test_target = list()

for i in range(max(len(train_spam_idx), len(train_ham_idx))):
    train_data.append(train_spam_idx[i%len(train_spam_idx)])
    train_target.append([1])

    train_data.append(train_ham_idx[i%len(train_ham_idx)])
    train_target.append([0])

for i in range(max(len(test_spam_idx), len(test_ham_idx))):
    test_data.append(test_spam_idx[i%len(test_spam_idx)])
    test_target.append([1])

    test_data.append(test_ham_idx[i%len(test_ham_idx)])
    test_target.append([0])

def train(model, input_data, target_data, batch_size=500, iterations=5):
    n_batches = int(len(input_data) / batch_size)
    for iter in range(iterations):
        iter_loss = 0
        for b_i in range(n_batches):
            # дополняющая лексема не должна оказывать влияния на прогноз
            model.weight.data[w2i['<unk>']] *= 0
            input = Tensor(input_data[b_i*bs:(b_i+1)*bs], autograd=True)
            target = Tensor(target_data[b_i*bs:(b_i+1)*bs], autograd=True)

            pred = model.forward(input).sum(1).sigmoid()
            loss = criterion.forward(pred, target)
            loss.backward()
            optim.step()

            iter_loss += loss.data[0] / bs

        sys.stdout.write("\r\tLoss:" + str(iter_loss / (b_i+1)))
        print()
    return model

def test(model, test_input, test_output):
    model.weight.data[w2i['<unk>']] *= 0

    input = Tensor(test_input, autograd=True)
    target = Tensor(test_output, autograd=True)

```

```
pred = model.forward(input).sum(1).sigmoid()
return ((pred.data > 0.5) == target.data).mean()
```

Определив вспомогательные функции `train()` и `test()`, мы можем инициализировать нейронную сеть и обучить ее, написав всего несколько строк кода. Уже после трех итераций сеть оказывается в состоянии классифицировать контрольный набор данных с точностью 99.45 % (контрольный набор данных хорошо сбалансирован, поэтому этот результат можно считать превосходным):

```
model = Embedding(vocab_size=len(vocab), dim=1)
model.weight.data *= 0
criterion = MSELoss()
optim = SGD(parameters=model.parameters(), alpha=0.01)

for i in range(3):
    model = train(model, train_data, train_target, iterations=1)
    print("% Correct on Test Set: " + \
          str(test(model, test_data, test_target)*100))
```

---

```
Loss:0.037140416860871446
% Correct on Test Set: 98.65
Loss:0.011258669226059114
% Correct on Test Set: 99.15
Loss:0.008068268387986223
% Correct on Test Set: 99.45
```

## Сделаем модель федеративной

Выше было выполнено самое обычное глубокое обучение. Теперь добавим конфиденциальности

В предыдущем разделе мы реализовали пример анализа электронной почты. Теперь поместим все электронные письма в одно место. Это старый добрый метод работы (который все еще широко используется во всем мире). Для начала симулируем окружение федеративного обучения, в котором имеется несколько разных коллекций писем:

```
bob = (train_data[0:1000], train_target[0:1000])
alice = (train_data[1000:2000], train_target[1000:2000])
sue = (train_data[2000:], train_target[2000:])
```

Пока ничего сложного. Теперь мы можем выполнить ту же процедуру обучения, что и прежде, но уже на трех отдельных наборах данных. После каждой итерации мы будем усреднять значения в моделях Боба (Bob), Алисы (Alice) и Сью

(Sue) и оценивать результаты. Обратите внимание, что некоторые методы федеративного обучения предусматривают объединение после каждого пакета (или коллекции пакетов); я же решил сохранить код максимально простым:

```
for i in range(3):
    print("Starting Training Round...")
    print("\tStep 1: send the model to Bob")
    bob_model = train(copy.deepcopy(model), bob[0], bob[1], iterations=1)

    print("\n\tStep 2: send the model to Alice")
    alice_model = train(copy.deepcopy(model),
                        alice[0], alice[1], iterations=1)

    print("\n\tStep 3: Send the model to Sue")
    sue_model = train(copy.deepcopy(model), sue[0], sue[1], iterations=1)

    print("\n\tAverage Everyone's New Models")
    model.weight.data = (bob_model.weight.data + \
                        alice_model.weight.data + \
                        sue_model.weight.data)/3

    print("\t% Correct on Test Set: " + \
          str(test(model, test_data, test_target)*100))
    print("\nRepeat!!\n")
```

Ниже дан фрагмент с результатами. Эта модель достигла практически того же уровня точности, что и предыдущая, и теоретически у нас отсутствовал доступ к обучающим данным — или нет? Как бы там ни было, но каждый человек изменяет модель в процессе обучения, верно? Неужели мы действительно не сможем ничего выудить из их наборов данных?

```
Starting Training Round...
Step 1: send the model to Bob
Loss:0.21908166249699718
.....
Step 3: Send the model to Sue
Loss:0.015368461608470256

Average Everyone's New Models
% Correct on Test Set: 98.8
```

## Взламываем федеративную модель

Рассмотрим простой пример, как извлечь информацию из обучающего набора данных

Федеративное обучение страдает двумя большими проблемами, особенно трудноразрешимыми, когда у каждого человека имеется лишь маленькая горстка

обучающих примеров, – скорость и конфиденциальность. Как оказывается, если у кого-то имеется лишь несколько обучающих примеров (или модель, присланная вам, была обучена лишь на нескольких примерах: обучающем пакете), вы все еще можете довольно много узнать об исходных данных. Если представить, что у вас есть 10 000 человек (и у каждого имеется очень небольшой объем данных), большую часть времени вы потратите на пересылку модели туда и обратно и не так много – на обучение (особенно если модель очень большая).

Но не будем забегать вперед. Давайте посмотрим, что можно узнать после того, как пользователь выполнит обновление весов на одном пакете:

```
import copy

bobs_email = ["my", "computer", "password", "is", "pizza"]

bob_input = np.array([[w2i[x] for x in bobs_email]])
bob_target = np.array([[0]])

model = Embedding(vocab_size=len(vocab), dim=1)
model.weight.data *= 0

bobs_model = train(copy.deepcopy(model),
                    bob_input, bob_target, iterations=1, batch_size=1)
```

Боб создает и обучает модель на электронных письмах в своем почтовом ящике. Но так случилось, что он сохранил свой пароль, послав самому себе письмо с текстом: «My computer password is pizza»<sup>1</sup>. Наивный Боб! Посмотрев, какие весовые коэффициенты изменились, мы можем выяснить словарь (и понять смысл) письма Боба:

```
for i, v in enumerate(bobs_model.weight.data - model.weight.data):
    if(v != 0):
        print(vocab[i])
```

Таким несложным способом мы узнали сверхсекретный пароль Боба (и, возможно, его кулинарные предпочтения). И что же делать? Как доверять федеративному обучению, если так легко узнать, какие обучающие данные вызвали изменение весов?

```
is
pizza
computer
password
my
```

<sup>1</sup> Мой пароль от компьютера – pizza. – *Примеч. пер.*

## Безопасное агрегирование

Усредним веса от миллионов людей до того, как кто-то сможет увидеть их

Решение заключается в том, чтобы Боб никогда не передавал градиенты в открытом виде, как в примере выше. Но как передать градиенты, чтобы никто их не увидел? В социологии для этого используется один интересный метод, который называется *рандомизированным ответом*.

Суть в следующем. Представьте, что вы проводите опрос и 100 респондентам задаете вопрос: совершали ли они отвратительные поступки. Конечно, все они ответят «Нет», даже если вы пообещаете им, что никому не расскажете. Но можно предложить им подбросить монету дважды (так, чтобы вы не видели результат) и попросить сказать правду, если после первого броска выпала «решка»; а если выпал «орел», они должны ответить «Да» или «Нет» в соответствии с результатом второго броска.

В этом случае вы фактически не просите людей честно ответить на вопрос о том, совершали ли они отвратительные поступки. Истинные ответы оказываются скрытыми в случайном шуме первого и второго бросков монеты. Если 60 % людей сказали «Да», вы можете определить (используя простые вычисления), что примерно 70 % опрошенных совершали такие поступки (плюс-минус несколько процентов). Идея состоит в том, что случайный шум помогает каждому скрыть истинный ответ и получить общую правдоподобную картину.

### КОНФИДЕНЦИАЛЬНОСТЬ ЧЕРЕЗ ПРАВДОПОДОБНОЕ ОТРИЦАНИЕ

Высокая вероятность, что ответ каждого обусловлен случайным шумом, защищает их конфиденциальность, давая возможность правдоподобного отрицания. Этот прием формирует основу для безопасного агрегирования и, в более общем смысле, дифференциальной конфиденциальности.

Мы видим только общую статистику. (Мы никогда не увидим чьих-либо прямых ответов — только пары или более крупные группы ответов.) Поэтому, чем больше людей будет участвовать в объединении перед добавлением шума, тем меньше шума придется добавить, чтобы скрыть индивидуальные ответы (и тем точнее будут результаты).

В контексте федеративного обучения мы могли бы (при желании) добавить сколько угодно шума, но это навредило бы обучению. Поэтому сначала сложим

все градиенты от всех участников так, чтобы никто не мог видеть никаких градиентов, кроме своих собственных. Класс задач, связанных с этим, называется *безопасным агрегированием* (secure aggregation), и для их решения нам понадобится еще один (очень мощный) инструмент: *гомоморфное шифрование*.

## Гомоморфное шифрование

### Есть возможность выполнять вычисления с зашифрованными данными

Одно из интереснейших достижений лежит на пересечении искусственного интеллекта (включая глубокое обучение) и криптографии. В центре этого пересечения находится очень мощная технология с названием гомоморфное шифрование. Выражаясь простым языком, гомоморфное шифрование позволяет выполнять вычисления с зашифрованными значениями, не расшифровывая их.

В частности, нас интересует сложение таких значений. Чтобы объяснить суть этого приема, потребуется целая книга, но я покажу вам, как она работает, используя лишь несколько определений. Во-первых, шифрование чисел производится с помощью *открытого ключа*. Расшифровывание зашифрованных чисел осуществляется с помощью *закрытого ключа*. Зашифрованное значение называется *зашифрованным текстом*, а расшифрованное — *открытым текстом*.

Рассмотрим пример гомоморфного шифрования с использованием библиотеки `phe`. Чтобы установить ее, выполните команду `pip install phe` или загрузите библиотеку из репозитория GitHub: <https://github.com/n1analytics/python-paillier>:

```
import phe

public_key, private_key = phe.generate_paillier_keypair(n_length=1024)

x = public_key.encrypt(5)  ← Зашифрует число 5
y = public_key.encrypt(3)  ← Зашифрует число 3
z = x + y  ← Сложит зашифрованные числа

z_ = private_key.decrypt(z)  ← Расшифрует результат
print("The Answer: " + str(z_))
```

---

The Answer: 8

Этот код шифрует два числа (5 и 3) и складывает их в зашифрованном виде. Необычно, правда? Есть еще один метод, в чем-то схожий с гомоморфным

шифрованием: *безопасные многосторонние вычисления*. Познакомиться с ним поближе можно в блоге «Cryptography and Machine Learning» (<https://mortendahl.github.io>).

А теперь вернемся к задаче безопасного агрегирования. Зная о существовании возможности складывать числа, не видя их, мы легко можем решить эту задачу. Человек, инициализирующий модель, посылает Бобу, Алисе и Сью открытый ключ, чтобы они могли зашифровать свои градиенты. Затем Боб, Алиса и Сью (не имеющие закрытого ключа) общаются друг с другом непосредственно и складывают свои градиенты, результат сложения посылается обратно владельцу модели, который расшифровывает их с помощью закрытого ключа.

## Федеративное обучение с гомоморфным шифрованием

Используем гомоморфное шифрование для защиты объединяемых градиентов

```
model = Embedding(vocab_size=len(vocab), dim=1)
model.weight.data *= 0

# обратите внимание, что при использовании этого алгоритма в действующем
# программном обеспечении значение n_length должно быть не меньше 1024
public_key, private_key = phe.generate_paillier_keypair(n_length=128)

def train_and_encrypt(model, input, target, pubkey):
    new_model = train(copy.deepcopy(model), input, target, iterations=1)

    encrypted_weights = list()
    for val in new_model.weight.data[:,0]:
        encrypted_weights.append(public_key.encrypt(val))
    ew = np.array(encrypted_weights).reshape(new_model.weight.data.shape)

    return ew

for i in range(3):
    print("\nStarting Training Round...")
    print("\tStep 1: send the model to Bob")
    bob_encrypted_model = train_and_encrypt(copy.deepcopy(model),
                                             bob[0], bob[1], public_key)

    print("\n\tStep 2: send the model to Alice")
    alice_encrypted_model=train_and_encrypt(copy.deepcopy(model),
                                             alice[0],alice[1],public_key)

    print("\n\tStep 3: Send the model to Sue")
    sue_encrypted_model = train_and_encrypt(copy.deepcopy(model),
```

```

sue[0], sue[1], public_key)

print("\n\tStep 4: Bob, Alice, and Sue send their")
print("\tencrypted models to each other.")
aggregated_model = bob_encrypted_model + \
    alice_encrypted_model + \
    sue_encrypted_model

print("\n\tStep 5: only the aggregated model")
print("\tis sent back to the model owner who")
print("\tcan decrypt it.")
raw_values = list()
for val in sue_encrypted_model.flatten():
    raw_values.append(private_key.decrypt(val))
new = np.array(raw_values).reshape(model.weight.data.shape)/3
model.weight.data = new

print("\t% Correct on Test Set: " + \
    str(test(model, test_data, test_target)*100))

```

Теперь можно попробовать запустить новую схему обучения, которая включает дополнительный шаг. Алиса, Боб и Сью складывают свои гомоморфно зашифрованные модели перед передачей нам, поэтому мы никогда не узнаем, какие градиенты получены от каждого из них (форма правдоподобного отрицания). В промышленном варианте можно также добавить дополнительный случайный шум, чтобы достичь определенного порога конфиденциальности, требуемого Бобу, Алисе и Сью (в соответствии с их личными предпочтениями). Подробнее об этом я расскажу в будущей книге.

```

Starting Training Round...
Step 1: send the model to Bob
Loss:0.21908166249699718

Step 2: send the model to Alice
Loss:0.2937106899184867
...
...
...
% Correct on Test Set: 99.15

```

## Итоги

**Федеративное обучение — одно из самых захватывающих достижений в глубоком обучении**

Я абсолютно уверен, что в ближайшие годы федеративное обучение изменит ландшафт глубокого обучения. Этот подход откроет доступ к новым наборам



данных, которые раньше считались слишком чувствительными для этого, и принесет много пользы обществу, расширяя возможности для бизнеса. Это пересечение исследований в области криптографии и искусственного интеллекта, на мой взгляд, является самым захватывающим пересечением десятилетия.

Главное, что сдерживает практическое использование этих методов, — это отсутствие их поддержки в современных инструментах глубокого обучения. Перелом наступит, когда любой сможет выполнить команду `pip install...` и получить доступ к фреймворкам глубокого обучения, где конфиденциальность и безопасность стоят на первом месте, и в которые встроены такие технологии, как федеративное обучение, гомоморфное шифрование, дифференциальная конфиденциальность и безопасные многосторонние вычисления (и вам не надо быть экспертом, чтобы использовать их).

Следуя этому убеждению, весь прошлый год я работал с разработчиками открытого ПО в рамках проекта OpenMined, добавляя эти примитивы в основные фреймворки глубокого обучения. Если вы верите в важность этих инструментов обеспечения конфиденциальности и безопасности, посетите сайт проекта по адресу <http://openmined.org> или страницу в GitHub (<https://github.com/OpenMined>). Выкажите свою поддержку, хотя бы просто поставив звезду нескольким репозиториям; и присоединяйтесь к нам, если у вас есть такая возможность (slack.openmined.org — это комната в чате).

# 16 Куда пойти дальше: краткий путеводитель



## В этой главе

- ✓ Шаг 1: начните изучать PyTorch.
- ✓ Шаг 2: начните изучать следующий курс по глубокому обучению.
- ✓ Шаг 3: купите учебник по математике глубокого обучения.
- ✓ Шаг 4: заведите блог и рассказывайте в нем о глубоком обучении.
- ✓ Шаг 5: Twitter.
- ✓ Шаг 6: напишите руководство на основе академической статьи.
- ✓ Шаг 7: получите доступ к GPU.
- ✓ Шаг 8: найдите оплачиваемую работу, связанную с глубоким обучением.
- ✓ Шаг 9: присоединитесь к открытому проекту.
- ✓ Шаг 10: ищите единомышленников.

Если ты уверен, что сможешь, — ты прав; если ты думаешь, что не сможешь, — тоже прав.

*Генри Форд, промышленник,  
владелец автомобилестроительных заводов*

## Поздравляю!

Если вы дочитали до этого места, значит, у вас за спиной более 300 страниц текста о глубоком обучении

Вы сделали это! Вы освоили массу материала. Я вами горжусь, и вы тоже должны гордиться собой. Сегодня отличный повод устроить вечеринку. Теперь вы понимаете основные идеи, лежащие в основе искусственного интеллекта, и должны быть совершенно уверены в своей готовности обсуждать эту тему и в своей способности изучать сложные понятия.

Эта последняя глава состоит из нескольких коротких разделов, где рассматриваются дальнейшие возможные шаги, особенно если эта книга стала для вас первым знакомством с глубоким обучением. Я предполагаю, что вы заинтересованы продолжить развиваться в этом направлении или хотя бы продолжить заниматься им как хобби, и надеюсь, что мои замечания помогут вам выбрать правильное направление (даже притом, что эти очень общие рекомендации могут не иметь прямого отношения к вам).

## Шаг 1: начните изучать PyTorch

Созданный нами фреймворк глубокого обучения очень похож на PyTorch

Вы узнали, как реализовать глубокое обучение с использованием NumPy — простой библиотеки матричных операций. Создали свой фреймворк глубокого обучения и попробовали применить его на практике. Но далее вам нужно не только познакомиться с новыми архитектурами нейронных сетей, но также освоить настоящие фреймворки, которые вы могли бы использовать в своих экспериментах. Они надежнее и с ними вы будете допускать меньше ошибок. Они работают (*намного*) быстрее и дают возможность использовать и изучать код, написанный другими.

Почему я советую начать с PyTorch? Есть много хороших фреймворков, но имеющим опыт использования NumPy фреймворк PyTorch покажется наиболее знакомым. Кроме того, фреймворк, который мы создали в главе 13, имеет API, очень похожий на API фреймворка PyTorch. Я намеренно сделал так, чтобы подготовить вас к встрече с настоящим фреймворком. Выбрав PyTorch, вы будете чувствовать себя как дома. С другой стороны, выбор фреймворка глубокого обучения похож на выбор факультета в Хогвартсе: они все хороши (но PyTorch — это определенно Гриффиндор).

Следующий вопрос: как осваивать PyTorch? Лучший способ: пройти курс, где преподается глубокое обучение с использованием фреймворка. Он освежит в памяти понятия, с которыми вы познакомились здесь, и покажет, где в PyTorch находятся те или иные инструменты. (Например, вы вспомните стохастический градиентный спуск и узнаете, где он находится в PyTorch API.) Лучшими на момент написания этих строк были: курс глубокого обучения Udacity Deep Learning Nanodegree (впрочем, здесь я могу быть необъективен, потому что сам участвовал в его разработке) и fast.ai. Также пригодятся такие замечательные ресурсы, как <https://pytorch.org/tutorials> и <https://github.com/pytorch/examples>.

## Шаг 2: начните изучать следующий курс по глубокому обучению

Я сам осваивал глубокое обучение, снова и снова изучая одни и те же понятия

Удобно думать, что одной книги или курса достаточно, чтобы полностью освоить глубокое обучение, но это не так. Даже если бы в этой книге мы рассмотрели все понятия (на самом деле это далеко не так), все равно, чтобы по-настоящему освоить их, вы должны познакомиться с разными точками зрения на них (понимаете, как я трудился?). Я изучил, наверное, с полдюжины разных курсов (и серий видеоуроков на YouTube), формируясь как разработчик, а также посмотрел тысячи роликов на YouTube и прочитал массу статей в блогах, описывающих основные понятия.

Найдите онлайн-курсы о глубоком обучении от крупных университетов или лабораторий искусственного интеллекта (Стэнфорд, MIT, Оксфорд, Монреальский университет, Нью-Йоркский университет и так далее). Посмотрите все видеоуроки. Выполните все упражнения. Пройдите курс обучения в fast.ai и Udacity, если сможете. Изучайте одни и те же понятия снова и снова. Применяйте их на практике. Познакомьтесь с ними как можно ближе. Вы должны прочно усвоить основы.

## Шаг 3: купите книгу по математике глубокого обучения

Изучите математику глубокого обучения

В университете я получил степень бакалавра в области прикладной дискретной математики, но, занимаясь глубоким обучением, я узнал об алгебре, численных

методах и математической статистике намного больше, чем в университетских аудиториях. Кроме того, и это может показаться удивительным, я многое узнал, изучая код NumPy и математические задачи, которые он реализует, стремясь понять ход решения. Так я освоил математику глубокого обучения на более высоком уровне. Это хороший совет, который, я надеюсь, вы примете к сведению.

Если вы сомневаетесь в выборе книги, могу посоветовать лучшую, пожалуй, из имевшихся в продаже на момент написания этих строк: *Deep Learning*<sup>1</sup>, написанную Яном Гудфеллоу (Ian Goodfellow), Иошуа Бенджио (Yoshua Bengio) и Аароном Курвиллем (Aaron Courville) и изданную в MIT Press (2016). Это не самая безумная книга с точки зрения математики, но следующий шаг по сравнению с данной книгой (а вводная часть с описанием математических обозначений чудо как хороша).

## Шаг 4: заведите блог и рассказывайте в нем о глубоком обучении

Ничто так не помогло мне в получении знаний или в карьере, как блог

Возможно, я должен был поставить этот шаг на первое место, но все же поместил его на четвертое. Ничто не помогло мне в освоении глубокого обучения (и в карьере специалиста по глубокому обучению) больше, чем попытки поделиться своими знаниями с другими в своем блоге. Роль учителя заставляет объяснять сложные понятия простыми словами, а боязнь оказаться в глупом положении заставит вас хорошо потрудиться.

Со мной приключилась забавная история: одна из моих первых публикаций в блоге попала на Hacker News, но она была написана так ужасно, что крупный исследователь, работающий в одной из известных лабораторий искусственного интеллекта, разбил меня в пух и прах в комментариях. Это поколебало мою уверенность в себе, но одновременно и помогло. Я понял, что когда что-то читаю и мне трудно понять суть написанного, это не моя проблема — просто человек, написавший это, не уделил достаточно времени, чтобы объяснить все подробности, необходимые для понимания всей идеи. Он не привел аналогий, которые помогли бы понять его слова.

В общем, заведите свой блог. Попробуйте попасть на главную страницу Hacker News или ML Reddit. Начните с рассказа об основных понятиях. Попробуйте

<sup>1</sup> Гудфеллоу Ян, Бенджио Иошуа, Курвилль Аарон, «Глубокое обучение». ДМК-Пресс, 2018. — Примеч. пер.

сделать это лучше других. Пусть вас не волнует, если выбранная тема уже неоднократно освещалась. На сегодняшний день наибольшей популярностью пользуется моя статья в блоге «A Neural Network in 11 Lines of Python» («Нейронная сеть в 11 строках на Python»)<sup>1</sup>, в которой рассказывается о самой хорошо изученной теме глубокого обучения — простой нейронной сети прямого распространения. Но я смог преподнести эту тему по-новому, что помогло некоторым людям. Главная причина популярности этой статьи: я написал ее так, что она помогла мне самому понять описываемый предмет. В этом вся суть. Учитесь других так, как вам хотелось бы, чтобы учили вас.

И старайтесь не писать кратких обзоров на тему глубокого обучения! Краткие обзоры неинформативны и неинтересны. Пишите руководства. Каждая статья должна включать нейронную сеть, обучаемую чему-то, которую читатель сможет загрузить и запустить. В статье должно содержаться также подробное описание, что делает каждая часть, чтобы даже пятилетний ребенок мог понять происходящее. Это непреложное правило. Иногда, поработав три дня над двухстраничной статьей, у вас может появиться желание бросить все, но не сдавайтесь: приложите все силы и удивите других! Одна хорошая статья в блоге может изменить вашу жизнь. Верьте мне.

Если вы захотите устроиться на работу в магистратуру или аспирантуру, чтобы заняться исследованиями в области искусственного интеллекта, выберите исследователя, с которым хотели бы работать, и напишите руководства на основе его работ. Всякий раз, когда я поступал так, это влекло за собой встречу с этим исследователем. Таким способом вы покажете, что понимаете его работы, а это одно из обязательных условий, необходимых, чтобы у исследователя появилось желание работать с вами. Это намного лучше, чем простое письмо, — если ваше руководство попадет на Reddit, Hacker News или куда-то еще, кто-нибудь другой обязательно напишет об этом исследователю первым. Иногда исследователь может даже сам обратиться к вам.

## Шаг 5: Twitter

**Много рассуждений об искусственном интеллекте можно найти в Twitter**

В Twitter я встречал исследователей чаще, чем где-либо еще, и там же я узнал почти обо всех статьях, которые читал, потому что следовал за теми, кто писал об этом. Вы должны оставаться в курсе последних достижений и, что особенно

<sup>1</sup> <http://iamtrask.github.io/2015/07/12/basic-python-network/>.

важно, участвовать в общении. Я начал с того, что нашел нескольких исследователей искусственного интеллекта и подписался на них, а потом на их подписчиков. Так ко мне начала поступать информация, что очень помогло мне. (Только не превращайте зависание в Twitter в зависимость!)

## Шаг 6: напишите руководство на основе академической статьи

**Twitter + ваш блог = руководство на основе академической статьи**

Регулярно просматривайте ленту в Twitter, пока не встретите статью, которая выглядит интересной и не требует для проверки ее идей безумного количества графических процессоров (GPU). Напишите на ее основе руководство. Для этого вам придется внимательно прочитать статью, расшифровать представленный в ней математический аппарат и разобраться в настройках, через которые прошли исследователи. Нет лучшего упражнения, если вам интересны абстрактные исследования. Моя первая статья, опубликованная на международной конференции по машинному обучению International Conference on Machine Learning (ICML), появилась потому, что я прочитал статью другого исследователя и затем перепроектировал код, используя алгоритм word2vec. В какой-то момент, прочитав чью-нибудь статью, вы воскликнете: «Минутку! Я думаю, что смогу сделать это лучше!» И вот вы уже исследователь.

## Шаг 7: получите доступ к GPU

**Чем быстрее будут протекать ваши эксперименты, тем быстрее вы будете учиться**

Не секрет, что графические процессоры (GPU) ускоряют обучение нейронных сетей от 10 до 100 раз, но это также означает, что до 100 раз быстрее вы сможете перебирать свои идеи (плохие или хорошие). Это очень ценно для глубокого обучения. Одна из ошибок, которые я допустил в своей карьере, была в том, что я слишком долго выжидал, прежде чем начал использовать GPU. Не уподобляйтесь мне: купите хороший графический процессор NVIDIA или используйте графические ускорители K80, доступные через сервис Google Colab. Кроме того, компания NVIDIA иногда позволяет бесплатно использовать свое оборудование студентам на разных состязаниях по искусственному интеллекту, но вы должны следить за ними.

## Шаг 8: найдите оплачиваемую работу, связанную с глубоким обучением

**Чем больше времени вы будете уделять глубокому обучению, тем быстрее вы будете учиться**

Еще один поворот в моей карьере случился, когда я получил работу, которая позволила мне заняться изучением инструментов и исследований в области глубокого обучения. Станьте специалистом по обработке данных, исследователем или внештатным консультантом по статистике. Вам нужна возможность зарабатывать деньги и продолжать учиться в рабочее время. Такая работа существует, просто надо приложить усилия, чтобы ее найти.

Ваш блог пригодится вам в поиске такой работы. Напишите в блоге хотя бы пару сообщений, показывающих, что вы умеете все, для чего кто-то мог бы нанять вас. Это идеальный способ отрекомендовать себя (намного лучше, чем заявить о наличии ученой степени по математике). Идеальный кандидат — это тот, кто уже показал, что может справиться с работой.

## Шаг 9: присоединитесь к открытому проекту

**Лучший способ наладить контакты и построить карьеру в области искусственного интеллекта — стать основным разработчиком проекта с открытым исходным кодом**

Выберите понравившийся вам фреймворк и примите участие в его развитии. Вы начнете контактировать с исследователями из ведущих лабораторий (которые будут читать/утверждать предлагаемые вами изменения) раньше, чем узнаете об этом. Я знаю многих, кто получил хорошую работу (казалось бы, на пустом месте), выбрав такой подход.

Учтите, что при этом вам придется потратить немало времени. Читайте код. Заводите друзей. Начните с добавления модульных тестов и документации с описанием кода, затем поработайте над ошибками и, наконец, присоединитесь к более крупным проектам. Да, это случится не быстро, но это инвестиции в ваше же будущее. Если не знаете, с чего начать, начните с известного фреймворка глубокого обучения, например PyTorch, TensorFlow или Keras, или приходите ко мне в проект OpenMined (который я считаю самым крупным проектом с открытым исходным кодом). Мы очень доброжелательны к новичкам.



## Шаг 10: ищите единомышленников

Я многое узнал о глубоком обучении еще и потому, что люблю общаться с друзьями

О глубоком обучении многое я узнал, посиживая в ресторанчике Bongo Java со своими друзьями, которые тоже интересовались этой темой. Я старался не изменять этой традиции, даже когда бился в поисках трудной ошибки (однажды я потратил два дня, чтобы найти недостающую точку) или не мог усвоить какие-то понятия, потому что мне нравилось общаться с людьми, вызывающими у меня симпатию. Не стоит недооценивать благотворного влияния такого общения. Если вы находитесь в месте, которое вам нравится, и с людьми, с которыми хотели бы общаться, вы сможете работать дольше и продвигаться вперед быстрее. Это не сложно, нужно лишь поддерживать общение. А еще вы можете даже немного повеселиться, пока вместе!

## грокаем

# Глубокое обучение

**Г**лубокое обучение — это раздел искусственного интеллекта, цель которого — научить компьютеры обучаться с помощью нейронных сетей — технологии, созданной по образу и подобию человеческого мозга. Онлайн-переводчики, беспилотные автомобили, рекомендации по выбору товаров именно для вас и виртуальные голосовые помощники — вот лишь несколько достижений, которые стали возможны благодаря глубокому обучению.

*«Грокаем глубокое обучение»* научит конструировать нейронные сети с нуля! Эндрю Траск знакомит со всеми деталями и тонкостями этой нелегкой задачи. Python и библиотека NumPy способны научить ваши нейронные сети видеть и распознавать изображения, переводить любые тексты на все языки мира и даже писать не хуже Шекспира!

### Что вы найдете внутри книги:

- Теоретические основы глубокого обучения
- Приемы создания и обучения нейронных сетей
- Работа с естественным языком
- Федеративное обучение и работа с конфиденциальными данными

Вам не понадобятся специальные навыки, выходящие за рамки школьного курса математики и базовых навыков программирования.

*Эндрю Траск* — аспирант Оксфордского университета, научный сотрудник в DeepMind. Он занимался обучением крупнейшей в мире искусственной нейронной сети в Digital Reasoning и отвечал за разработку планов анализа для платформы когнитивных вычислений Synthesys.

«Сложная тема простым языком!»

— Винуа Гупта, Microsoft

«Отличный обзор всех вопросов глубокого обучения, написанный искусным учителем, который поможет проложить ваш путь в эту тему».

— Келвин Д. Минс, International Technology Ventures

«Все понятия даются на прекрасных иллюстрациях и примерах».

— Калян Редди, AnsiGlobal

«Превосходный путеводитель, который в простой и понятной форме знакомит читателя с основами глубокого обучения».

— Еремей Валетон, Lancaster University/Fermilab

«Пошаговое руководство по изучению искусственного интеллекта».

— Ян Спарк, I and K Consulting



Заказ книг:  
тел.: (812) 703-73-74  
books@piter.com

WWW.PITER.COM  
каталог книг и интернет-магазин

 [instagram.com/piterbooks](https://www.instagram.com/piterbooks)

 [youtube.com/ThePiterBooks](https://www.youtube.com/ThePiterBooks)

 [vk.com/piterbooks](https://vk.com/piterbooks)

 [facebook.com/piterbooks](https://facebook.com/piterbooks)

 MANNING

ISBN: 978-5-4461-1334-7



9 785446 113347